

1. Memory Efficient Doubly Linked List – Introduction

In conventional implementation, we need to keep a forward pointer to the next item on the list and a backward pointer to the previous item. That means elements in doubly linked list implementations consist of data, a pointer to the next node and a pointer to the previous node in the list as shown below:

```
typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
} Node;
```

But in XOR (Memory Efficient) List we have :

```
typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
```

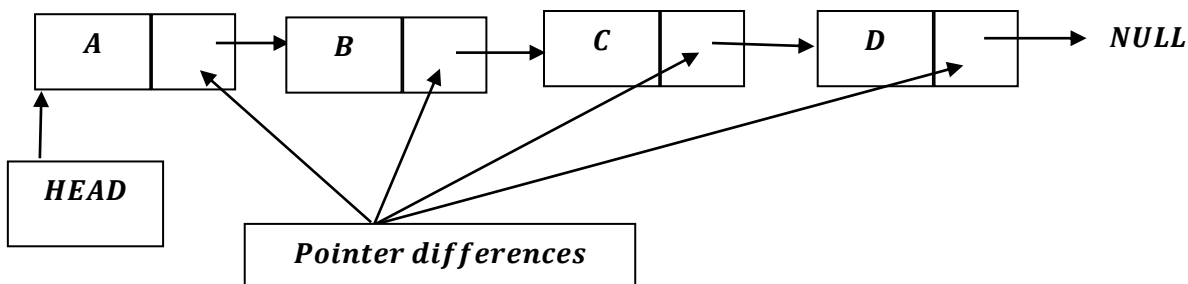
The *'ptrdiff'* pointer field contains the difference between the pointer to the next node and the pointer to the previous node. The pointer difference is calculated by using exclusive – or ($\oplus$) operation.

$$\text{ptrdiff} = \text{pointer to previous node} \oplus \text{pointer to next node}.$$

The *'ptrdiff'* = *'npx'* as NPX is stands for NEXT – PREVIOUS XOR.

The same *'ptrdiff'* definition for *'NPX'* : the difference between the pointer to the next node and the pointer to the previous node.

The *'ptrdiff'* of the start node(head node) is the $\oplus$ of NULL and next node (next node to head). Similarly , the *ptrdiff* of end node is the $\oplus$ of previous node(previous to end node) and NULL.



In the example above,

- The next pointer of A is : $\text{NULL} \oplus B$
- The next pointer of B is: $A \oplus C$
- The next pointer of C is : $B \oplus D$
- The next pointer of D is $C \oplus \text{NULL}$

How does it work?

To find the answer to this question let us consider the properties of $\oplus$.

- $X \oplus X = 0$
- $X \oplus 0 = X$
- $X \oplus Y = Y \oplus X$ (Symmetric)
- $(X \oplus Y) \oplus Z = X \oplus (Y \oplus Z)$ (transitive)

For the example above, let us assume that we are at C node and want to move to B. We know that C's ptrdiff is defined as $B \oplus D$. If we want to move to B, performing $\oplus$ on C's ptrdiff with D would give B. This due to the fact that:

$$(B \oplus D) \oplus D = B \text{ (since , } D \oplus D = 0 \text{)}$$

Similarly , if we want to move to D, then we have to apply $\oplus$ to C's ptrdiff with D would give B. This is due to fact that:

$$(B \oplus D) \oplus B = D \text{ (since, } B \oplus B = 0 \text{)}$$

Form the above discussion, we can see that just by using a single pointer, we can move back and forth.

A memory – efficient implementation of a doubly linked list is possible with minimal compromising of timing efficiency.

2. a. Memory Efficient Doubly Linked List
–Creation of Empty List and Time Complexity

Program:

```
#include <stdint> // Required for uintptr_t (essential for
pointer XOR)
#include <stdlib> // Required for malloc and free
#include <iostream>

typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

Node *head;

Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}

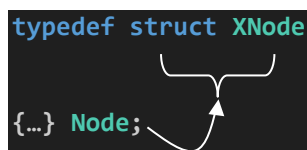
void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```


→ *Node*, is a structure variable i. e. the name given to struct *XNode* achieved through *typedef* keyword, gives new name to existing data types, here for struct *XNode*. Struct is a composite data type which contains group of several variables or different data types , grouped under a single name or single type.

Node is given name to struct *XNode* , As shown below:

```
typedef struct XNode
{
    ...
} Node;
```

A diagram with a curly brace grouping the text 'struct XNode' in the line 'typedef struct XNode'. An arrow points from this brace down to the word 'Node' in the line '} Node;'. Another arrow points from the curly brace of the closing brace '}' in the same line down to the word 'Node'.

We can also write it as:

```
typedef struct XNode
{
    int data;
    XNode *npx; // XOR of next and prev pointer
} Node;
```

And , now :

```
Node *head;
```

is a global variable, which are accessible to any part of the program, and as we need to start from head and keep track of it , its easy to perform by making the variable global otherwise with local scope we have to create each seperate head variable while calling each modular functions.

```

#include <cstdlib> // Required for uintptr_t (essential for
pointer XOR)
#include <stdlib.h> // Required for malloc and free
#include <iostream>

typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

Node *head;

```

The Rule: Zero – Initialization

In C ++, any variable declared at the global scope (outside of all functions) has static storage duration. The C ++ standard guarantees that variables with static storage duration are zero – initialized by the compiler if you do not explicitly initialize them.

For a pointer, "zero – initialization" means it is automatically set to a null pointer.

Node * head

<i>Address</i>	<i>Head</i>
0x021	<i>NULLPTR</i>

*And , then * head initialized to null pointer(explicitly).*

```
void createEmptyList()  
{  
    head = nullptr;  
}
```



Note: It is just explicitly assigning `Node * head` to `nullptr`, which is already assigned with `NULLPTR` , since `Node * head` is global pointer variable, following :The Zero Initialization Rule.

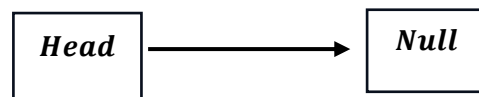
C + + version 11 → Null Pointer vs MACRO NULL

In C + +03 and earlier versions, there was no dedicated null pointer literal. Developers often used the constant 0 or macros like `NULL` to represent null pointers. The problem with this approach is that an integer value of 0 could accidentally be assigned to a pointer variable, leading to unexpected behavior and potential crashes.

In essence, `nullptr` is a special literal value that represents the absence of a valid pointer in C + +. It was introduced in the C + +11 standard to enhance type safety and improve code clarity. The `nullptr`, however, is a distinct type, `std :: nullptr_t`, designed explicitly for pointer absence. By using `nullptr`, the compiler can enforce type safety and distinguish between an intentionally null pointer and an integer value. This reduces the potential

for errors caused by accidental assignment of integers to pointers. The nullptr enhances code readability and self – documentation. Its explicit representation of a null pointer makes the code's intent clearer. Using nullptr makes the code more readable and less prone to errors. This helps prevent subtle bugs that could arise from accidentally assigning the wrong value to a pointer. The nullptr is a dedicated keyword that represents the null pointer value.

The nullptr is a keyword introduced in C++11 to represent a null pointer. It is used to indicate that a pointer does not point to any object or function. The nullptr has its own type, std::nullptr_t, which can be implicitly converted to any pointer type but not to integral types. This helps prevent errors where a null pointer might be confused with an integer. We can use nullptr wherever you need to assign or compare a pointer to a null value. Hence Creation of Empty Linked List , according to Linked List representation:



Head points to Null , is creation of empty linked list.

XORing Memory Address

```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
```

*In C++, `uintptr_t` stands for **Unsigned Integer Pointer Type**.*

It is a special integer type (defined in the `<stdint.h>` header) that is guaranteed to be **large enough to hold a memory address (pointer)** without losing data.

Here is the breakdown of why it exists and why it is critical for operations like the XOR Linked List.

1. The Problem: "How big is a pointer?"

In C++, the size of a pointer (`Node*`, `int*`, etc.) depends on your computer's architecture.

- **32-bit System:** A pointer is **4 bytes** (32 bits).
- **64-bit System:** A pointer is **8 bytes** (64 bits).

If you try to store a pointer inside a standard integer like `int`, you run a huge risk:

- An `int` is usually **4 bytes**.
- If you are on a 64-bit computer, your address is **8 bytes**.
- **Result:** If you cast the pointer to an `int`, you chop off half the address (truncation), causing the program to crash.

2. The Solution: `uintptr_t`

`uintptr_t` is a "smart" integer. The compiler automatically changes its size based on the system you are compiling for.

- *Compiling on 32-bit? `uintptr_t` becomes a 32-bit unsigned integer.*
- *Compiling on 64-bit? `uintptr_t` becomes a 64-bit unsigned integer.*

It guarantees: `sizeof(uintptr_t) == sizeof(void*)`

3. Visualizing the Safety

Imagine a 64 – bit Pointer Address: 0x7FFF5555 (Big number).

Scenario A: Using int (Dangerous)

Pointer (8 bytes): [0x7FFF5555]
int (4 bytes): [0x5555] < – – DATA LOSS! The '7FFF' is gone.

Scenario B: Using uintptr_t (Safe)

Pointer (8 bytes): [0x7FFF5555]
uintptr_t (8 bytes): [0x7FFF5555] < – – Perfect fit.

4. Why did we use it in the XOR Linked List?

In C ++, you are not allowed to use bitwise operators (like XOR ^) on pointers.

```
Node * a = ...;  
Node * b = ...;  
Node * c = a ^ b; // ERROR! Compiler forbids math on pointers.
```

To perform the math, we must:

- 1. Treat the pointers as just raw numbers (Integers).*
- 2. Perform the math.*
- 3. Turn the result back into a pointer.*

*Because we need an integer type that is **guaranteed** to hold the pointer safely, we cast to uintptr_t.*

// 1. Cast pointers to uintptr_t (Safe Integers)

```
uintptr_t addrA = (uintptr_t)a;  
uintptr_t addrB = (uintptr_t)b;
```

// 2. Perform Math

```
uintptr_t result = addrA ^ addrB;
```

// 3. Cast back to Pointer

```
return (Node *)result;
```

Summary Table

<i>Type</i>	<i>Size (Typical 64 – bit System)</i>	<i>Can hold a pointer?</i>	<i>Safe for XOR Logic?</i>
<i>int</i>	<i>4 bytes</i>	<i>No (Too small)</i>	<i>No</i>
<i>long long</i>	<i>8 bytes</i>	<i>Yes</i>	<i>Yes (but wastes space on 32 – bit systems)</i>
<i>uintptr_t</i>	<i>8 bytes</i>	<i>Yes (Perfect fit)</i>	<i>Yes (Portable)</i>

Using `long long` **might work accidentally on some systems**, but it is **NOT safe** and can **break your XOR linked list** depending on:

- CPU architecture (32-bit / 64-bit)
- Compiler
- Platform
- Pointer size

Let's break it down clearly.

1. Pointer size vs long long size

Case 1: 64-bit system

- Pointer size = 8 bytes (64 bits)
- `long long` = **also** 8 bytes (usually 64 bits)

Here, XOR might seem to work:

```
long long a = (long long)ptr; // allowed  
long long b = (long long)ptr2;  
long long x = a ^ b;  
Node * result = (Node *)x;
```

This will work on many 64-bit machines.

BUT not guaranteed by the C++ standard.

Case 2: 32-bit system

- *Pointer size = 4 bytes (32 bits)*
- *long long = 8 bytes*

*Now casting pointer → long long **extends** the pointer:*

Example pointer:

ptr = 0x8040ABCD

Cast to long long:

00000000 00000000 08040ABC D0000000 (64 – bit) // implementation dependent

Different compilers may fill the upper bits differently.

→ XOR breaks

→ Converted pointer becomes invalid

→ Program crashes while traversing

Case 3: Exotic architectures

On some systems:

- *Pointer = 128 bits (rare embedded or capability – based)*
- *long long = still 64 bits*

*Now pointer → long long is **guaranteed loss of data** → XOR list becomes corrupt.*

Case 4: Strict aliasing / alignment issues

Casting pointer → long long violates:

- **Pointer strict aliasing rules:**

The strict aliasing rule is a C/C++ optimization guideline that allows compilers to assume pointers of different types do not point to the same memory location. This rule enables aggressive optimization, but violating it (by accessing memory through a pointer of a different type, with exception → `char *`) can lead to undefined behavior and

incorrect program results. Programs must follow this rule unless specific compiler flags (like `-fno-strict-aliasing` in GCC) are used to disable the optimization, which can break other optimizations.

```
int foo(int * a, float * b) {  
    * a = 5;  
    * b = 3.14;  
    return * a; // < -- -- The Critical Line  
}
```

Strict Aliasing (The Fast Way):

1. Write 5 to address a.
2. Write 3.14 to address b.
3. Assume a and b cannot overlap because int and float are different types.
4. Return 5 immediately (no memory reload needed).

The "Universal Key" Exception (char*)

There is one major exception to this rule. `char*` (or `unsigned char*`) is allowed to alias anything. The standard acknowledges that sometimes need to look at the raw bytes of an object (like when copying memory or saving to a file).

```
int i = 5;  
  
char * ptr = (char *)&i; // This is LEGAL.  
  
cout << * ptr;          // This reads the first byte of i. Safe.
```

This is why functions like `memcpy` work perfectly – they cast everything to `char*` internally.

- **Alignment constraints** (*violates*):

"Violating alignment constraints" *refers to a hardware limitation regarding how the CPU accesses memory.*

When we use a long long (which is typically 8 bytes), the CPU expects it to be stored at a memory address that is a multiple of 8.

If we try to read or write a long long at an address like 0x1003 (which is not divisible by 8), we are violating alignment constraints.

The Rules of Alignment

Every data type has a "natural alignment" requirement:

Type	Size	Alignment Requirement	Allowed Addresses (Example)
Char	1 byte	Any address	0x01, 0x02, 0x03...
Int	4 bytes	Multiple of 4	0x00, 0x04, 0x08, 0x0C (INT = 12)...
long long	8 bytes	Multiple of 8	0x00, 0x08, 0x10 (INT = 16), 0x18 (INT = 24)...

Final Safe Rule

✓ `uintptr_t` is ***the only correct type*** for pointer arithmetic and XOR

✓ It is ***legally guaranteed*** to hold a pointer

✓ Works on ***every architecture***

✓ No alignment problems

Why C + + created `uintptr_t`?

Because no other integer type is guaranteed to:

- be the same size as a pointer*
- be able to hold a pointer without losing bits*
- allow safe bitwise operations*

Even `size_t` is not guaranteed to match pointer size.

But `uintptr_t` is explicitly guaranteed to:

- ✓store pointer bit patterns*
- ✓XOR safely*
- ✓convert back safely*

Summary Table

<i>Type</i>	<i>Guaranteed to hold pointer bit?</i>	<i>Safe for XOR list?</i>
<i><code>uintptr_t</code></i>	<i>✓YES</i>	<i>✓SAFE</i>
<i><code>long long</code></i>	<i>✗NO</i>	<i>✗UNSAFE</i>
<i><code>unsigned long</code></i>	<i>✗NO</i>	<i>✗UNSAFE</i>
<i><code>size_t</code></i>	<i>✗NO</i>	<i>✗UNSAFE</i>
<i><code>unsigned int</code></i>	<i>✗NO</i>	<i>✗UNSAFE</i>

2. b. Memory Efficient Doubly Linked List

–Creation of Empty List – Space Complexity and Algorithm

Program:

```
#include <stdint> // Required for uintptr_t (essential for
pointer XOR)
#include <stdlib> // Required for malloc and free
#include <iostream>

typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

Node *head;

Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}

void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```

Space Complexity

XOR Linked List Analysis

1. Input Space Complexity (Parameter – Only): This measures the space required by the parameters passed to the functions.

- ***XOR(a, b): Takes two pointers as parameters. Each pointer occupies a fixed amount of memory (4 or 8 bytes).***
- ***createEmptyList(): Takes no parameters.***
- ***main(): Takes no parameters.***
- ***Conclusion: The Input Space Complexity (Parameter – Only) is $O(1)$.***

2. Input Space Complexity (Full – Data): This considers the space occupied by the data structure upon which the program operates.

- ***Global Variable (head): The program declares a single global pointer head. A pointer occupies a fixed size.***
- ***The List Nodes: At this specific stage (initialization), the list is empty. There are $n = 0$ nodes.***
- ***Analysis: Since the list is currently empty, the only "data" present is the global head pointer.***

- ***Conclusion: The Input Space Complexity (Full – Data) is $O(1)$.***

3. Auxiliary Space Complexity

This measures the extra space used during the execution of the functions (stack space, local variables, dynamic allocation).

- ***Variables: * createEmptyList uses no local variables.***
 - ***XOR uses type casting internally to perform the bitwise operation, which happens in registers or a tiny amount of stack space.***
- ***Dynamic Allocation: There is no use of malloc or new in this specific snippet.***
- ***Stack: The function call overhead for createEmptyList is constant. Even if XOR is called, it does not use recursion.***
- ***Conclusion: The Auxiliary Space Complexity is $O(1)$.***

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm for Creation of Empty Memory Efficient Doubly Linked List

- ***Step 1: DEFINE STRUCTURE*** Define a structure DLL with three members:
 - ***An integer data to store the value of the node.***
 - ***A pointer next of type DLL * to point to the next node in the list.***
 - ***A pointer prev of type DLL * to point to the previous node in the list.***
- ***Step 2: TYPE ALIAS:*** Use typedef to create an alias Node for struct DLL to simplify the syntax throughout the program.
- ***Step 3: DECLARE GLOBAL POINTER:*** Declare a global pointer head of type Node *. This will act as the entry point to the list and allow all functions to access the list's starting position.
- ***Step 4: CREATE INITIALIZATION FUNCTION:*** Define a function createEmptyList that takes no arguments:
 - ***SET head := nullptr (or NULL in C) to indicate that the list currently contains no nodes.***
- ***Step 5: EXECUTE INITIALIZATION:*** In the main function, call createEmptyList() to initialize the data structure before any nodes are added.

Memory Efficient Doubly Linked List

– Insert At Beginning And Time Complexity.

Program :

```
typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;
Node *head;
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    // Logic:
    // New Node's npx = NULL ^ current_head
    newNode->npx = XOR(nullptr, head);

    // If the list was not empty, we must update the old
head's npx
    if (head != nullptr)
    {
        // Old head's npx was: NULL ^ Next
        // We need it to be: newNode ^ Next

        // 1. Decode the 'next' node of the current head
```

```

    Node *next = XOR(nullptr, head->npx);

    // 2. Update head's npx
    head->npx = XOR(newNode, next);
}

head = newNode;
cout << data << " inserted at the beginning." << endl;
}
... .

case 1:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtBeginning(data);
    break;

```

Program Analysis:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ data(an integer) is typically 4 bytes.

→ npx(a pointer to another Node) is typically 8 bytes.

→ Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npx) .

And we already know:

```
#include <stdint> // Required for uintptr_t (essential for
pointer XOR)
#include <stdlib> // Required for malloc and free
#include <iostream>

typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

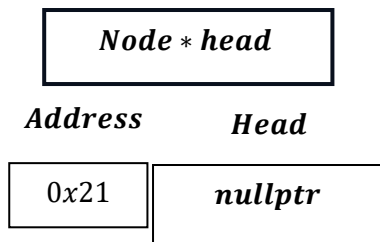
Node *head;

Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}

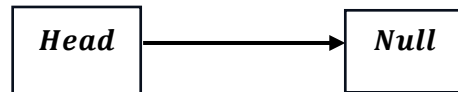
void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```

Head will point to nullptr , at creation of linked list.



i. e.



Now coming to insert at beginning :

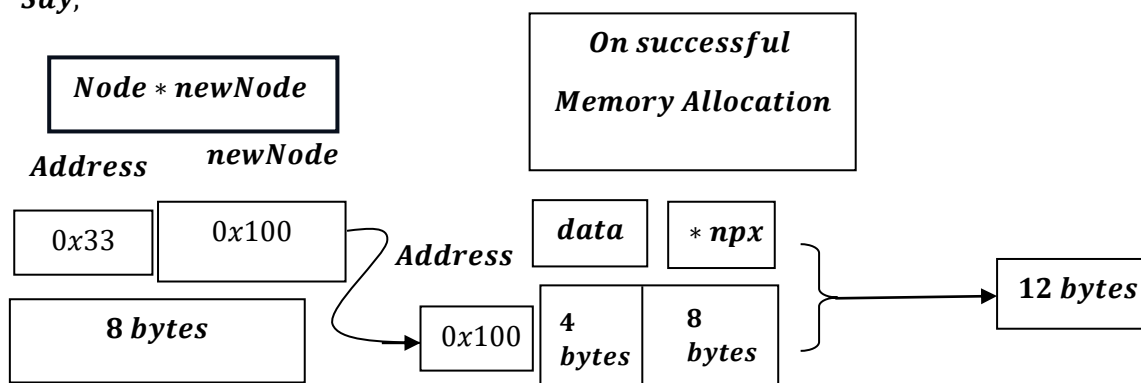
```

Node *newNode = (Node *)malloc(sizeof(Node));
if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}
  
```

What does it mean?

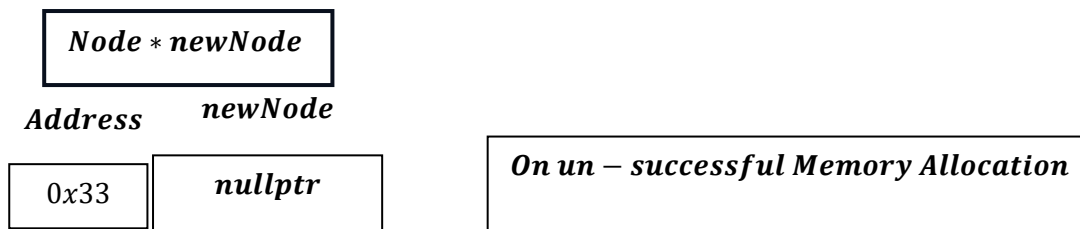
→ *newNode will try to allocate memory for data i.e. 4 bytes , npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



But on Un – Successful memory allocation:

→ if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.



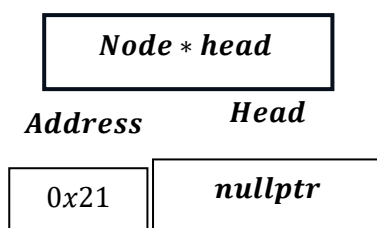
Now, if the value of the pointer variable newNode is nullptr, then:

One get output : `Error: Memory Allocation Failed as a message.`

Now,

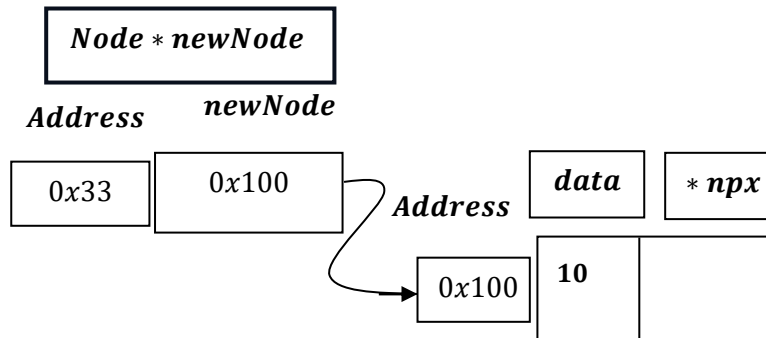
```
case 1:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtBeginning(data);
    break;
```

Say , data = 10 and currently head is pointed at nullptr.



```
newNode->data = data;
```

Say,



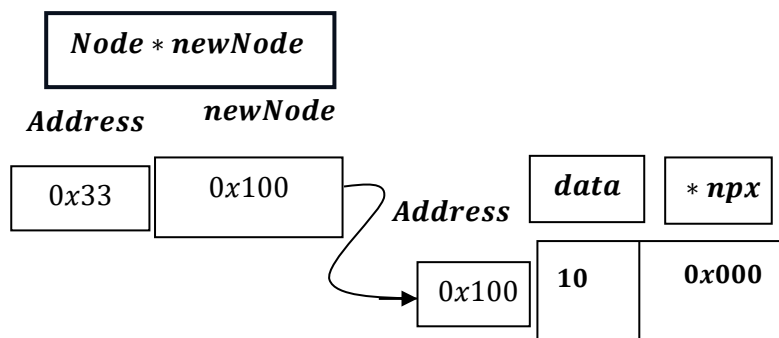
```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}

newNode->npx = XOR(nullptr, head);
```

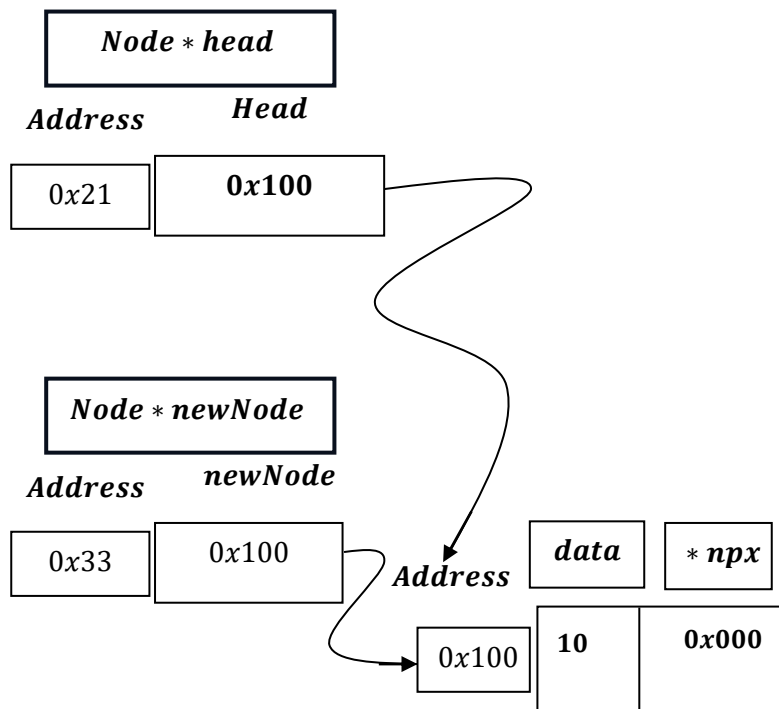
Represent : NULL $\rightarrow$ 0x000

Current Head: NULL $\rightarrow$ 0x0000

Hence, newNode $\rightarrow$ npx = XOR(NULL: 0x000 $\oplus$ HEAD: 0x000) = 0x000



```
head = newNode;
```

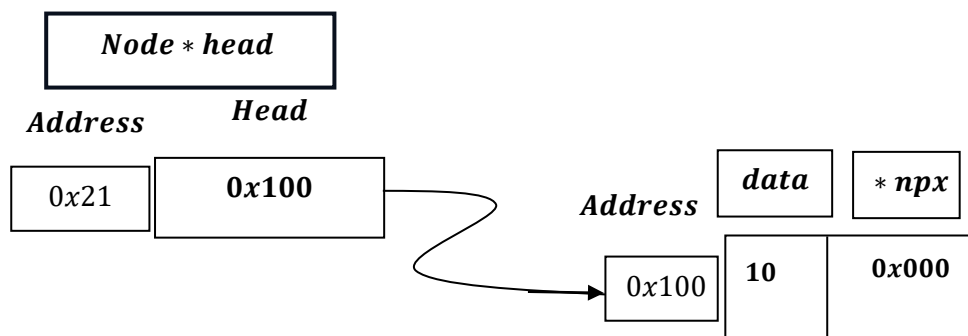
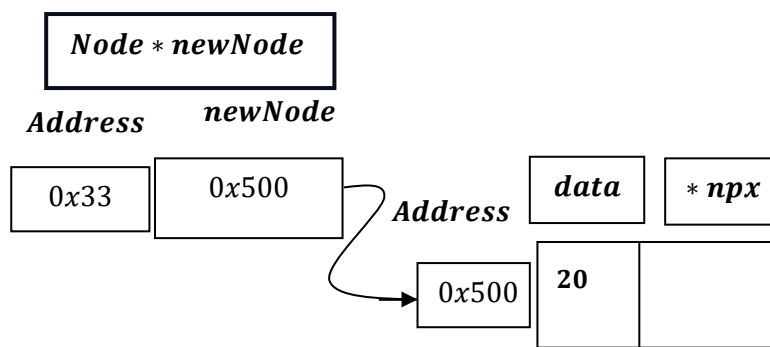


When Head is not NULL!

```
newNode->data = data;
newNode->npx = XOR(nullptr, head);
if (head != nullptr)
{
    Node *next = XOR(nullptr, head->npx);

    // 2. Update head's npx
    head->npx = XOR(newNode, next);
}
head = newNode;
cout << data << " inserted at the beginning." << endl;
}
```

Say, next data we want to : 20.



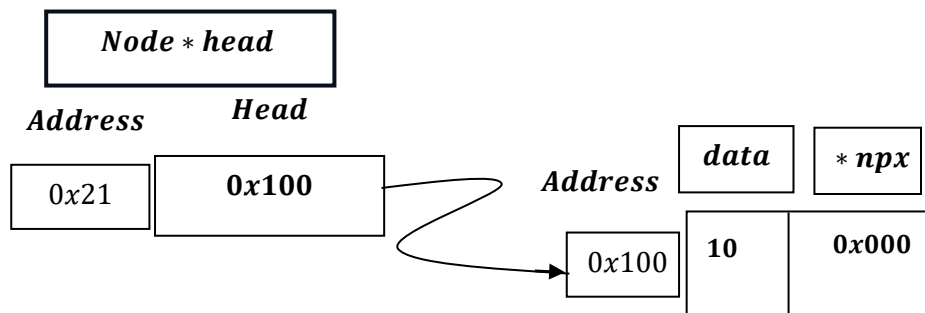
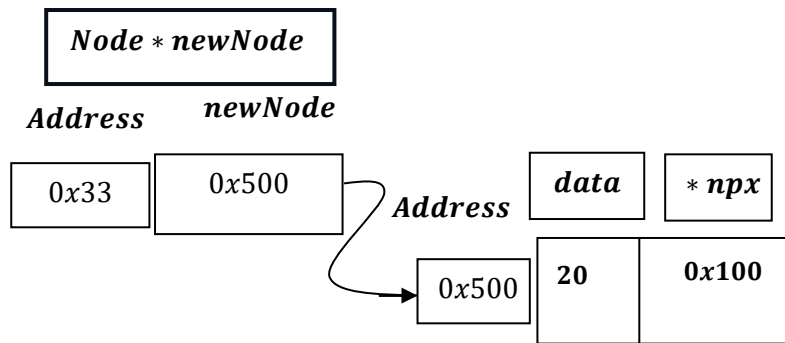
```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}

newNode->npx = XOR(nullptr, head);
```

Represent : $NULL \rightarrow 0x000$

Current Head: 0x100

Hence, $newNode \rightarrow npx = XOR(NULL: 0x000 \oplus HEAD: 0x100) = 0x100$

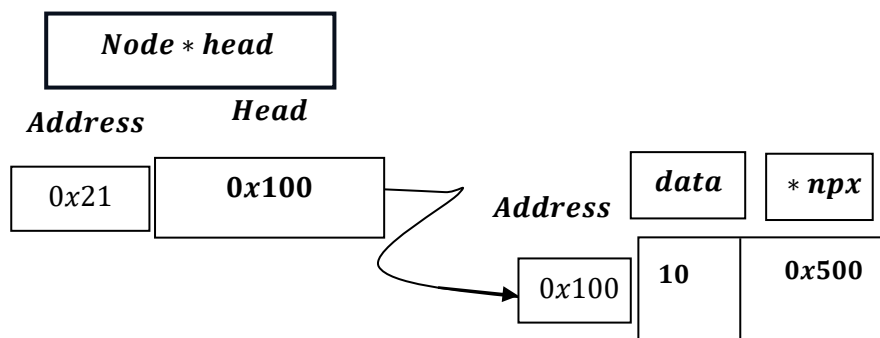
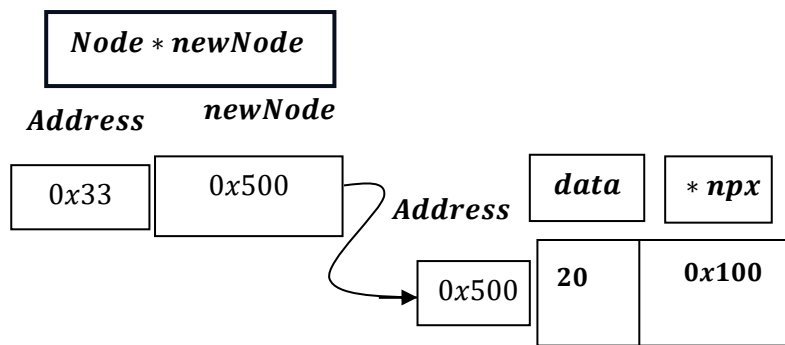


```
Node *next = XOR(nullptr, head->npx);
```

$Node * next = XOR(NULL: 0x000, Head \rightarrow NPX: 0x000); \Rightarrow 0x000 \oplus 0x000 = 0x000.$

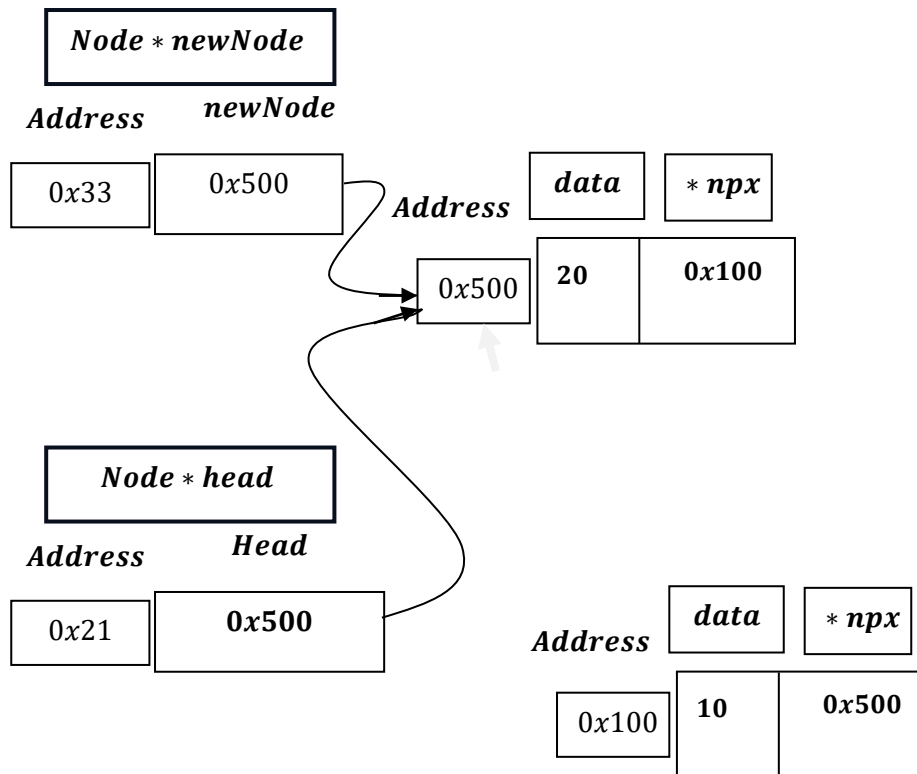
```
head->npx = XOR(newNode, next);
```

$Head \rightarrow NPX = XOR(newNode: 0x500, next: 0x000) = 0x500 \oplus 0x000 = 0x500.$



```
head = newNode;
```

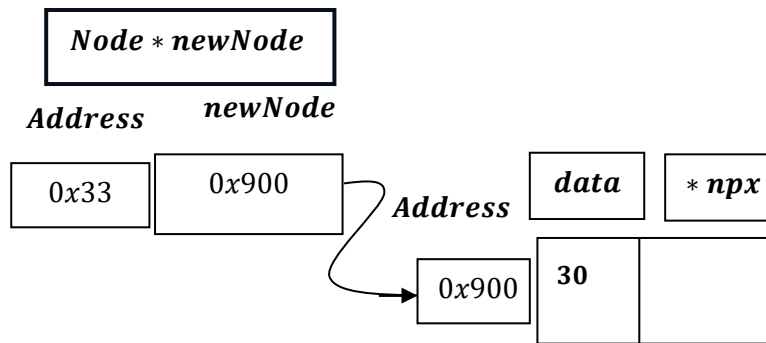
head = *newNode* = 0x500;



Say, we want to enter another data: 30.

```
newNode->data = data;
newNode->npx = XOR(nullptr, head);
if (head != nullptr)
{
    Node *next = XOR(nullptr, head->npx);

    // 2. Update head's npx
    head->npx = XOR(newNode, next);
}
head = newNode;
cout << data << " inserted at the beginning." << endl;
}
```



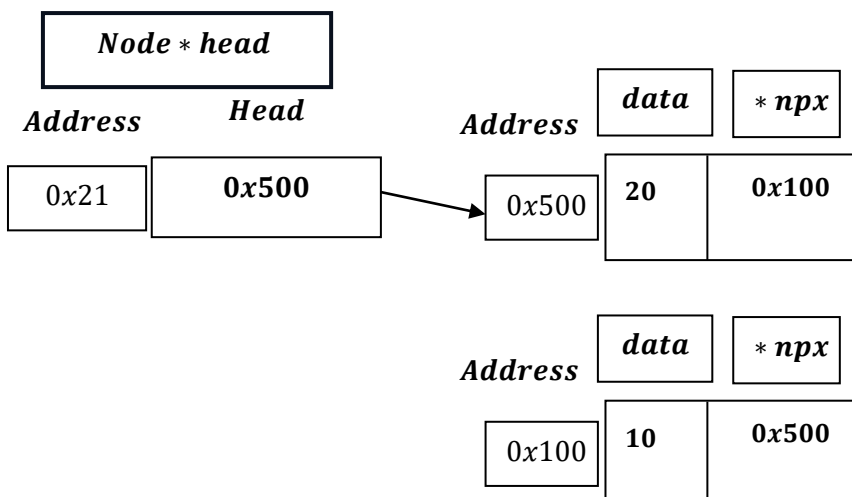
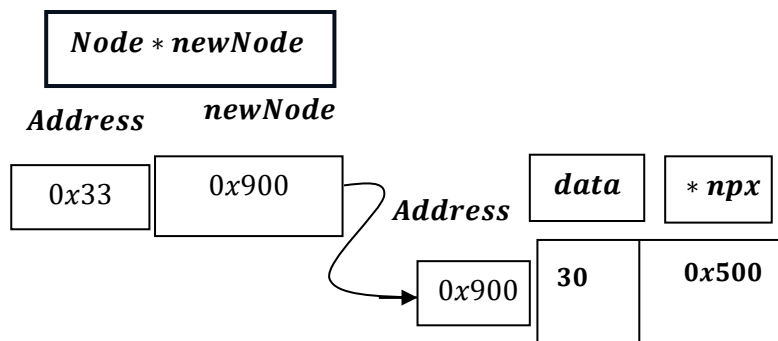
```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}

newNode->npx = XOR(NULLptr, head);
```

Represent : NULL → 0x000

Current Head: 0x500

Hence, newNode → npx = XOR(NULL: 0x000 ⊕ HEAD: 0x500) = 0x500



```
Node *next = XOR(nullptr, head->npx);
```

$Node * next = XOR(NULL: 0x000, Head \rightarrow NPX: 0x100); \Rightarrow 0x000 \oplus 0x100 = 0x100.$

```
head->npx = XOR(newNode, next);
```

$Head \rightarrow NPX = XOR(newNode: 0x900, next: 0x100) = 0x900 \oplus 0x100 = 0x800.$

$0x900 = 1001\ 0000\ 0000_2$

$0x100 = 0001\ 0000\ 0000_2$

XORing we get:

$1001\ 0000\ 0000_2$

$\oplus$

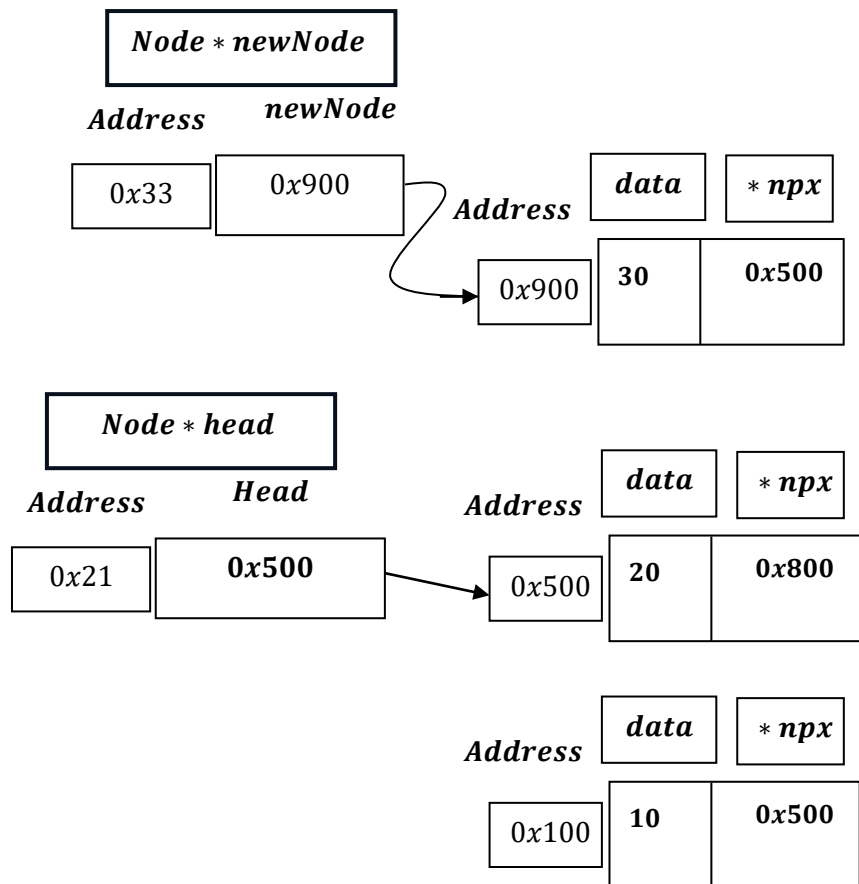
$0001\ 0000\ 0000_2$

$1000\ 0000\ 0000_2$

$A[INPUT]$	$B[INPUT]$	$XOR(\oplus)$
0	0	0
0	1	1
1	0	1
1	1	0

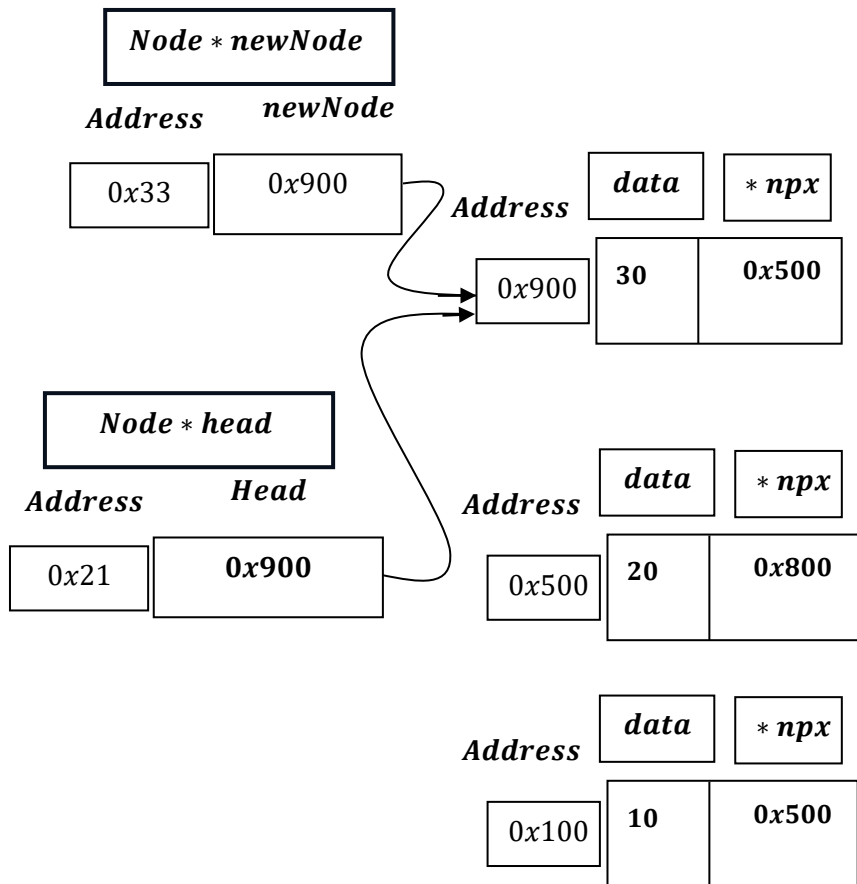
i. e. in 4 bits : $1001_2 \oplus 0001_2 = 1000_2 = 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 8_{10}.$

$\Rightarrow 0x900 \oplus 0x100 = 0x800.$

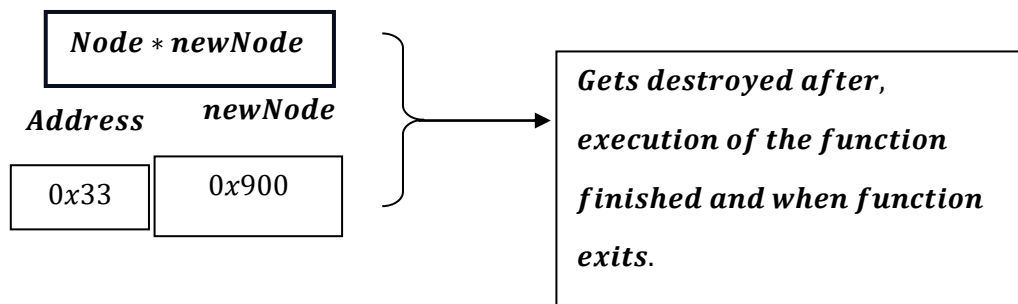


```
head = newNode;
```

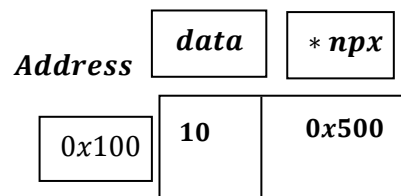
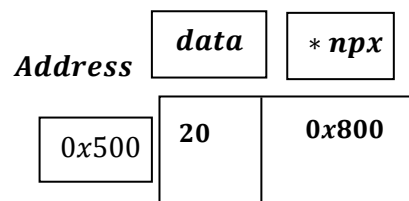
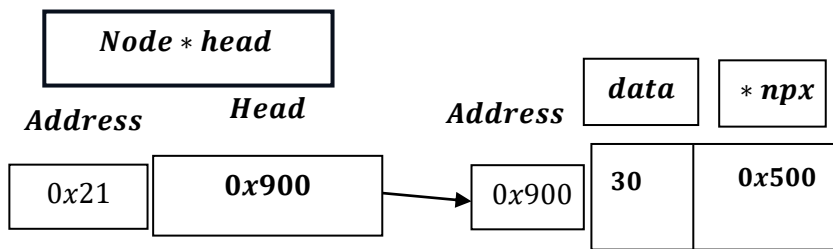
head = newNode = 0x900.



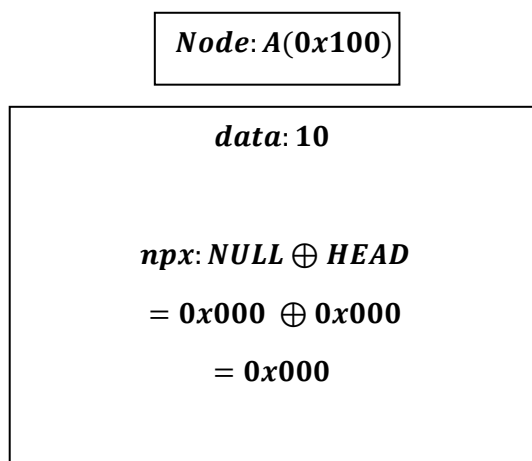
*After Function ends ,temporary pointer Node * newNode gets destroyed:*



And ,we are left with: –



Therefore, what we have observed, here:



Node: A(0x500)

$$\begin{aligned} \text{data: } 20 \\ \\ \text{npx} = \\ \text{NULL} \oplus B \\ = 0x000 \oplus 0x100 \\ = 0x100 \end{aligned}$$

Node: B(0x100)

$$\begin{aligned} \text{data: } 10 \\ \\ \text{npx} = \\ (NULL \oplus B) \oplus A \\ (0x000 \oplus 0x000) \oplus 0x500 \\ = 0x000 \oplus 0x500 \\ = 0x500 \end{aligned}$$

Node: A(0x900)

$$\begin{aligned} \text{data: } 30 \\ \\ \text{npx} = \\ \text{NULL} \oplus B \\ = 0x000 \oplus 0x500 \\ = 0x500 \end{aligned}$$

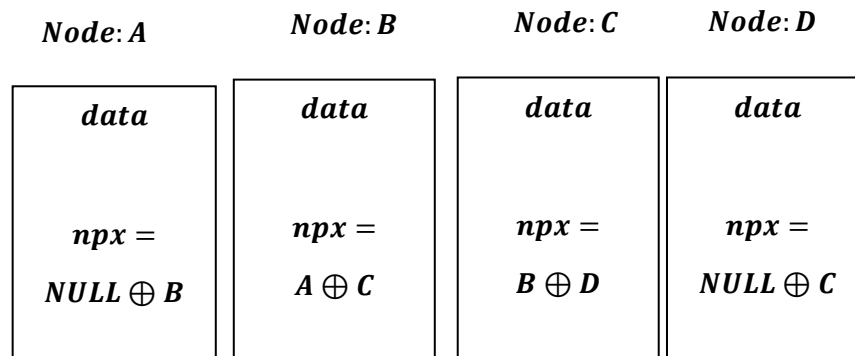
Node: B(0x500)

$$\begin{aligned} \text{data: } 20 \\ \\ \text{npx} = \\ (NULL \oplus A) \oplus C \\ = (0x000 \oplus 0x900) \oplus 0x100 \\ = 0x900 \oplus 0x100 \\ = 0x800 \end{aligned}$$

Node: C(0x100)

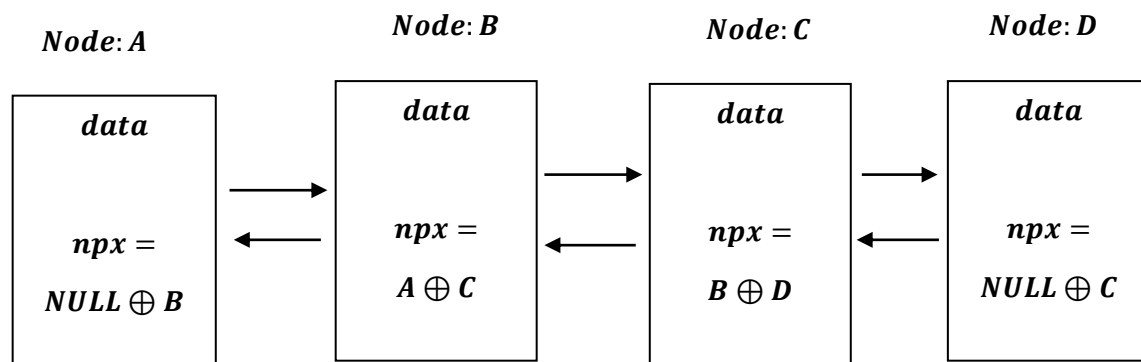
$$\begin{aligned} \text{data: } 10 \\ \\ \text{npx} = \\ \text{NULL} \oplus B \\ 0x000 \oplus 0x500 \\ = 0x500 \end{aligned}$$

Similarly,



- *There is no separate prev pointer and no separate next pointer stored.*
- *The npx pointer by itself is not ``a pointer to the previous node`` and not ``a pointer to the next node``.*
- *It's just a mixed value: the bitwise XOR of two addresses.*

Hence , by XORing only we can achieve:



- *In C ++, ``npx``, is not functioning as a pointer. It is functioning as a Storage Container.*
- *We declare it as `Node * npx` only to tell the compiler: "Reserve 8 bytes of space here, because I plan to turn this numbers back into a pointer eventually."*
- *It is effectively an Integer masquerading as a Pointer.*

The "Envelope" Analogy

Imagine a normal Linked List is a treasure hunt.

- *Normal Pointer: You open an envelope, and inside is a slip of paper that says: "Go to House #100." You go there, and you find the house.*
- *XOR Pointer: You open the envelope, and inside is a slip of paper that says: "800".*
 - *If you try to go to House #800, you will find an empty field or a cliff edge (Crash).*
 - *Instead, you are supposed to take your current house number (100) and do math: $800 \text{ XOR } 100 = 900$.*

"Ah! The real destination is House #900."

In a standard sense, the npx pointer violates the logical definition of a pointer.

- *Standard Definition: A pointer holds the memory address of a specific, valid object.*
- *XOR Definition: The npx variable holds a math result (a number) that does not point to any valid memory location.*

Why does the compiler allow this violation?

C + +: It assumes you know what you are doing. It allows you to store any number in a pointer variable, as long as you cast it correctly. It only checks validity when you try to dereference ($\rightarrow$ or $$) it.*

Summary:

- ***Does npx point to a memory address? No. It points to "garbage" (mathematically calculated noise).***
 - ***Does it violate pointer properties? Yes, it violates the property of "Dereferencability." You cannot look inside npx directly.***
 - ***Why do we do it? To save space. We sacrifice the "safety" and "direct access" of a pointer to save 50% of our memory usage.***
-

Memory Efficient Doubly Linked List - Insert At Beginning [Working Summary]:

1. Allocation & Setup

- **Action:** The function allocates memory for a newNode using malloc.
- **Safety:** It checks if the allocation failed (nullptr).
- **Data:** It assigns the input data to the new node.

2. Establishing the New Head's Links

- **Goal:** Determine the npx value for the newNode.
- **Logic:** Since this node is being inserted at the **very beginning**:
 - Its **Previous** node is nullptr.
 - Its **Next** node is the current head.
- **Code:** newNode->npx = XOR(nullptr, head);

3. Updating the Old Head (The "Stitching")

If the list was not empty (head != nullptr), the old head needs to know that it is no longer the first node. Its "Previous" link must change from nullptr to newNode.

- **Step A: Decode the Neighbor**
 - We cannot just write to head->npx directly because it contains a compressed value (Prev $\oplus$ Next). We need to isolate the Next part first.
 - **Math:** next = nullptr $\oplus$ head->npx (This recovers the address of the 2nd node).
- **Step B: Re-encode with New Previous**
 - Now we calculate the new XOR checksum for the old head.
 - **New Previous:** newNode
 - **Next:** next (unchanged)
 - **Code:** head->npx = XOR(newNode, next);

4. Updating the Global Pointer

- **Action:** Finally, the global head pointer is updated to point to the newNode, officially making it the start of the list.

Visual Logic Summary

Scenario: Inserting **A** before **B**.

1. **Create A.**
 2. **Calculate A->npx:** $NULL \oplus Address(B)$
 3. **Update B (Old Head):**
 - *Old B->npx:* $NULL \oplus Address(C)$
 - *New B->npx:* $Address(A) \oplus Address(C)$
 4. **Set Head = A.**
-

Time Complexity of the program

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node)); →  $O(1)$ 
    if (newNode == nullptr) →  $O(1)$ 
    {
        cout << "Error: Memory allocation failed." << endl; →  $O(1)$ 
        return; →  $O(1)$ 
    }

    newNode->data = data; →  $O(1)$ 
    newNode->npx = XOR(nullptr, head); →  $O(1)$ 

    if (head != nullptr) →  $O(1)$ 
    {
        Node *next = XOR(nullptr, head->npx); →  $O(1)$ 
        head->npx = XOR(newNode, next); →  $O(1)$ 
    }

    head = newNode; →  $O(1)$ 
    cout << data << " inserted at the beginning." << endl; →  $O(1)$ 
}
```

Best Case = Worst Case = Average Case = $O(1)$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>InsertAt Beginning</i>	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(1)$
	3. Average Case	$\Theta(1)$

Memory Efficient Doubly Linked List – Insert At Beginning –Space Complexity and Algorithm

Program :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    // Logic:
    // New Node's npx = NULL ^ current_head
    newNode->npx = XOR(nullptr, head);

    // If the list was not empty, we must update the old
    head's npx
    if (head != nullptr)
    {
        // Old head's npx was: NULL ^ Next
        // We need it to be: newNode ^ Next

        // 1. Decode the 'next' node of the current head
        Node *next = XOR(nullptr, head->npx);

        // 2. Update head's npx
        head->npx = XOR(newNode, next);
    }

    head = newNode;
    cout << data << " inserted at the beginning." << endl;
}
```



```
... • •
    case 1:
        cout << "Enter data to insert: ";
        cin >> data;
        insertAtBeginning(data);
        break;
```

Space Complexity Analysis:

XOR Linked List (Insert At Beginning)

1. Input Space Complexity

- ***Parameter – only Definition:***
 - ***Function Parameter: int data.***
 - ***Analysis: The data parameter is a single integer. It occupies a fixed amount of memory (typically 4 bytes) regardless of the size of the list.***
 - ***Complexity: $O(1)$***

2. Full Data Definition:

- ***Function Data: The int data parameter + the existing XOR list.***
- ***Analysis: To perform the insertion, the function interacts with the existing list (specifically the current head). If the list contains n nodes, the total input data context is proportional to n .***

- **Complexity: $O(n)$.**

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Insert at Beginning (XOR Doubly Linked List)

XORINSBEG(START, ITEM):

((Node *)malloc(sizeof(Node)) represents the availability of space for the data; let's name it AVAIL.))

1. [ALLOCATE NEW NODE] Set NEWPTR := AVAIL

2. [MEMORY ALLOCATED?]

If NEWPTR = NULL, then:

Write: "Error: Memory allocation failed."

Return and EXIT.

[End of If structure]

3. [COPY DATA] Set INFO[NEWPTR] := ITEM

4. [INITIALIZE NEW NODE'S POINTER]

Set NEXTXORPTR[NEWPTR] := NULL $\oplus$ START ***$\left(\begin{array}{l} \text{This sets the new node's} \\ \text{pointer field to the address} \\ \text{of the current head,} \\ \text{as its previous is NULL} \end{array} \right)$***

5. [UPDATE OLD HEAD IF LIST NOT EMPTY]

If START $\neq$ NULL, then:

a. Set NEXTTEMPPTR := NULL $\oplus$ NEXTXORPTR[START] ***$\left(\begin{array}{l} \text{Decode the} \\ \text{address of} \\ \text{the node} \\ \text{following the} \\ \text{current head} \end{array} \right)$***

b. Set NEXTXORPTR[START] := NEWPTR $\oplus$ NEXTTEMPPTR

*Update the
old head
to point
back
to the new
node
and forward
to its
original
successor*

[End of If structure]

6. [UPDATE HEAD POINTER]

Set START := NEWPTR (Update the global head to the newly created node)

7. [PRINT MESSAGE]

Write: ITEM, " inserted at the beginning."

8. EXIT

-----*****

Memory Efficient Doubly Linked List

– Insert At END And Time Complexity.

Program :

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    if (head == nullptr)
    {
        newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    while (curr != nullptr)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr){ break; }
        prev = curr;
        curr = next;
    }
    curr->npx = XOR(prev, newNode);
    newNode->npx = XOR(curr, nullptr);
    cout << data << " inserted at the end." << endl;
}

.....

case 2:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtEnd(data);
    break;
```

Program Analysis:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *npx(a pointer to another Node) is typically 8 bytes.*

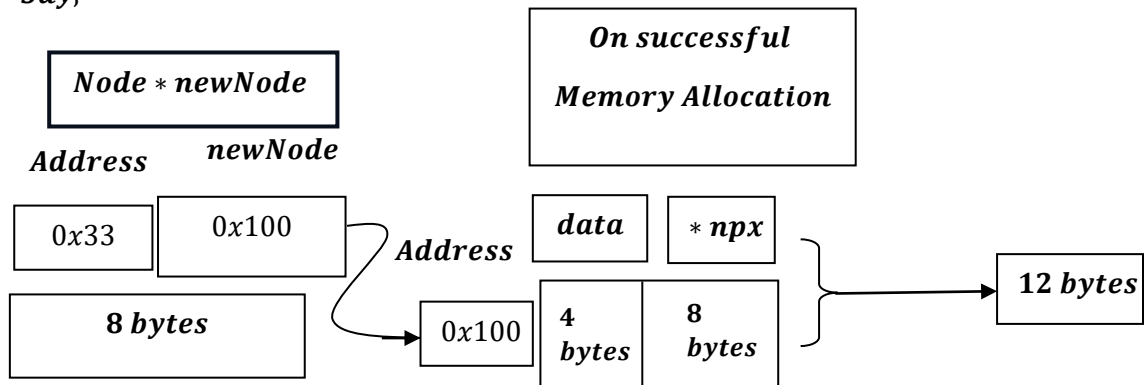
→ *Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npv) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

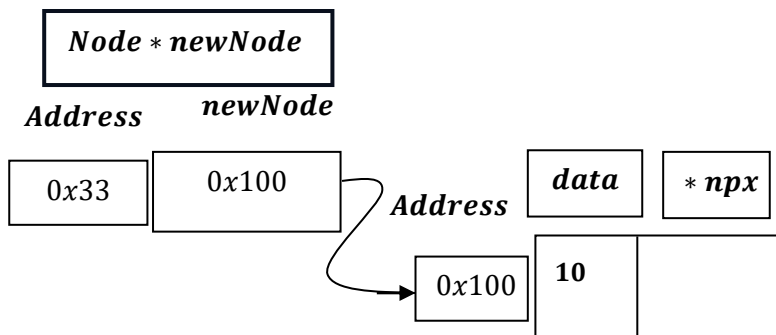
→ *newNode will try to allocate memory for data i.e. 4 bytes ,npv pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



Lets say ,there is data → 10 ,to be entered.

```
newNode->data = data;
```



When List is Empty

```
if (head == nullptr)
{
    newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
    head = newNode;
    cout << data << " inserted at the end." << endl;
    return;
}
```

Node * head

Address Head

0x21

nullptr

```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
```

$newNode \rightarrow npx = XOR(NULL: 0x000, NULL: 0x000) \Rightarrow 0x000 \oplus 0x000 = 0x000.$

Node * newNode

Address newNode

0x33

0x100

Address

data

*** npx**

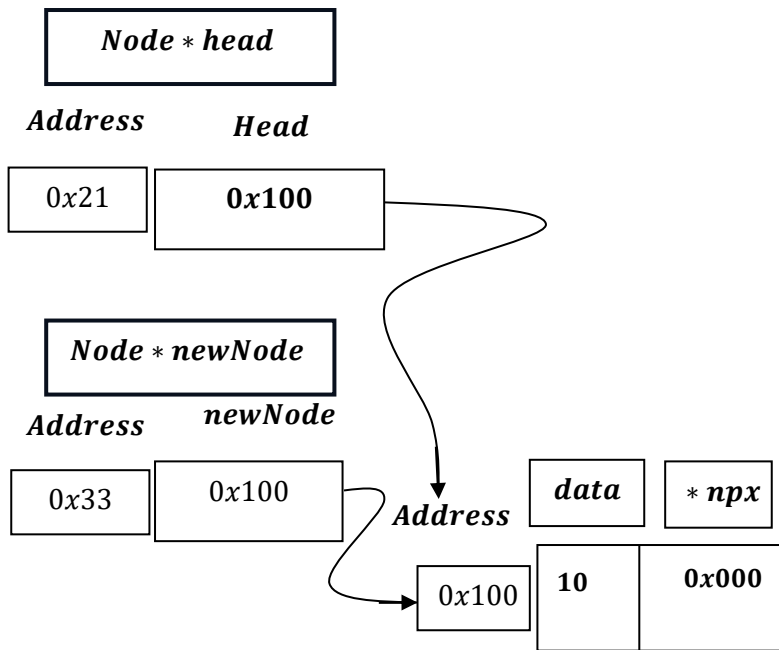
0x100

10

0x000

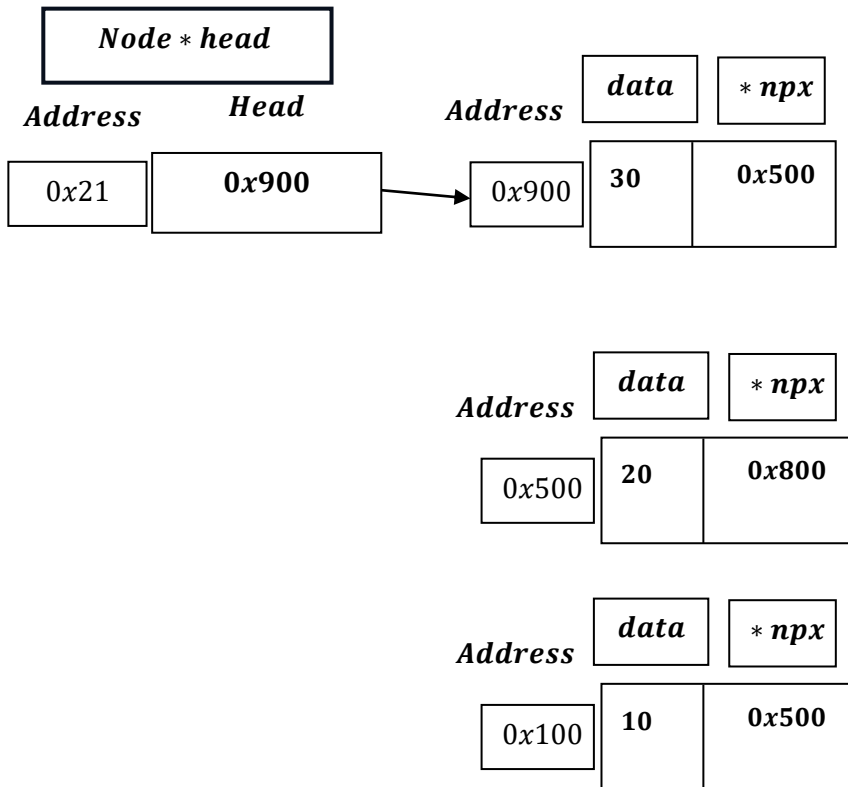
```
head = newNode;
```

$head = newNode = 0x100.$



And, it prints: ``10 inserted at end.`` and Return end the if block and exit from the function.

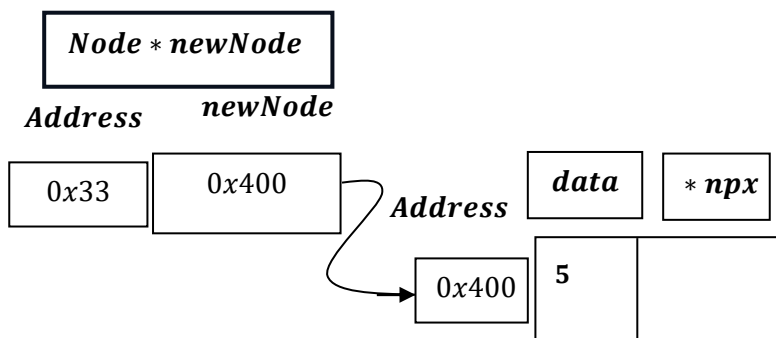
When List is not Empty



And,

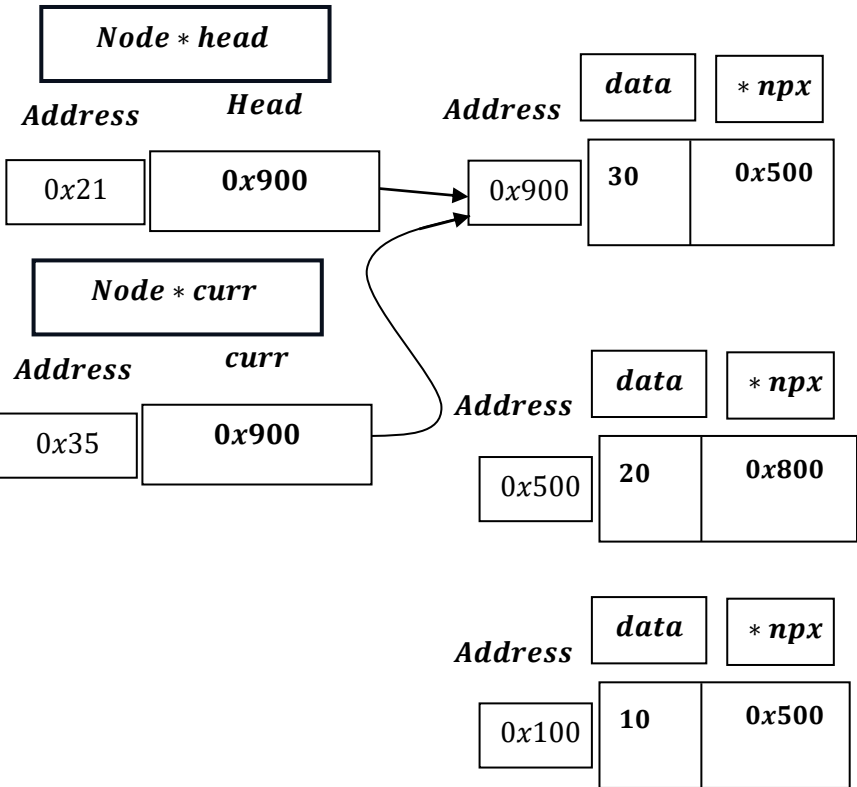
Lets say , there is data $\rightarrow 5$, to be entered.

```
newNode->data = data;
```



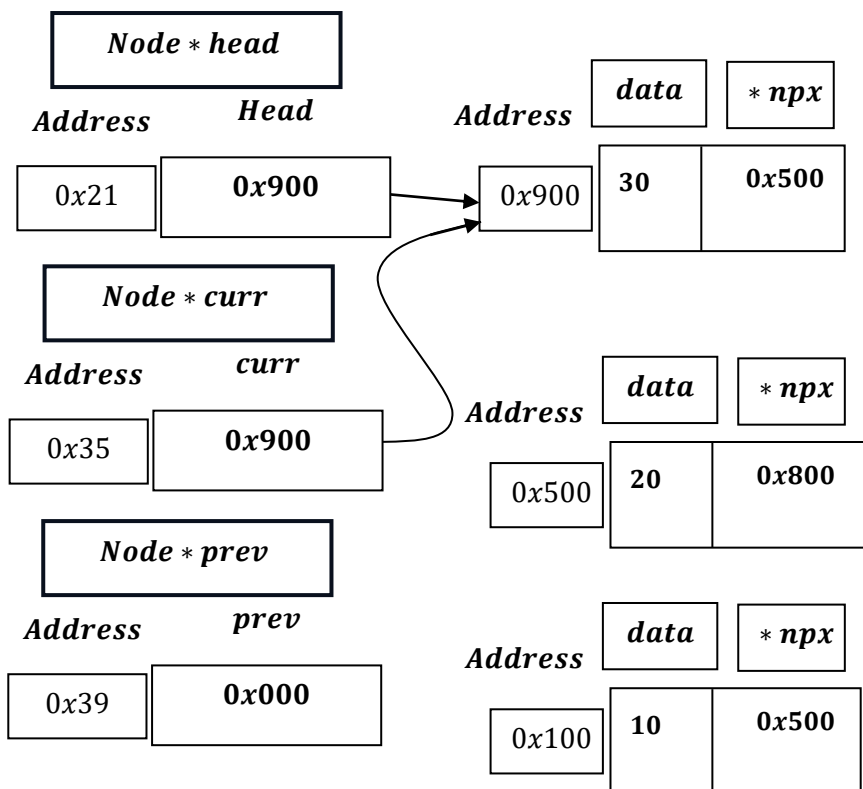
```
Node *curr = head;
```

*Node * curr = head = 0x900.*

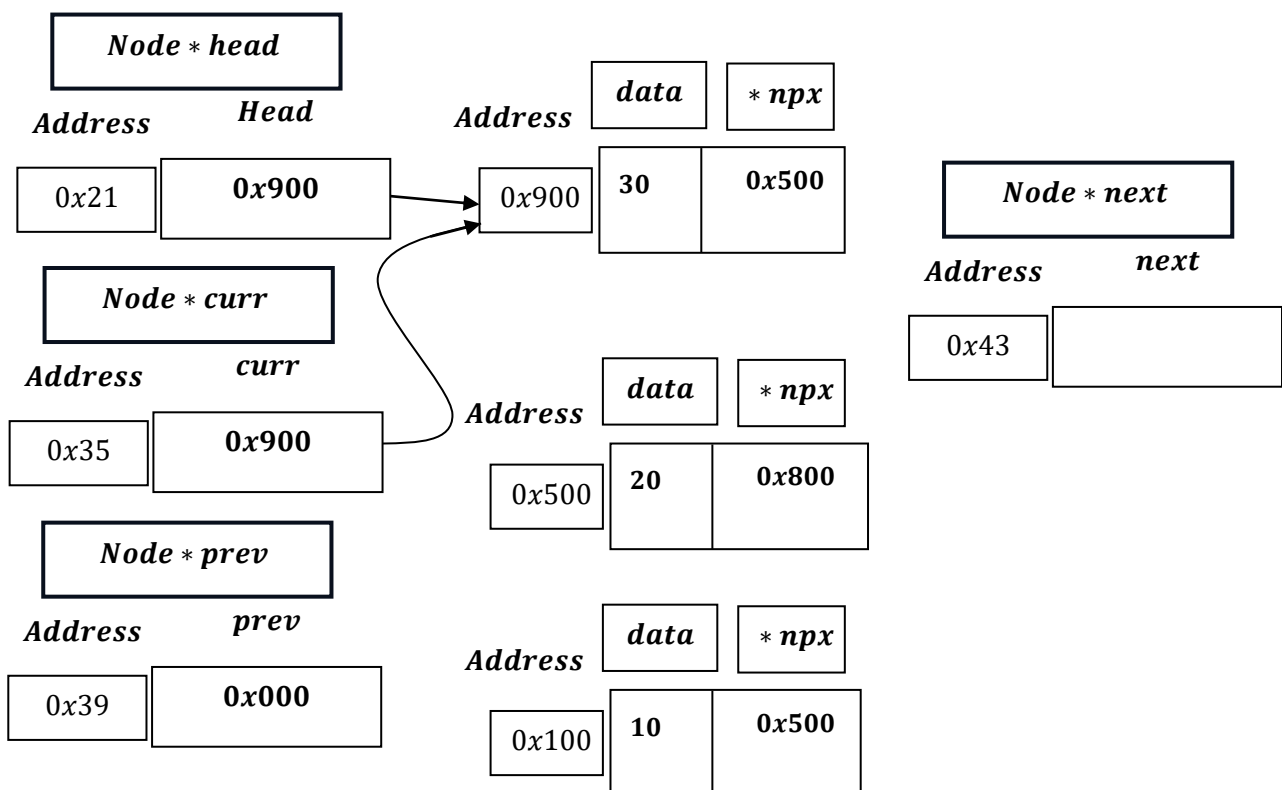


```
Node *prev = nullptr;
```

*Node * prev = nullptr = 0x000;*



```
Node *next;
```



```

while (curr != nullptr)
{
    next = XOR(prev, curr->npx);
    if (next == nullptr){ break; }
    prev = curr;
    curr = next;
}

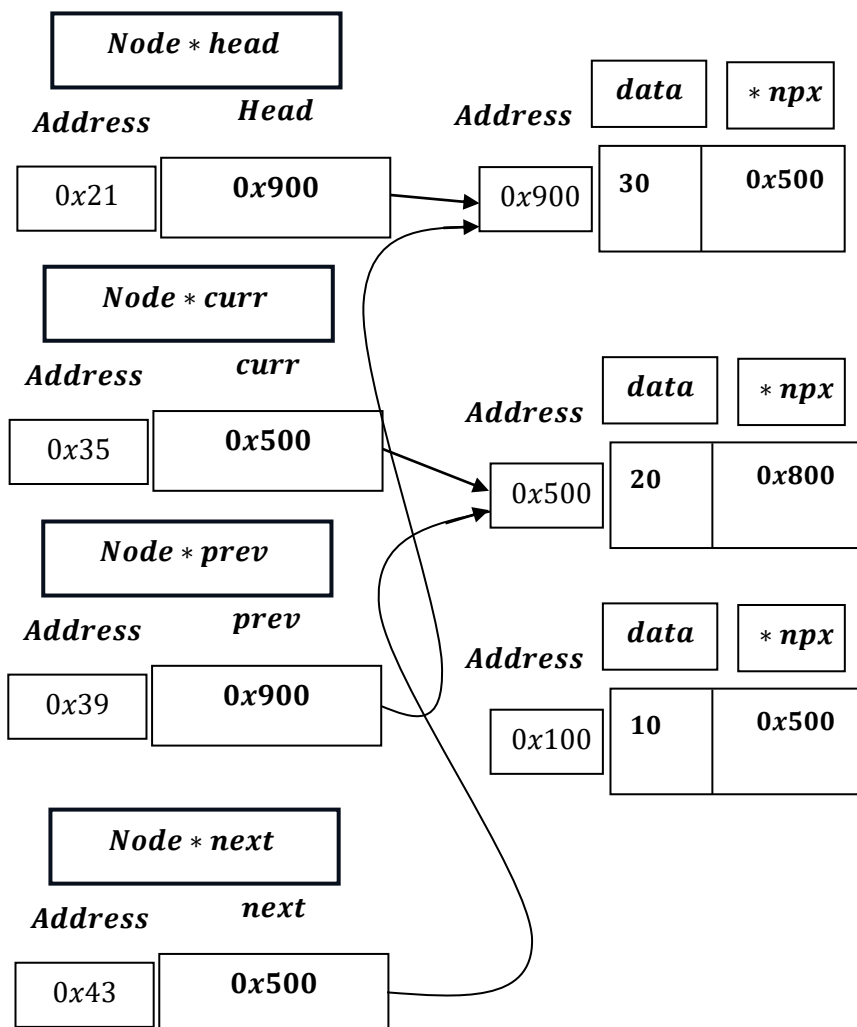
```

curr = 0x900:

next = XOR(prev, curr → npx) ; ⇒ XOR(0x000, 0x500) ⇒ 0x500;

prev = curr = 0x900;

curr = next = 0x500;



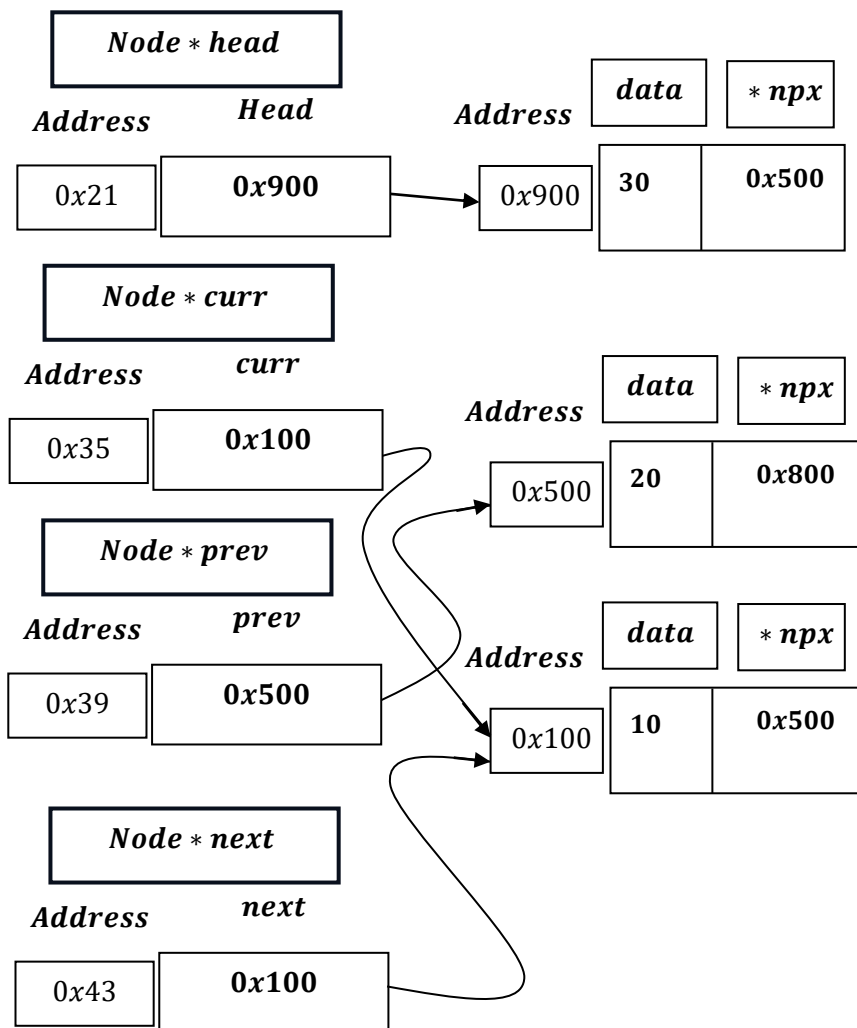
curr = 0x500;

next = XOR(prev, curr → npx) ; ⇒ XOR(0x900, 0x800)

⇒ 1001₂ ⊕ 1000₂ = 0001₂ = 0x100;

prev = curr = 0x500;

curr = next = 0x100;



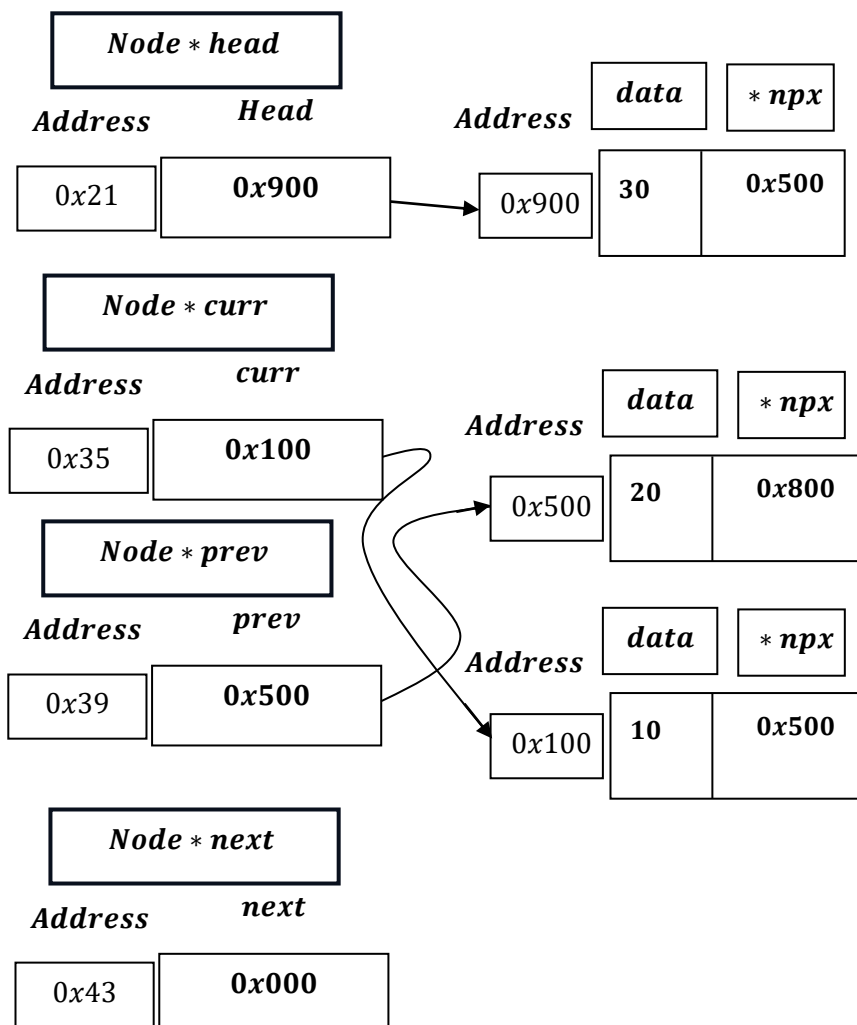
curr = 0x100:

next = XOR(prev, curr → npx) ; ⇒ XOR(0x500, 0x500)

⇒ 0101₂ ⊕ 0101₂ = 0000₂ = 0x000;

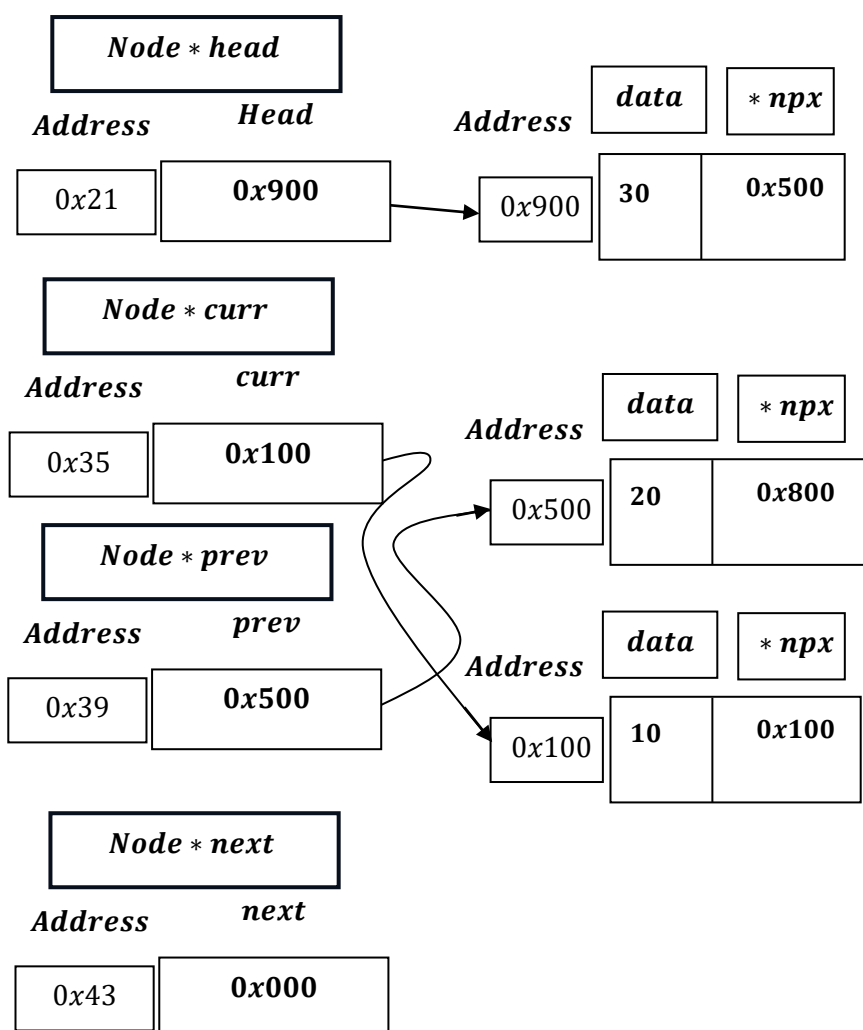
```
if (next == nullptr){ break; }
```

As, break control flow statement used → it exits from the loop.



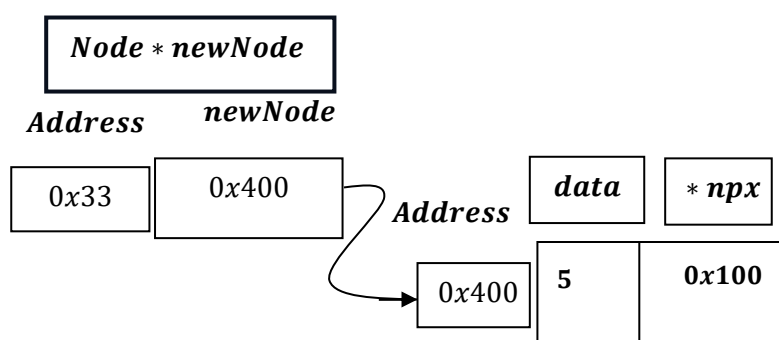
```
curr->npx = XOR(prev, newNode);
```

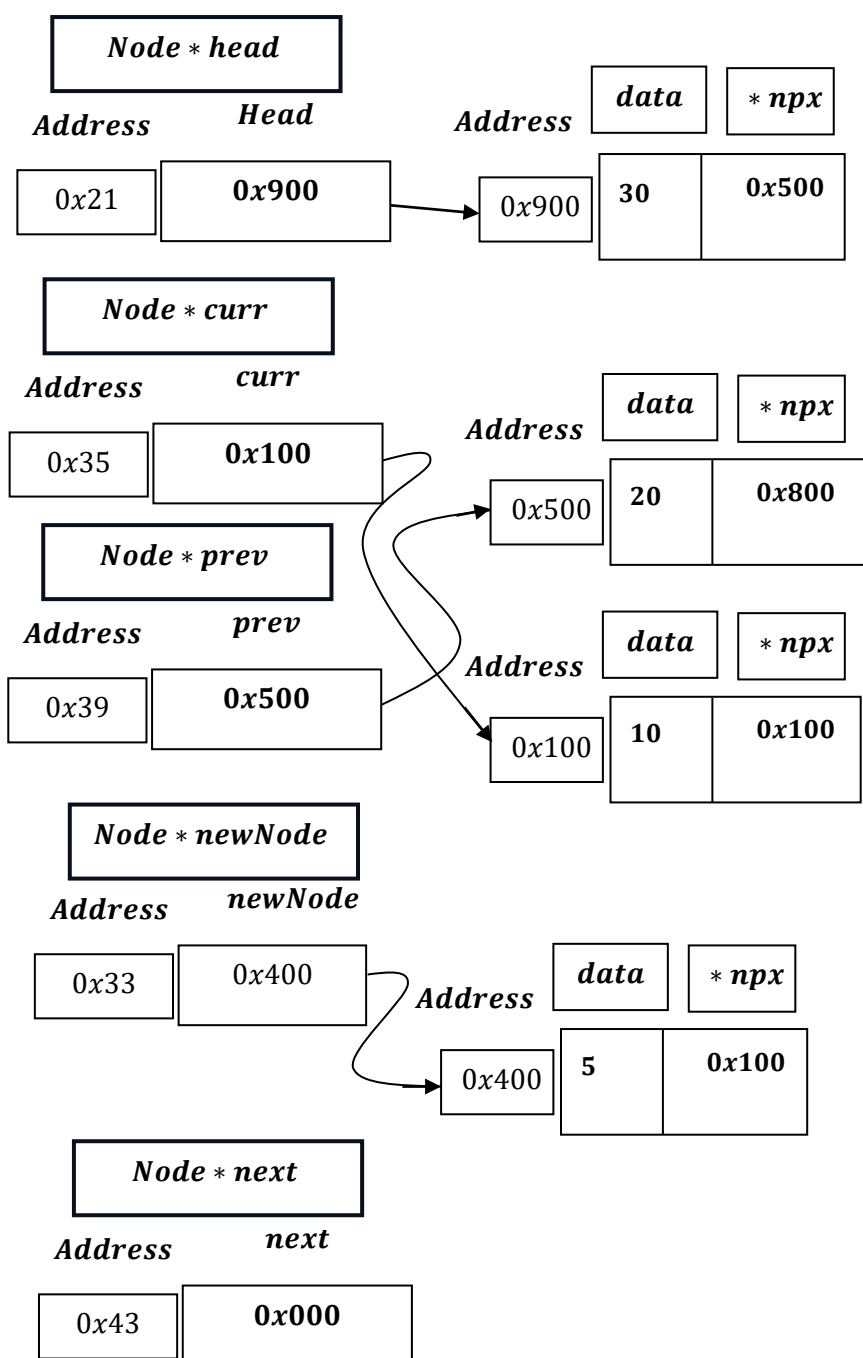
curr → npx = XOR(0x500, 0x400); = 0101₂ ⊕ 0100₂ = 0001₂ = 0x100₂.



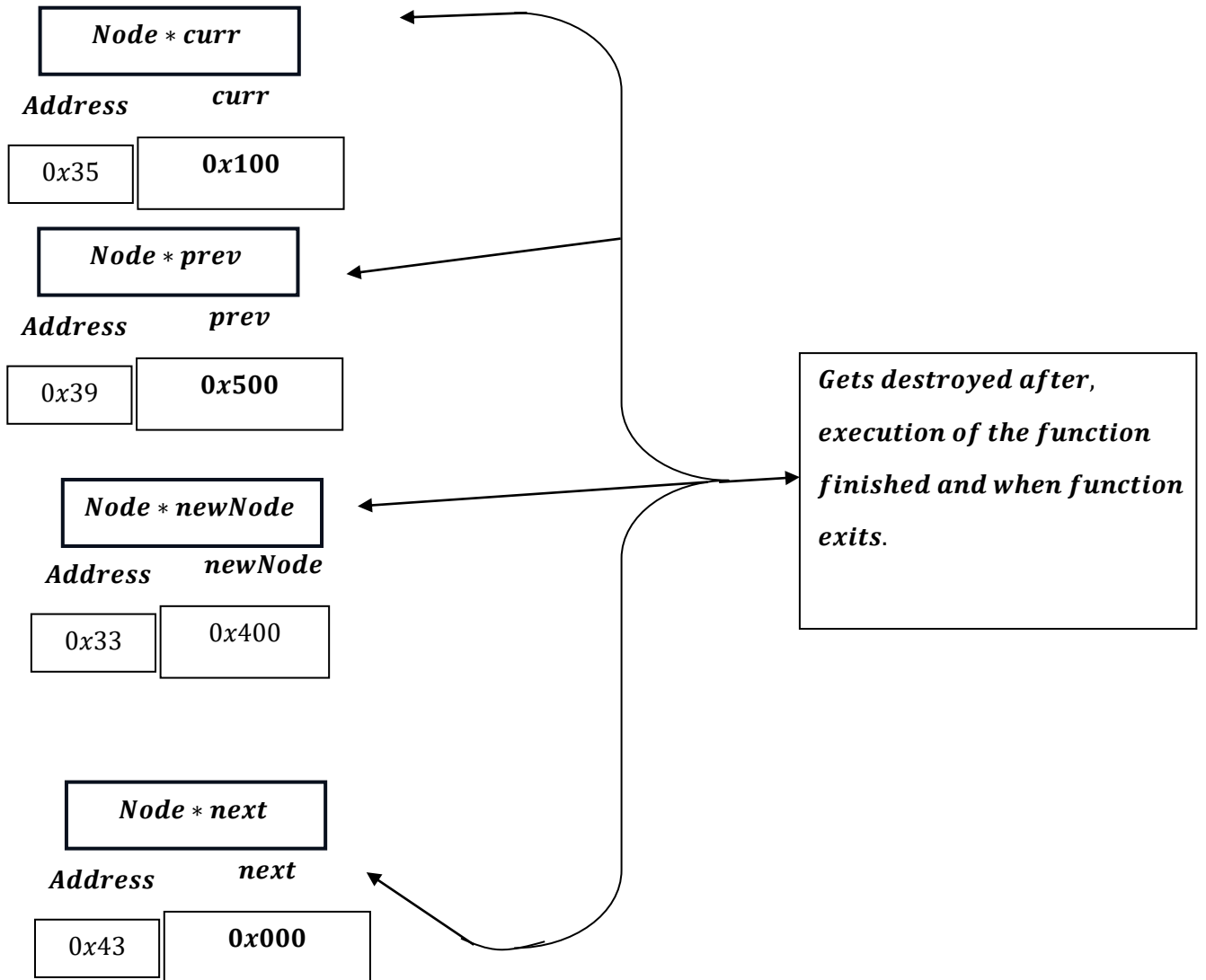
```
newNode->npx = XOR(curr, nullptr);
```

$newNode \rightarrow npx = XOR(0x100, 0x000) = 0x100 \oplus 0x000 = 0x100.$

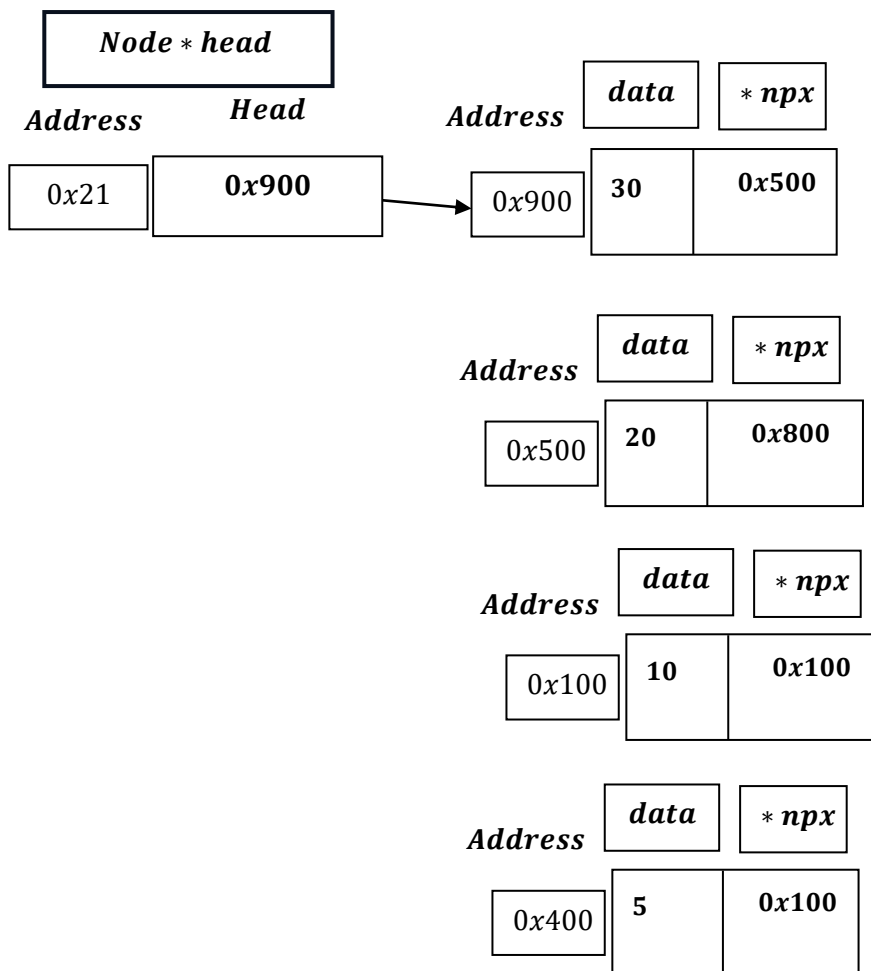




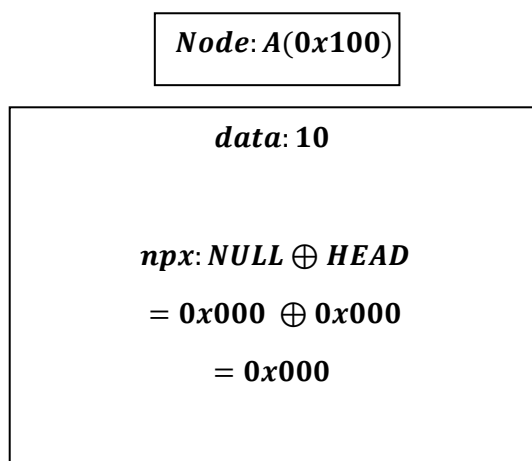
*After Function ends ,temporary pointer Node * newNode ,
Node * next,Node * prev,and Node * curr gets destroyed:*



And we are left with:



Therefore, what we have observed, here:



Node: A(0x500)

$$\begin{aligned} &\textbf{data: 20} \\ & \\ &\textbf{npx =} \\ &\textbf{NULL} \oplus \textbf{B} \\ &= \textbf{0x000} \oplus \textbf{0x100} \\ &= \textbf{0x100} \end{aligned}$$

Node: B(0x100)

$$\begin{aligned} &\textbf{data: 10} \\ & \\ &\textbf{npx =} \\ &(\textbf{NULL} \oplus \textbf{B}) \oplus \textbf{A} \\ &(\textbf{0x000} \oplus \textbf{0x000}) \oplus \textbf{0x500} \\ &= \textbf{0x000} \oplus \textbf{0x500} \\ &= \textbf{0x500} \end{aligned}$$

Node: A(0x900)

$$\begin{aligned} &\textbf{data: 30} \\ & \\ &\textbf{npx =} \\ &\textbf{NULL} \oplus \textbf{B} \\ &= \textbf{0x000} \oplus \textbf{0x500} \\ &= \textbf{0x500} \end{aligned}$$

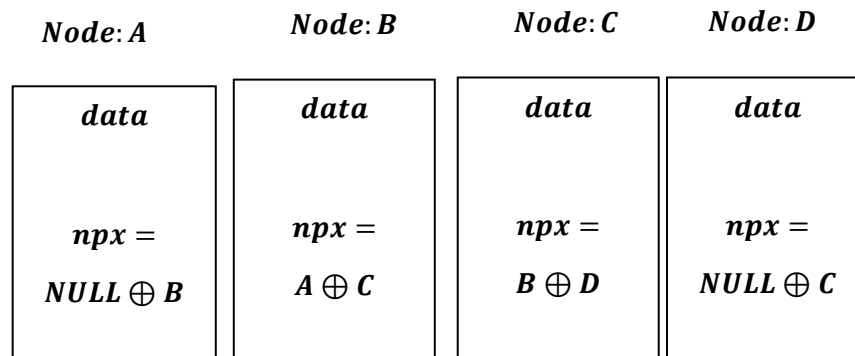
Node: B(0x500)

$$\begin{aligned} &\textbf{data: 20} \\ & \\ &\textbf{npx =} \\ &(\textbf{NULL} \oplus \textbf{A}) \oplus \textbf{C} \\ &= (\textbf{0x000} \oplus \textbf{0x900}) \oplus \textbf{0x100} \\ &= \textbf{0x900} \oplus \textbf{0x100} \\ &= \textbf{0x800} \end{aligned}$$

Node: C(0x100)

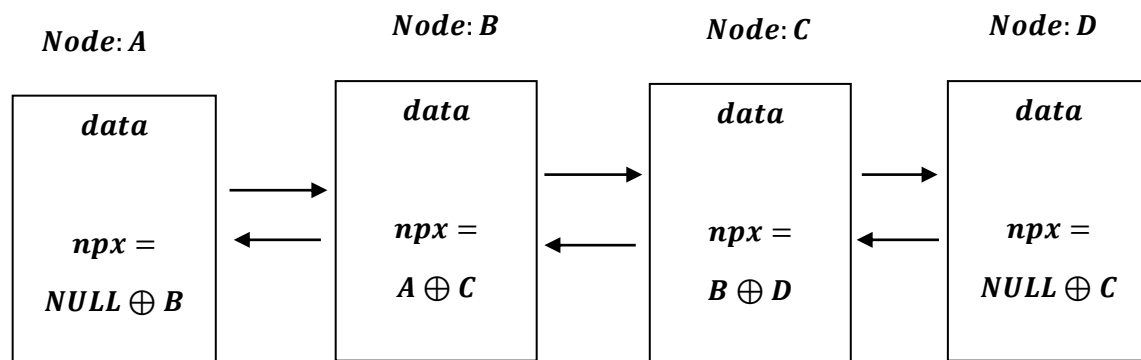
$$\begin{aligned} &\textbf{data: 10} \\ & \\ &\textbf{npx =} \\ &\textbf{NULL} \oplus \textbf{B} \\ &\textbf{0x000} \oplus \textbf{0x500} \\ &= \textbf{0x500} \end{aligned}$$

Similarly,



- *There is no separate prev pointer and no separate next pointer stored.*
- *The npx pointer by itself is not ``a pointer to the previous node`` and not ``a pointer to the next node``.*
- *It's just a mixed value: the bitwise XOR of two addresses.*

Hence , by XORing only we can achieve:



- *In C ++, ``npx``, is not functioning as a pointer. It is functioning as a Storage Container.*
- *We declare it as `Node * npx` only to tell the compiler: "Reserve 8 bytes of space here, because I plan to turn this numbers back into a pointer eventually."*
- *It is effectively an Integer masquerading as a Pointer.*

The "Envelope" Analogy

Imagine a normal Linked List is a treasure hunt.

- *Normal Pointer: You open an envelope, and inside is a slip of paper that says: "Go to House #100." You go there, and you find the house.*
- *XOR Pointer: You open the envelope, and inside is a slip of paper that says: "800".*
 - *If you try to go to House #800, you will find an empty field or a cliff edge (Crash).*
 - *Instead, you are supposed to take your current house number (100) and do math: $800 \text{ XOR } 100 = 900$.*

"Ah! The real destination is House #900."

In a standard sense, the npx pointer violates the logical definition of a pointer.

- *Standard Definition: A pointer holds the memory address of a specific, valid object.*
- *XOR Definition: The npx variable holds a math result (a number) that does not point to any valid memory location.*

Why does the compiler allow this violation?

C++: It assumes you know what you are doing. It allows you to store any number in a pointer variable, as long as you cast it correctly. It only checks validity when you try to dereference ($\rightarrow$ or $$) it.*

Summary:

- ***Does npx point to a memory address? No. It points to "garbage" (mathematically calculated noise).***
 - ***Does it violate pointer properties? Yes, it violates the property of "Dereferencability." You cannot look inside npx directly.***
 - ***Why do we do it? To save space. We sacrifice the "safety" and "direct access" of a pointer to save 50% of our memory usage.***
-

Time Complexity Of The Program

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    if (head == nullptr)
    {
        newNode->npx = XOR(nullptr, nullptr);
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    while (curr != nullptr)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr){ break; }
        prev = curr;
        curr = next;
    }
    curr->npx = XOR(prev, newNode);
    newNode->npx = XOR(curr, nullptr);
    cout << data << " inserted at the end." << endl;
}
```

Time complexity analysis for the `insertAtEnd` function:

- `Node *newNode = (Node *)malloc(sizeof(Node));` → $O(1)$
- `newNode->data = data;` → $O(1)$
- `if (head == nullptr)` → $O(1)$
- Inside the `if` block:
 - `newNode->npx = XOR(nullptr, nullptr);` → $O(1)$
 - `head = newNode;` → $O(1)$
 - `cout << data << " inserted at the end." << endl;` → $O(1)$
 - `return;` → $O(1)$
- `Node *curr = head;` → $O(1)$
- `Node *prev = nullptr;` → $O(1)$
- `Node *next;` → $O(1)$
- `while (curr != nullptr)` → checks `n` times in worst complexity. $1 + 1 + 1 + \dots n = O(n)$.
- Inside the `while` loop:
 - `next = XOR(prev, curr->npx);` → runs 1 time = $O(1)$.
 - `if (next == nullptr){ break; }` → $1 + 1 + 1 + \dots n - 1 = O(n)$.
 - `prev = curr;` → $O(1)$
 - `curr = next;` → $O(1)$
- `curr->npx = XOR(prev, newNode);` → $O(1)$
- `newNode->npx = XOR(curr, nullptr);` → $O(1)$
- `cout << data << " inserted at the end." << endl;` → $O(1)$

- **Best Case** is when , List is empty, and single node is inserted = $\Omega(1)$.
- **Worst Case** when list is not empty especially the loop, traversing part:
 $= O(cn + (cn + c + (cn - c) + (cn - c))),$ where c is constant
or, $c \times O(n + (n + 1 + (n - 1) + (n - 1)))$
 $= O(n)$.
- **Average Case** , there is no true average case as either it will be `c` (constant), when list is empty or traverse through list i.e. `n` times in worst complexity , hence here Average Case = Worst Case = $O(n)$.

Memory Efficient Doubly Linked List	Cases	Time Complexity
InsertAt End	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(n)$
	3. Average Case	$\Theta(n)$

Memory Efficient Doubly Linked List – Insert At END –Space Complexity and Algorithm

Program :

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    if (head == nullptr)
    {
        newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    while (curr != nullptr)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr){ break; }
        prev = curr;
        curr = next;
    }
    curr->npx = XOR(prev, newNode);
    newNode->npx = XOR(curr, nullptr);
    cout << data << " inserted at the end." << endl;
}

.....

case 2:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtEnd(data);
    break;
```

Space Complexity Analysis: XOR Linked List (Insert At End)

1. Input Space Complexity

- **Parameter – only Definition:**
 - **Function Parameter:** *int data.*
 - **Analysis:** *The data parameter is a single integer. It takes a constant amount of space regardless of the list size.*
 - **Complexity:** **$O(1)$**
- **Full Data Definition:**
 - **Function Data:** *The int data parameter + the existing XOR list.*
 - **Analysis:** *To find the end of the list, the function must access every node in the sequence. If there are n nodes, the input data structure being operated on is proportional to n .*
 - **Complexity:** **$O(n)$**

2. Auxiliary Space Complexity

Definition: *This measures the extra /temporary space used during the execution of the function.*

- **Node Allocation:** *`Node *newNode = (Node *)malloc(sizeof(Node));`. This creates exactly **one** new node.*
- **Local Pointer Variables:** *The function declares three local pointers: `curr`, `prev`, and `next`.*
 - *Even though the while loop runs n times, these variables are **reused** in each iteration. They do not multiply as the list grows.*
- **Traversal:** *The traversal is iterative, not recursive. This means no extra memory is added to the call stack as the list size increases.*
- **Operations:** *The XOR calculation is a bitwise operation that happens in CPU registers.*

Conclusion: *The amount of extra memory used by this function is fixed. It doesn't matter if the list has 5 nodes or 5,000 nodes; the function only uses space for one new node and three pointers.*

- **Auxiliary Space Complexity:** **$O(1)$**

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter – only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full – data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

*In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity** ($O(1)$) is the expected answer.*

Algorithm: Insert at End (XOR Doubly Linked List)

XORINSEND(START, ITEM):

*((Node *)malloc(sizeof(Node)) represents the availability of space for the data; let's name it AVAIL.))*

1. **[ALLOCATE NEW NODE]**
Set NEWPTR := AVAIL
2. **[MEMORY ALLOCATED?]**
If NEWPTR = NULL, then:
Write: "Error: Memory allocation failed."
Return and EXIT.
[End of If structure]
3. **[COPY DATA]**
Set INFO[NEWPTR] := ITEM
4. **[CHECK IF LIST IS EMPTY]**
If START = NULL, then:
 - a. *Set NEXTXORPTR[NEWPTR] := NULL $\oplus$ NULL*
 - b. *Set START := NEWPTR*
 - c. *Write: ITEM, " inserted at the end."*
 - d. *Return and EXIT.***[End of If structure]**
5. **[INITIALIZE TRAVERSAL]**
Set CURRPTR := HEAD
Set PREVPTR := NULL
6. **[TRAVERSE TO THE LAST NODE]**
Repeat while CURRPTR $\neq$ NULL:
 - a. *Set NEXTTEMPPTR := PREVPTR $\oplus$ NEXTXORPTR[CURRPTR]*
 - b. *If NEXTTEMPPTR = NULL, then:*
Break and Exit from the loop
 - c. *Set PREVPTR := CURRPTR*
 - d. *Set CURRPTR := NEXTTEMPPTR***[End of Loop]**

7. [UPDATE POINTERS FOR INSERTION]

a. Set NEXTXORPTR[CURRPTR] := PREVPTR $\oplus$ NEWPTR $\left(\begin{array}{l} \text{Update} \\ \text{old last node's NPX} \\ \text{to point to the new node} \end{array} \right)$

b. Set NEXTXORPTR [NEWPTR] := CURRPTR $\oplus$ NULL $\left(\begin{array}{l} \text{New node's NPX} \\ \text{points back} \\ \text{to old last node} \\ \text{and forward to NULL} \end{array} \right)$

8. [PRINT MESSAGE]

Write: ITEM, " inserted at the end. "

9. EXIT

Memory Efficient Doubly Linked List
–Insert At Position and Time Complexity
–Part 2

***When if(next $\neq$ NULL) $\rightarrow$ Condition is False. When we enter the
last node***

let , pos = 5

```
while (curr != nullptr && i < pos - 1)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

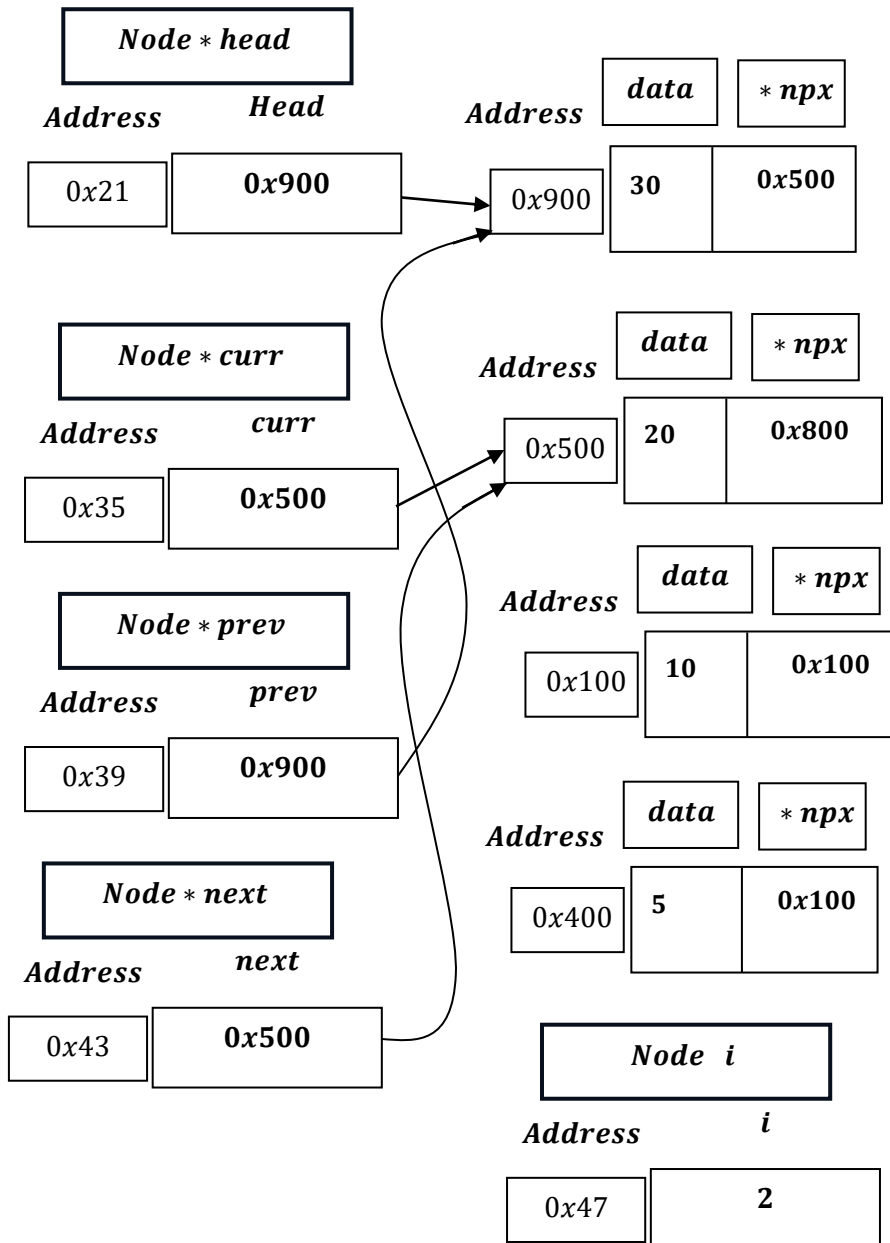
$curr(0x900) \neq 0x000$ And $i < (5 - 1 = 4) \Rightarrow 1 < 4$ is true:

$next = XOR(0x000, curr \rightarrow npx \Rightarrow 0x500) = 0x500$.

$prev = curr = 0x900$;

$curr = next = 0x500$;

$i = i + 1 = 1 + 1 = 2$;



$curr(0x500) \neq 0x000$ And $i < (5 - 1 = 4) \Rightarrow 2 < 4$ is true:

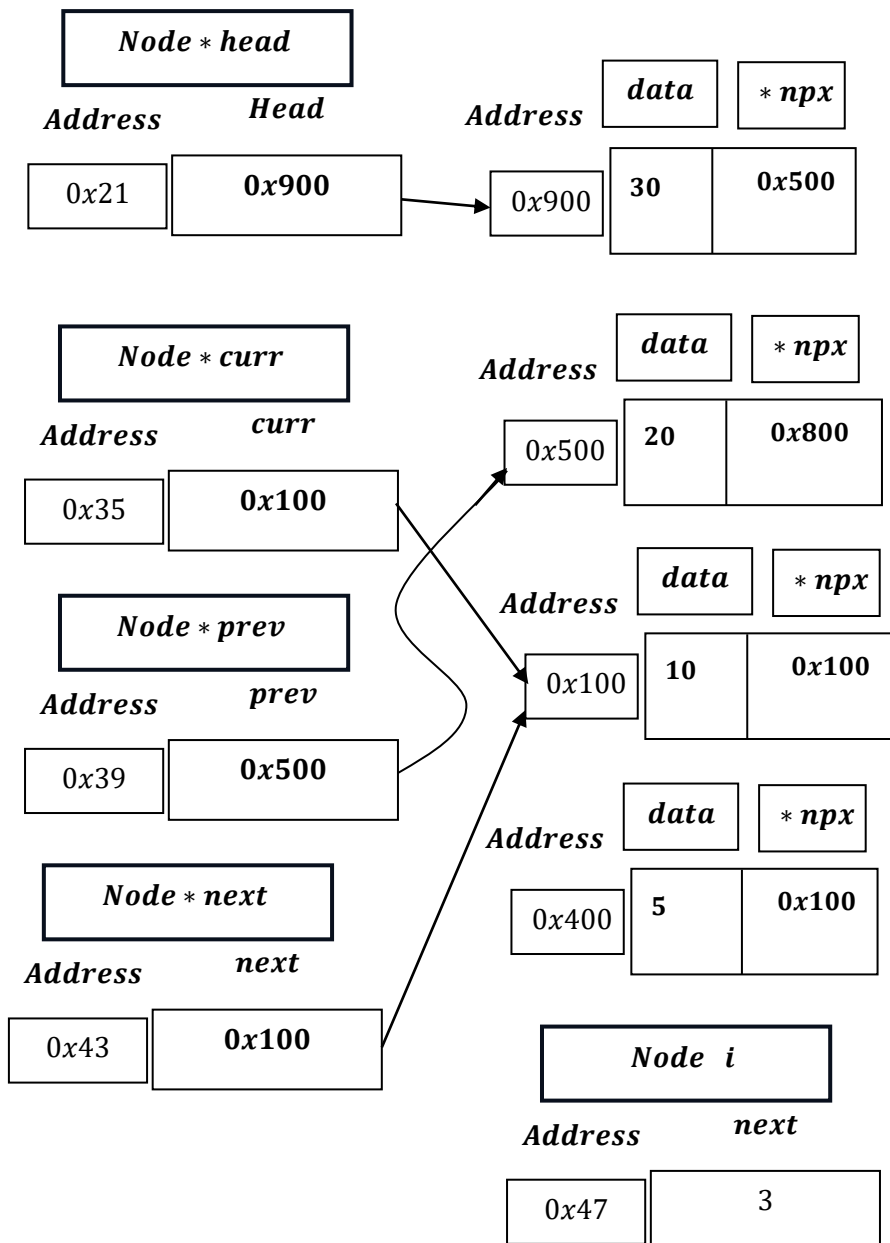
$next = XOR(0x900, curr \rightarrow npx \Rightarrow 0x800)$

$= 1001_2 \oplus 1000_2 = 0001_2 = 0x100$

$prev = curr = 0x500;$

$curr = next = 0x100;$

$i = i + 1 = 2 + 1 = 3;$



$curr(0x100) \neq 0x000$ And $i < (5 - 1 = 4) \Rightarrow 3 < 4$ is true:

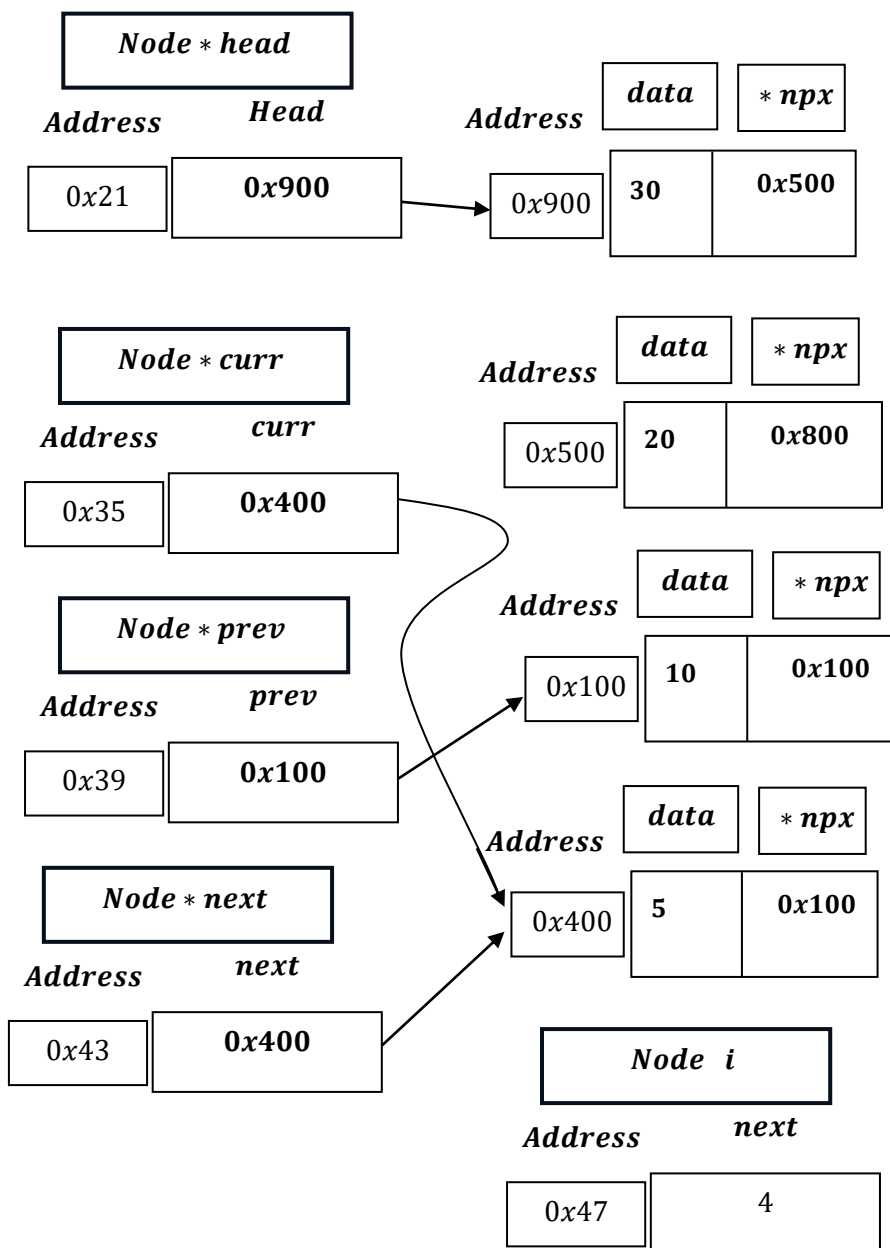
$next = XOR(0x500, curr \rightarrow npx \Rightarrow 0x100)$

$= 0101_2 \oplus 0001_2 = 0100_2 = 4_{10} = 0x400$

$prev = curr = 0x100;$

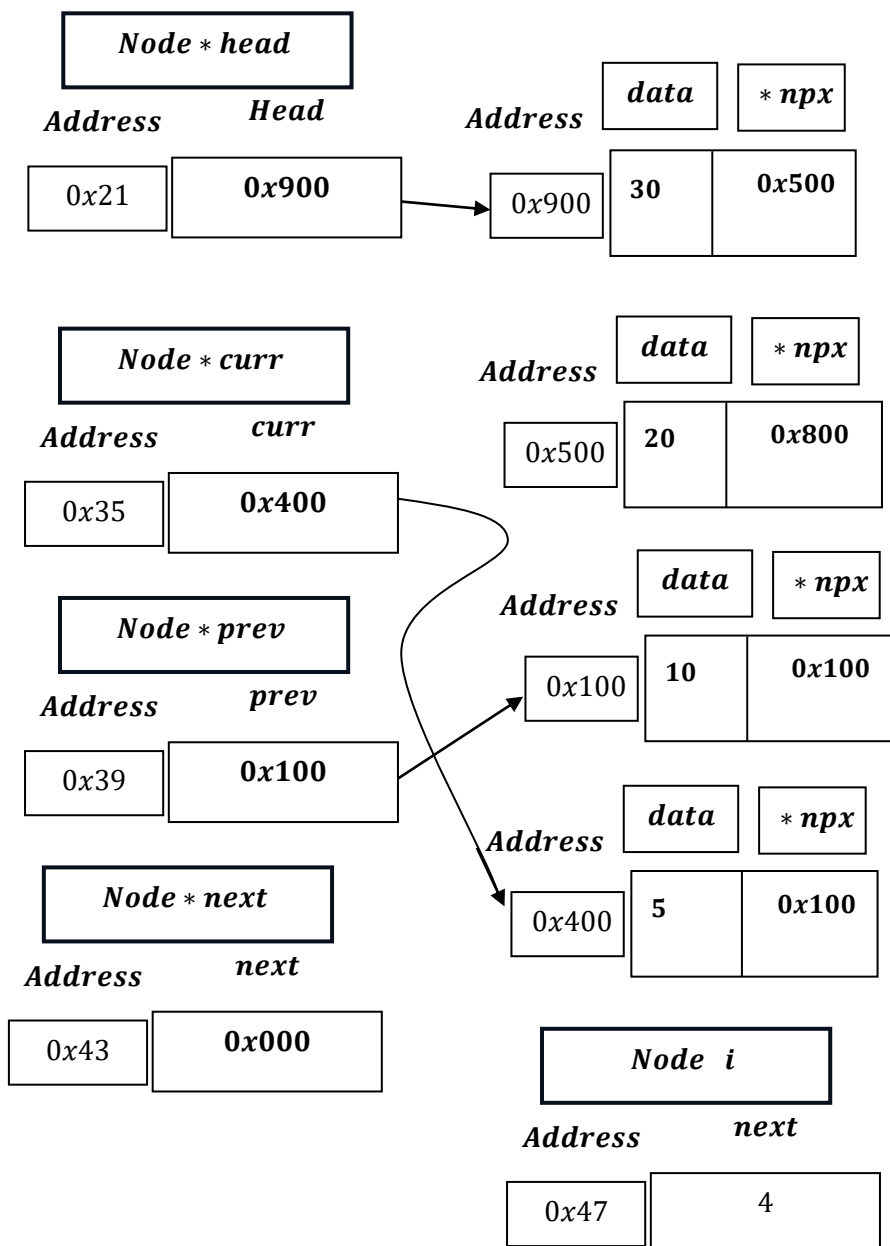
$curr = next = 0x400;$

$i = i + 1 = 3 + 1 = 4;$



$curr(0x400) \neq 0x000$ And $i < (5 - 1 = 4) \Rightarrow 4 < 4$ is false. Hence , loop exits.

```
next = XOR(prev, curr->npx);
next = XOR(0x100, 0x100) = 00012  $\oplus$  00012 = 00002 = 010 = 0x000
```



```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *npx(a pointer to another Node) is typically 8 bytes.*

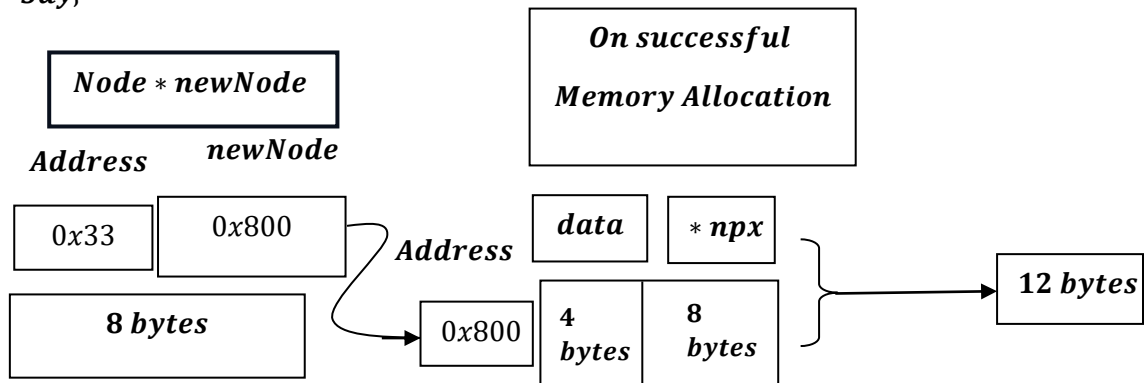
→ *Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npx) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

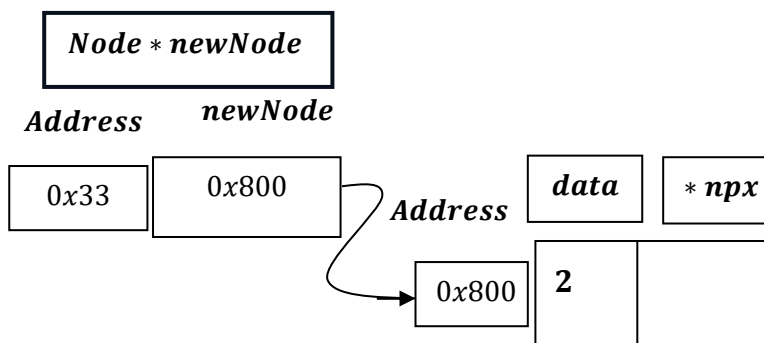
→ *newNode will try to allocate memory for data i.e. 4 bytes ,npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier ,the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



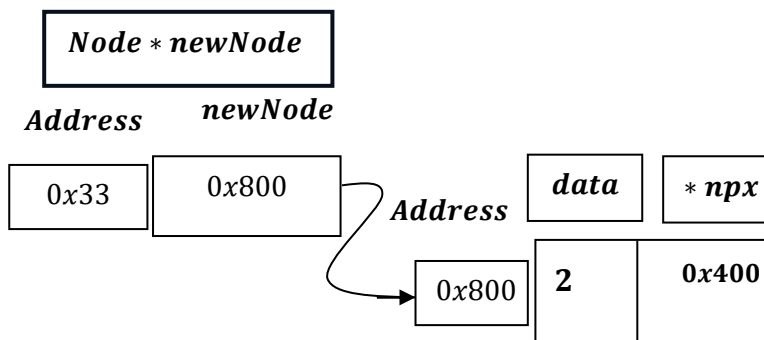
```
newNode->data = data;
```

Say, $data = 2$. Then, $newNode \rightarrow data = data = 2$.

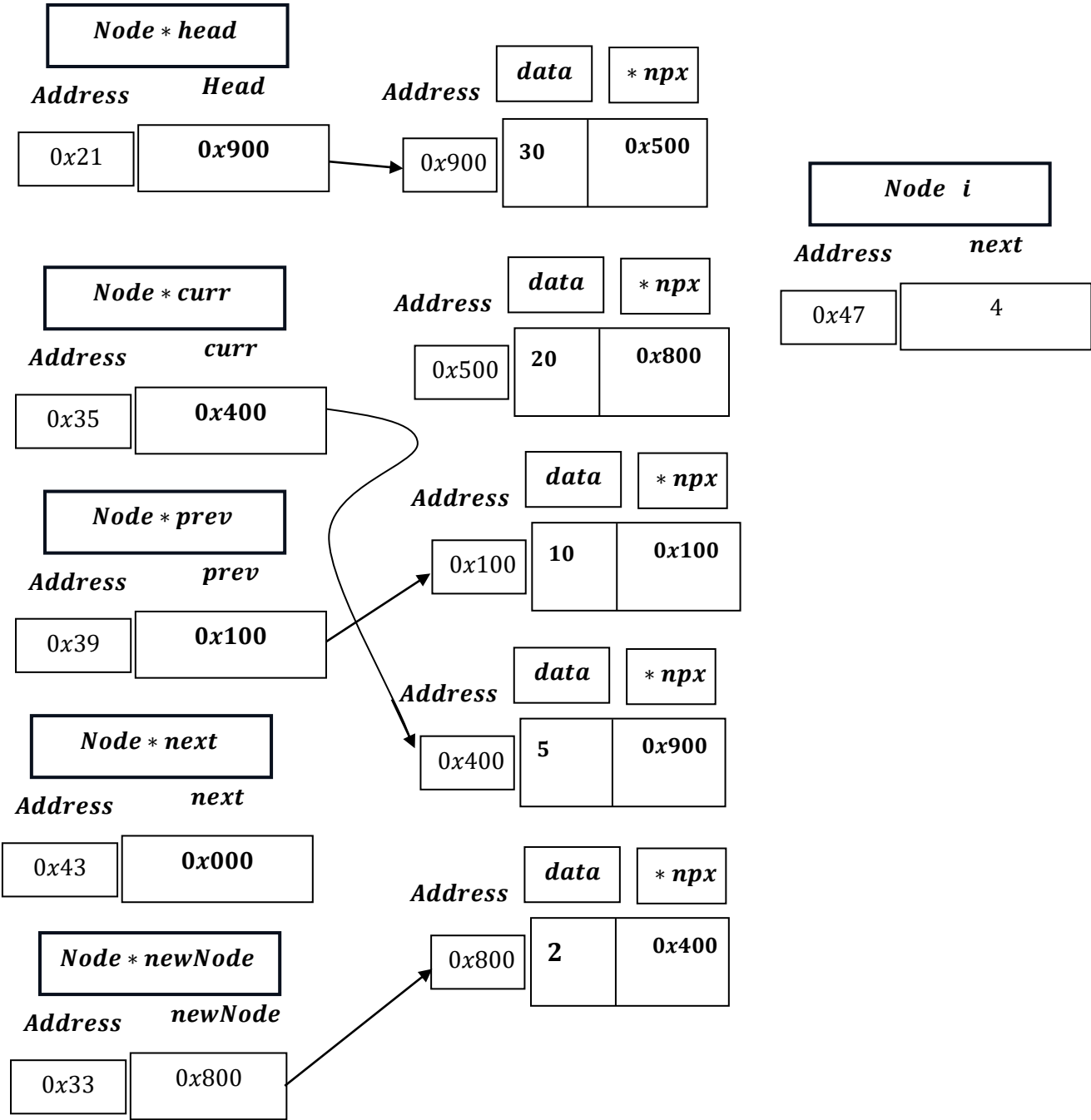


```
newNode->npx = XOR(curr, next);
```

Say, $newNode \rightarrow npx = XOR(0x400, 0x000) = 0x400$



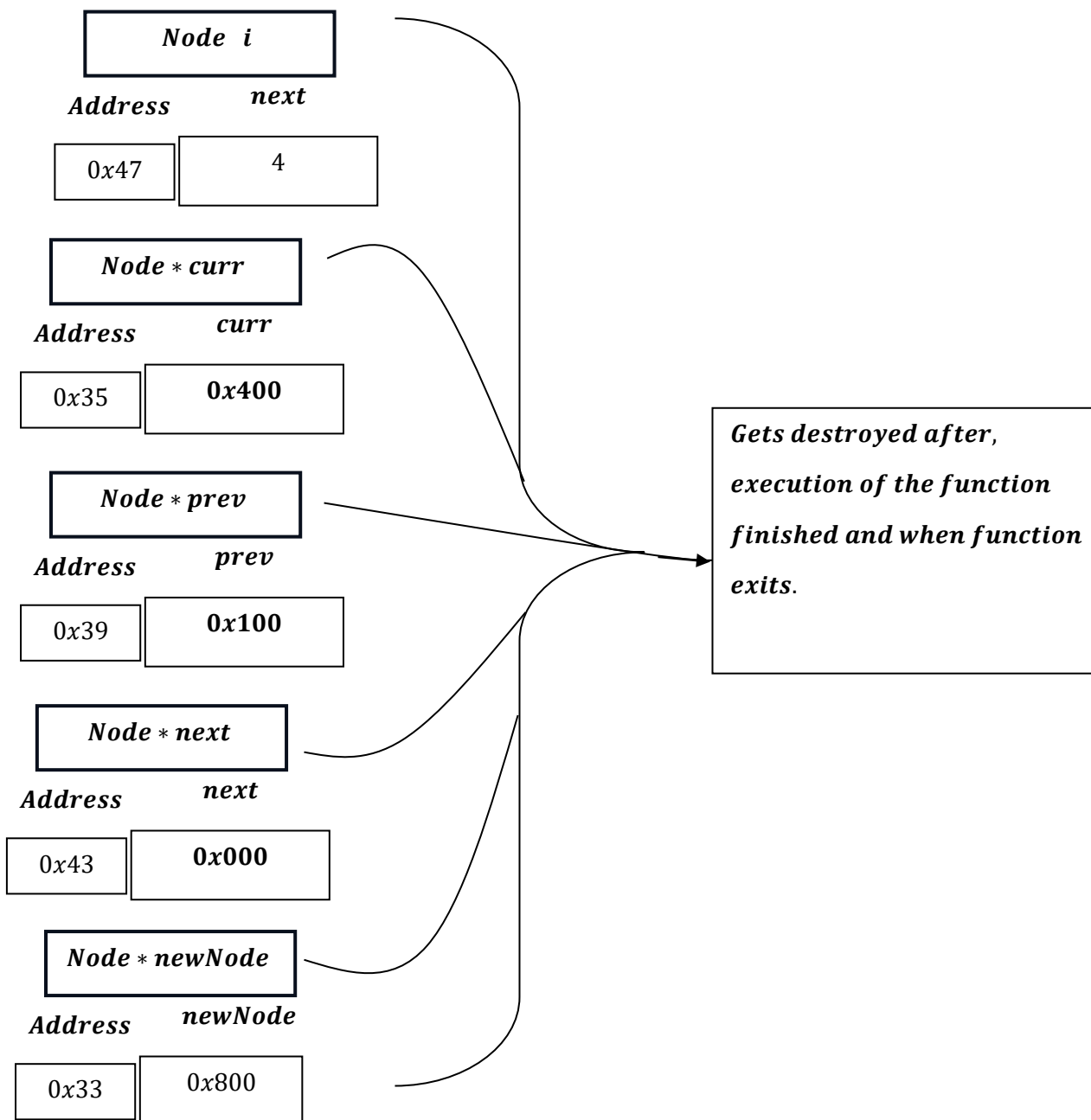
```
curr->npx = XOR(prev, newNode);  
curr → npx = XOR(0x100, 0x800) = 00012 ⊕ 10002 = 10012 = 910 = 0x900
```



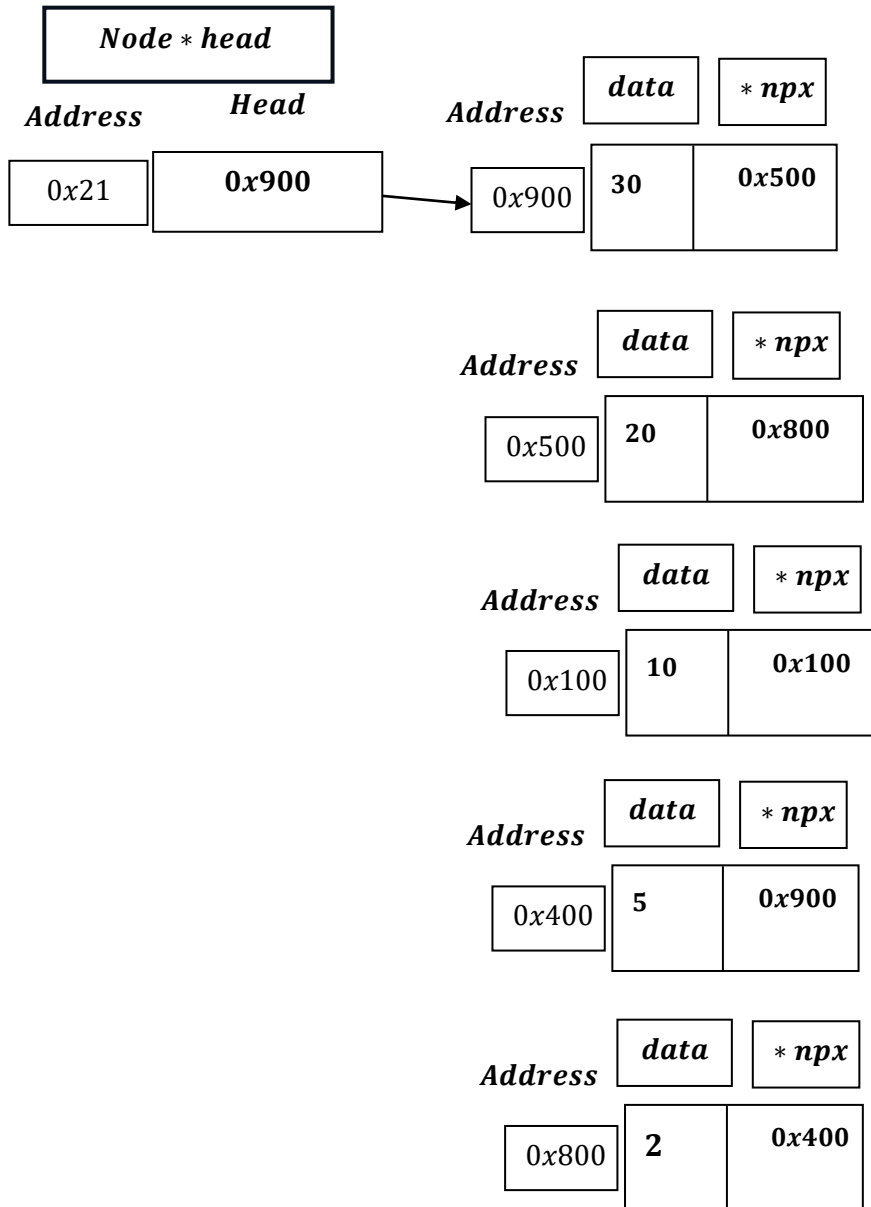
Now, we see that, $next = 0x000$ i.e. $NULLPTR$ hence ,

When $if(next \neq NULL) \rightarrow$ Condition is False.

As , functions ends:



And we are left with:



Memory Efficient Doubly Linked List
–Insert At Position and Time Complexity
–Part 3

***When if(curr == NULL) → Condition is True. [When Position
is out of bounds]***

let , pos = 6

```
while (curr != nullptr && i < pos - 1)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

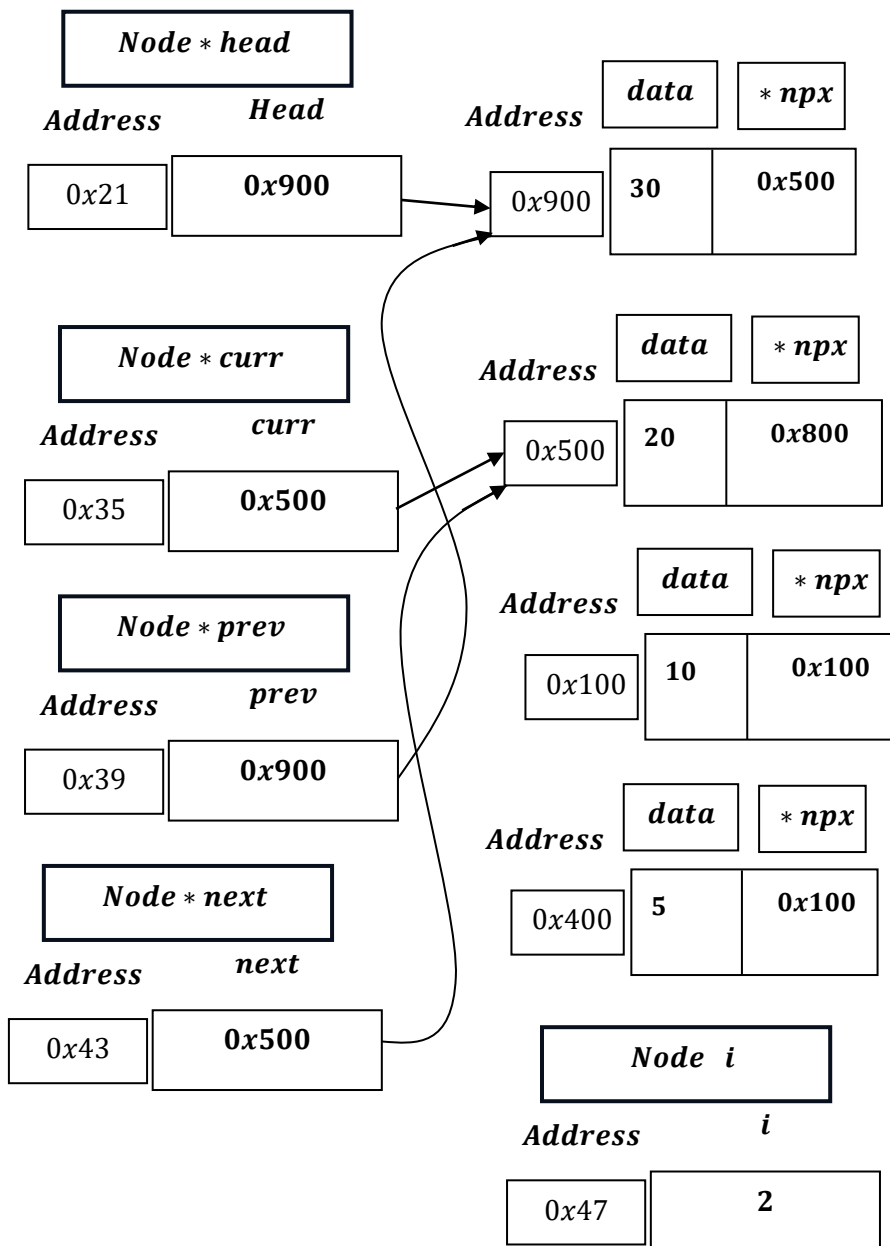
$curr(0x900) \neq 0x000$ And $i < (6 - 1 = 5) \Rightarrow 1 < 5$ is true:

$next = XOR(0x000, curr \rightarrow npx \Rightarrow 0x500) = 0x500.$

$prev = curr = 0x900;$

$curr = next = 0x500;$

$i = i + 1 = 1 + 1 = 2;$



$curr(0x500) \neq 0x000$ And $i < (6 - 1 = 5) \Rightarrow 2 < 5$ is true:

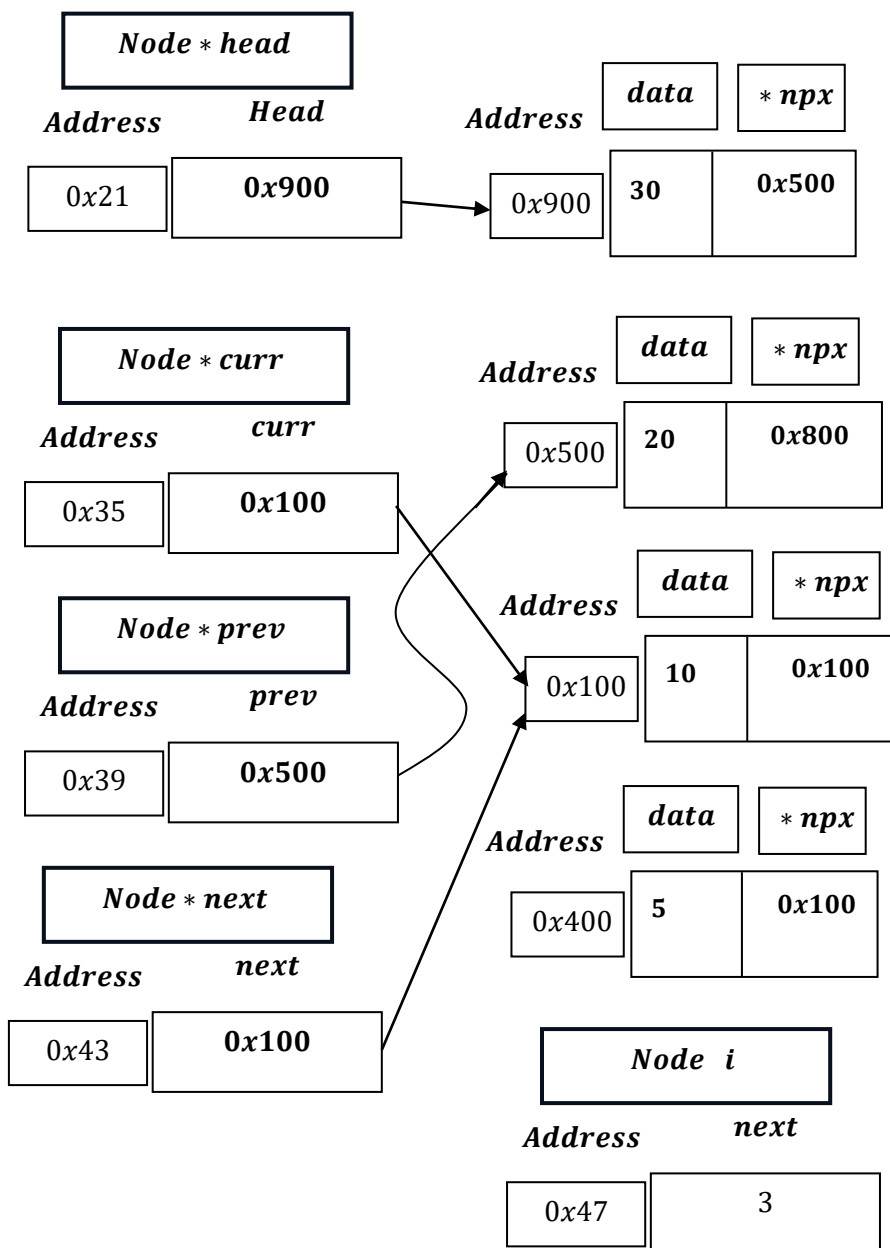
$next = XOR(0x900, curr \rightarrow npx \Rightarrow 0x800)$

$= 1001_2 \oplus 1000_2 = 0001_2 = 0x100$

$prev = curr = 0x500;$

$curr = next = 0x100;$

$i = i + 1 = 2 + 1 = 3;$



$curr(0x100) \neq 0x000$ And $i < (6 - 1 = 5) \Rightarrow 3 < 5$ is true:

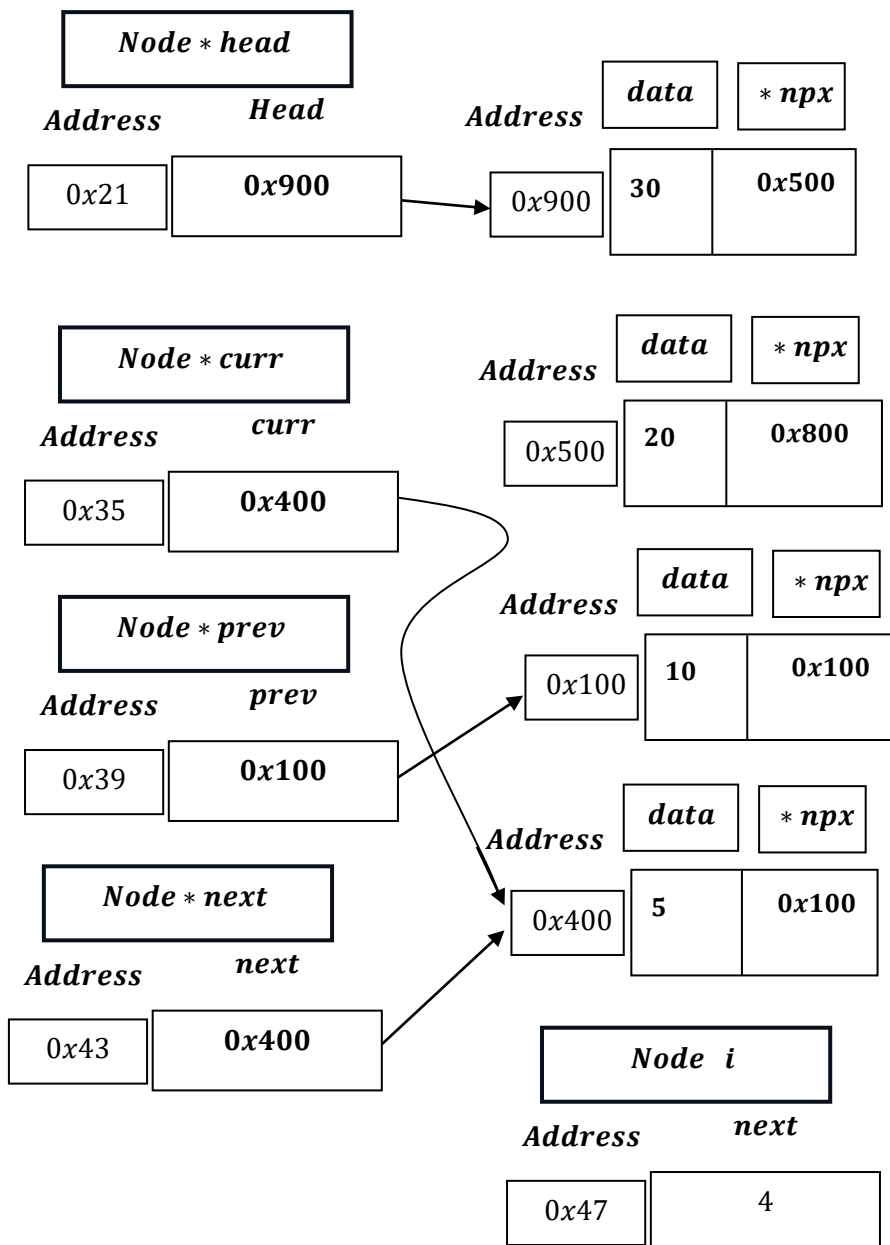
$next = XOR(0x500, curr \rightarrow npx \Rightarrow 0x100)$

$= 0101_2 \oplus 0001_2 = 0100_2 = 4_{10} = 0x400$

$prev = curr = 0x100;$

$curr = next = 0x400;$

$i = i + 1 = 3 + 1 = 4;$



$curr(0x400) \neq 0x000$ And $i < (6 - 1 = 5) \Rightarrow 4 < 5$ is true:

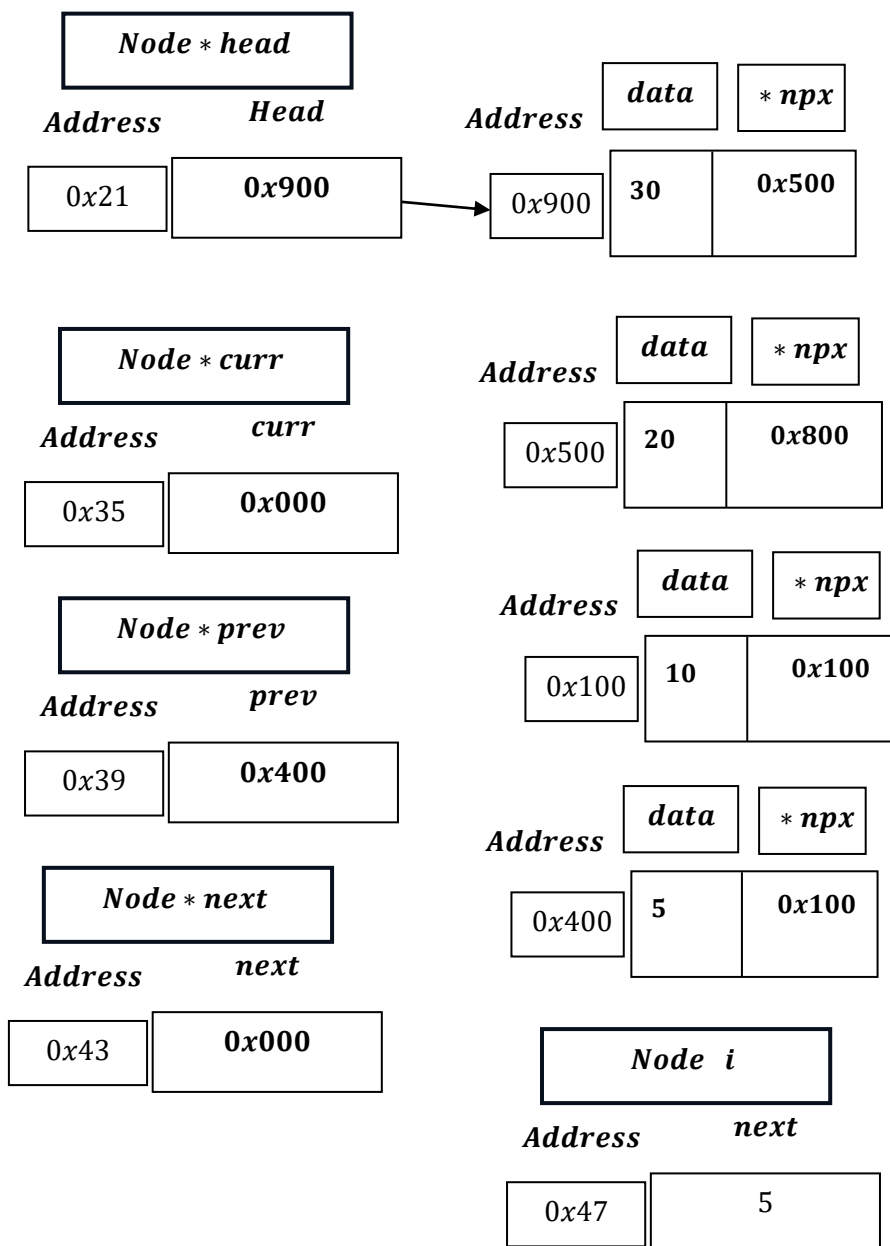
$next = XOR(0x100, curr \rightarrow npx \Rightarrow 0x100)$

$= 0001_2 \oplus 0001_2 = 0000_2 = 0_{10} = 0x000$

$prev = curr = 0x400;$

$curr = next = 0x000;$

$i = i + 1 = 4 + 1 = 5;$



$\text{curr}(0x000) \neq 0x000$ is false And $i < (6 - 1 = 5) \Rightarrow 5 < 5$ is false.

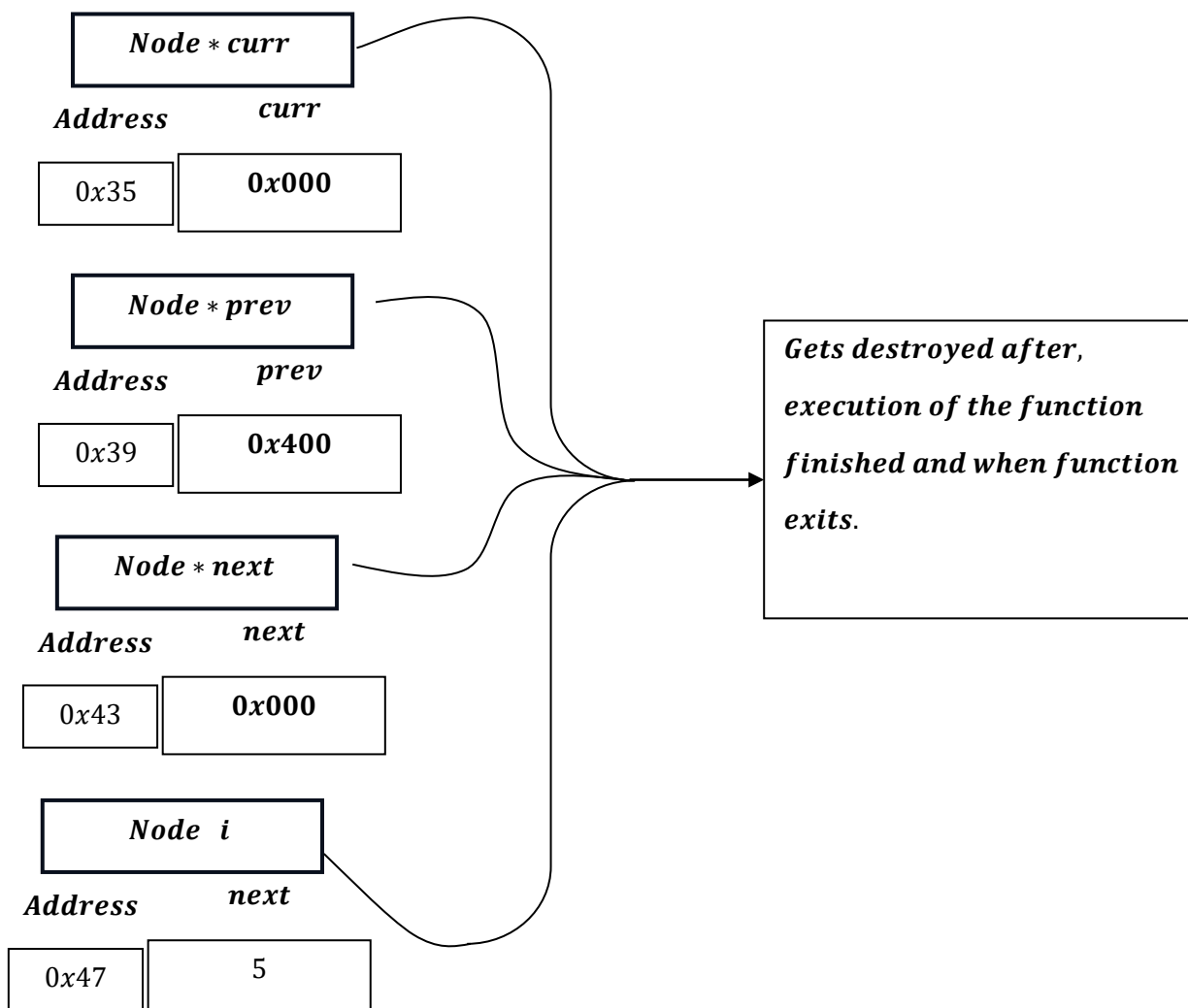
Hence , loop exits.

```
if (curr == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

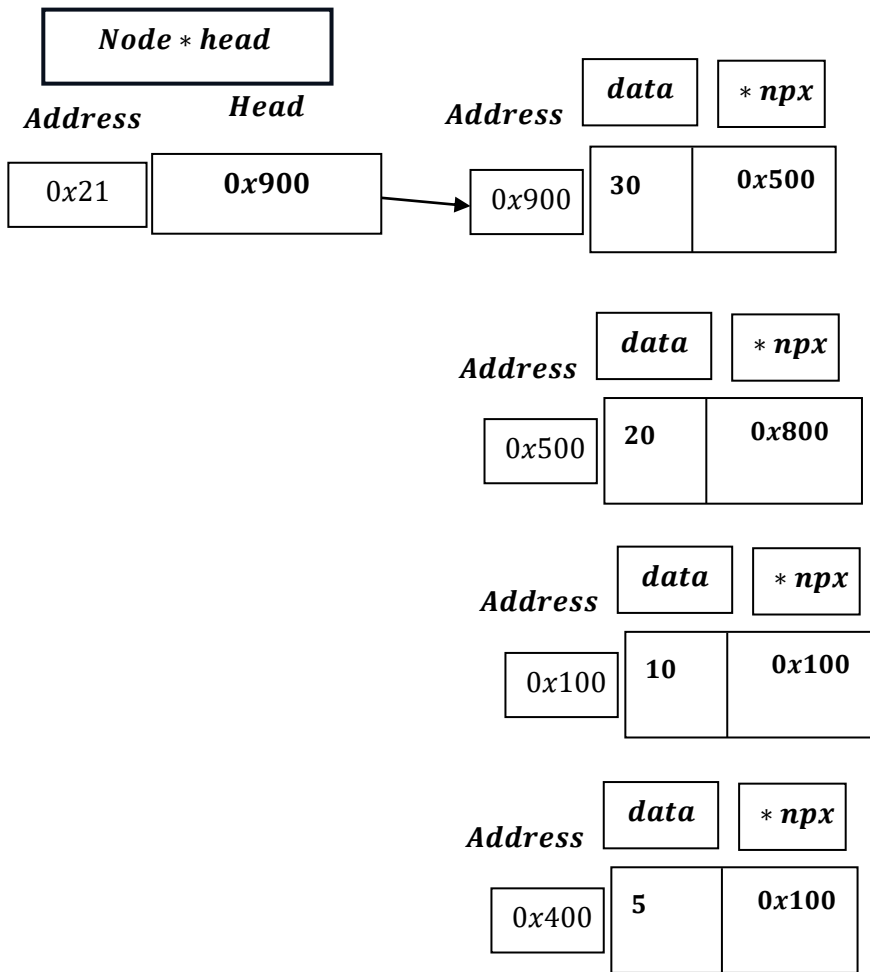
As, $\text{curr} = \text{NULL}$, the it prints "Position out of bounds!"

Then, return void and exits from the function.

As , the function exits :



And we are left with:



Time Complexity of Insert At Position

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    while (curr != nullptr && i < pos - 1)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    // We are inserting AFTER 'curr'.
    // Let 'next' be the node currently after 'curr'.
}
```

The diagram illustrates the time complexity of the `insertAtPosition` function. It uses brackets and arrows to group code segments and link them to their respective time complexities:

- The first `if` block (checking for invalid position) is labeled $O(1)$.
- The second `if` block (checking for insertion at the beginning) is labeled $O(1)$.
- The initialization of `Node *curr = head;`, `Node *prev = nullptr;`, `Node *next;`, and `int i = 1;` is collectively labeled $O(1)$.
- The `while` loop, which traverses the list to find the insertion point, is labeled *Runs n times at worst time complexity .* (representing $O(n)$).
- The final `if` block (checking for out-of-bounds) is labeled $O(1)$.


```

next = XOR(prev, curr->npx); —————→ O(1)

Node *newNode = (Node *)malloc(sizeof(Node)); —————→ O(1)
newNode->data = data; —————→ O(1)

// Linkage Logic:
// 1. Setup newNode: npx = curr ^ next
newNode->npx = XOR(curr, next); —————→ O(1)

// 2. Update curr: npx was (prev ^ next), now becomes
(prev ^ newNode)
curr->npx = XOR(prev, newNode); —————→ O(1)

// 3. Update next (if it exists): npx was (curr ^
nextNext), now becomes (newNode ^ nextNext)
if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
} —————→ O(1)

cout << data << " inserted at position " << pos << "."
<< endl; —————→ O(1)
}

```

Best Case

Best Case : Absolute Best Case $\rightarrow POS < 1, = O(1)$.

Also, pos == 1 also Best Case but not Absolute Best Case.

Worst Case

The loop takes `n` times at worst case ,when position `pos` is out of bounds. $\approx O(n)$.

Average Case

→ If the position is 1, the loop's inner statement runs 0 time.

→ If the position is 2, the loop's inner statement runs 0 time.

→ If the position is 3, the loop's inner statement runs 1 times.

→ If the position is 4, the loop's inner statement runs 2 times.

.

.

→ If the position is $(N + 1)$, the loop's inner statement runs : $N - 1$ times ,

As when $i = N < N$ becomes false ,hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

Now, as we take all the possibilites then ,we take the best case also i.e.

$O(1)$, when position = 1 and 2. Considering `T(P)` is time complexity of position `P` .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ 1 & , \text{when } P = 2. \\ P - 2 \text{ or } N - 1 & , \text{when } P = 3, 4, \dots, N + 1. \end{cases}$$

Now, If there is `N` no. of nodes there we can insert data at $N + 1$ positions.

For position 1, the probability of insertion is $\frac{1}{N + 1}$.

For position 2, the probability of insertion is $\frac{1}{N + 1}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N + 1}$.

Hence probability of insertion will always remain $\frac{1}{N + 1}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N + 1} \times 1 + \frac{1}{N + 1} \times 2 + \dots + \frac{1}{N + 1} \times N - 1 \right) + \frac{1}{N + 1} \times 1 + \frac{1}{N + 1} \times 1$$

$$= \frac{1}{N + 1} \times \{(1 + 2 + 3 + \dots + N - 1) + 1 + 1\}$$

$$= \frac{1}{N + 1} \times \left\{ \sum_{i=1}^{N-1} i + 2 \right\}$$

$$= \frac{1}{N + 1} \times \left(\frac{N(N - 1)}{2} + 2 \right)$$

$$= \frac{N(N - 1)}{2(N + 1)} + \frac{2}{N + 1}$$

$$= \frac{N^2 - N}{2N + 2} + \frac{2}{N + 1}$$

Applying Big Theta in the equation:

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

Now, taking out the dominant term of numerator and denominator:

- ***In the numerator, $N^2 - N$, the term N^2 grows much faster than N .***

So, for large N , the dominant term is N^2 .

- ***In the denominator, $2N + 2$, the term $2N$ grows much faster than the constant 2. The dominant term is $2N$.***

$$= \Theta\left(\frac{N^2}{2N}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + 2 \times \Theta\left(\frac{1}{N}\right)$$

Now removing the constant `2` we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

$\rightarrow \Theta(N)$ grows linearly with N .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ decreases to a constant, i.e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant} \right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note, by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Apporach]:
(Simplest approach)

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.

.

For position $N+1$, the probability of insertion is $\frac{1}{N+1}$.

***Hence probability of insertion will always remain $\frac{1}{N+1}$,
for position 1 to $N+1$.***

Average case complexity : Probability of insertion $\times$ $\left(\begin{array}{l} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N+1} \times \left(\sum_{i=1}^{N+1} i \right)$$

$$= \frac{1}{N+1} \times \left(\frac{1}{2} (N+1) ((N+1) + 1) \right) \left[\text{As, } \sum_{k=1}^n k = \frac{1}{2} (n)(n+1) \right]$$

$$= \frac{1}{N+1} \times \left(\frac{(N+1)(N+2)}{2} \right)$$

$$= \frac{N+2}{2}$$

$$= \frac{N}{2} + \frac{2}{2}$$

$$= \frac{N}{2} + 1$$

Applying Big Theta Θ we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta(1)$$

$$= \frac{1}{2} \times \Theta(N)$$

$$= \Theta(N)$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N}{2} + 1\right)$ or $\Theta(N)$.

Now,

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N}{2} + 1 \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Insert At Position</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

Memory Efficient Doubly Linked List

–Insert At Position and Time Complexity

Program:

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    // Traverse to find the node BEFORE the insertion point
    (pos-1)
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    while (curr != nullptr && i < pos - 1)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }
}
```

```

    }

    // We are inserting AFTER 'curr'.
    // Let 'next' be the node currently after 'curr'.
    next = XOR(prev, curr->npx);

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    // Linkage Logic:
    // 1. Setup newNode: npx = curr ^ next
    newNode->npx = XOR(curr, next);

    // 2. Update curr: npx was (prev ^ next), now becomes
    (prev ^ newNode)
    curr->npx = XOR(prev, newNode);

    // 3. Update next (if it exists): npx was (curr ^
    nextNext), now becomes (newNode ^ nextNext)
    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(newNode, nextNext);
    }

    cout << data << " inserted at position " << pos << "."
    << endl;
}

...
case 3:
    cout << "Enter data to insert: ";
    cin >> data;
    cout << "Enter position: ";
    cin >> pos;
    insertAtPosition(data, pos);
    break;

```

Program Analysis:

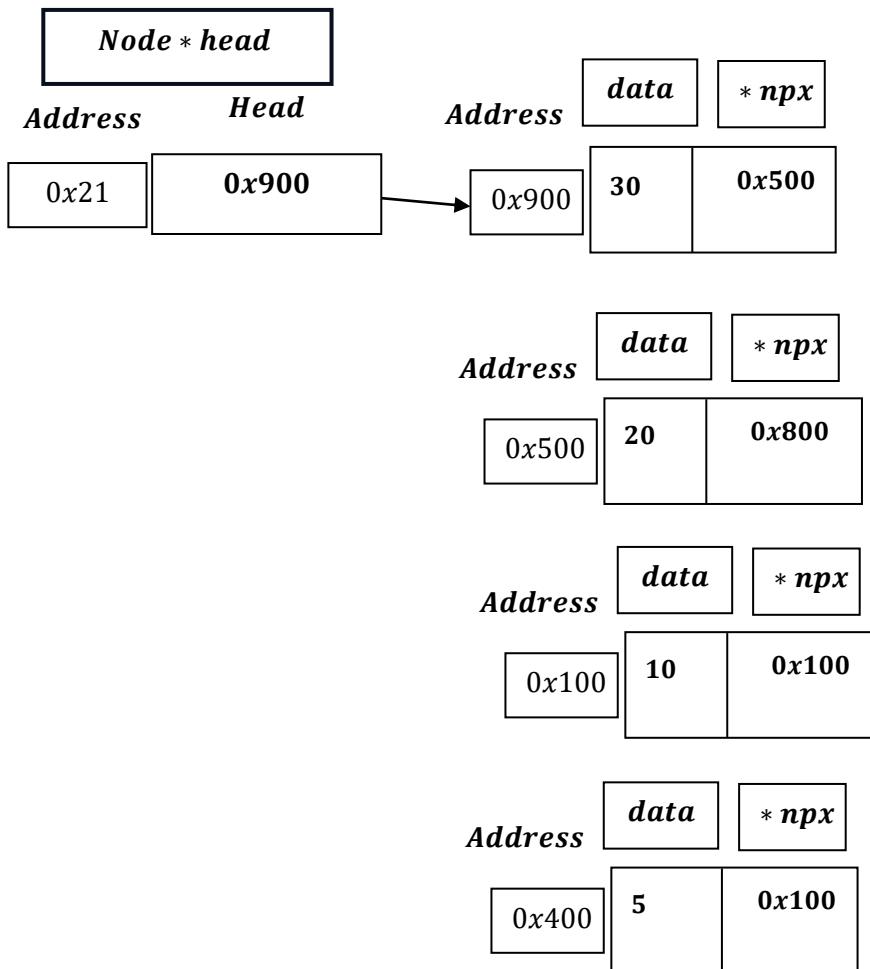
```
if (pos < 1)
{
    cout << "Invalid position!" << endl;
    return;
}
```

*if pos is less than 1 ,it will print ``Invalid position!``
and return void and exit from the function.*

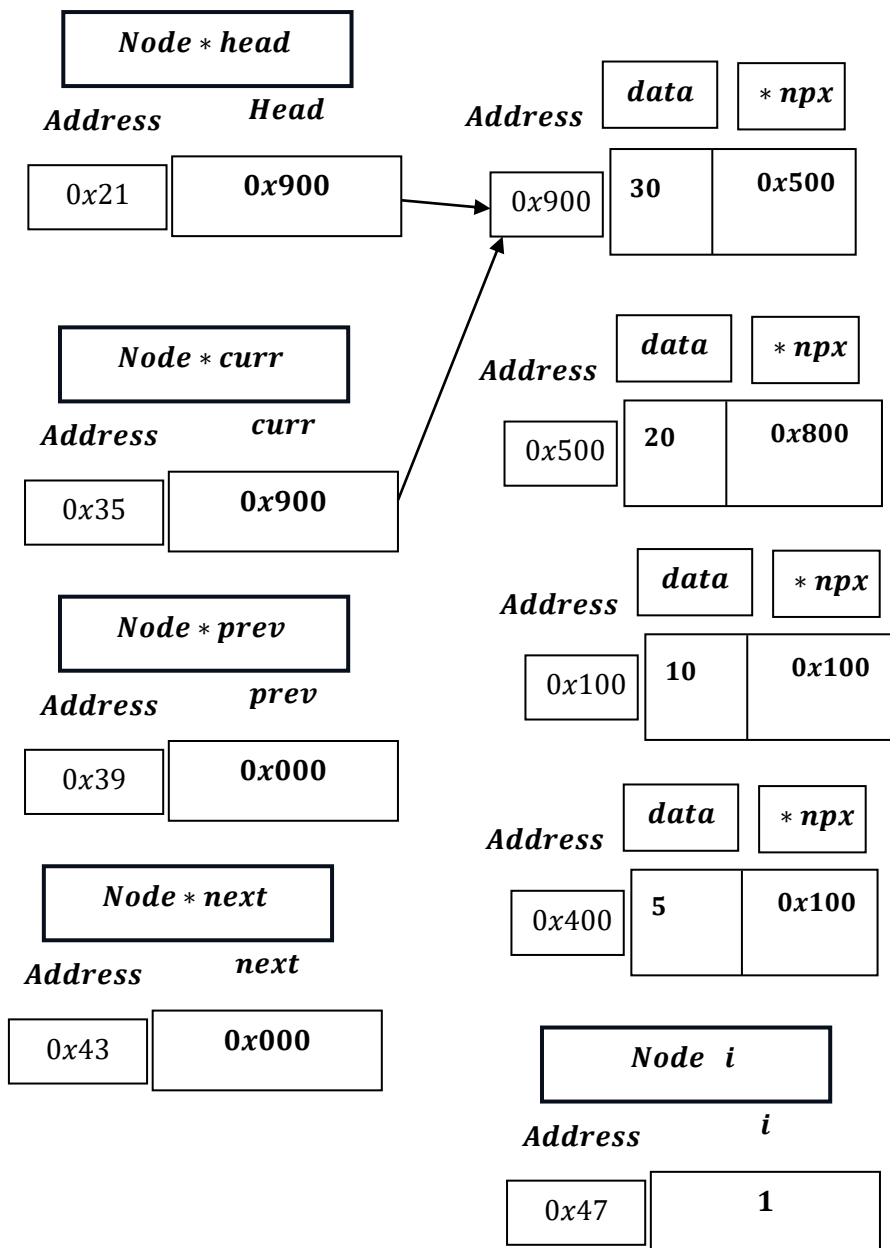
```
if (pos == 1)
{
    insertAtBeginning(data);
    return;
}
```

*if pos is equal to 1 ,it will run: insertAtBeginning(data);
and return void and exit from the function.*

If $pos > 1$, then we have 4 nodes, say:



```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;  
int i = 1;
```



```
while (curr != nullptr && i < pos - 1)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

let , pos = 4

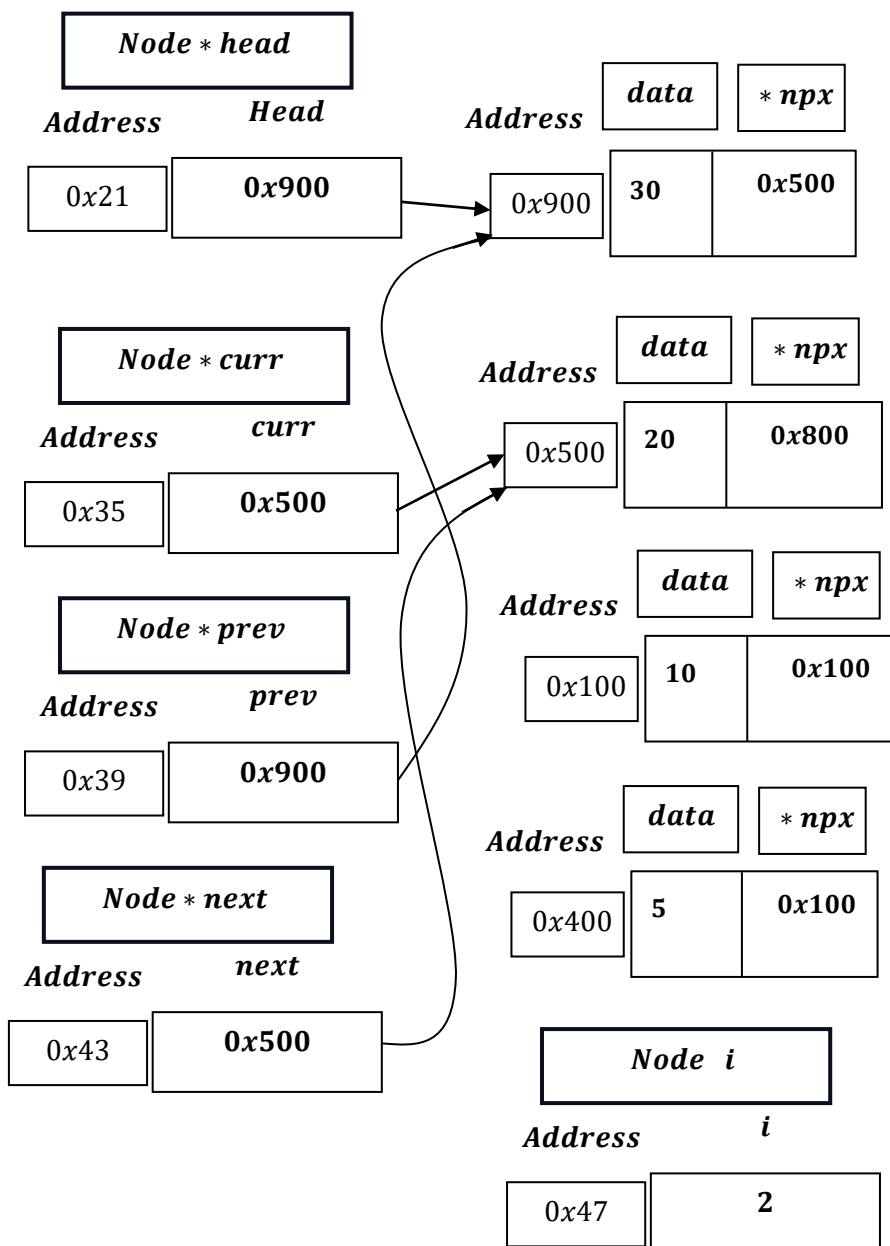
curr(0x900) ≠ 0x000 And i < (4 − 1 = 3) ⇒ 1 < 3 is true:

next = XOR(0x000, curr → npx ⇒ 0x500) = 0x500.

prev = curr = 0x900;

curr = next = 0x500;

i = i + 1 = 1 + 1 = 2;



$curr(0x500) \neq 0x000$ And $i < (4 - 1 = 3) \Rightarrow 2 < 3$ is true:

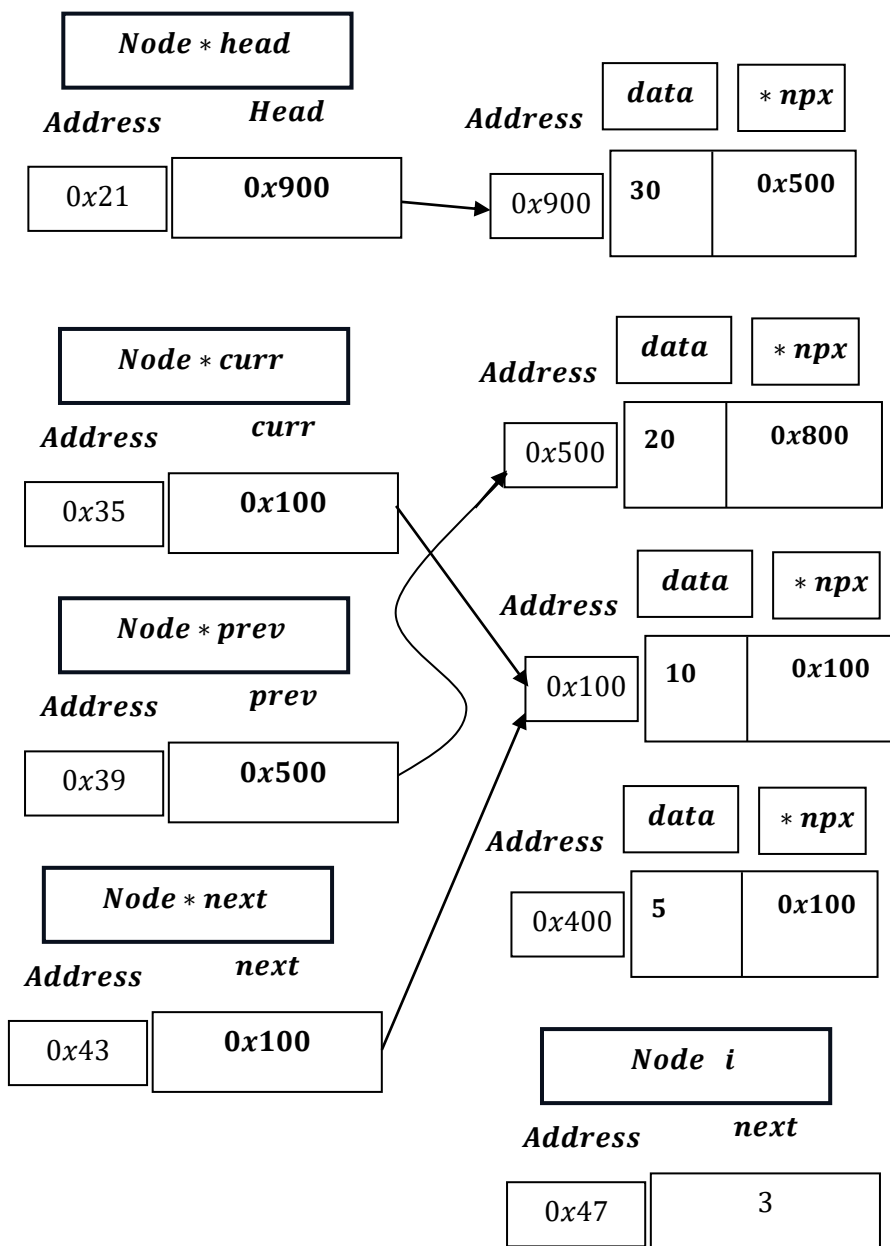
$next = XOR(0x900, curr \rightarrow npx \Rightarrow 0x800)$

$= 1001_2 \oplus 1000_2 = 0001_2 = 0x100$

$prev = curr = 0x500;$

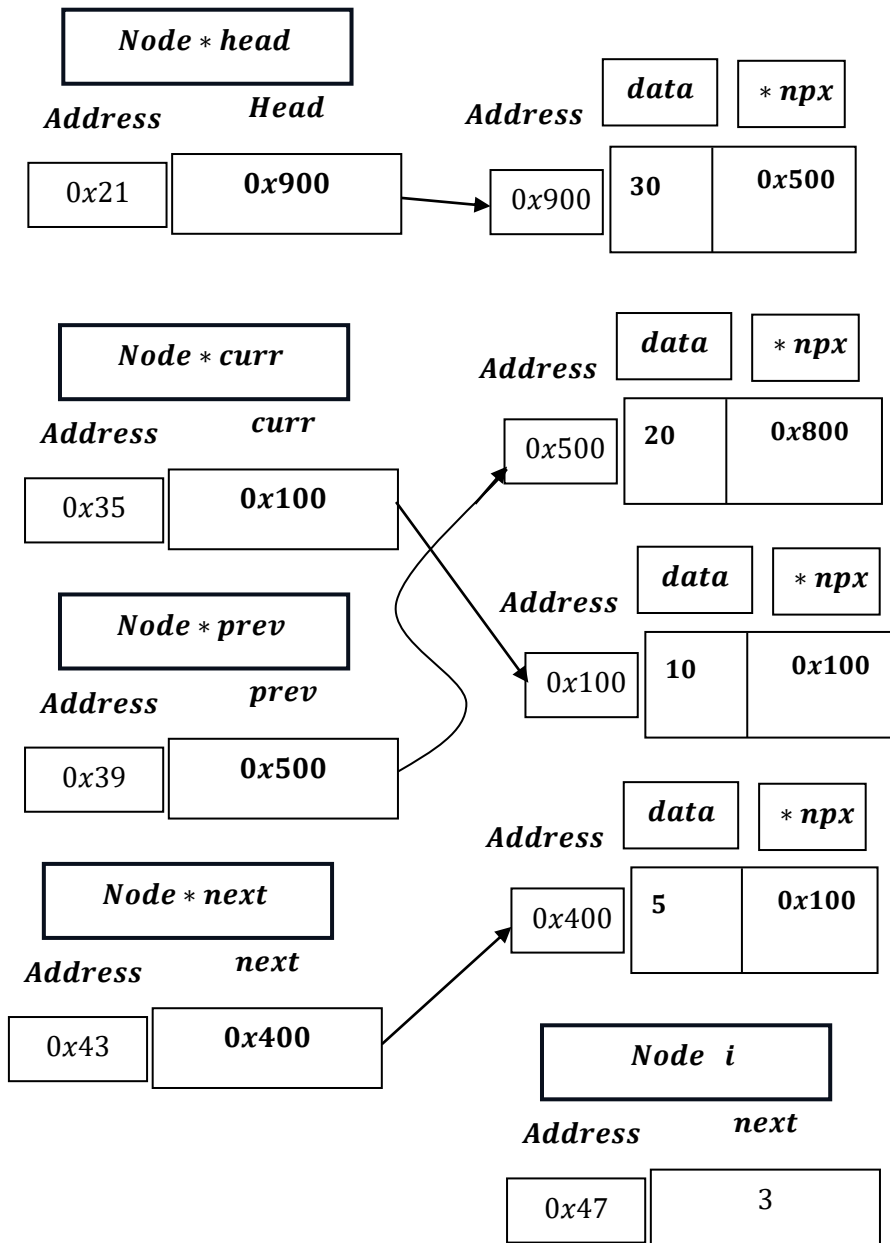
$curr = next = 0x100;$

$i = i + 1 = 2 + 1 = 3;$



$curr(0x100) \neq 0x000$ And $i < (4 - 1 = 3) \Rightarrow 3 < 3$ is false. Hence , loop exits.

```
next = XOR(prev, curr->npx);
next = XOR(0x500, 0x100) = 01012 ⊕ 00012 = 01002 = 410 = 0x400
```




```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *npx(a pointer to another Node) is typically 8 bytes.*

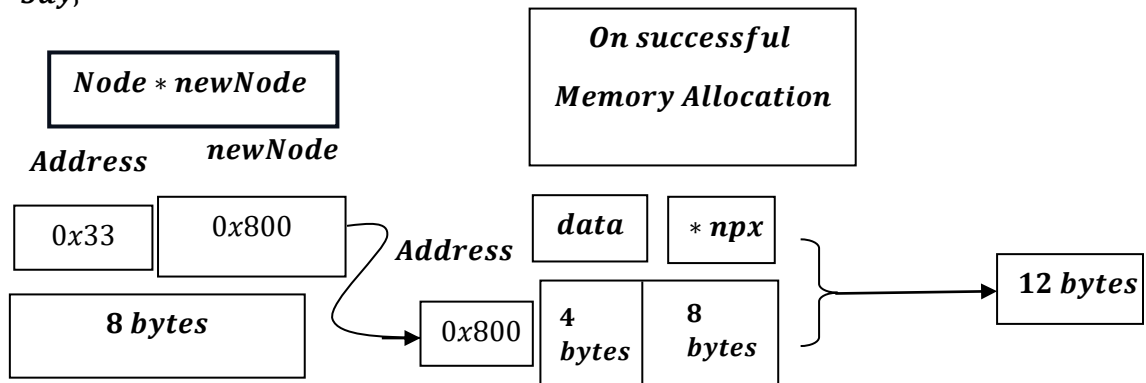
→ *Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npx) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

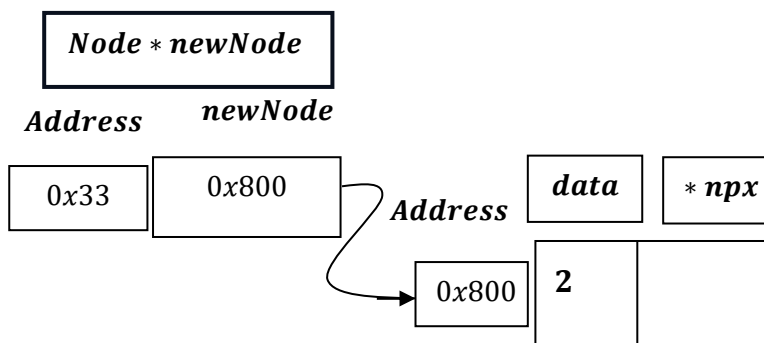
→ *newNode will try to allocate memory for data i.e. 4 bytes ,npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier ,the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



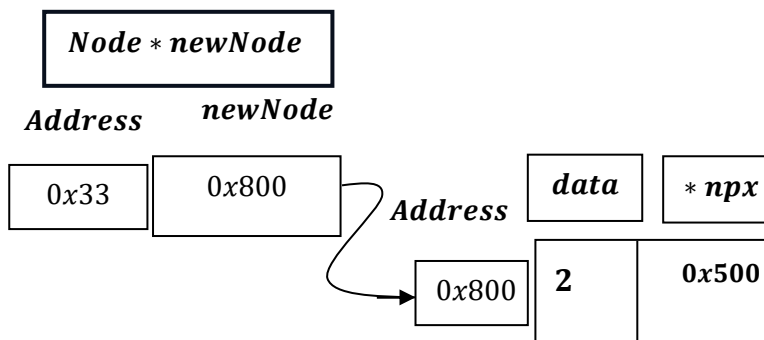
```
newNode->data = data;
```

Say, $data = 2$. Then, $newNode \rightarrow data = data = 2$.

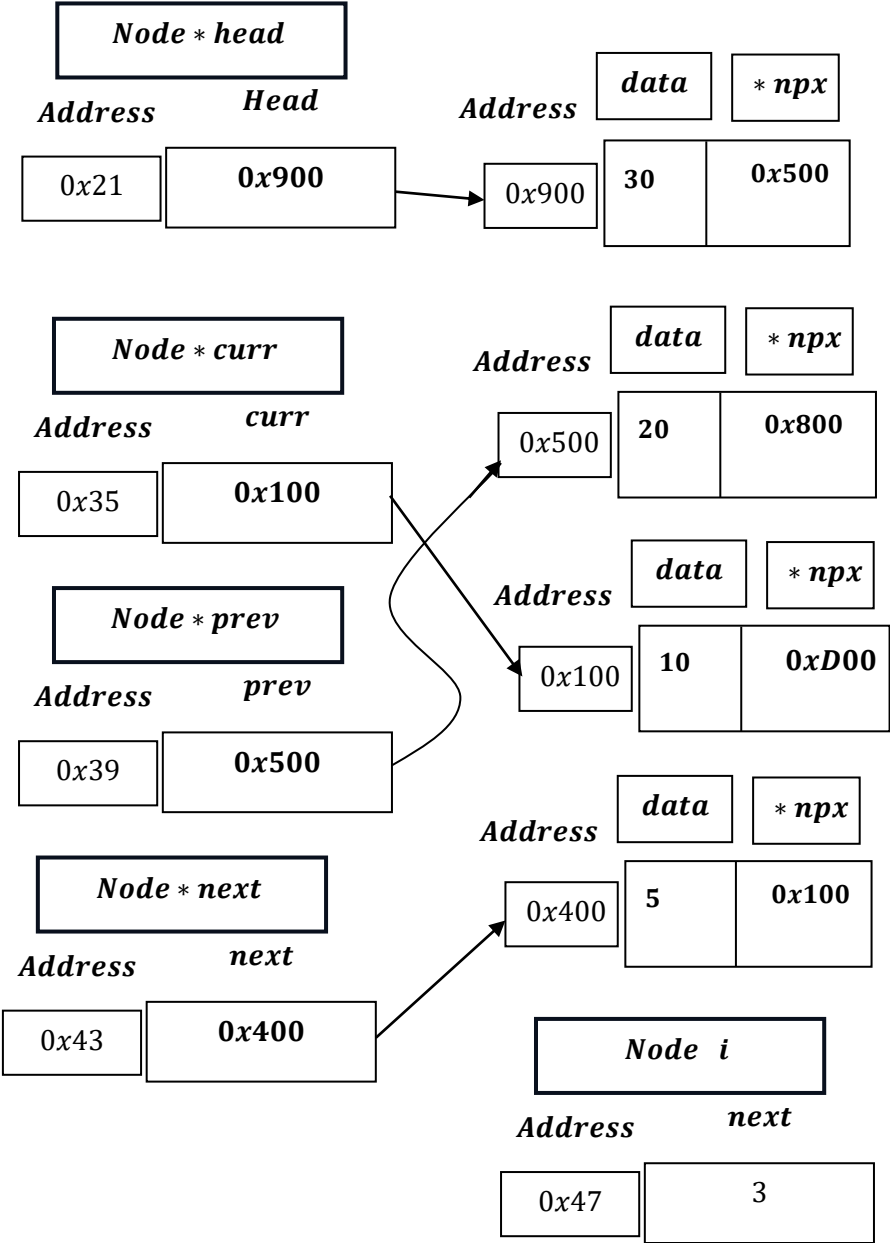


```
newNode->npx = XOR(curr, next);
```

Say, $newNode \rightarrow npx = XOR(0x100, 0x400) = 0001_2 \oplus 0100_2 = 0101_2 = 0x500$



```
curr->npx = XOR(prev, newNode);  
curr → npx = XOR(0x500, 0x800) = 01012 ⊕ 10002 = 11012 = 1310 = 0xD00
```



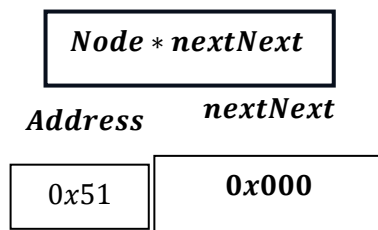
```

if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}

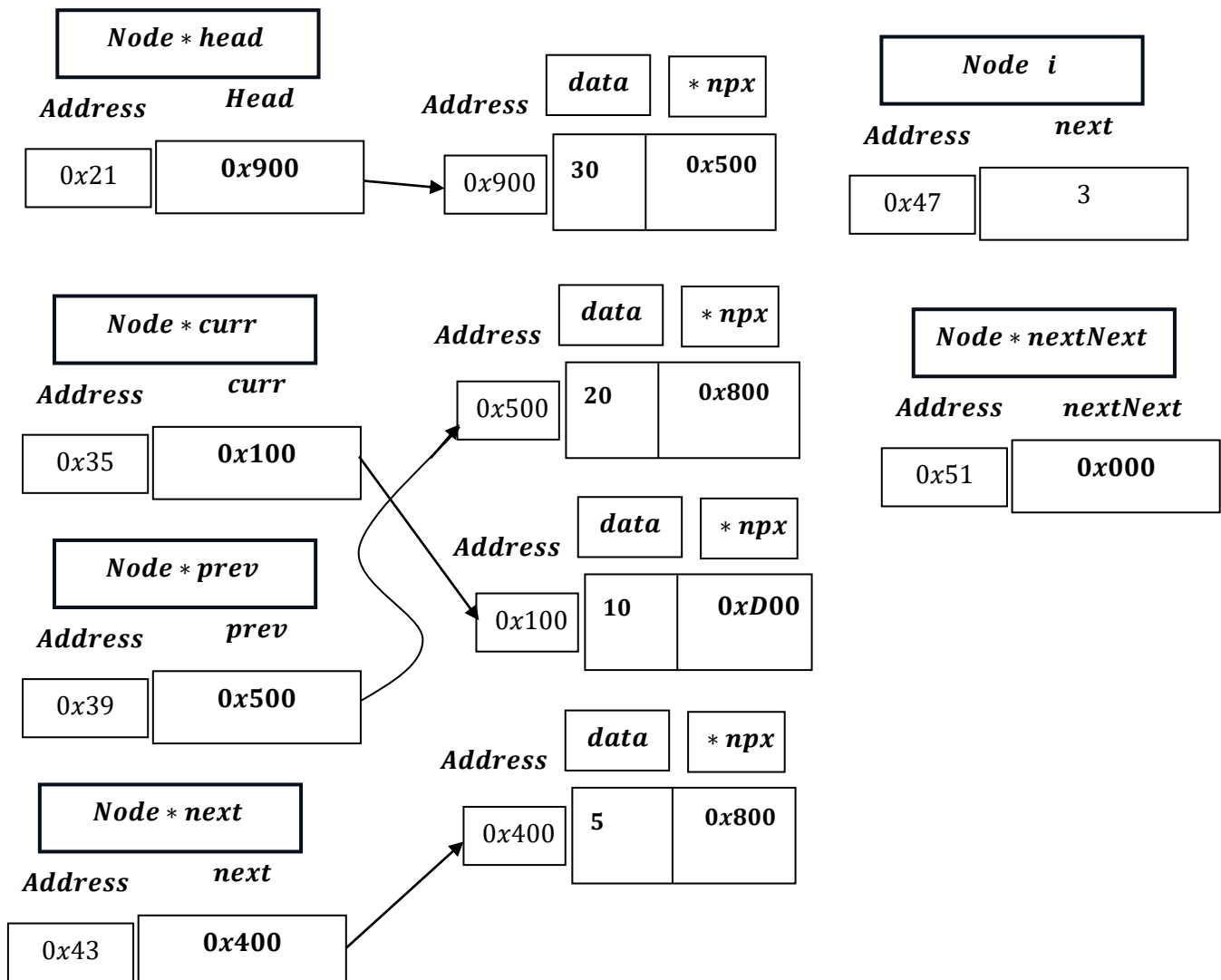
```

if next $\neq$ nullptr is true:

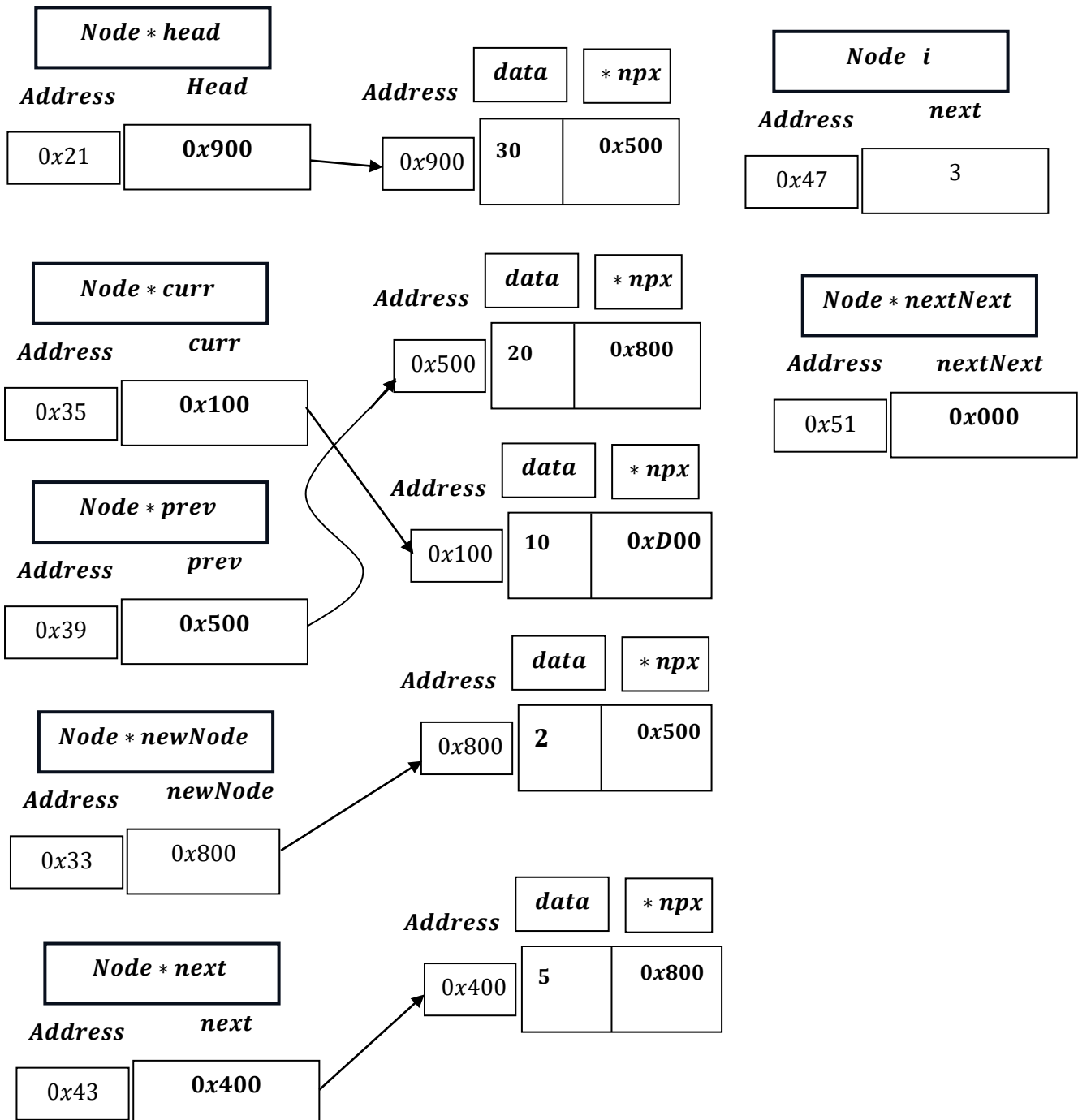
$$\text{Node} * \text{nextNext} = \text{XOR}(\text{curr}, \text{next} \rightarrow \text{npx}) = 0x100 \oplus 0x100 = 0x000$$



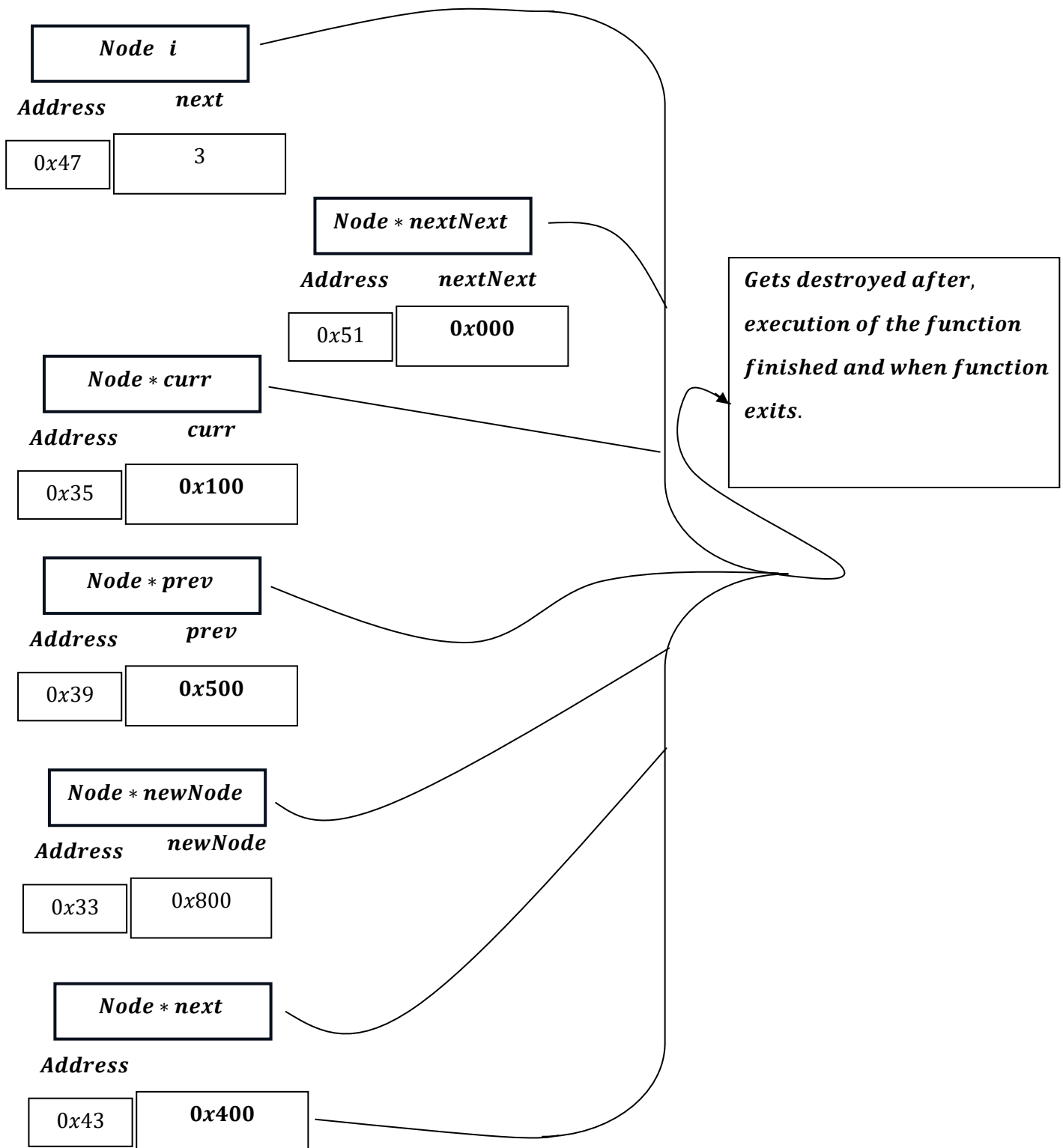
$$\text{next} \rightarrow \text{npx} = \text{XOR}(\text{newNode}, \text{nextNext}) = 0x800 \oplus 0x000 = 0x800(\text{Direct})$$



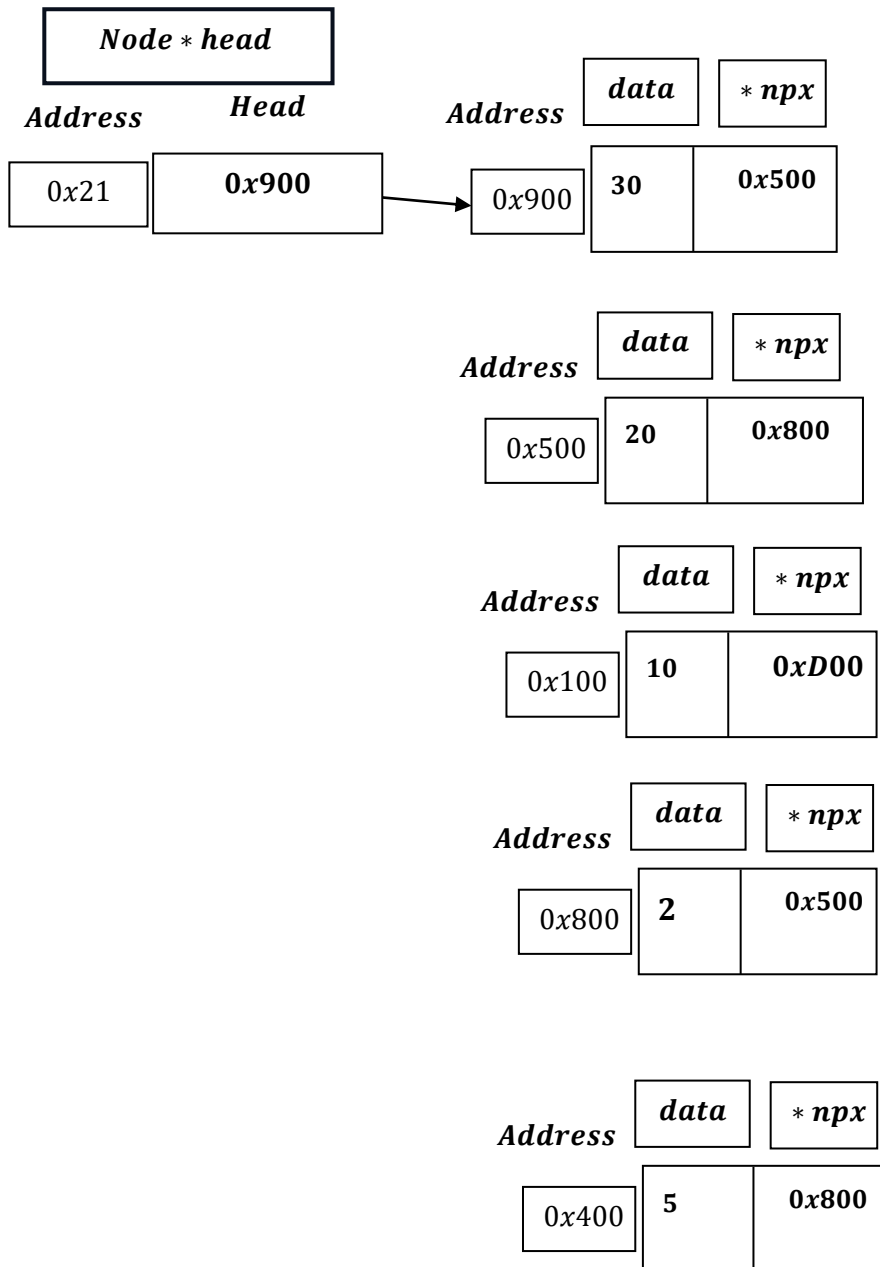
Now, we have:



As , functions ends:



And we are left with:



Memory Efficient Doubly Linked List – Insert At Position –Space Complexity and Algorithm

Program :

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    // Traverse to find the node BEFORE the insertion point
    (pos-1)
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    while (curr != nullptr && i < pos - 1)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }
}
```



```

    }

    // We are inserting AFTER 'curr'.
    // Let 'next' be the node currently after 'curr'.
    next = XOR(prev, curr->npx);

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    // Linkage Logic:
    // 1. Setup newNode: npx = curr ^ next
    newNode->npx = XOR(curr, next);

    // 2. Update curr: npx was (prev ^ next), now becomes
    (prev ^ newNode)
    curr->npx = XOR(prev, newNode);

    // 3. Update next (if it exists): npx was (curr ^
    nextNext), now becomes (newNode ^ nextNext)
    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(newNode, nextNext);
    }

    cout << data << " inserted at position " << pos << "."
    << endl;
}

...
case 3:
    cout << "Enter data to insert: ";
    cin >> data;
    cout << "Enter position: ";
    cin >> pos;
    insertAtPosition(data, pos);
    break;

```

Space Complexity

1. Input Space Complexity

- **Parameter – only Definition:**
 - **Function Parameters:** *int data and int pos.*
 - **Analysis:** *Both parameters are simple integers. Regardless of the list's size (n) or the target position (pos), these occupy a fixed amount of memory (constant space).*
 - **Complexity:** $O(1)$
- **Full Data Definition:**
 - **Function Data:** *The parameters (data, pos) + the existing XOR linked list.*
 - **Analysis:** *The function operates on a list of n nodes. To reach the insertion point, it may need to traverse up to pos nodes. The entire list is the context for the operation.*
 - **Complexity:** $O(n)$

2. Auxiliary Space Complexity

Definition: Extra memory used by the algorithm during its execution (excluding input).

- **Node Allocation:** *Node *newNode = (Node *)malloc(sizeof(Node));. This allocates memory for exactly **one** new node.*
- **Local Variables:** * Pointers: curr, prev, next, and nextNext.
 - Counter: int i.
 - **Analysis:** *These are local stack variables. Their number does not grow based on the size of the list (n) or the value of pos. They are reused or discarded once the function ends.*
- **Traversal Logic:** *The traversal is iterative (a while loop). It does not use recursion, so it does not add to the call stack as the list size increases.*
- **Bitwise Operations:** *The XOR function performs bitwise math in CPU registers, requiring no extra memory.*

Conclusion: The function uses a constant amount of extra memory to create one node and manage a few pointers, no matter how large the list is.

- **Auxiliary Space Complexity: $O(1)$**

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Insert at Position (XOR Doubly Linked List)

XORINSPOS(START, ITEM, POS):

*(((Node *)malloc(sizeof(Node)) represents the availability of space
for the data; let's name it AVAIL.))*

1. [VALIDATE POSITION]

If POS < 1, then:

Write: "Invalid position!"

Return and EXIT.

[End of If structure]

2. [INSERT AT BEGINNING?]

If POS = 1, then:

Call XORINSBEG(START, ITEM)

Translating XORINSBEG(START, ITEM) :

XORINSBEG(START, ITEM):

*(((Node *)malloc(sizeof(Node)) represents the availability of
space for the data; let's name it AVAIL.))*

1. [ALLOCATE NEW NODE] Set NEWPTR := AVAIL

2. [MEMORY ALLOCATED?]

If NEWPTR = NULL, then:

Write: "Error: Memory allocation failed."

Return and EXIT.

[End of If structure]

3. [COPY DATA] Set INFO[NEWPTR] := ITEM

4. [INITIALIZE NEW NODE'S POINTER]

Set NEXTXORPTR[NEWPTR] := NULL $\oplus$ START $\left(\begin{array}{l} \text{This sets the new node's} \\ \text{pointer field to the address} \\ \text{of the current head,} \\ \text{as its previous is NULL} \end{array} \right)$

5. [UPDATE OLD HEAD IF LIST NOT EMPTY]

If START $\neq$ NULL, then:

a. Set NEXTTEMPPTR := NULL $\oplus$ NEXTXORPTR[START] $\left(\begin{array}{l} \text{Decode the} \\ \text{address of} \\ \text{the node} \\ \text{following the} \\ \text{current head} \end{array} \right)$

b. Set NEXTXORPTR[START] := NEWPTR $\oplus$ NEXTTEMPPTR $\left(\begin{array}{l} \text{Update the} \\ \text{old head} \\ \text{to point} \\ \text{back} \\ \text{to the new} \\ \text{node} \\ \text{and forward} \\ \text{to its} \\ \text{original} \\ \text{successor} \end{array} \right)$

[End of If structure]

6. [UPDATE HEAD POINTER]

Set START := NEWPTR (Update the global head to the newly created node)

7. [PRINT MESSAGE]

Write: ITEM, " inserted at the beginning. "

8. EXIT

Return and EXIT.

[End of If structure]

3. [INITIALIZE TRAVERSAL]

Set CURRPTR := START

Set PREVPTR := NULL

Set COUNT := 1

4. [TRAVERSE TO POSITION (POS - 1)]

Repeat while CURRPTR $\neq$ NULL AND COUNT < POS - 1:

a. Set NEXTTEMPPTR := PREVPTR $\oplus$ NEXTXORPTR[CURRPTR]

b. Set PREVPTR := CURRPTR

c. Set CURRPTR := NEXTTEMPPTR

d. Set COUNT := COUNT + 1

[End of Loop]

5. [CHECK BOUNDS]

If CURRPTR = NULL, then:

Write: "Position out of bounds!"

Return and EXIT.

[End of If structure]

6. [PREPARE NEW NODE AND SURROUNDINGS]

a. Set NEXTTEMPPTR := PREVPTR $\oplus$ NEXTXORPTR[CURRPTR]

b. Set NEWPTR := AVAIL

c. If NEWPTR = NULL, then:

Write: "Error: Memory allocation failed."

Return and EXIT.

d. Set INFO[NEWPTR] := ITEM

7. [LINK NEW NODE]

a. Set NEXTXORPTR[NEWPTR] :

$$= \text{CURRPTR} \oplus \text{NEXTTEMPPTR} \left(\begin{array}{l} \text{New node points back to current} \\ \text{and forward to the} \\ \text{original next node} \end{array} \right)$$

b. Set NEXTXORPTR[CURRPTR] :

$$= \text{PREVPTR} \oplus \text{NEWPTR} \left(\begin{array}{l} \text{Current node's NPX is updated} \\ \text{to point forward to the new node} \end{array} \right)$$

8. [UPDATE NEXT NODE IF IT EXISTS]

If NEXTTEMPPTR $\neq$ NULL, *then*:

a. Set NEXTNEXTTEMPPTR := CURRPTR $\oplus$ NEXTXORPTR [NEXTTEMPPTR]

b. Set NEXTXORPTR [NEXTTEMPPTR] := NEWPTR $\oplus$ NEXTNEXTTEMPPTR

(Next node's NPX is updated to point backward to the new node)

[End of If structure]

9. [PRINT MESSAGE]

Write: ITEM, " inserted at position ", POS

10. EXIT

Memory Efficient Doubly Linked List

–insertAfterElement and Time Complexity – Part 2

When next = NULL

–It occurs when Element is found and data is entered at end of the list, which have already elaborated.

When next $\neq$ NULL

–It occurs when Element is found but data is not enter at end of the list.

```
while (curr != nullptr && curr->data != element)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
}
```

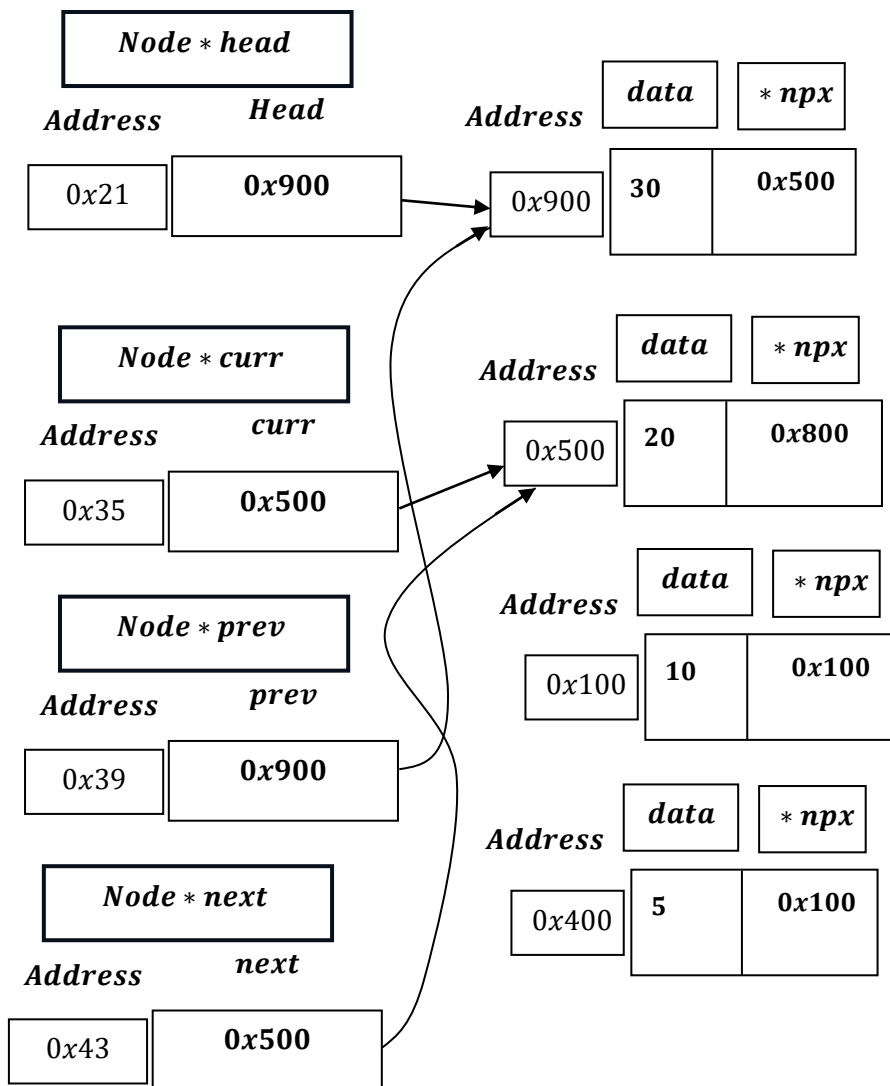
Lets say ,Element = 10 and data = 15;

curr: 0x900 $\neq$ nullptr: 0x000 AND curr $\rightarrow$ data = 30 $\neq$ element = 10 :

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x000 $\oplus$ 0x500 = 0x500.

prev = curr = 0x900

curr = next = 0x500

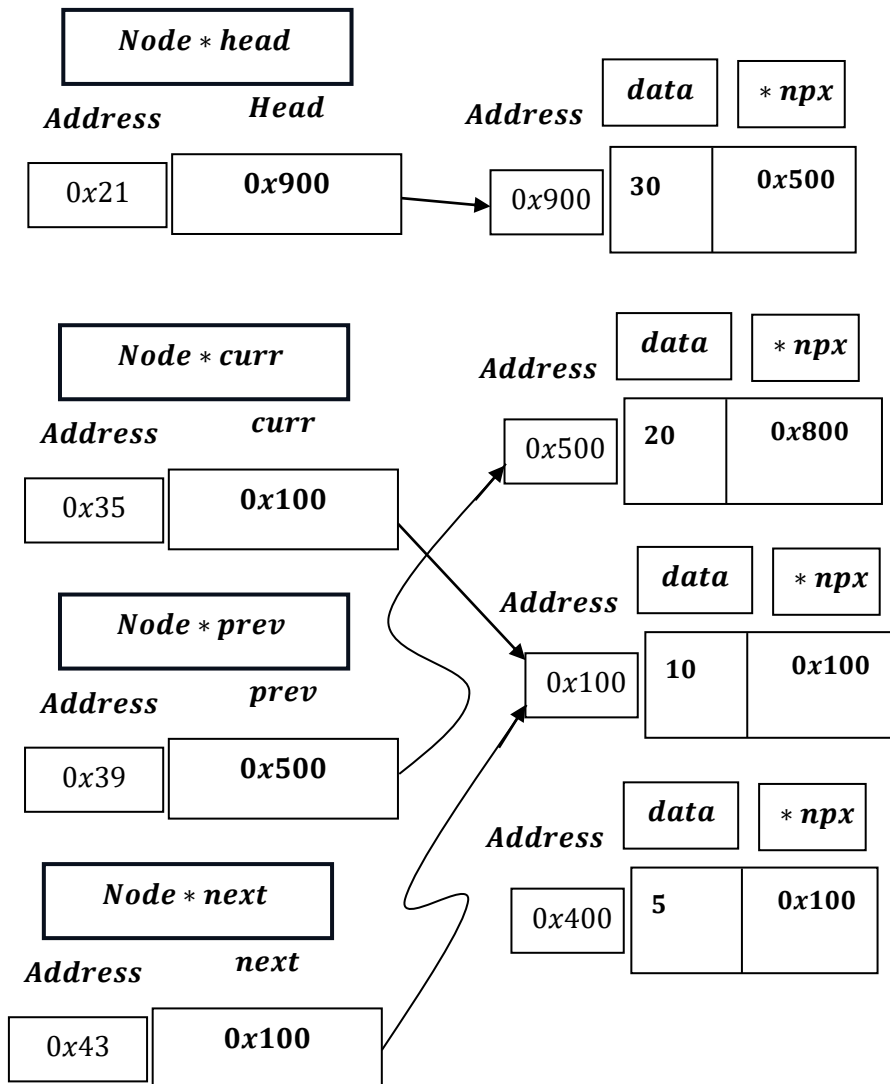


curr: 0x500 $\neq$ nullptr: 0x000 AND curr $\rightarrow$ data = 20 $\neq$ element = 10:

next = XOR(prev: 0x900, curr $\rightarrow$ npx: 0x800) = $1001_2 \oplus 1000_2 = 0001_2 = 0x100$

prev = curr = 0x500

curr = next = 0x100

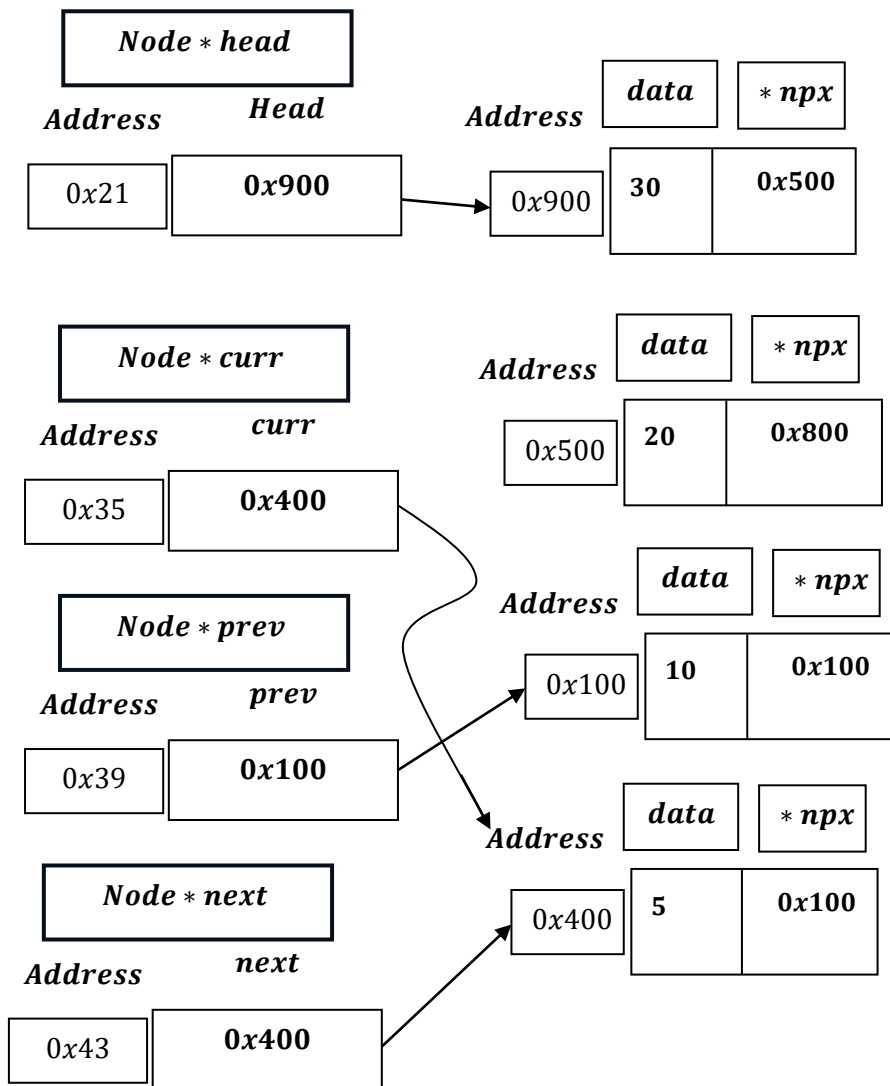


curr: 0x100 $\neq$ nullptr: 0x000[is TRUE] AND curr $\rightarrow$ data = 10 $\neq$ element = 10[is False] $\Rightarrow$ Condition is False.

```
next = XOR(prev, curr->npx);
```

$next = XOR(prev: 0x500 \oplus curr \rightarrow npx: 0x100);$

$0101_2 \oplus 0001_2 = 0100_2 = 4_{10} = 0x400.$



```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ *data*(an integer) is typically 4 bytes.

→ *npx*(a pointer to another Node) is typically 8 bytes.

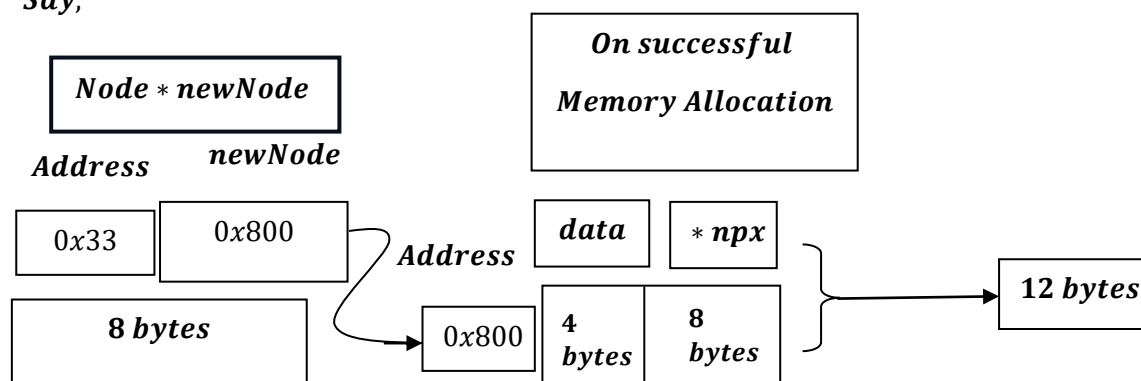
→ Therefore , the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for *npx*) .

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

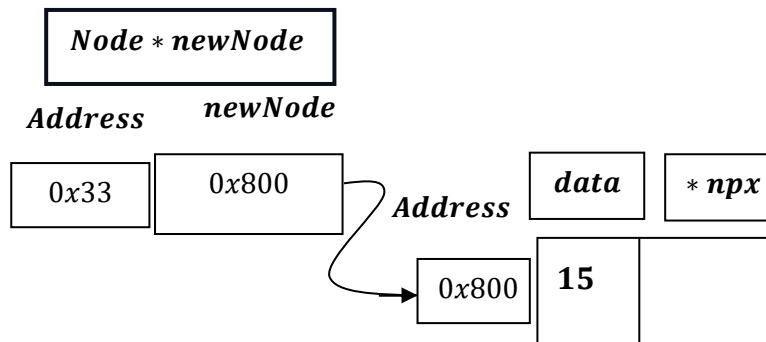
→ *newNode* will try to allocate memory for data i.e. 4 bytes , *npx* pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

Say,



```
newNode->data = data;
```

Say, $data = 15$. Then, $newNode \rightarrow data = data = 15$.



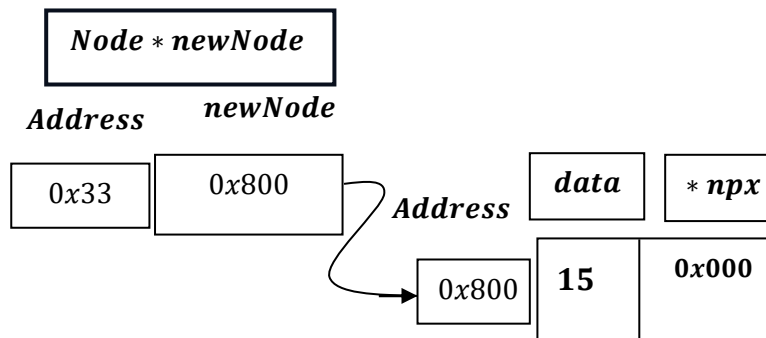
```
newNode->npx = XOR(curr, next);
curr->npx = XOR(prev, newNode);
```

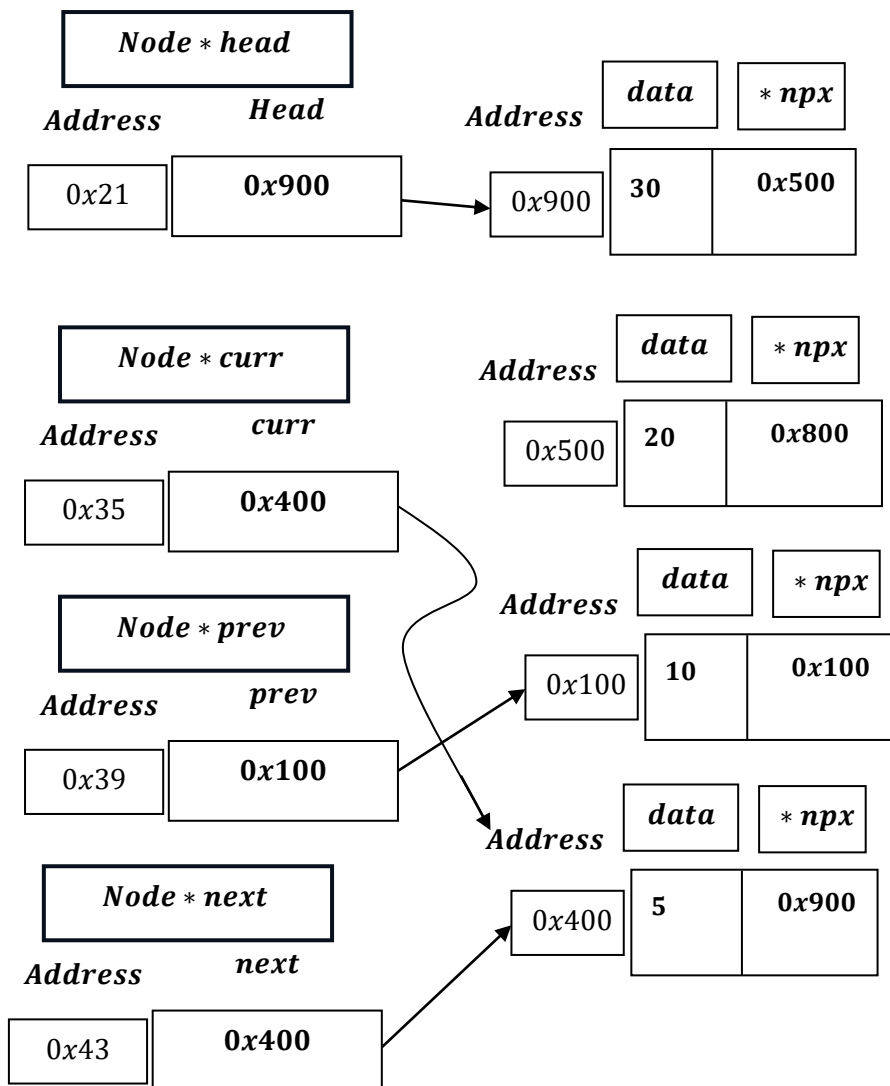
$newNode \rightarrow npx = XOR(curr: 0x400, next: 0x400)$

$= 0100_2 \oplus 0100_2 = 000_2 = 0_{10} = 0x000$.

$curr \rightarrow npx = XOR(prev, newNode) = (prev: 0x100, newNode: 0x800) =$

$0001_2 \oplus 1000_2 = 1001_2 = 0x900$.





```
if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}
```

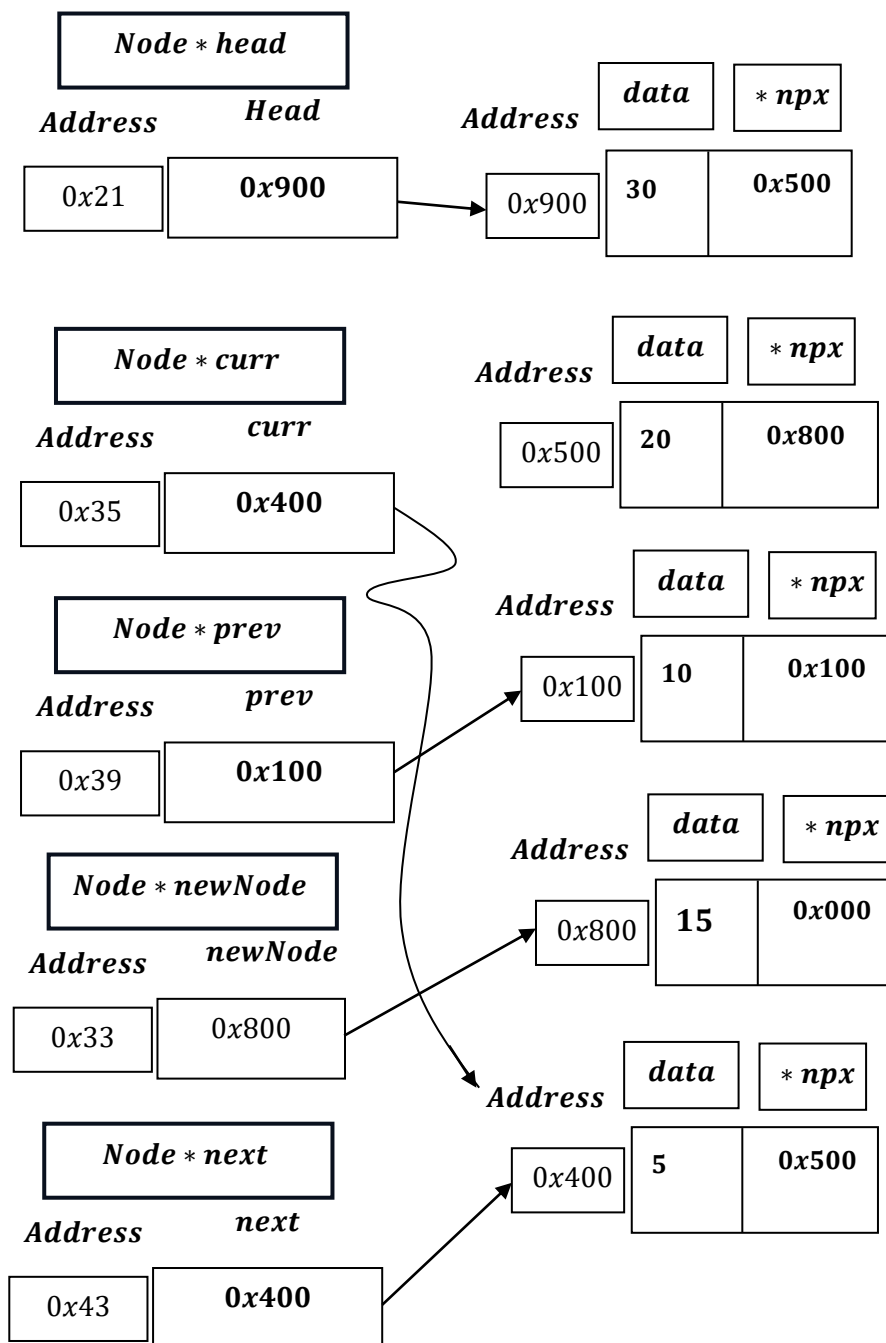
next $\neq$ *nullptr* is *TRUE* :

*Node * nextNext* = *XOR*(*curr*: 0x400, *next* $\rightarrow$ *npx*: 0x900)

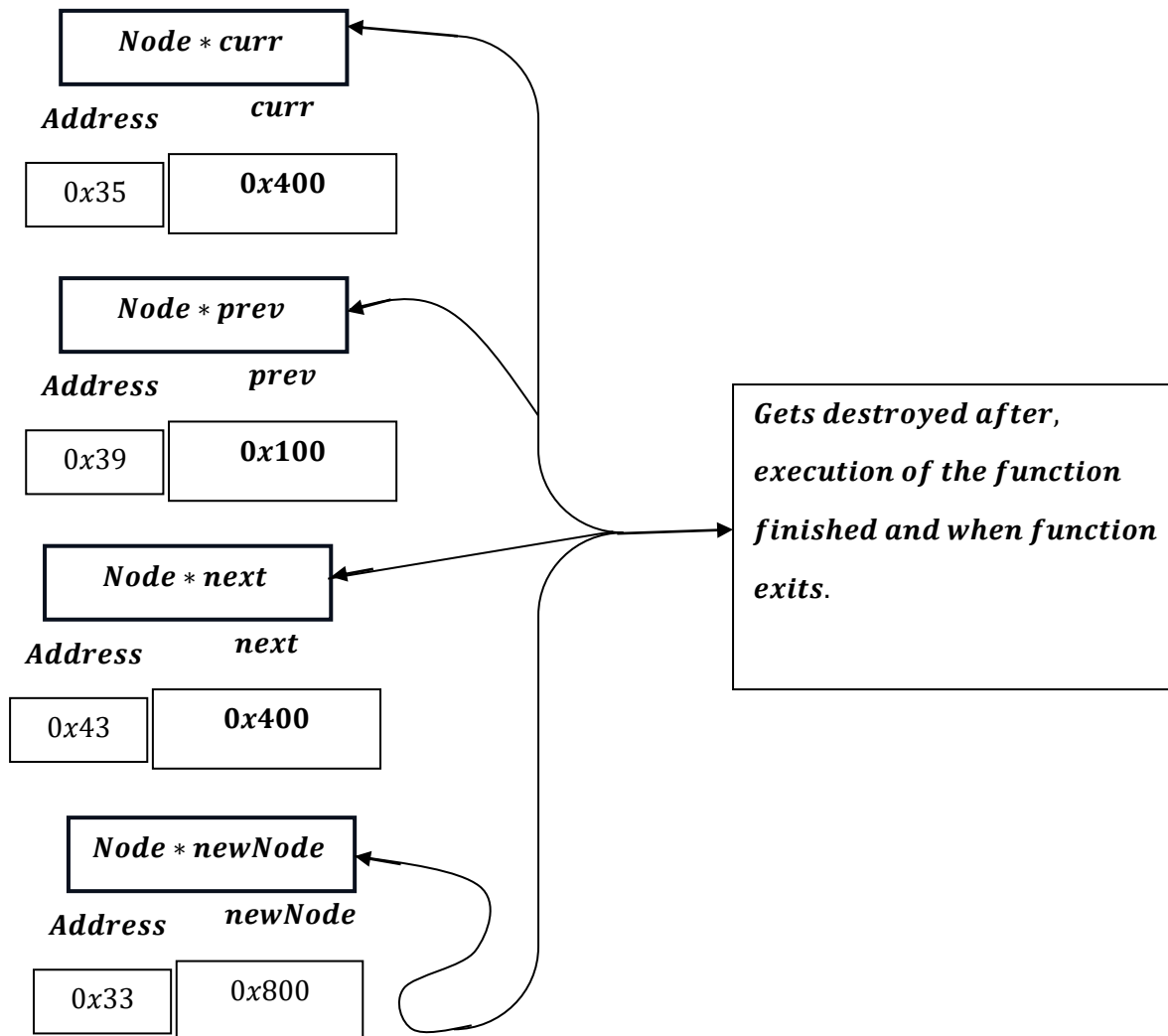
$$= 0100_2 \oplus 1001_2 = 1101_2 = 13_{10} = 0xD00$$

next $\rightarrow$ *npx* = *XOR*(*newNode*: 0x800, *nextNext*: 0xD00)

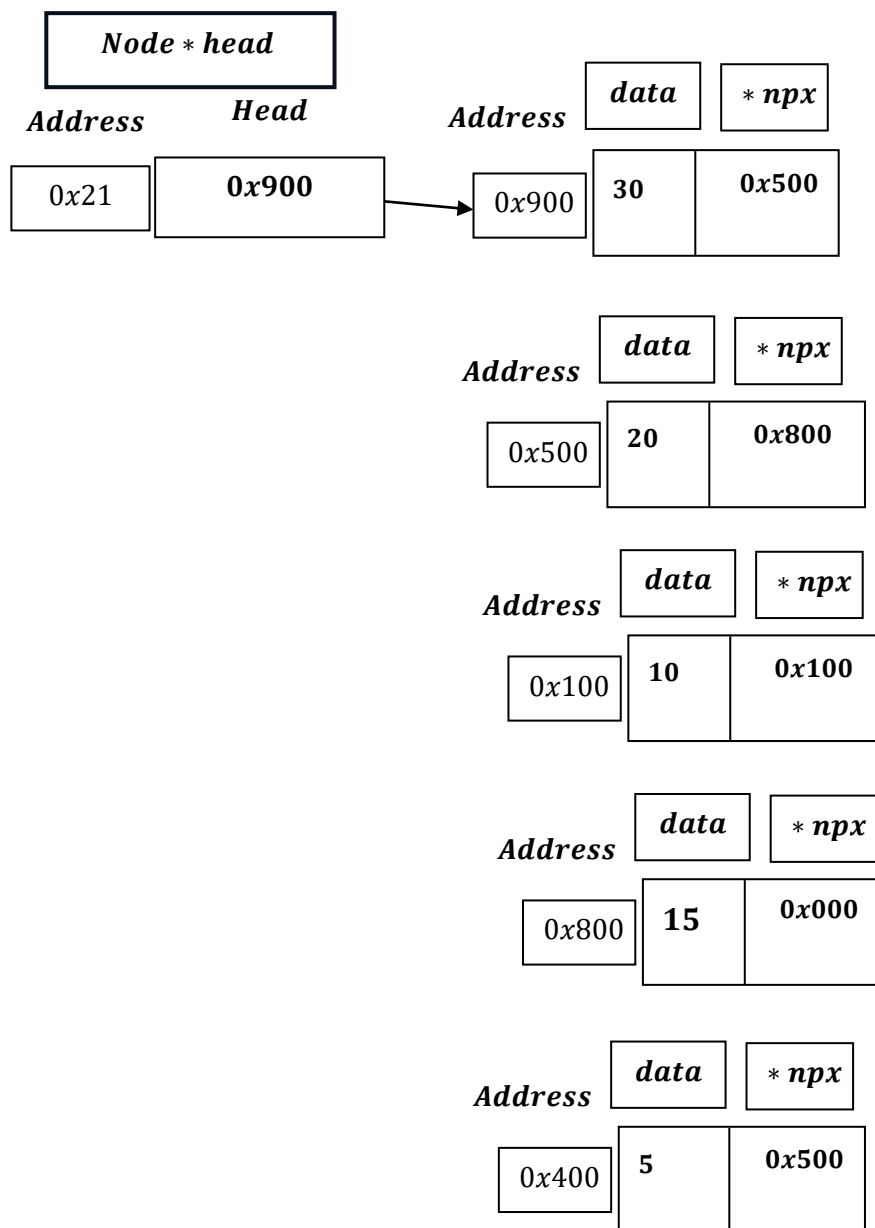
$$= 1000_2 \oplus 1101_2 = 0101_2 = 5_{10} = 0x500$$



As , functions ends:



And we are left with:



Memory Efficient Doubly Linked List –insertAfterElement and Time Complexity – Part 3

```
while (curr != nullptr && curr->data != element)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
}
```

When Curr = NULLPTR, when Element is not found in the list.

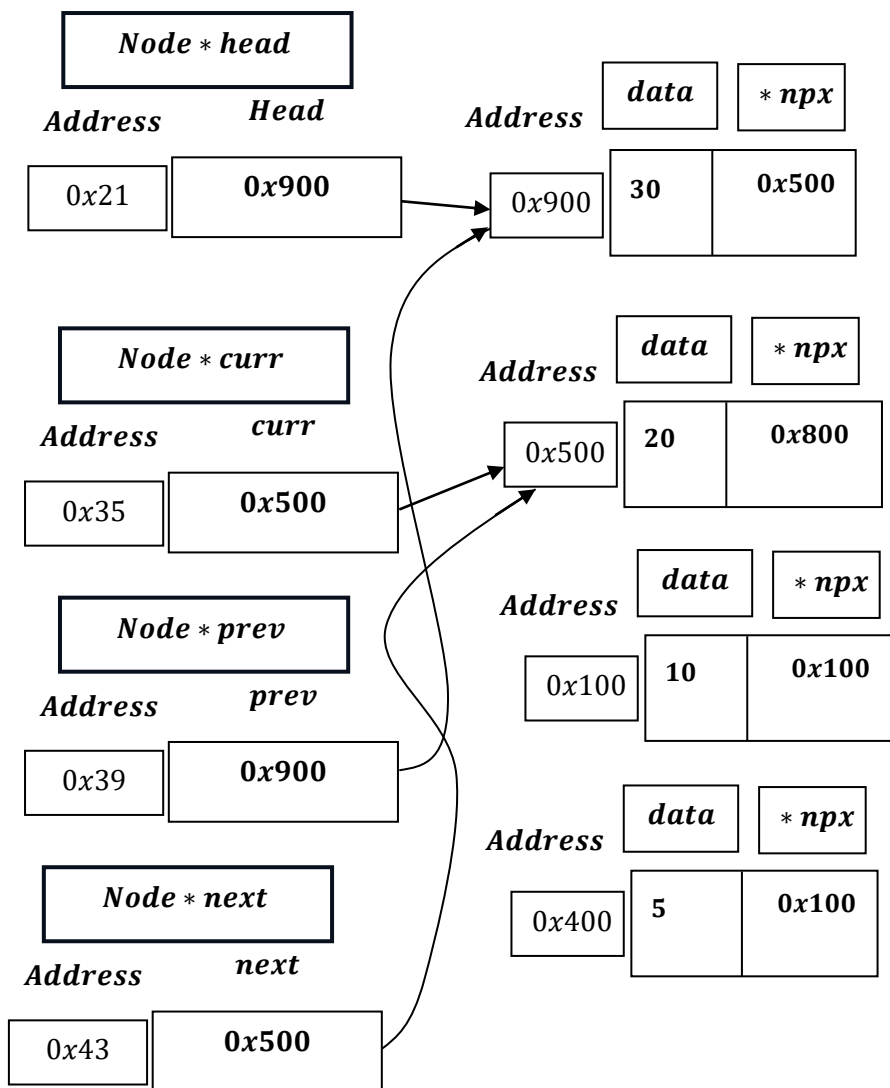
Lets say, Element = 14 and data = 15;

curr: 0x900 ≠ nullptr: 0x000 AND curr → data = 30 ≠ element = 14 :

next = XOR(prev: 0x000, curr → npx: 0x500) = 0x000 ⊕ 0x500 = 0x500.

prev = curr = 0x900

curr = next = 0x500

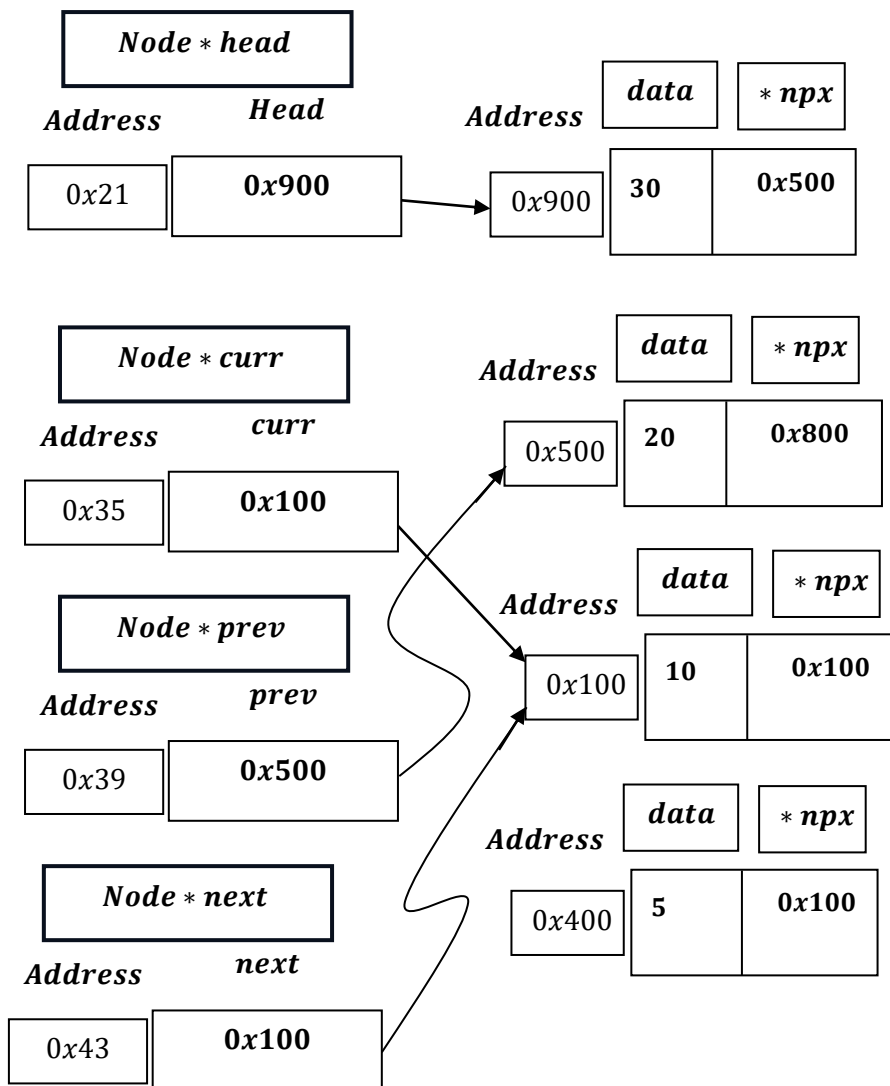


curr: 0x500 ≠ nullptr: 0x000 AND curr → data = 20 ≠ element = 14 :

next = XOR(prev: 0x900, curr → npx: 0x800) = 1001₂ ⊕ 1000₂ = 0001₂ = 0x100

prev = curr = 0x500

curr = next = 0x100

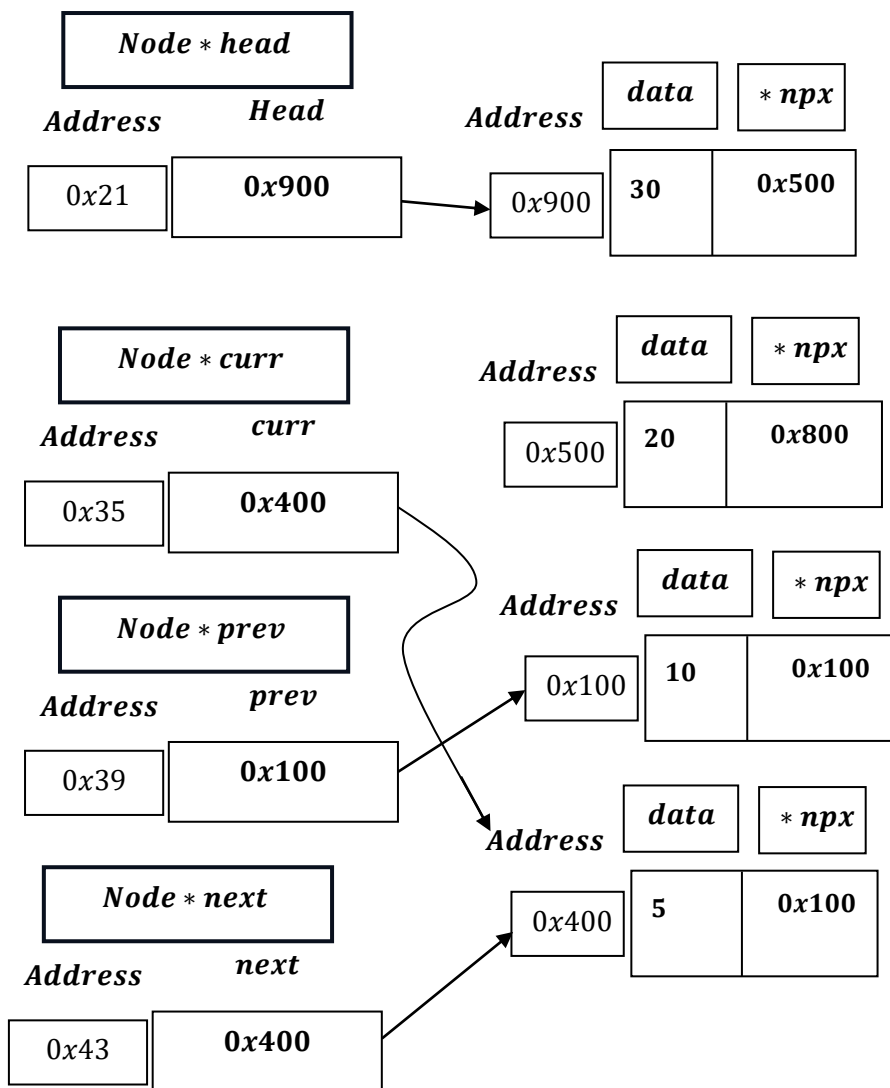


$curr: 0x100 \neq nullptr: 0x000$ AND $curr \rightarrow data = 10 \neq element = 14$:

$next = XOR(prev: 0x500, curr \rightarrow npx: 0x100) = 0101_2 \oplus 0001_2 = 0100_2 = 0x400$

$prev = curr = 0x100$

$curr = next = 0x400$

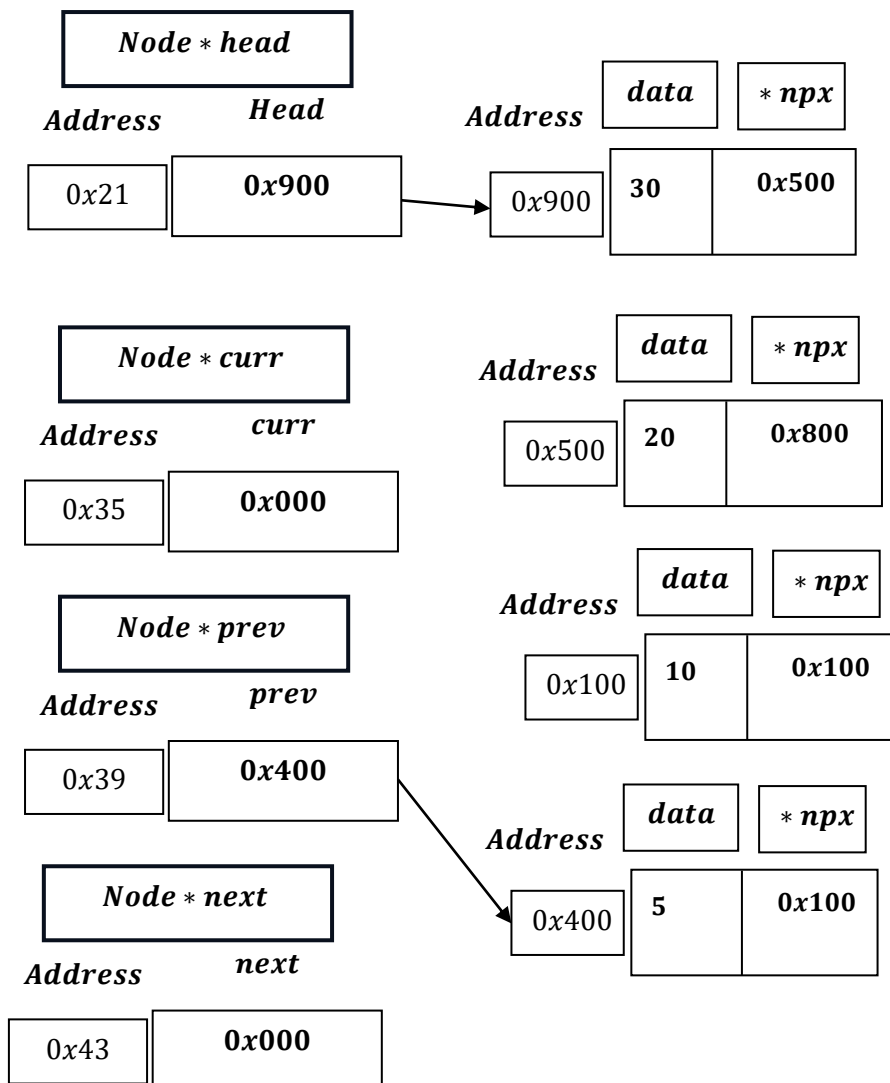


$curr: 0x400 \neq nullptr: 0x000[True]$ AND $curr \rightarrow data = 5 \neq element = 14$:

$next = XOR(prev: 0x100, curr \rightarrow npx: 0x100) = 0001_2 \oplus 0001_2 = 0000_2 = 0x000$

$prev = curr = 0x400$

$curr = next = 0x000$



curr: 0x000 $\neq$ nullptr: 0x000[False] hence ,condition is false and loop exits.

```
if (curr == nullptr)
{
    cout << "Element " << element << " not found in the
list." << endl;
    return;
}
```

As, $curr = nullptr = 0x000$:

Prints: Element 14 not found in the list.

Returns statement return void and exits from the function.

Time Complexity

```
void insertAfterElement(int data, int element)
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    // Search for element
    while (curr != nullptr && curr->data != element)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
    }

    if (curr == nullptr)
    {
        cout << "Element " << element << " not found in the list." << endl;
        return;
    }

    next = XOR(prev, curr->npx);

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
}
```

$O(1)$

$O(1)$

$O(1)$

condition checks `n + 1` times at Worst Case.

Each run `n` times = 3n

$O(1)$

$O(1)$

$O(1)$

```

// Update pointers using XOR logic
newNode->npx = XOR(curr, next);
curr->npx = XOR(prev, newNode);

if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}

cout << data << " inserted after " << element << "." << endl;
}

```

Complexity analysis for the code above:

- `newNode->npx = XOR(curr, next);` → $O(1)$
- `curr->npx = XOR(prev, newNode);` → $O(1)$
- The `if` block contains two XOR operations, each taking $O(1)$ time.
- `cout << data << " inserted after " << element << "." << endl;` → $O(1)$

Best Case Analysis

The target `element` is present in the **first node (head)**. The `while` loop condition fails immediately.

Hence complexity : $\Omega(1)$.

Worst Case Analysis

*Just if we focus on the loop ,Worst case occurs when
`Element not Found situation` .*

Loop condition checks $n + 1$ times including when it becomes false, the innerstatement runs:

$next = XOR(prev, curr \rightarrow npx); \Rightarrow 1 + 1 + \dots n \text{ times} = n.$

$prev = curr; \Rightarrow 1 + 1 + \dots n \text{ times} = n.$

$curr = next; \Rightarrow 1 + 1 + \dots n \text{ times} = n.$

Hence ,time complexity at worst complexity: $n + n + n = 3n = 3 \times O(n) = O(n)$.

Average Case

Inserted position from 1 to $N + 1$, while loop runs from 1 to $N - 1$.

And when the position is uniformly distributed from 1 to $(N + 1)$:

(Average Case i. e. Undetermined Position)

→ If the Element is inserted after 1st node, the loop's inner statement runs 0 time.

→ If the Element is inserted after 2nd node, the loop's inner statement runs 1 time.

→ If the Element is inserted after 3rd node, the loop's inner statement runs 2 time.

→ If the Element is inserted after 4th node, the loop's inner statement runs 3 time.

.

.

→ If the Element is inserted at $(n + 1)^{th}$ node, the loop's inner statement runs $N - 1$ time.

$$\therefore, \sum_{i=0}^{N-1} i = \frac{N(N-1)}{2}$$

Now, If there is 'N' no. of nodes there we can insert data at 2 to $N + 1$ positions or repositioning 1 to N .

For position 1, the probability of insertion is $\frac{1}{N}$.

For position 2, the probability of insertion is $\frac{1}{N}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N}$.

Hence probability of insertion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations)

$$= \frac{1}{N} \times \left(\sum_{i=0}^{N-1} i \right)$$

$$= \frac{1}{N} \times \frac{N(N-1)}{2}$$

$$= \frac{N-1}{2}$$

Applying , Big O on the above equation:

$$= \frac{1}{2} \times O(N) - O\left(\frac{1}{2}\right)$$

$$= O(N)$$

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Approach]:

(Simplest approach)

**(If the elements in the linked list are uniformly distributed
and the key is equally likely to be any element in the list)**

**Note: –The checking that Element or Key is found or not , done in the
condtion of while loop, and the condition checks `n` times .**

–if key is found at 1st Node , Key/Element searched for 1 time .

–if key is found at 2nd Node , Key/Element searched for 2 time .

–if key is found at 3rd Node, Key/Element searched for 3 time .

.

.

–if key is found at Nth Node , Key/Element searched for N time .

As , the key is equally likely to be any element in the list,

Probablity of finding 1st element (p_1), if key equals to 1st element = $\frac{1}{N}$.

Probablity of finding 2nd element (p_2), if key equals to 2nd element = $\frac{1}{N}$.

.

.

Probability of finding N^{th} element (p_2), if key equals to N^{th} element $= \frac{1}{N}$.

Therefore,

$$\text{Probability}(P) = p_1 = p_2 = \dots = p_n = \frac{1}{N}.$$

Therefore, Average Case = Number of times the Key is searched $\times$ probability of finding elements.

That is the average number of executions is the sum of all possible executions weighted by their probabilities. This is given by:

$$\begin{aligned} \therefore \text{Average Case} &= \frac{1}{N} \times \left(\sum_{i=1}^N i \right) \\ &= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^N i = \frac{N(N+1)}{2} \right] \end{aligned}$$

$$= \frac{N+1}{2}$$

$$= \frac{N}{2} + \frac{1}{2}$$

Applying , Big Theta Θ we get :

$$= \frac{1}{2} \times (\Theta(N) + \Theta(1))$$

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Insert After Element</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

Memory Efficient Doubly Linked List
–insertAfterElement and Time Complexity

Program:

```
void insertAfterElement(int data, int element)
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    // Search for element
    while (curr != nullptr && curr->data != element)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
    }

    if (curr == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    // Element found at 'curr'. Insert 'newNode' between
'curr' and 'next'.
    next = XOR(prev, curr->npx); // Decode the node after
curr

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    // Update pointers using XOR logic
    newNode->npx = XOR(curr, next);
    curr->npx = XOR(prev, newNode);
```

```

    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(newNode, nextNext);
    }

    cout << data << " inserted after " << element << "." <<
endl;
}
... .

    case 4:
        cout << "Enter data to insert: ";
        cin >> data;
        cout << "Enter the element to insert after: ";
        cin >> element;
        insertAfterElement(data, element);
        break;

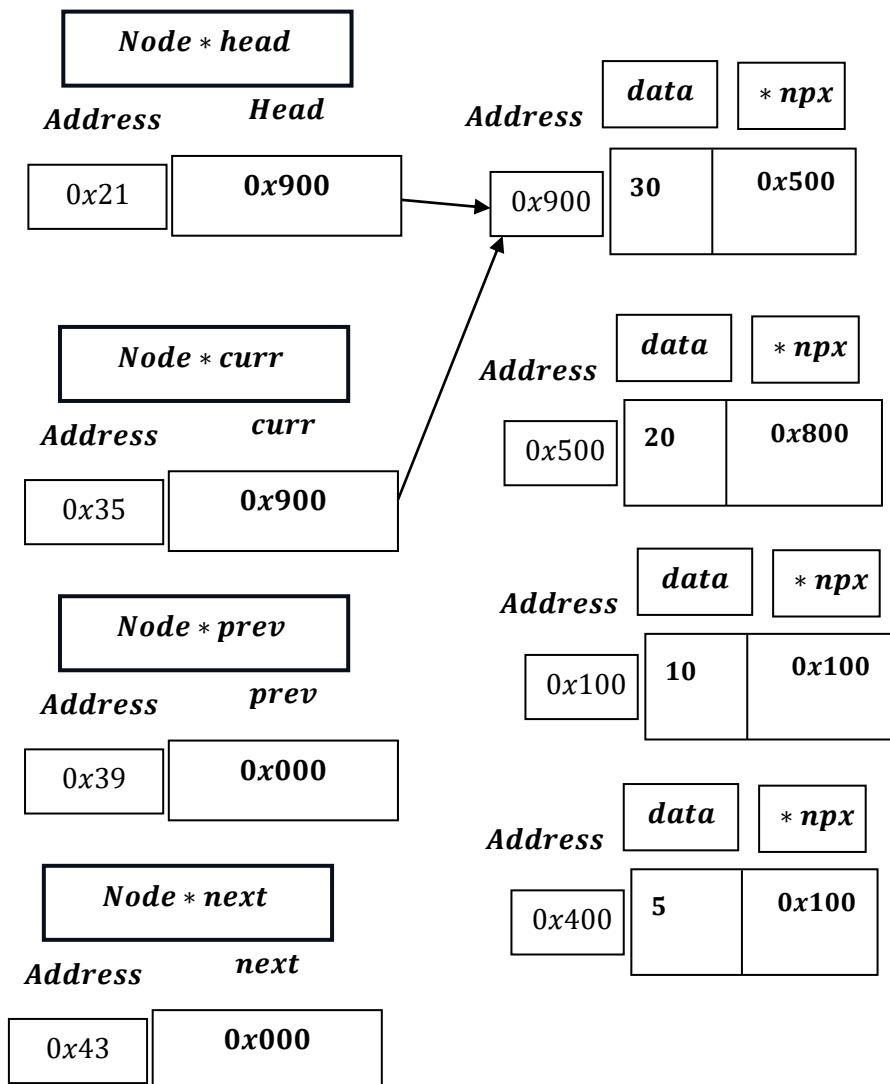
```

Program Analysis:

```

Node *curr = head;
Node *prev = nullptr;
Node *next;

```



```
while (curr != nullptr && curr->data != element)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
}
```

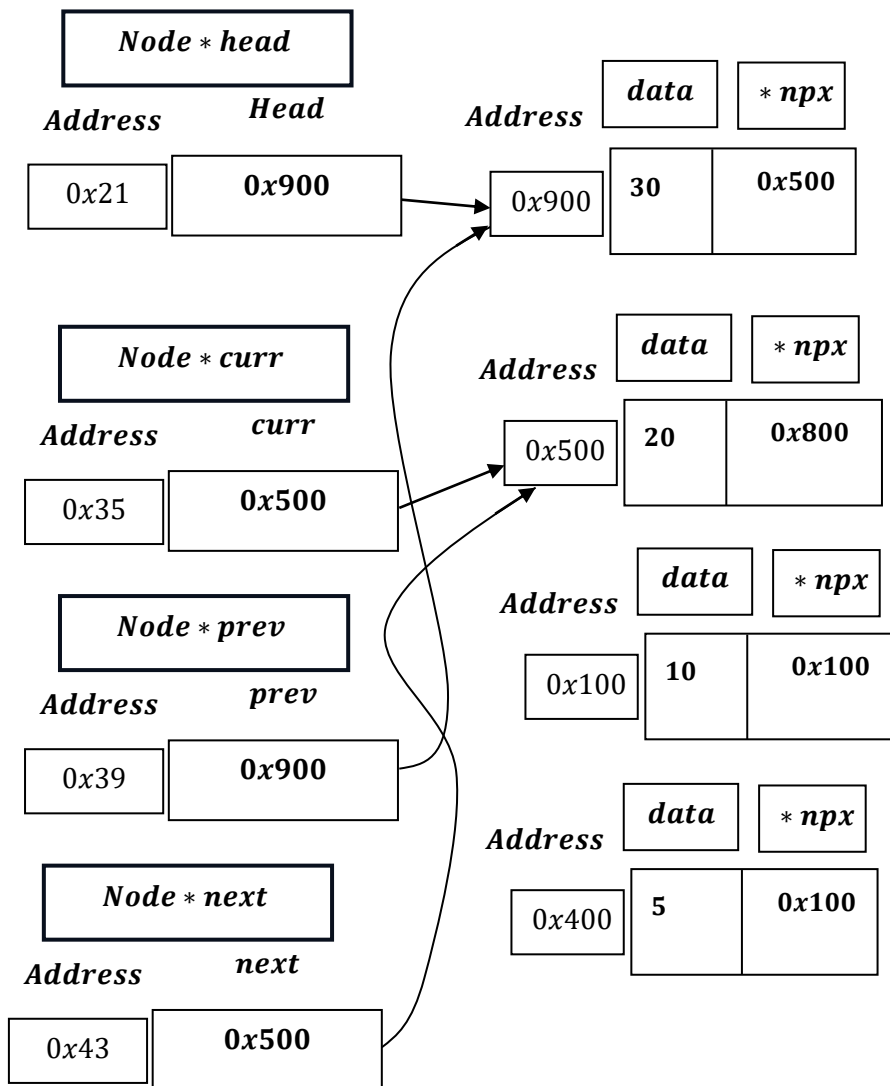
Lets say ,Element = 5 and data = 15;

curr: 0x900 $\neq$ nullptr: 0x000 AND curr $\rightarrow$ data = 30 $\neq$ element = 5 :

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x000 $\oplus$ 0x500 = 0x500.

prev = curr = 0x900

curr = next = 0x500

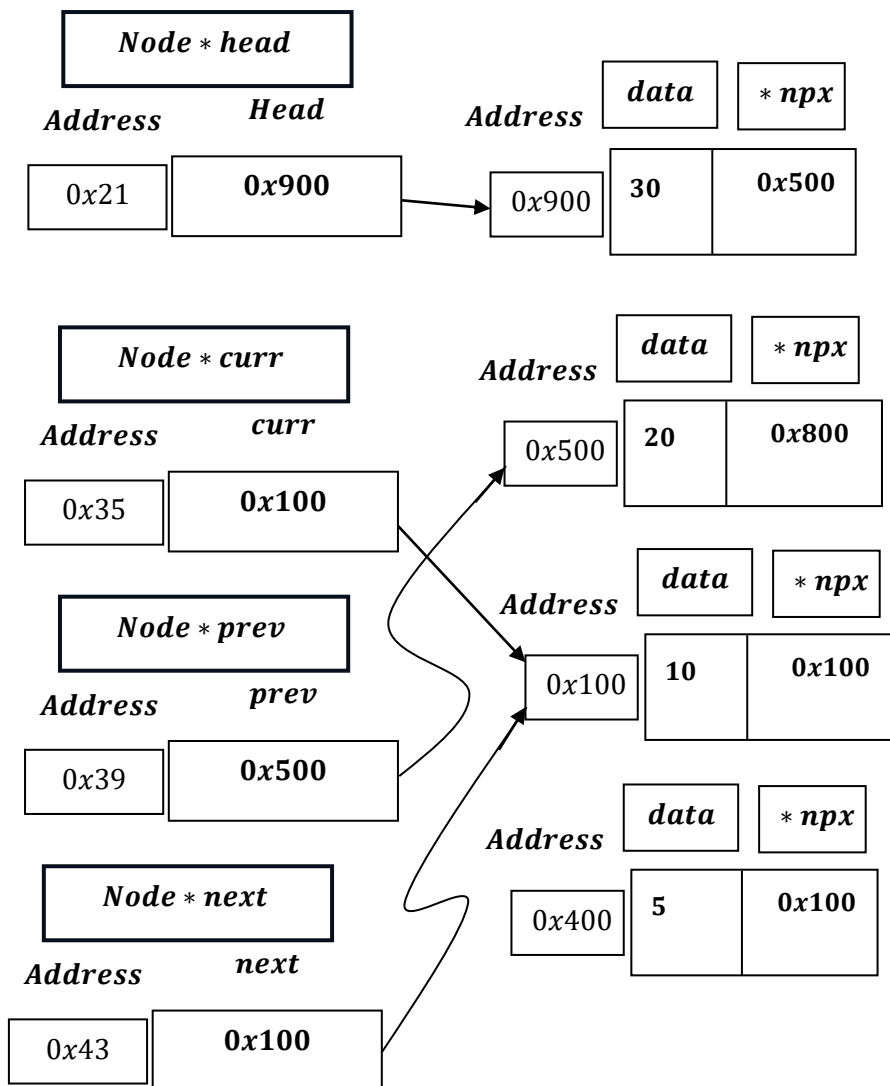


curr: 0x500 $\neq$ nullptr: 0x000 AND curr $\rightarrow$ data = 20 $\neq$ element = 5 :

next = XOR(prev: 0x900, curr $\rightarrow$ npx: 0x800) = 1001₂ $\oplus$ 1000₂ = 0001₂ = 0x100

prev = curr = 0x500

curr = next = 0x100

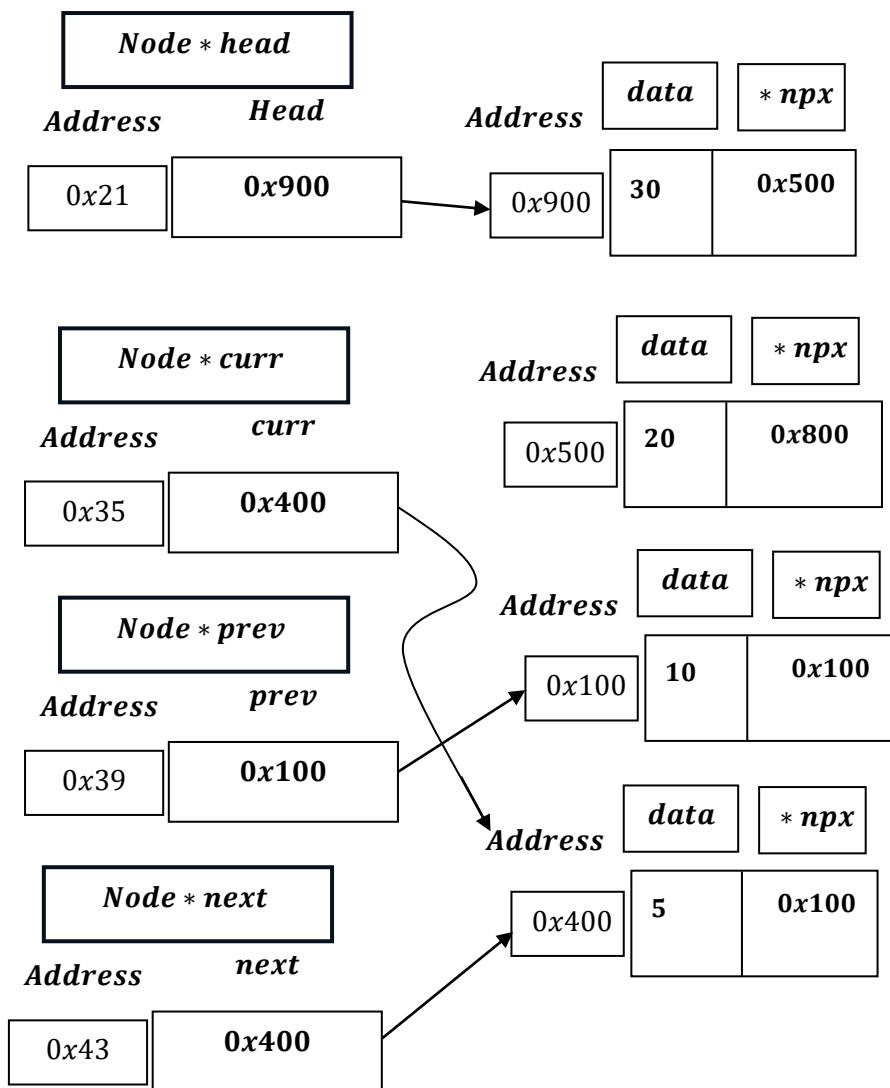


$curr: 0x100 \neq nullptr: 0x000$ AND $curr \rightarrow data = 10 \neq element = 5$:

$next = XOR(prev: 0x500, curr \rightarrow npx: 0x100) = 0101_2 \oplus 0001_2 = 0100_2 = 0x400$

$prev = curr = 0x100$

$curr = next = 0x400$

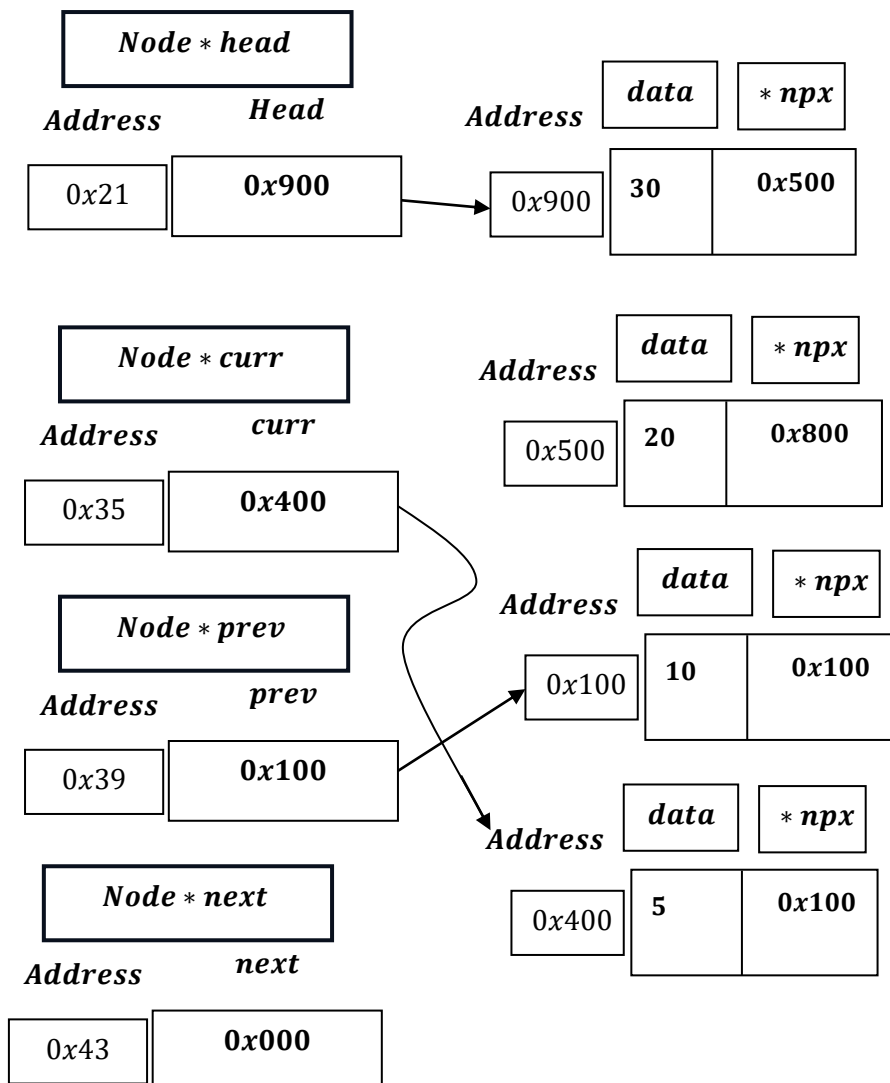


*curr: 0x400 $\neq$ nullptr: 0x000 [True] AND curr $\rightarrow$ data = 5 $\neq$ element = 5 [False]
 $\rightarrow$ is False , hence loop exits.*

```
next = XOR(prev, curr->npx);
```

next = XOR(prev: 0x100 $\oplus$ curr $\rightarrow$ npx: 0x100);

0001₂ $\oplus$ 0001₂ = 0000₂ = 0x000.



```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of *newNode* will be equal to the size of a *Node* structure and *newNode* is declared as a pointer to *Node* structure.

→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *npx(a pointer to another Node) is typically 8 bytes.*

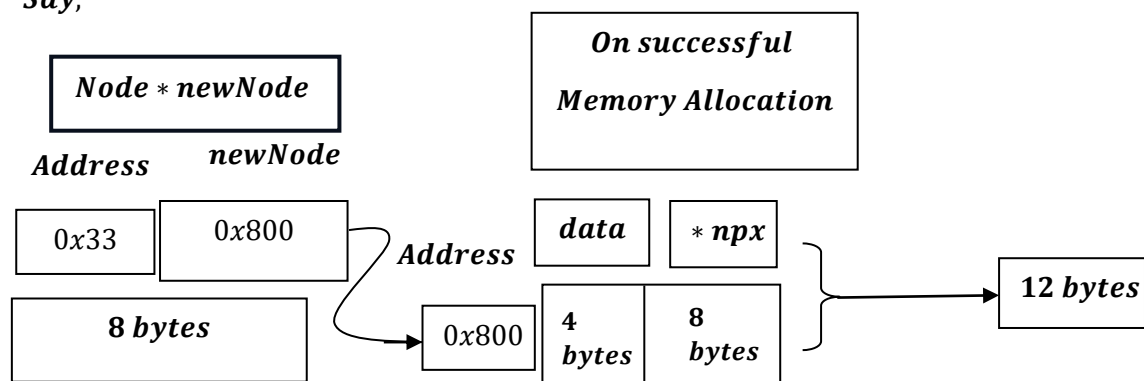
→ *Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npv) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

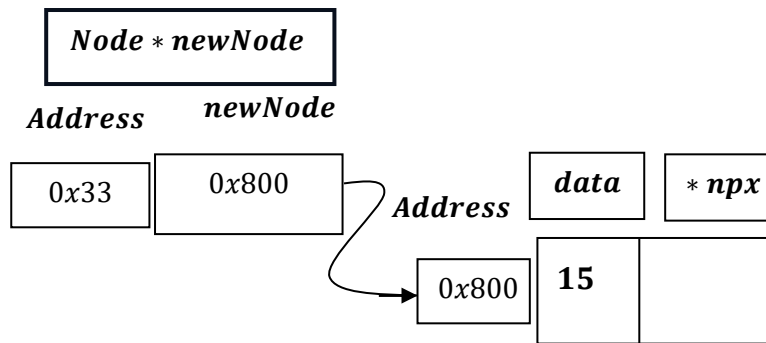
→ *newNode will try to allocate memory for data i.e. 4 bytes ,npv pointer variable i.e. typically 8 bytes,i.e. total 12 bytes,as discussed earlier ,the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



```
newNode->data = data;
```

Say, data = 15. Then ,newNode → data = data = 15.

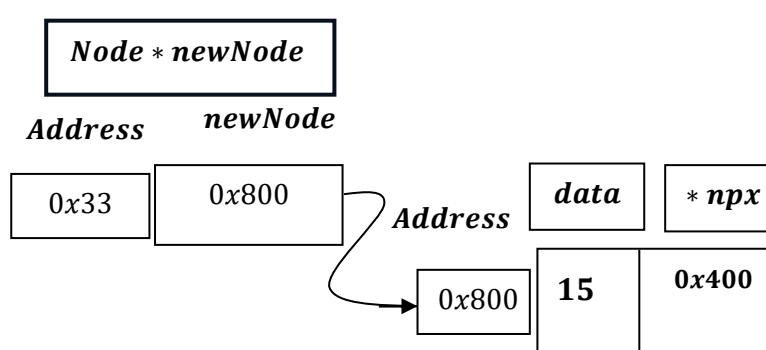


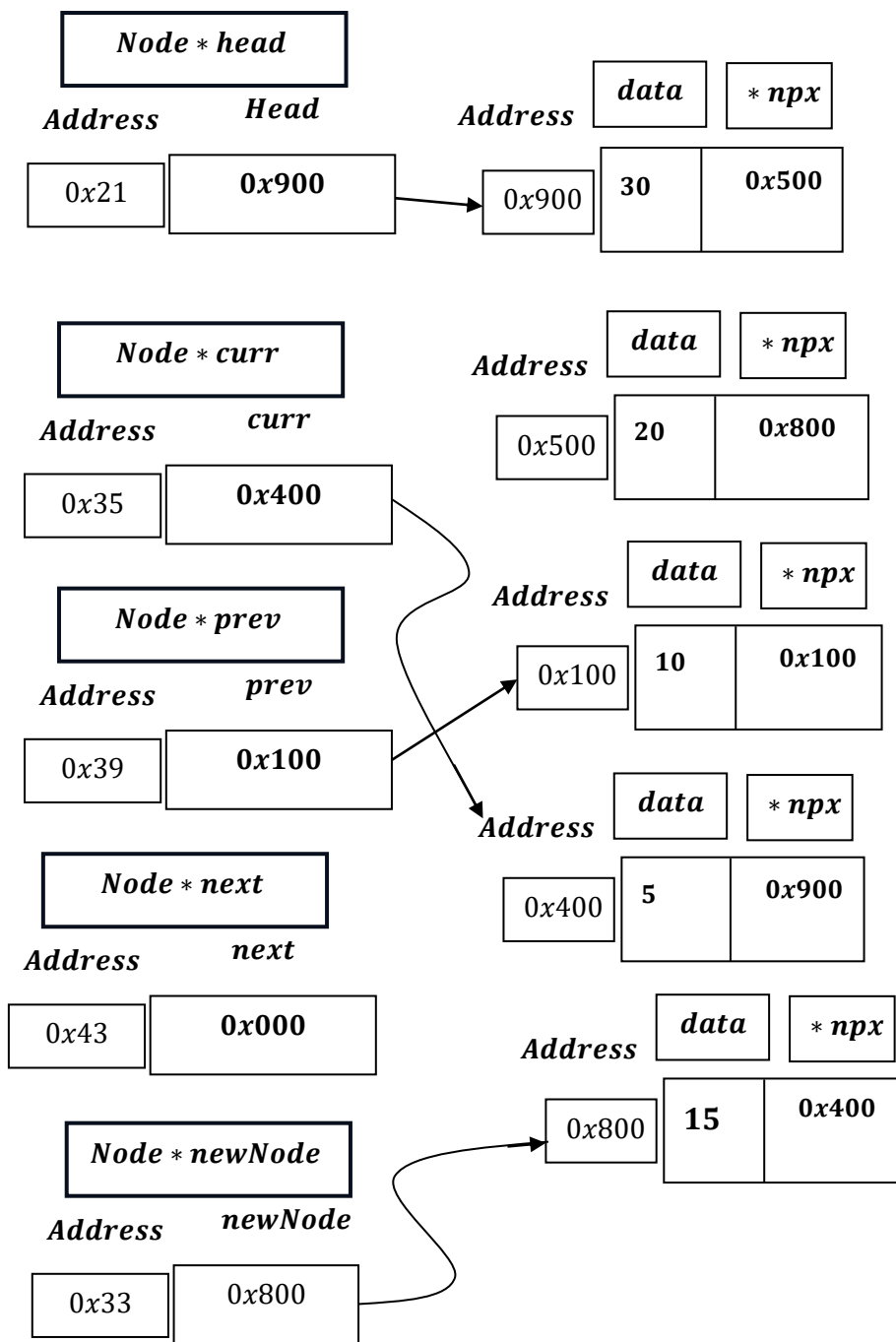
```
newNode->npx = XOR(curr, next);
curr->npx = XOR(prev, newNode);
```

$newNode \rightarrow npx = XOR(curr: 0x400, next: 0x000) = 0x400.$

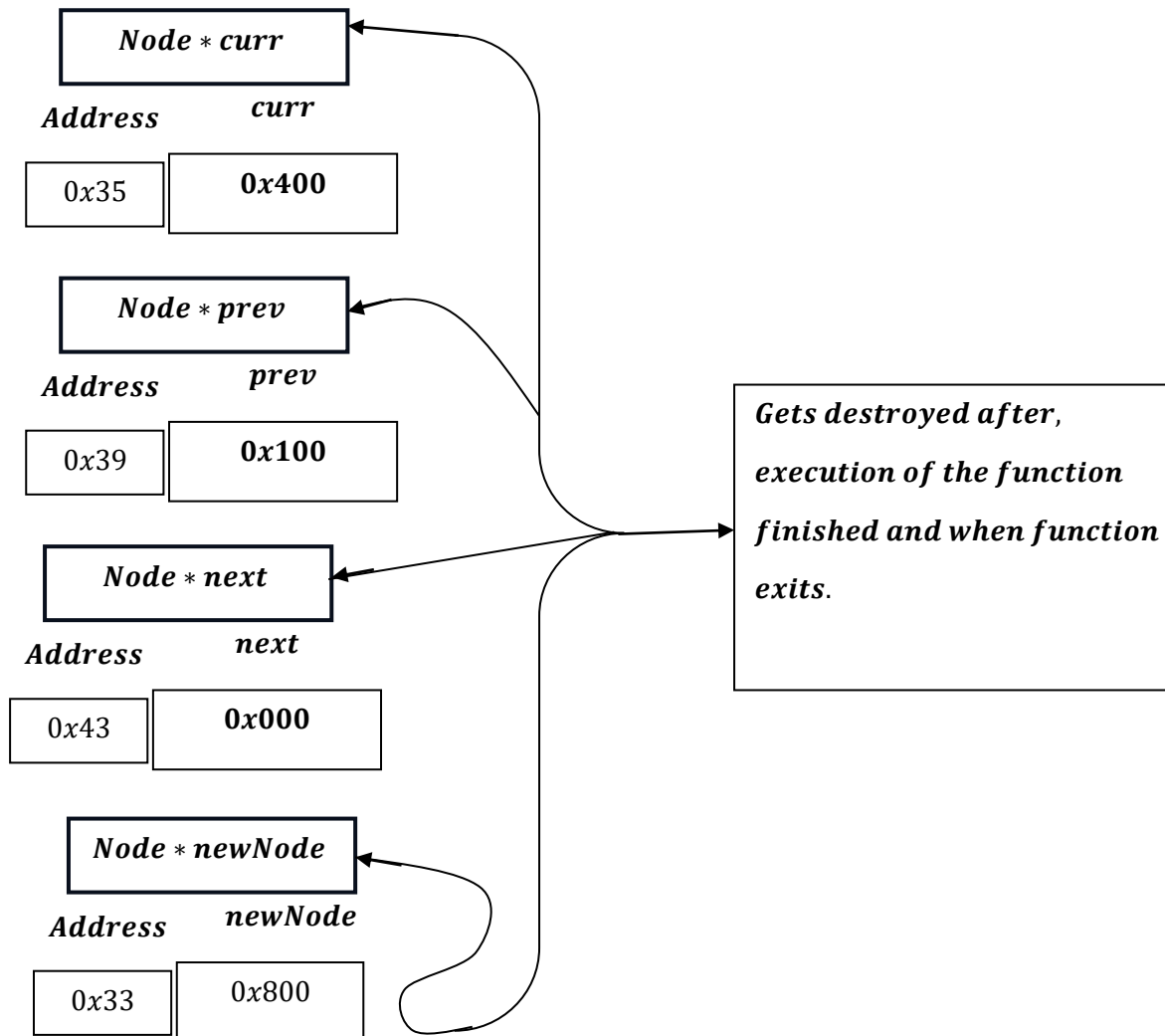
$curr \rightarrow npx = XOR(prev, newNode) = (prev: 0x100, newNode: 0x800) =$

$0001_2 \oplus 1000_2 = 1001_2 = 0x900.$

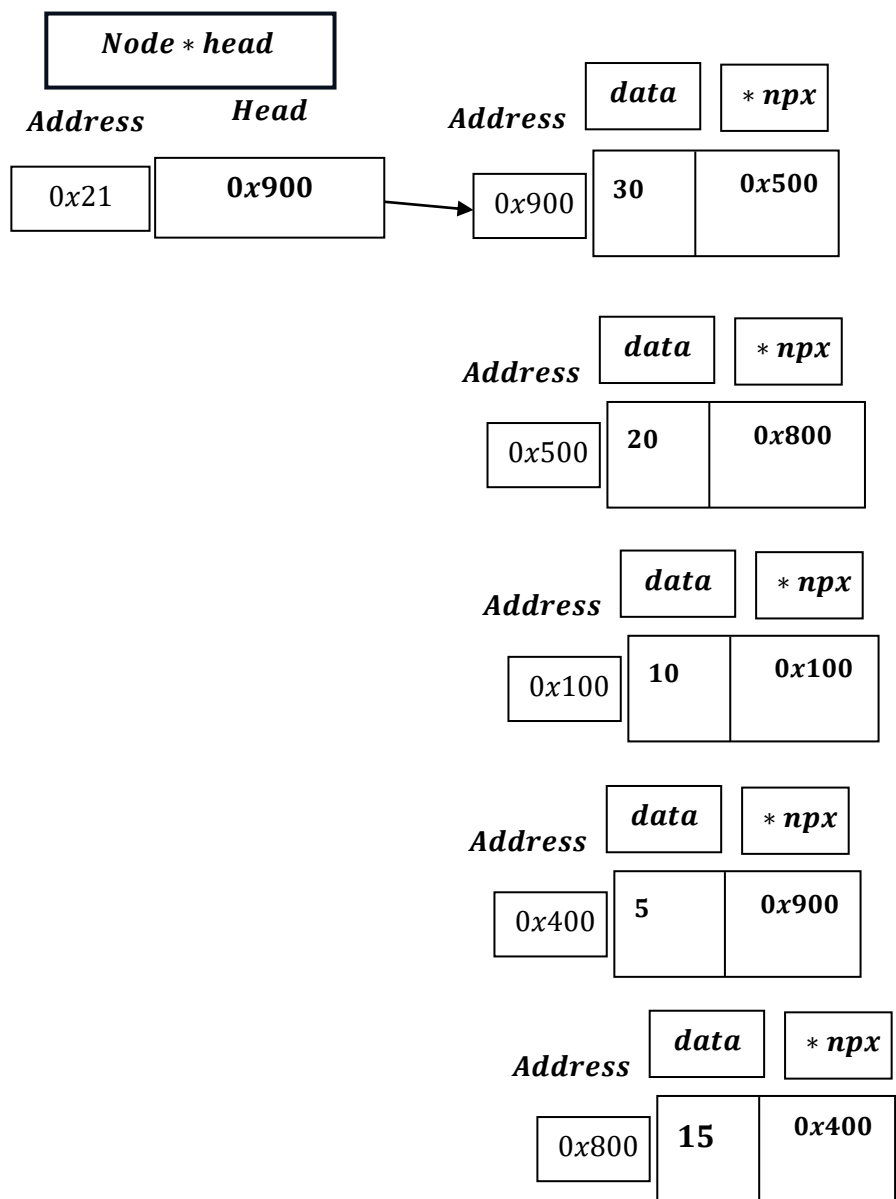




As , functions ends:



And we are left with:



Memory Efficient Doubly Linked List – Insert After Position –Space Complexity and Algorithm

Program :

```
void insertAfterElement(int data, int element)
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    // Search for element
    while (curr != nullptr && curr->data != element)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
    }

    if (curr == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    // Element found at 'curr'. Insert 'newNode' between
'curr' and 'next'.
    next = XOR(prev, curr->npx); // Decode the node after
curr

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    // Update pointers using XOR logic
    newNode->npx = XOR(curr, next);
    curr->npx = XOR(prev, newNode);
```

```

    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(newNode, nextNext);
    }

    cout << data << " inserted after " << element << "." <<
endl;
}
... .

    case 4:
        cout << "Enter data to insert: ";
        cin >> data;
        cout << "Enter the element to insert after: ";
        cin >> element;
        insertAfterElement(data, element);
        break;

```

Space Complexity Analysis

XOR Linked List (Insert After Element)

1. Input Space Complexity

- ***Parameter – only Definition:***
 - ***Function Parameters:*** *int data* (the value to insert) and *int element* (the target value to search for).
 - ***Analysis:*** Both are primitive integers. They take up a constant amount of space regardless of the list size.
 - ***Complexity:*** **$O(1)$**

- **Full Data Definition:**
 - **Function Data:** The parameters + the existing XOR linked list structure.
 - **Analysis:** The function must search through the existing list of n nodes. The input context for this operation is the entire data structure.
 - **Complexity:** $O(n)$
-

2. Auxiliary Space Complexity

Definition: This measures the extra or temporary memory the function uses to perform its task.

- **Node Allocation:** `Node * newNode = (Node *)malloc(sizeof(Node));`. This is the most significant part. We allocate memory for exactly **one** node.
- **Local Variables:** The function uses three pointers (`curr`, `prev`, `next`) and one temporary pointer `nextNext`.
 - **Analysis:** These are stack – allocated variables. Their space requirement is fixed (usually 8 bytes per pointer on a 64 – bit system) and does not increase even if the list has a billion nodes.
- **XOR Operations:** The bitwise math is performed in – place using CPU registers.
- **Traversal:** Since the search is done via an iterative while loop, there is no recursion depth to worry about.

Conclusion: The amount of extra memory used is fixed and does not grow as the size of the linked list (n) grows.

- **Auxiliary Space Complexity:** $O(1)$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-INFO input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Insert After Element (XOR Doubly Linked List)

XORINSAFTER(START, ITEM, ELEMENT):

$\left(\left(\begin{array}{l} \text{(Node *)} \text{malloc}(\text{sizeof}(\text{Node})) \text{represents} \\ \text{the availability of space for the data; let's name it AVAIL.} \end{array} \right) \right)$

1. [INITIALIZE SEARCH]

Set CURRPTR := START

Set PREVPTR := NULL

2. [SEARCH FOR ELEMENT ELEMENT]

Repeat while CURRPTRPTR $\neq$ NULL AND INFO[CURRPTR] $\neq$ ELEMENT:

a. Set NEXTTEMPPTR := PREVPTRPTR $\oplus$ NEXTXORPTR[CURRPTR]

b. Set PREVPTR := CURRPTR

c. Set CURRPTR := NEXTTEMPPTR

[End of Loop]

3. [ELEMENT FOUND?]

If CURRPTR = NULL, then:

Write: "Element ", ELEMENT, " not found in the list. "

Return and EXIT.

[End of If structure]

4. [PREPARE NEW NODE]

a. Set NEXTTEMPPTR :

= PREVPTR $\oplus$ NEXTXORPTR[CURRPTR] $\left(\begin{array}{l} \text{Decode the node} \\ \text{currently following} \\ \text{CURR} \end{array} \right)$

b. Set NEWPTR := AVAIL

c. If NEWPTR = NULL, then: Write: "Error: Memory allocation failed. "

Return and EXIT.

[End of If structure]

d. Set INFO[NEWPTR] := ITEM

5. [LINK NEW NODE]

a. Set NEXTXORPTR[NEWPTR] :

= CURRPTR $\oplus$ NEXTTEMPPTR $\left(\begin{array}{l} \text{The new node's neighbours} \\ \text{are the target node} \\ \text{and the original successor} \end{array} \right)$

b. Set NEXTXORPTR[CURRPTR] :

$$= PREVPTR \oplus NEWPTR \left(\begin{array}{l} \text{Update target node's NPX:} \\ \text{its previous stays the same,} \\ \text{but its NEXT} \\ \text{is now the new node} \end{array} \right)$$

6. [UPDATE SUCCESSOR NODE IF IT EXISTS]

If NEXTTEMPPTR $\neq$ NULL, then:

a. Set NEXTNEXTTEMPPTR :

$$= CURRPTR \oplus NEXTXORPTR[NEXTTEMPPTR] \left(\begin{array}{l} \text{Decode the node} \\ \text{following} \\ \text{NEXT} \end{array} \right)$$

b. Set NEXTXORPTR[NEXTTEMPPTR] :

$$= NEWPTR \oplus NEXTNEXTTEMPPTR \left(\begin{array}{l} \text{Update successor's NPX:} \\ \text{its previous is now the} \\ \text{new node instead of CURR} \end{array} \right)$$

[End of If structure]

7. [PRINT MESSAGE]

Write: ITEM, " inserted after ", ELEMENT

8. EXIT

Memory Efficient Doubly Linked List – Delete At Beginning –Program Analysis and Time Complexity

Program:

```
void deleteFromBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *temp = head;
    // Decode the second node (which will become head)
    // Next = NULL ^ head->npx
    Node *next = XOR(nullptr, head->npx);

    if (next != nullptr)
    {
        // Update the new head's npx.
        // Currently next->npx is: XOR(head, nextNext)
        // We need it to be: XOR(NULL, nextNext)
        Node *nextNext = XOR(temp, next->npx);
        next->npx = XOR(nullptr, nextNext);
    }

    head = next;
    cout << "Deleted " << temp->data << " from the
beginning." << endl;
    free(temp);
}
```


Program – Analysis

1. When List Is Empty

```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

*Node * head*

<i>Address</i>	<i>Head</i>
----------------	-------------

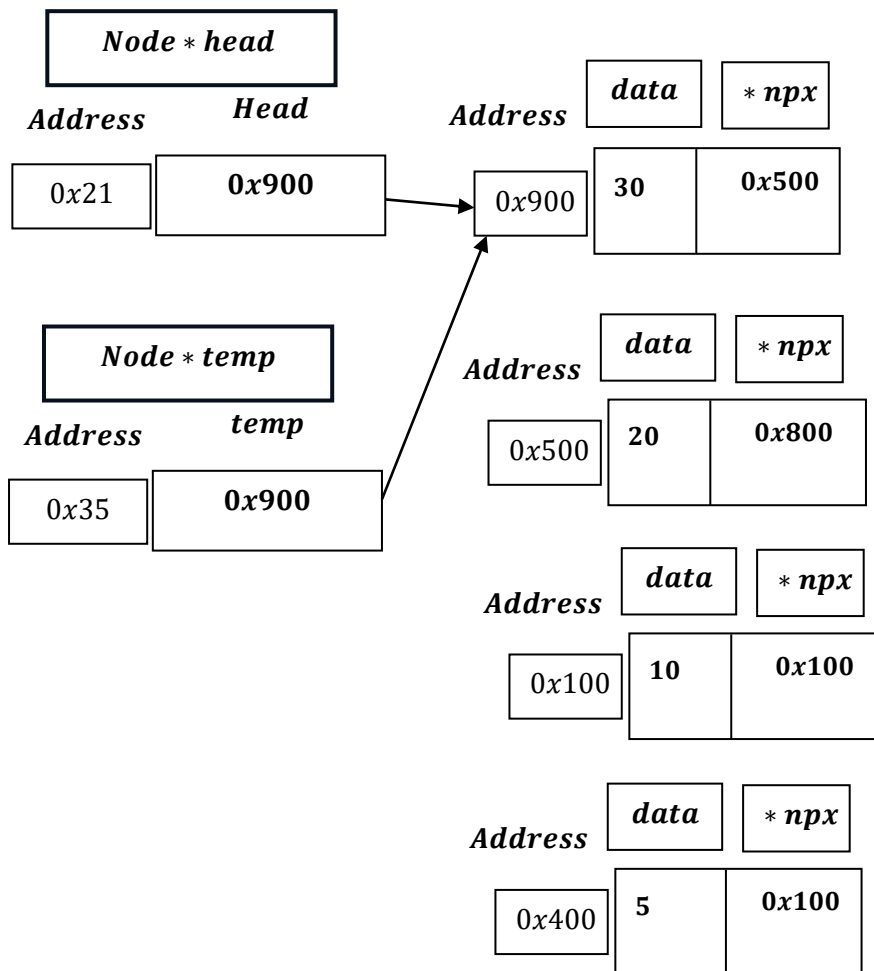
0x21	
------	--

	<i>nullptr</i>
--	----------------

Then , it prints : "List is empty. Nothing to delete."

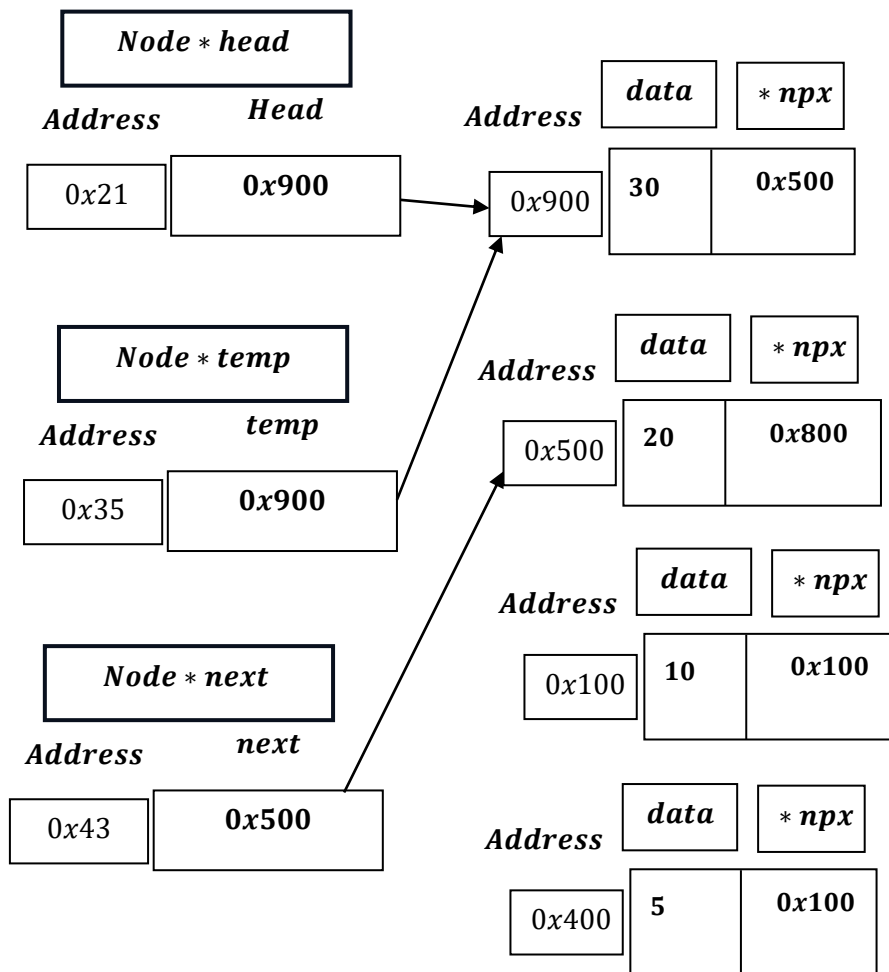
Return Void and Exit.

2. When List Is Not Empty



```
Node *next = XOR(nullptr, head->npx);
```

$$Node * next = XOR(nullptr, head \rightarrow npx) \Rightarrow 0x000 \oplus 0x500 = 0x500$$



Node * next = 0x500 ≠ NULLPTR (0x000).

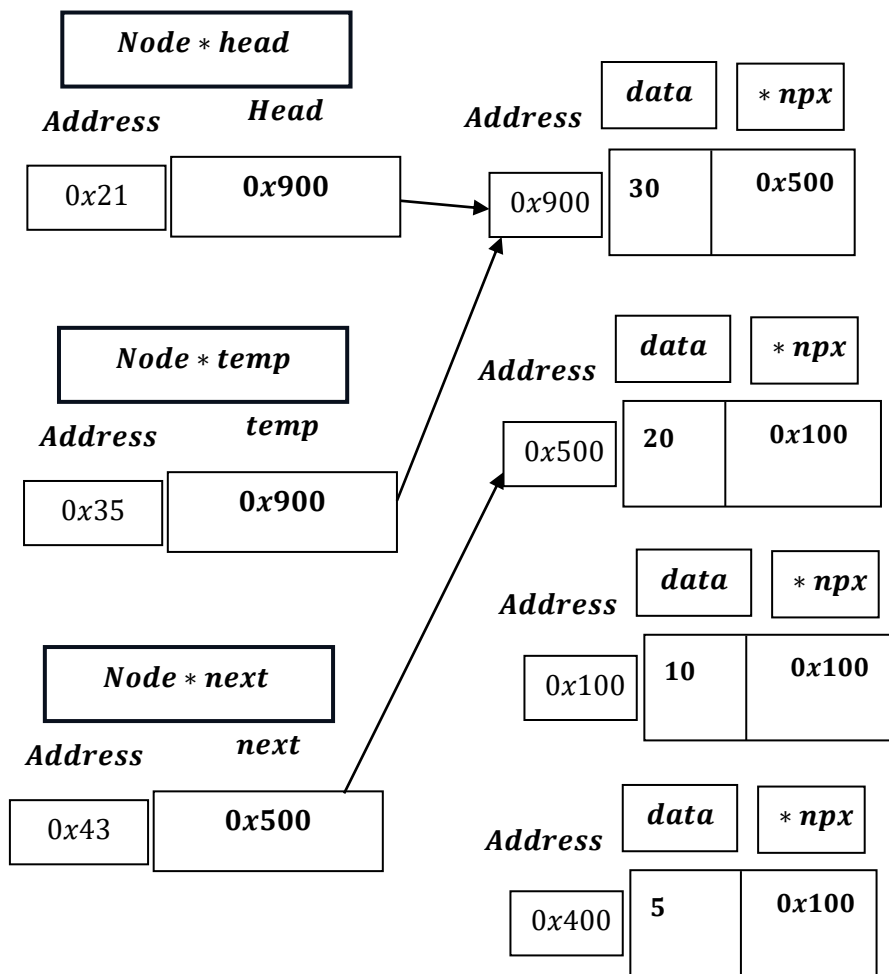
```

if (next != nullptr)
{
    Node *nextNext = XOR(temp, next->npx);
    next->npx = XOR(nullptr, nextNext);
}

```

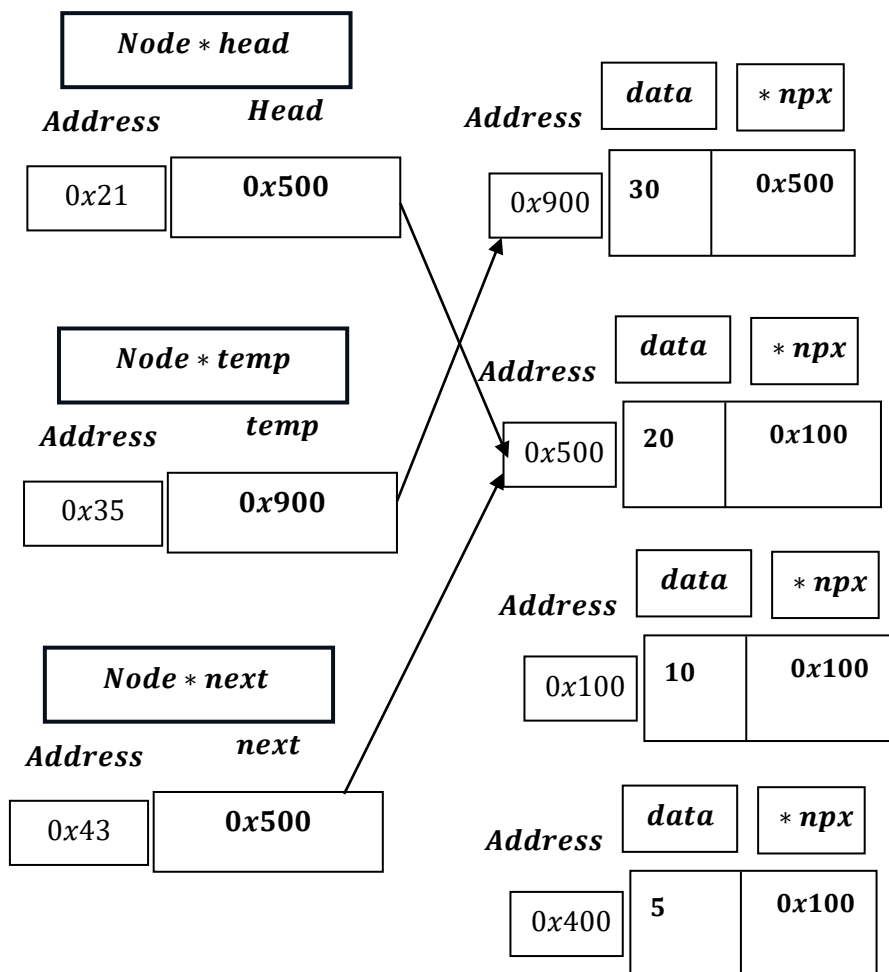
$$Node * nextNext = 0x900 \oplus 0x800 = 1001_2 \oplus 1000_2 = 0001_2 \\ = 0x100$$

$$next \rightarrow npx = XOR (nullptr, nextNext) = 0x000 \oplus 0x100 = 0x100 .$$



```
head = next;
```

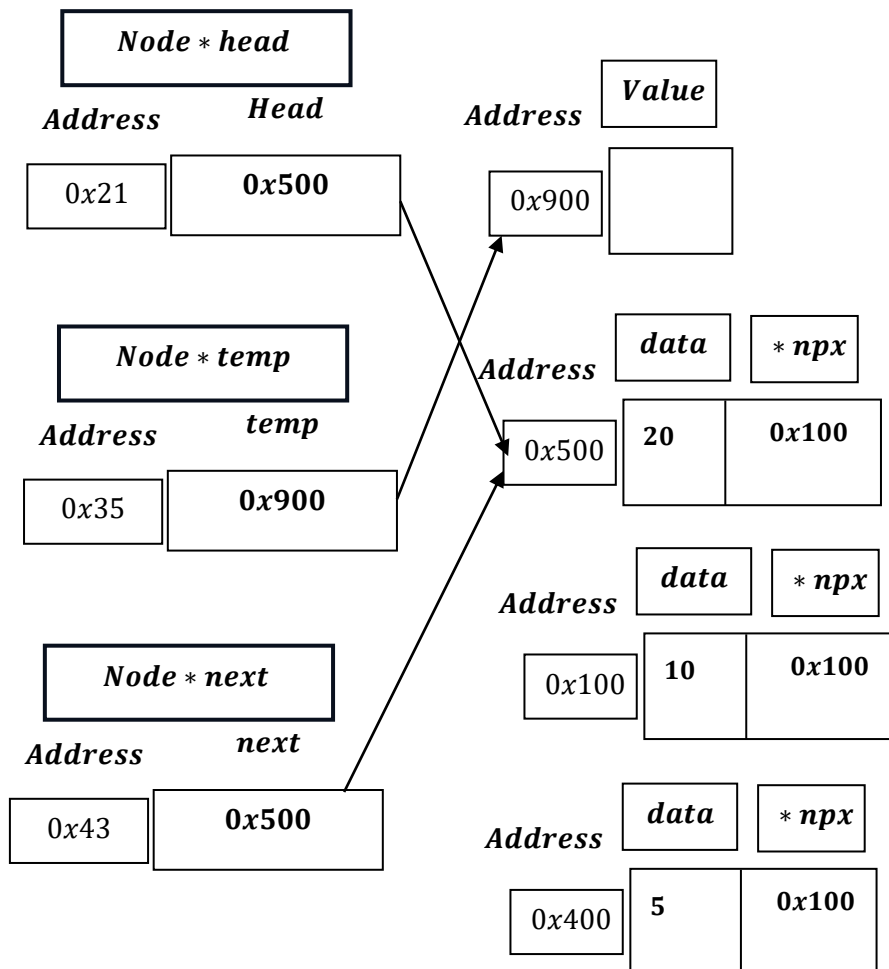
head = next = 0x500.



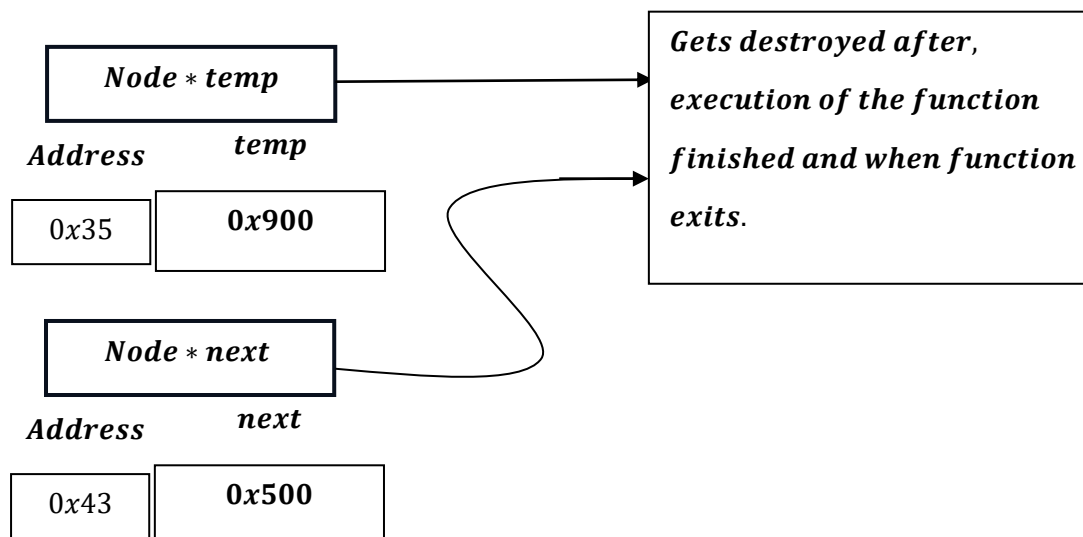
```
cout << "Deleted " << temp->data << " from the
beginning." << endl;
```

Print: Deleted 30 from the beginning.

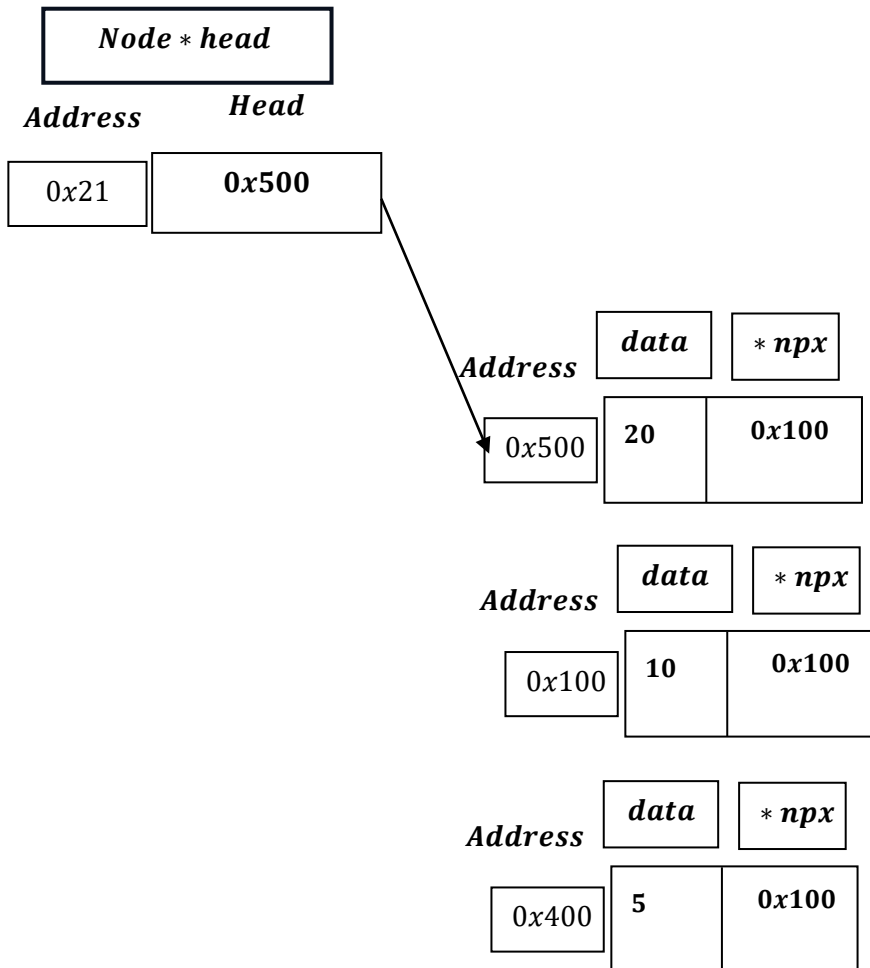
```
free(temp);
```



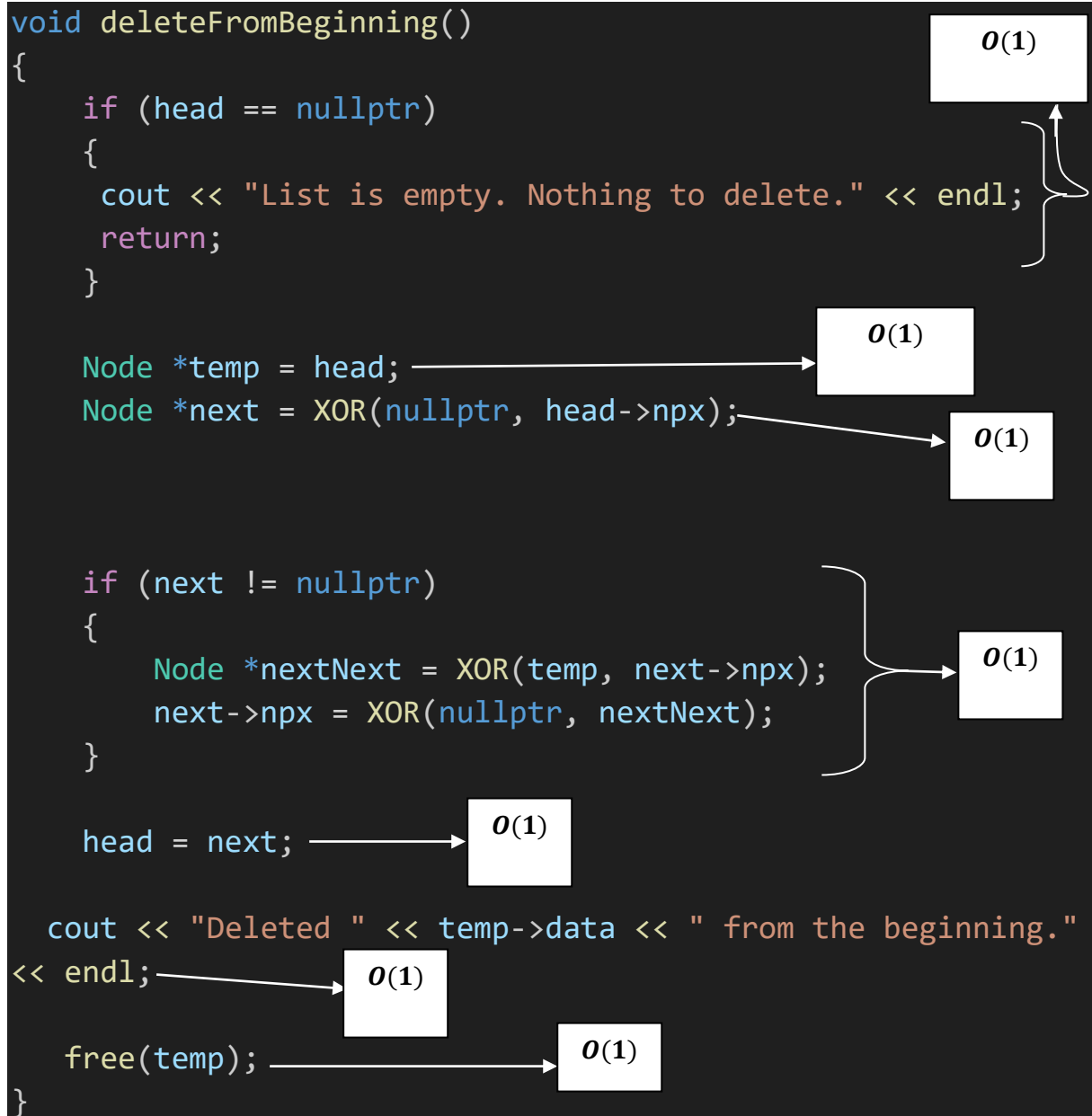
As , functions ends:



And we are left with:



Time Complexity



***Best Case = $\Omega(1)$ = Worst Case = $O(1)$
= Average Case = $\Theta(1)$.***

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Delete At Beginning</i>	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(1)$
	3. Average Case	$\Theta(1)$

***Memory Efficient Doubly Linked List – Delete At Beginning
–Space Complexity Analysis and Algorithm***

Program:

```
void deleteFromBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *temp = head;
    // Decode the second node (which will become head)
    // Next = NULL ^ head->npx
    Node *next = XOR(nullptr, head->npx);

    if (next != nullptr)
    {
        // Update the new head's npx.
        // Currently next->npx is: XOR(head, nextNext)
        // We need it to be: XOR(NULL, nextNext)
        Node *nextNext = XOR(temp, next->npx);
        next->npx = XOR(nullptr, nextNext);
    }

    head = next;
    cout << "Deleted " << temp->data << " from the
beginning." << endl;
    free(temp);
}
```

Space Complexity Analysis: XOR Linked List (Delete From Beginning)

1. Input Space Complexity

- **Parameter – only Definition:**
 - **Function Parameters:** None (takes no arguments).
 - **Analysis:** Since no data is passed into the function, the parameter space is zero.
 - **Complexity:** $O(1)$
- **Full Data Definition:**
 - **Function Data:** The existing XOR linked list structure (specifically the head and its immediate successor).
 - **Analysis:** The function operates on a list containing n nodes. The "input" context is the list itself.
 - **Complexity:** $O(n)$

2. Auxiliary Space Complexity

Definition: This measures the temporary memory used by the function to perform the deletion logic.

- **Local Variables:** The function uses three local pointers: *temp*, *next*, and *nextNext*.
 - **Analysis:** These are stack – allocated and take a fixed amount of memory (constant) regardless of how many nodes are in the list.
- **XOR Operations:** The bitwise XOR calculations are performed within CPU registers.
- **Memory Management:** The *free(temp)* operation releases memory back to the system. While this changes the total memory used by the program, it does not require extra auxiliary space to perform the deallocation.
- **No Recursion:** The logic is straightforward and iterative, so there is no growth in the call stack.

Conclusion: The extra space required to perform the deletion is constant.

- **Auxiliary Space Complexity:** $O(1)$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Delete From Beginning (XOR Doubly Linked List)

XORDELBEG(START):

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete."

Return and EXIT.

[End of If structure]

2. [INITIALIZE TEMPORARY POINTER]

Set TEMPPTR := START $\left(\begin{array}{l} \text{Store the current head to free} \\ \text{its memory later} \end{array} \right)$

3. [DECODE THE SECOND NODE]

Set NEXTTEMPPTR := NULL $\oplus$ NEXTXORPTR[START] $\left(\begin{array}{l} \text{Calculate the} \\ \text{address of the node} \\ \text{that will become} \\ \text{the new head} \end{array} \right)$

4. [UPDATE NEW HEAD'S LINKAGE]

If NEXTTEMPPTR $\neq$ NULL, then: $\left(\begin{array}{l} \text{If the list has more} \\ \text{than one node,} \\ \text{we must update} \\ \text{the successor's NPX} \end{array} \right)$

a. Set NEXTNEXTTEMPPTR :
= TEMPTR

$\oplus$ NEXTXORPTR[NEXTTEMPPTR] $\left(\begin{array}{l} \text{Decode the node following} \\ \text{the second node using} \\ \text{the current head} \\ \text{as the 'previous'} \\ \text{reference} \end{array} \right)$

b. Set NEXTXORPTR[NEXTTEMPPTR]:

= NULL $\oplus$ NEXTNEXTTEMPPTR $\left(\begin{array}{l} \text{Update the new head's NPX} \\ \text{to indicate its} \\ \text{previous neighbor} \\ \text{is now NULL} \end{array} \right)$

[End of If structure]

5. [UPDATE GLOBAL START]

Set $START := NEXTTEMPPTR$ $\left(\begin{array}{c} \text{The second node officially} \\ \text{becomes the} \\ \text{first node} \\ \text{of the list} \end{array} \right)$

6. [FREE MEMORY AND PRINT]

Write: Deleted , $INFO[TEMPPTR]$, from the beginning.

$FREE(TEMPPTR)$ $\left(\begin{array}{c} \text{Return the memory of the} \\ \text{deleted node to the system} \end{array} \right)$

7. EXIT

Memory Efficient Doubly Linked List

–Delete At End – Program Analysis

And Time Complexity.

Program:

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    // Traverse to end
    while (true)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr)
            break; // curr is the last node
        prev = curr;
        curr = next;
    }

    // curr is the last node. prev is the second to last.
    if (prev != nullptr)
    {
        // prev->npx was: XOR(prevPrev, curr)
        // Needs to be: XOR(prevPrev, NULL)

        // Decode prevPrev
        Node *prevPrev = XOR(prev->npx, curr);
```

```

        prev->npx = XOR(prevPrev, nullptr);
    }
    else
    {
        // List had only one node
        head = nullptr;
    }

    cout << "Deleted " << curr->data << " from the end." <<
endl;
    free(curr);
}

...

case 6:
    deleteFromEnd();
    break;

```

Program Analysis

1. When List Is Empty

```

if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}

```

Node * head

Address

Head

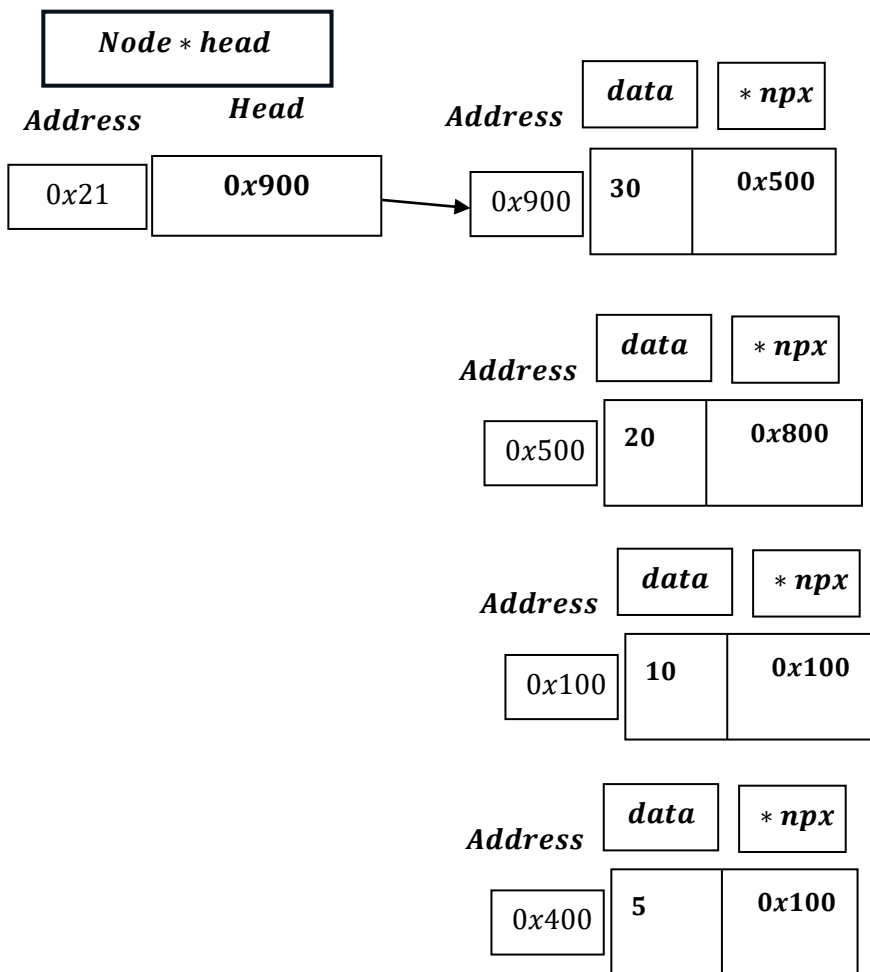
0x21

nullptr

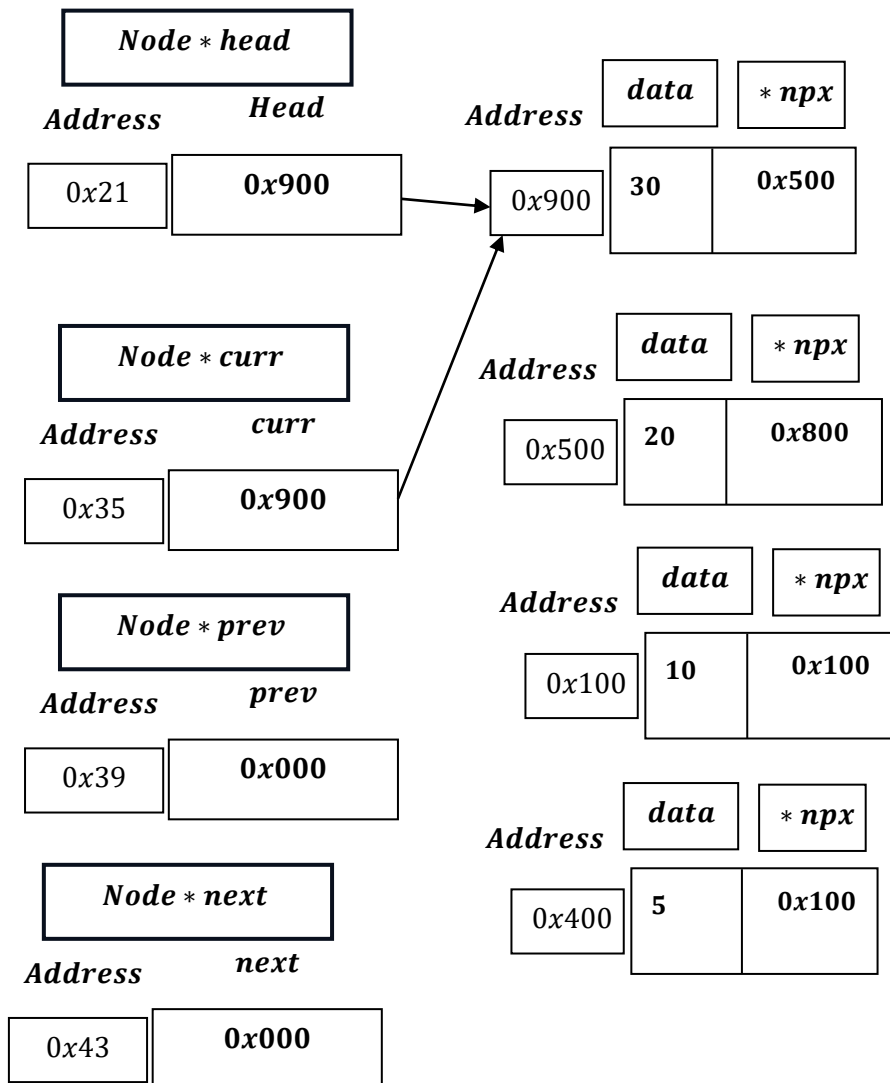
Then , it prints : "List is empty.Nothing to delete."

Return Void and Exit.

2. When List Is Not Empty



```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;
```



```
while (true)
{
    next = XOR(prev, curr->npx);
    if (next == nullptr)
        break; // curr is the last node
    prev = curr;
    curr = next;
}
```

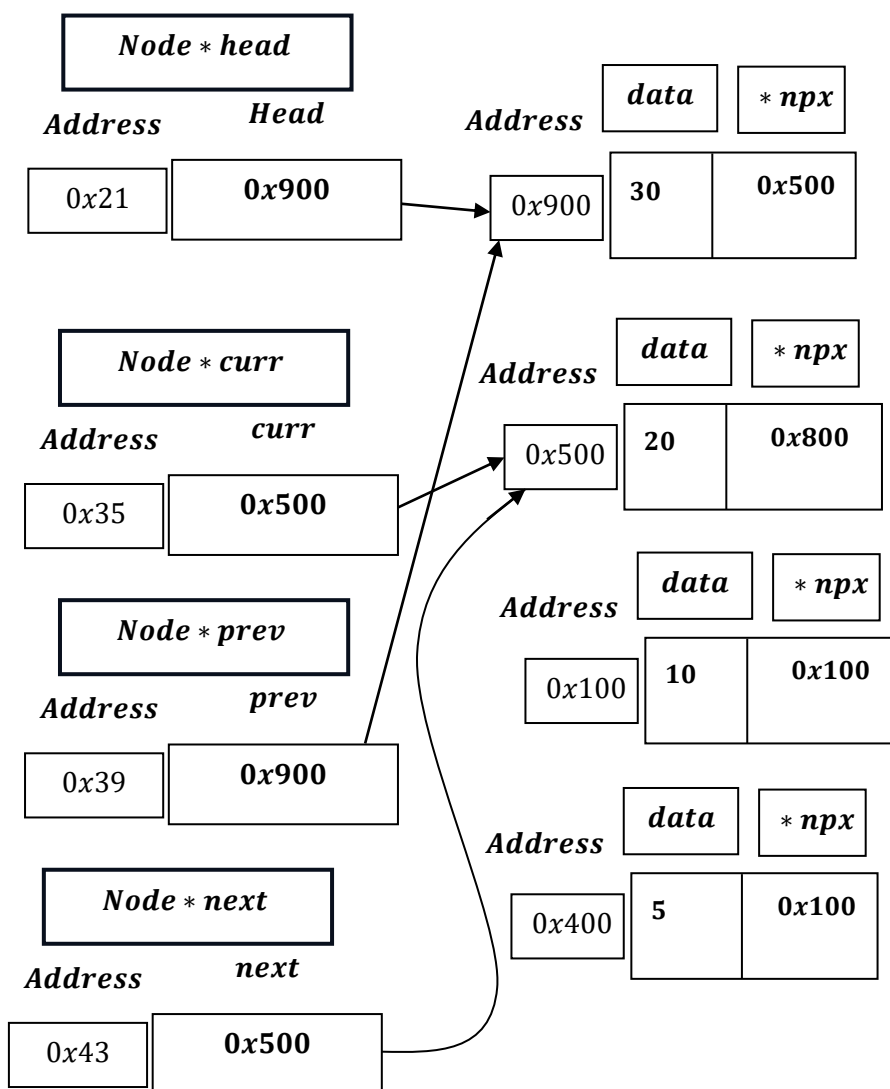
While(true):

next = XOR(prev: 0x000, curr → npx: 0x500) = 0x500.

if(next: 0x500 = nullptr: 0x000) is false.

prev = curr = 0x900

curr = next = 0x500.



While(true):

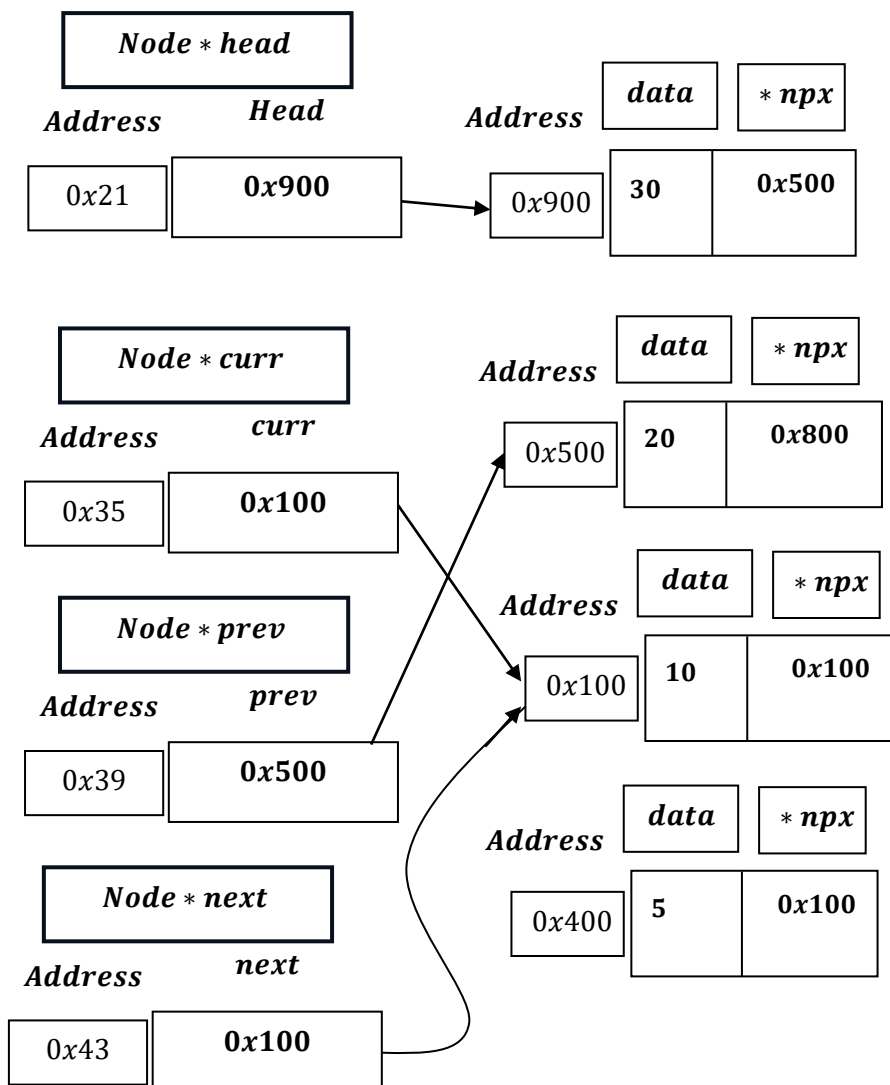
next = XOR(prev: 0x900, curr → npx: 0x800)

= 1001₂ ⊕ 1000₂ = 0001₂ = 0x100

if(next: 0x100 = nullptr: 0x000) is false.

prev = curr = 0x500

curr = next = 0x100.



While(true):

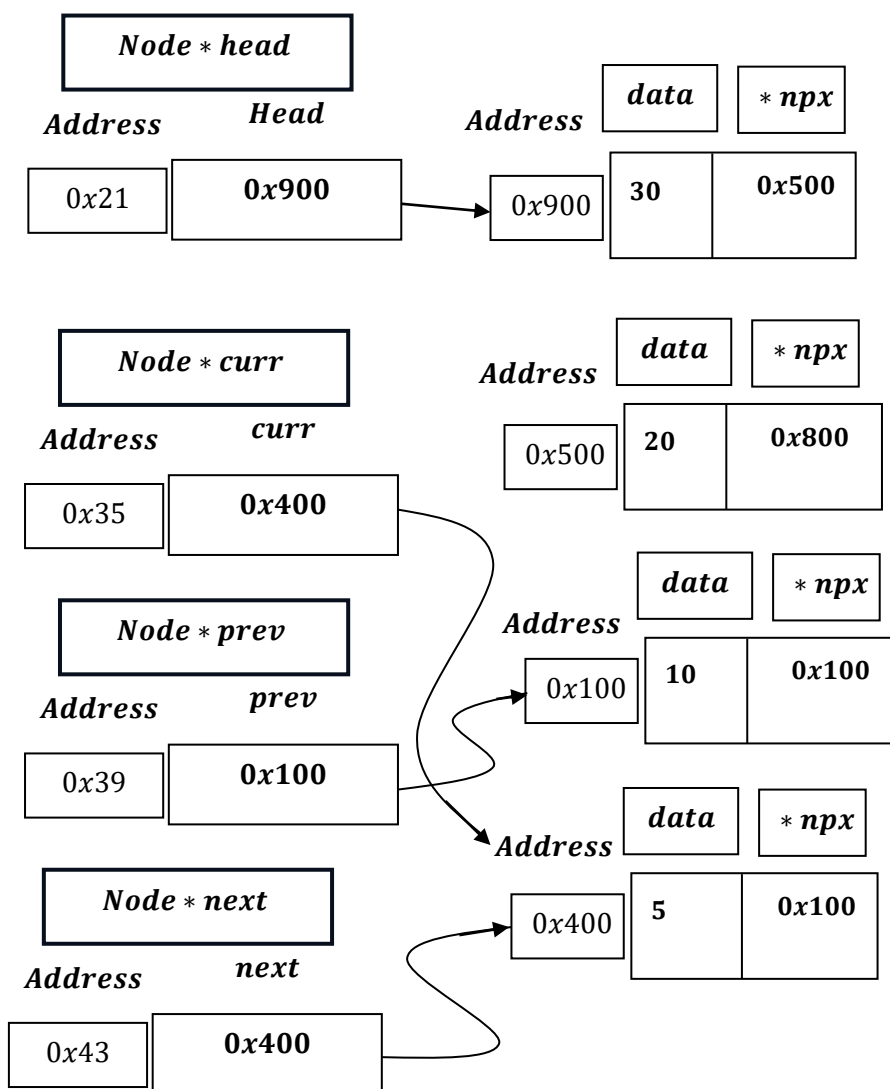
next = XOR(prev: 0x500, curr → npx: 0x100)

= 0101₂ ⊕ 0001₂ = 0100₂ = 0x400

if(next: 0x400 = nullptr: 0x000) is false.

prev = curr = 0x100

curr = next = 0x400.



While(true):

next = XOR(prev: 0x100, curr → npx: 0x100)

= 0001₂ ⊕ 0001₂ = 0000₂ = 0x000

if(next: 0x000 = nullptr: 0x000) is True:

break; → statement helps to exit from the loop.

... ..

... ..

```
if (prev != nullptr)
{
    Node *prevPrev = XOR(prev->npx, curr);
    prev->npx = XOR(prevPrev, nullptr);
}
else{..... }
```

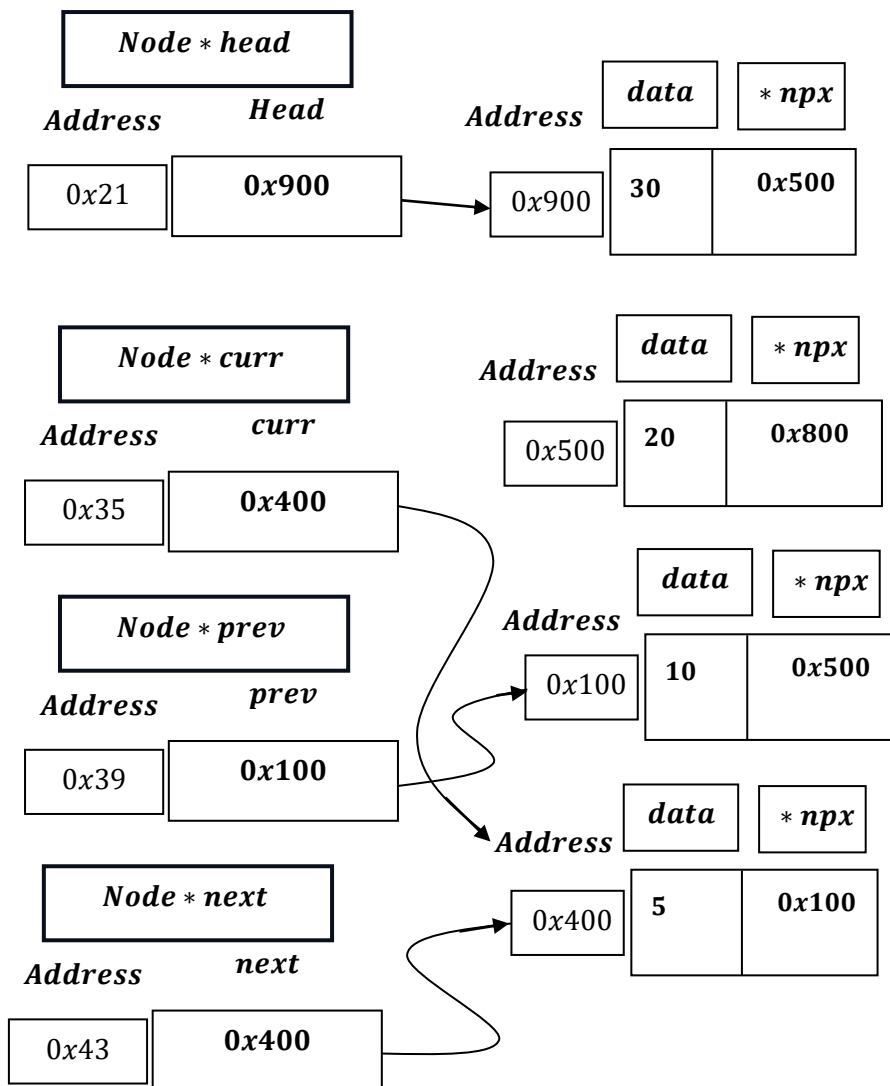
Prev: 0x100 ≠ NULLPTR[TRUE]:

Node * prevPrev = prev → npx: 0x100 ⊕ curr: 0x400

= 0001₂ ⊕ 0100₂ = 0101₂ = 0x500

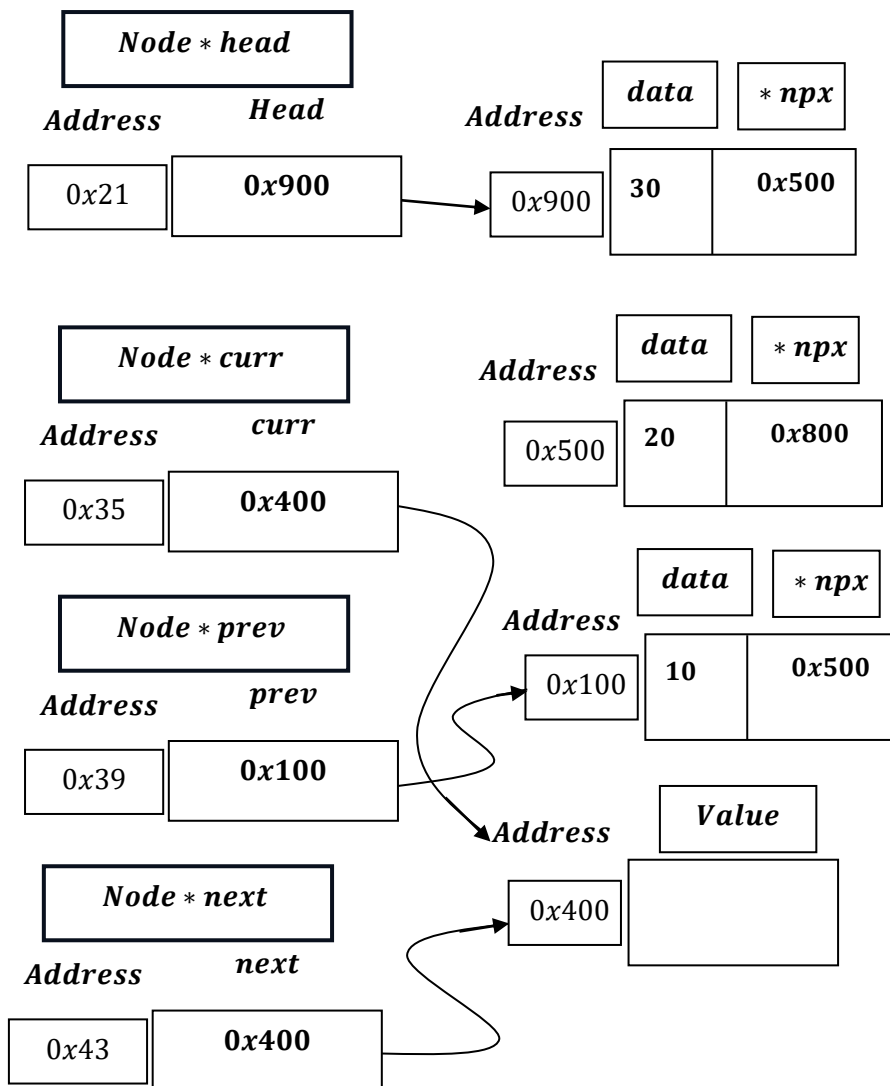
prev → npx = prevPrev: 0x500 ⊕ nullptr: 0x000

= 0101₂ ⊕ 0000₂ = 0101₂ = 0x500

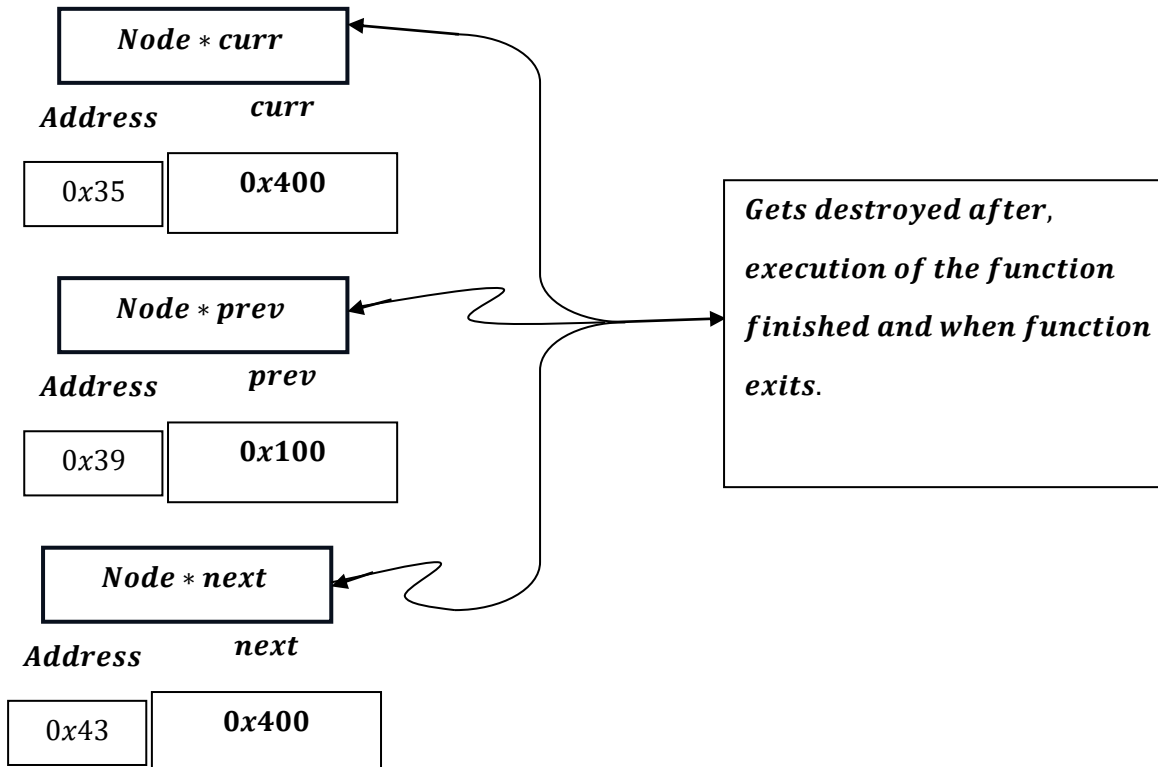


```
free(curr);
```

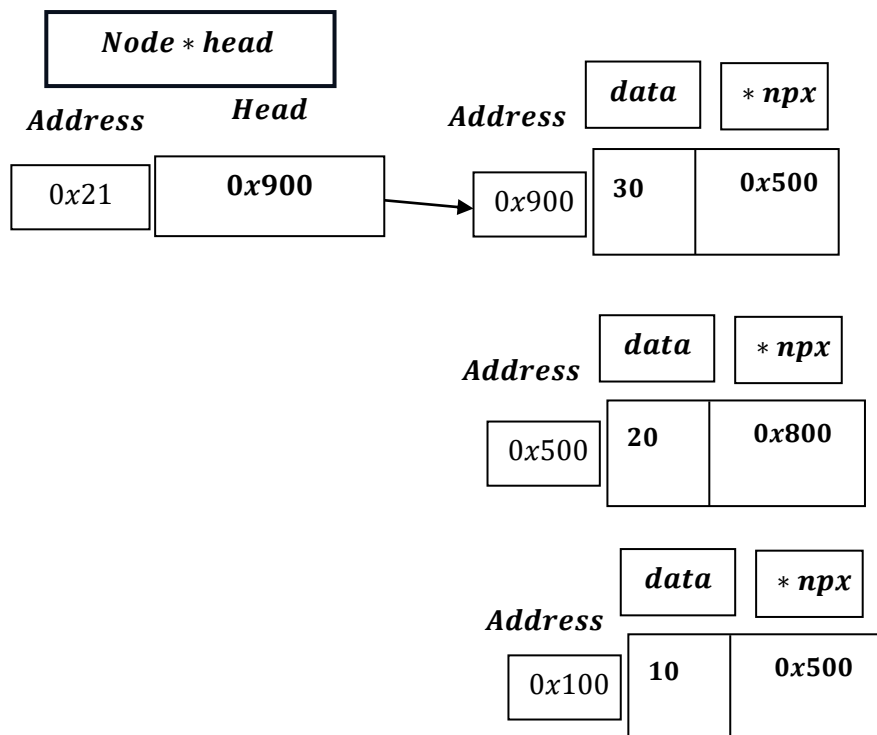
Free(curr: 0x400);



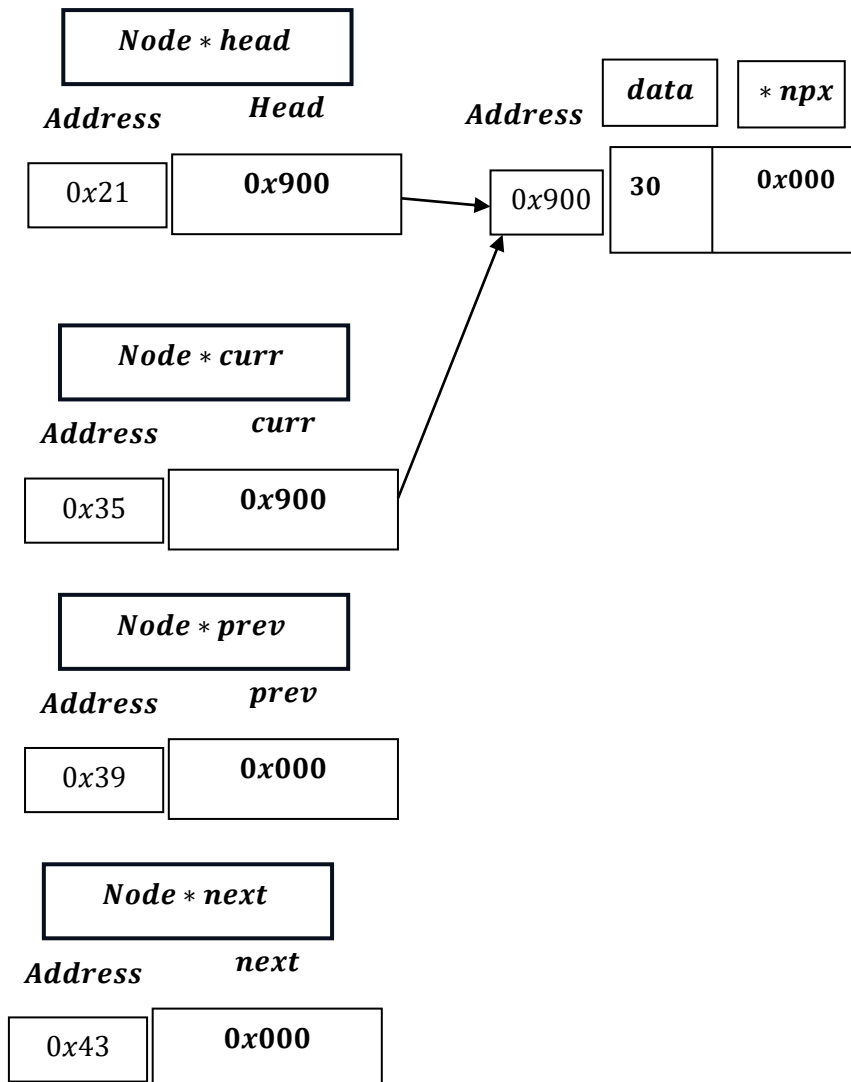
As , functions ends:



And we are left with:



Now say, when we have single node:



```
while (true)
{
    next = XOR(prev, curr->npx);
    if (next == nullptr)
        break; // curr is the last node
    prev = curr;
    curr = next;
}
```

While(true):

next = XOR(prev: 0x000, curr → npx: 0x000)

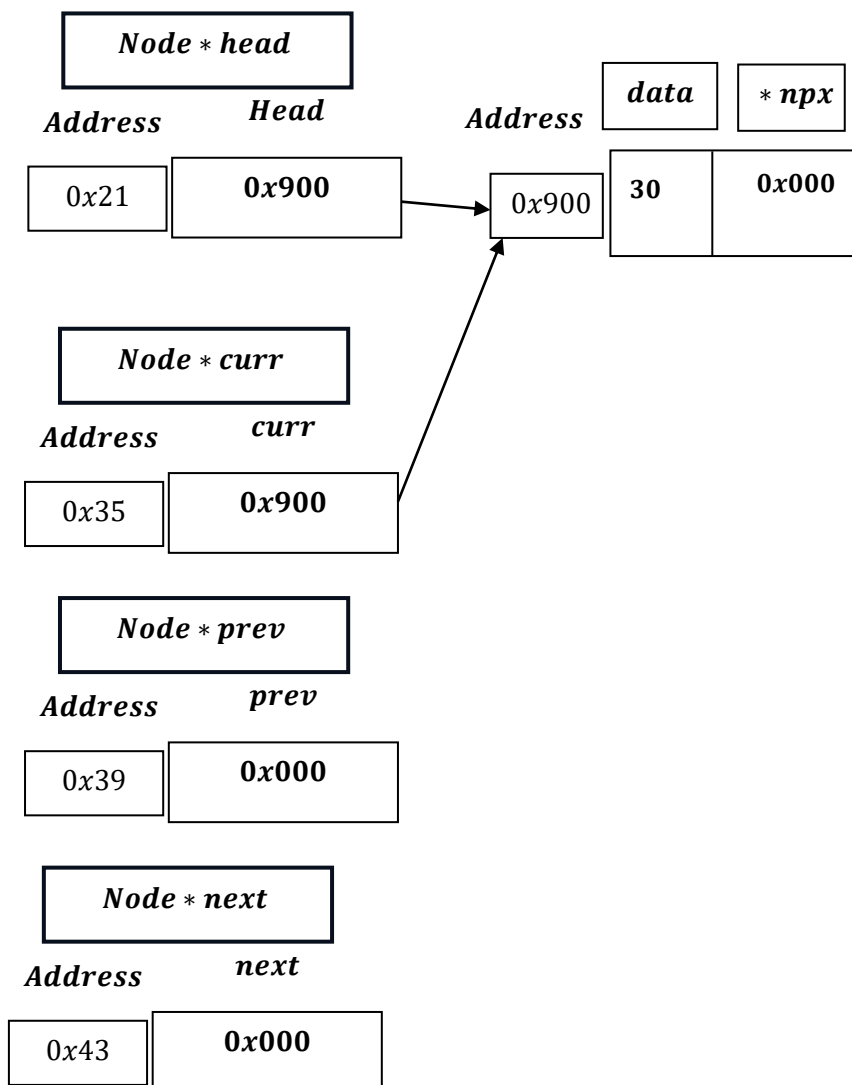
= 0000₂ ⊕ 0000₂ = 0000₂ = 0x000

if(next: 0x000 = nullptr: 0x000) is True:

break; → statement helps to exit from the loop.

... ..

... ..



```

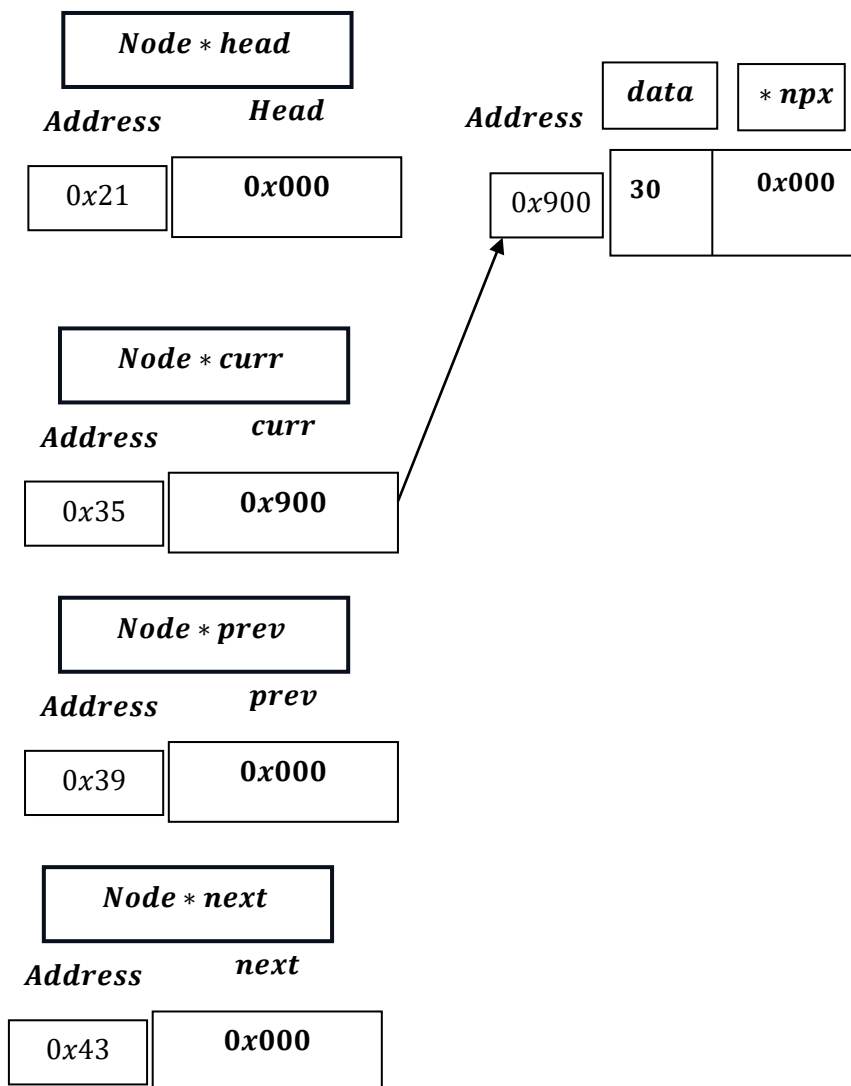
if (prev != nullptr)
{..... }
else
{
    head = nullptr;
}

```

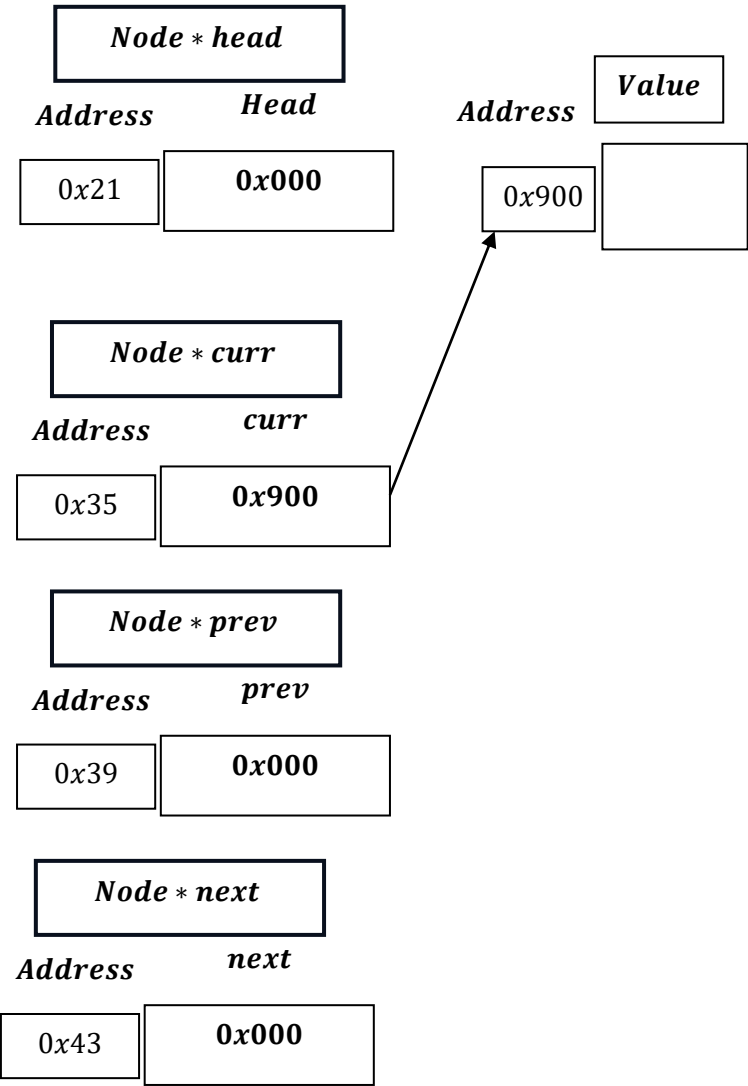
if (prev $\neq$ nullptr) false:

Else:

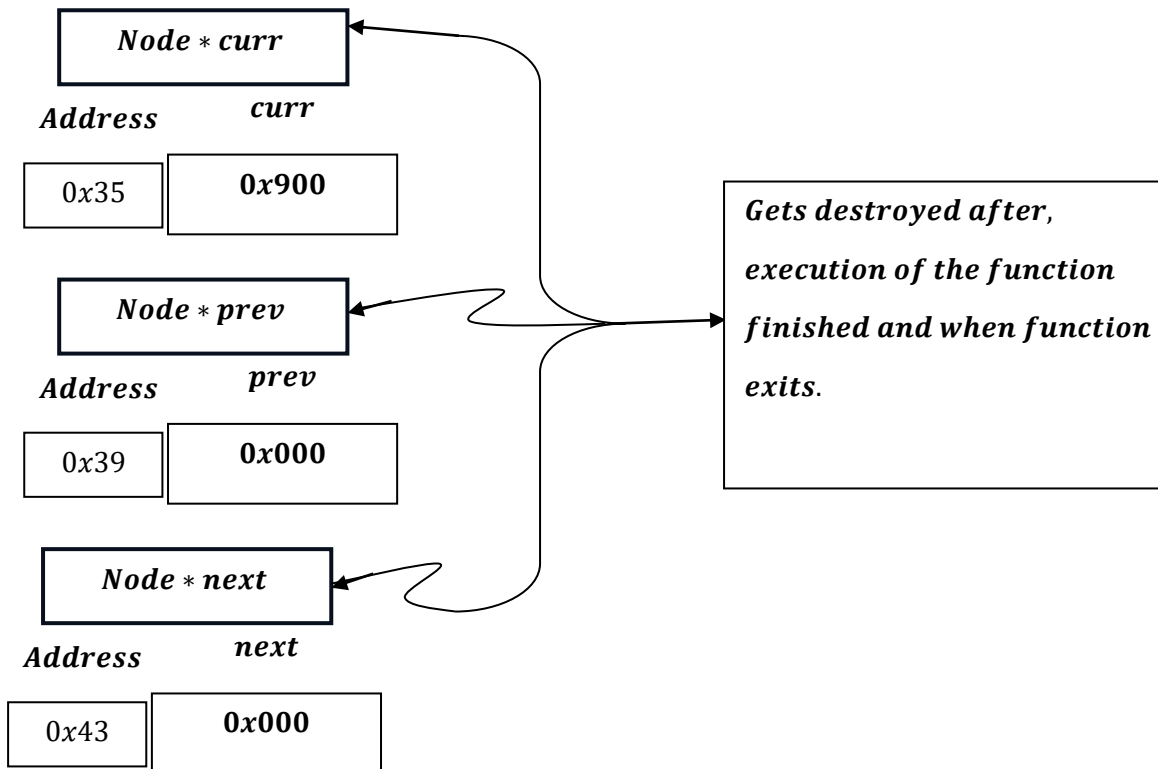
HEAD = NULLPTR = 0x000



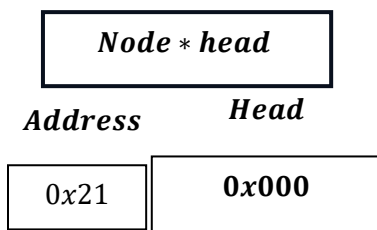
```
free(curr);  
Free(curr: 0x900);
```



As , functions ends:



And we are left with:



Time Complexity

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    while (true)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr)
            break;
        prev = curr;
        curr = next;
    }

    if (prev != nullptr)
    {
        Node *prevPrev = XOR(prev->npx, curr);
        prev->npx = XOR(prevPrev, nullptr);
    }
    else
    {
        head = nullptr;
    }

    cout << "Deleted " << curr->data << " from the end." << endl;
    free(curr);
}
```

The diagram illustrates the time complexity of the `deleteFromEnd()` function. It includes the following annotations:

- $O(1)$** : Points to the `if (head == nullptr)` block, indicating constant time for checking an empty list.
- $O(1)$** : Points to the initialization of `curr`, `prev`, and `next` pointers.
- Runs `n`` time.**: Points to the `while (true)` loop, indicating linear time complexity relative to the number of nodes.
- $O(1)$** : Points to the `if (next == nullptr) break;` condition and the `break;` statement.
- $O(1)$** : Points to the `prev = curr;` and `curr = next;` update statements.
- $O(1)$** : Points to the `if (prev != nullptr)` block, indicating constant time for the final update and `free(curr);` operation.

Best Case Analysis[When List has no Node]

When list has no node, head pointed to `nullptr`, it is the best case $\Omega(1)$.

Worst Case Analysis[When List has numerous Node]

for loop:

while(true) $\rightarrow$ [this condition runs `n` times]:

next = XOR(prev, curr $\rightarrow$ npx) $\rightarrow$ runs $1 + 1 + 1 \dots (n - 1)$ times $= n - 1$.

if(next = nullptr): $\rightarrow$ checks `n` time. [including when false and true]

break; $\rightarrow$ runs `1` time.

prev = curr; $\rightarrow$ runs $1 + 1 + 1 + \dots (n - 1)$ times $= n - 1$

curr = next; $\rightarrow$ runs $1 + 1 + 1 + \dots (n - 1)$ times $= n - 1$

*$\therefore$ We focus innerstatement $= n - 1 + n + 1 + n - 1 + n - 1 = 3n - 3 + 2$
 $= 4n - 2$.*

$\therefore$ Time Complexity $= O(4n - 2) = 4 \times O(n) - O(2) = O(n)$.

Average Case

*We can say , if List is Empty ,
, we get our best case scenario, and when list is not empty and there present multiple nodes , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the deletion to be made , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.*

In the case of DeleteAtEnd, the deletion always happens at the end, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the deletion position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence DeleteAtEnd doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>DeleteAt End</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

***Memory Efficient Doubly Linked List –
Delete At End – Space Complexity Analysis and Algorithm***

Program:

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    // Traverse to end
    while (true)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr)
            break; // curr is the last node
        prev = curr;
        curr = next;
    }

    // curr is the last node. prev is the second to last.
    if (prev != nullptr)
    {
        // prev->npx was: XOR(prevPrev, curr)
        // Needs to be: XOR(prevPrev, NULL)

        // Decode prevPrev
        Node *prevPrev = XOR(prev->npx, curr);
        prev->npx = XOR(prevPrev, nullptr);
    }
}
```

```

else
{
    // List had only one node
    head = nullptr;
}

cout << "Deleted " << curr->data << " from the end." <<
endl;
free(curr);
}

...

case 6:
    deleteFromEnd();
    break;

```

Space Complexity Analysis: XOR Linked List (Delete From End)

1. Input Space Complexity

- ***Parameter – only Definition:***
 - ***Function Parameters:*** None (takes no arguments).
 - ***Analysis:*** No additional data is passed into the function.
 - ***Complexity:*** $O(1)$
 - ***Full Data Definition:***
 - ***Function Data:*** The existing XOR linked list structure.
 - ***Analysis:*** The function must traverse all n nodes of the list to locate the last node. The entire list serves as the input context.
 - ***Complexity:*** $O(n)$
-

2. Auxiliary Space Complexity

Explanation: This measures the extra memory used by the algorithm during execution, excluding the input itself.

- **Local Variables:** The function utilizes three local pointers: *curr*, *prev*, and *next*. Additionally, it uses a temporary *prevPrev* pointer during the linkage update.
 - **Analysis:** These are stack – allocated pointers. Their number is fixed (constant) regardless of whether the list contains 10 nodes or 10,000 nodes.
- **Traversal:** The traversal is handled via a *while(true)* loop (iteration). Since it is not recursive, it does not add any frames to the call stack.
- **XOR Operations:** Bitwise XOR calculations are performed within CPU registers and do not require heap or additional stack allocation.
- **Deallocation:** *free(curr)* returns the memory of the last node to the heap. While this reduces the total memory footprint, the auxiliary space used to perform the logic is unchanged.

Conclusion: The amount of extra memory used is fixed and does not scale with *n*.

- **Auxiliary Space Complexity:** $O(1)$
-

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Delete From End (XOR Doubly Linked List)

XORDELEND(START):

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete."

Return and EXIT.

[End of If structure]

2. [INITIALIZE TRAVERSAL]

Set CURRPTR := START

Set PREVPTR := NULL

3. [TRAVERSE TO THE LAST NODE]

Repeat while TRUE:

a. Set NEXTTEMPPTR := PREVPTR $\oplus$ NEXTXORPTR[CURRPTR]

b. If NEXTTEMPPTR = NULL, then:

***Break** (CURRPTR is now the last node)*

c. Set PREVPTR := CURRPTR

d. Set CURRPTR := NEXTTEMPPTR

[End of Loop]

4. [CHECK IF LIST HAD ONLY ONE NODE]

If PREVPTR = NULL, then:

Set START := NULL $\left(\begin{array}{l} \text{The only node in the} \\ \text{list is being deleted} \end{array} \right)$

5. [UPDATE SECOND – TO – LAST NODE LINKAGE]

Else:

a. Set PREVPREVTEMPPTR := NEXTXORPTR[PREVPTR] $\oplus$

CURRPTR $\left(\begin{array}{l} \text{Decode the node} \\ \text{preceding} \\ \text{the second to} \\ \text{last node.} \end{array} \right)$

b. Set NEXTXORPTR[PREVPTR] := PREVPREVTEMPPTR $\oplus$

NULL $\left(\begin{array}{l} \text{Update the second – to – last node} \\ \text{to be the new last node} \\ \text{by setting} \\ \text{its `next` to NULL} \end{array} \right)$

[End of If structure]

6. [**FREE MEMORY AND PRINT**]

Write: "Deleted ", $INFO[CURRPTR]$, " from the end."

FREE($CURRPTR$) $\left(\begin{array}{l} \text{Return the memory} \\ \text{of the deleted node} \\ \text{to the system} \end{array} \right)$

7. **EXIT**

Memory Efficient Doubly Linked List

–Delete At Position

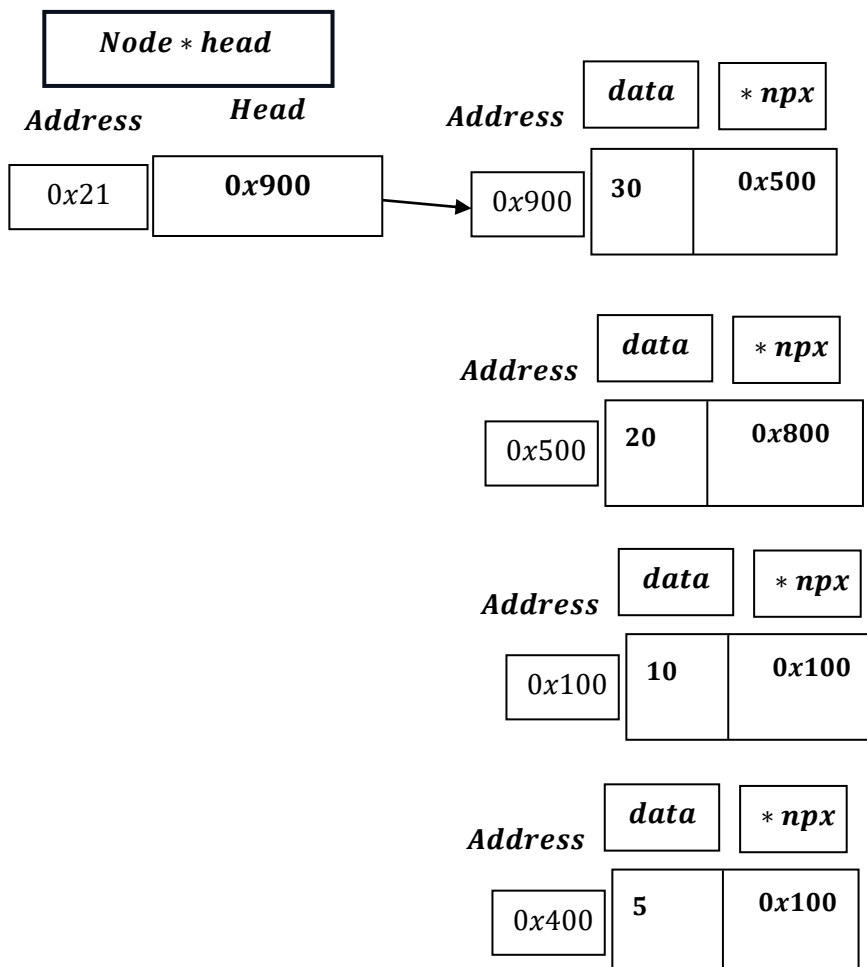
–Program Analysis – Part 2

When next = NULL

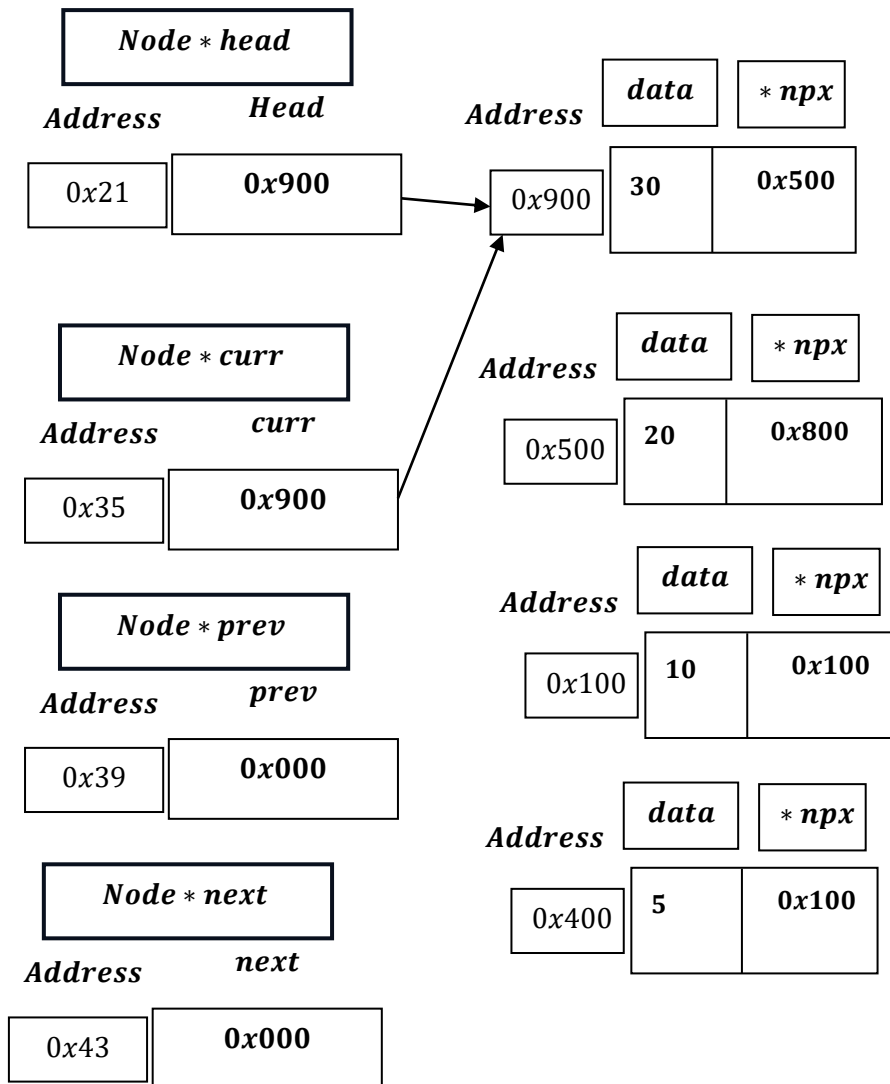
–It occurs when node is deleted at end of the list.

When next $\neq$ NULL

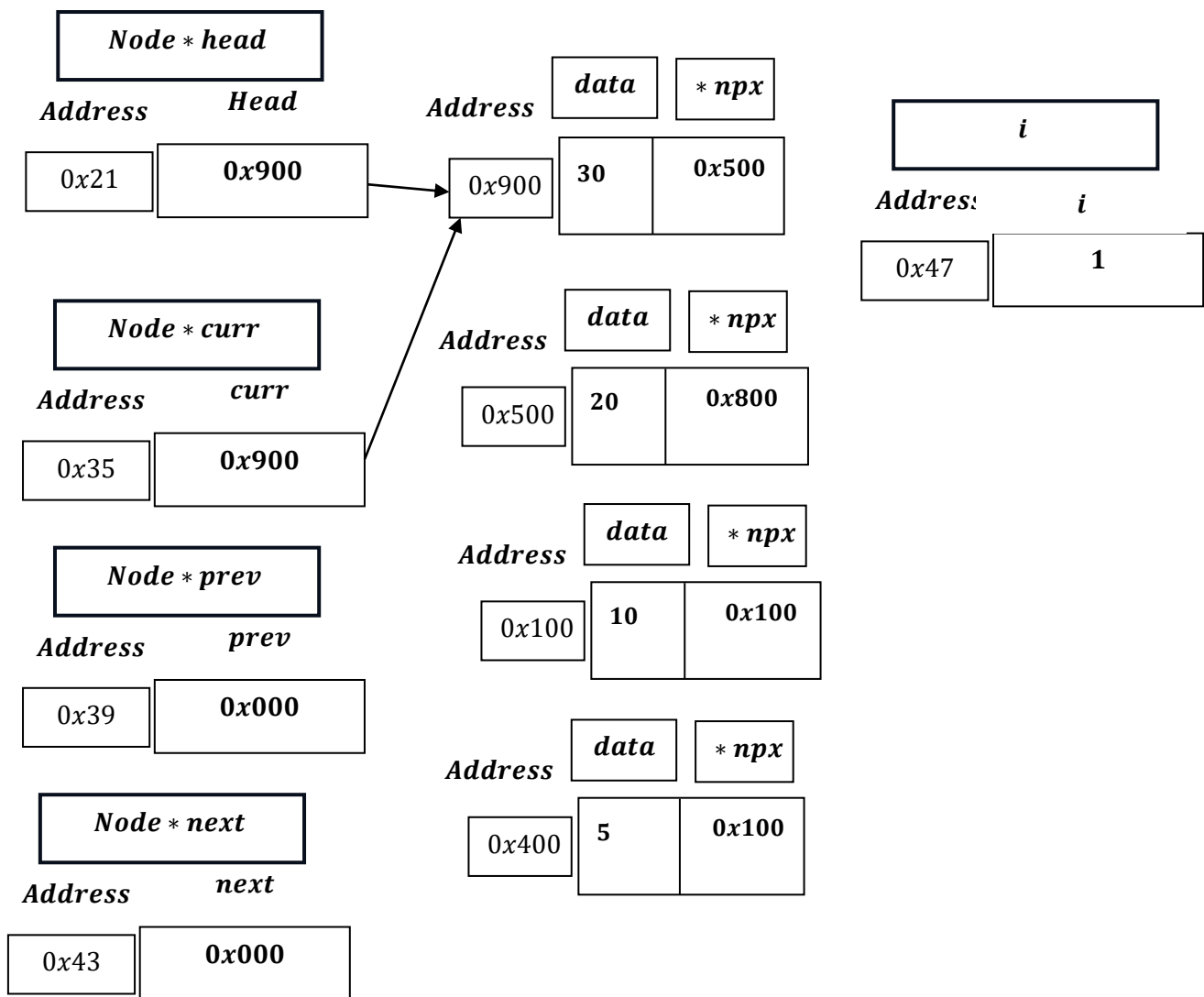
–It occurs when node is not deleted at end of the list.



```
Node *curr = head;
Node *prev = nullptr;
Node *next;
```



```
int i = 1;
```



```
while (curr != nullptr && i < pos)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

pos: 4.

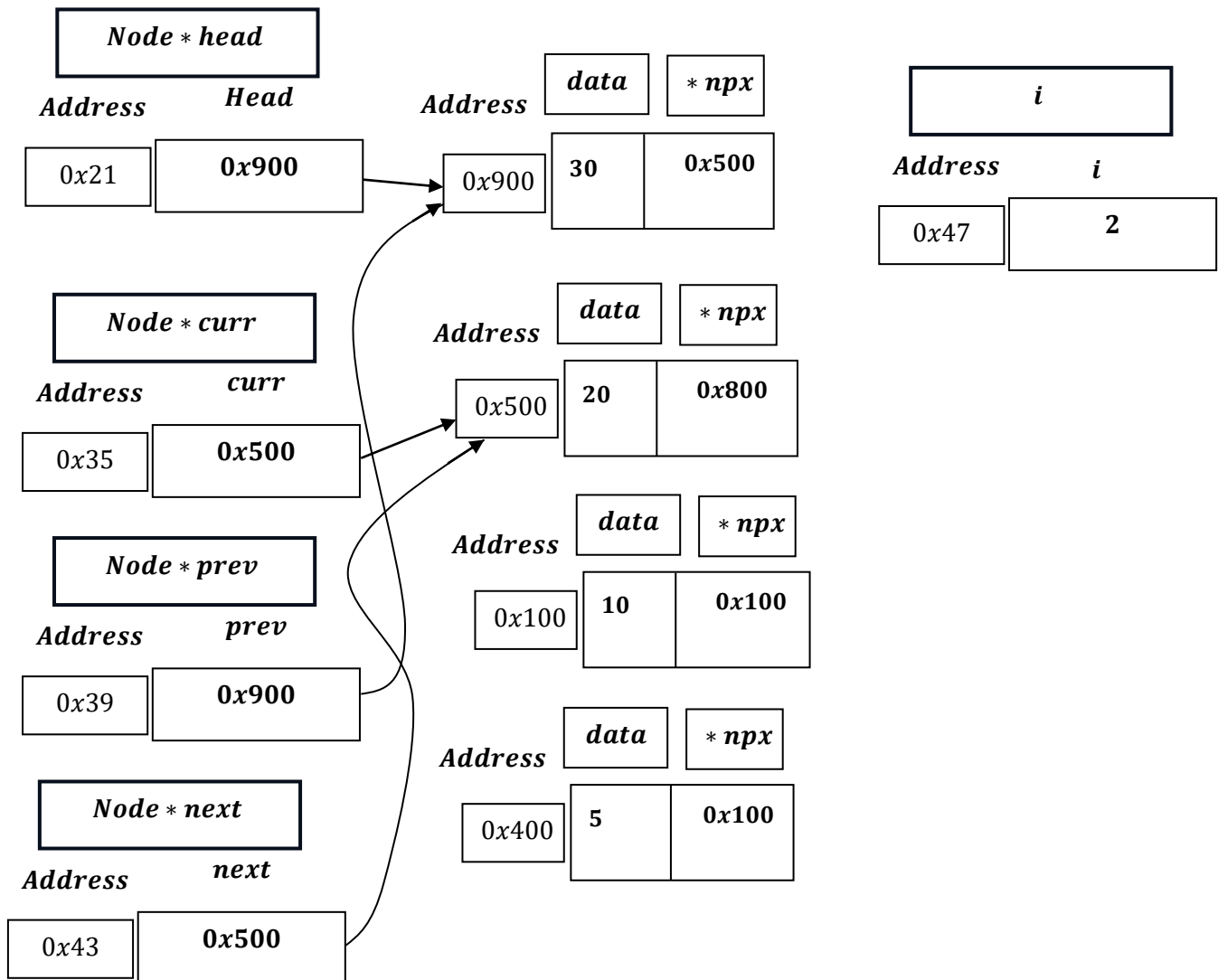
***curr: 0x900* $\neq$ *nullptr: 0x000* AND *i = 1* < *pos: 4* :**

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x000 $\oplus$ 0x500 = 0x500.

prev = curr = 0x900

curr = next = 0x500

i = i + 1 = 1 + 1 = 2.



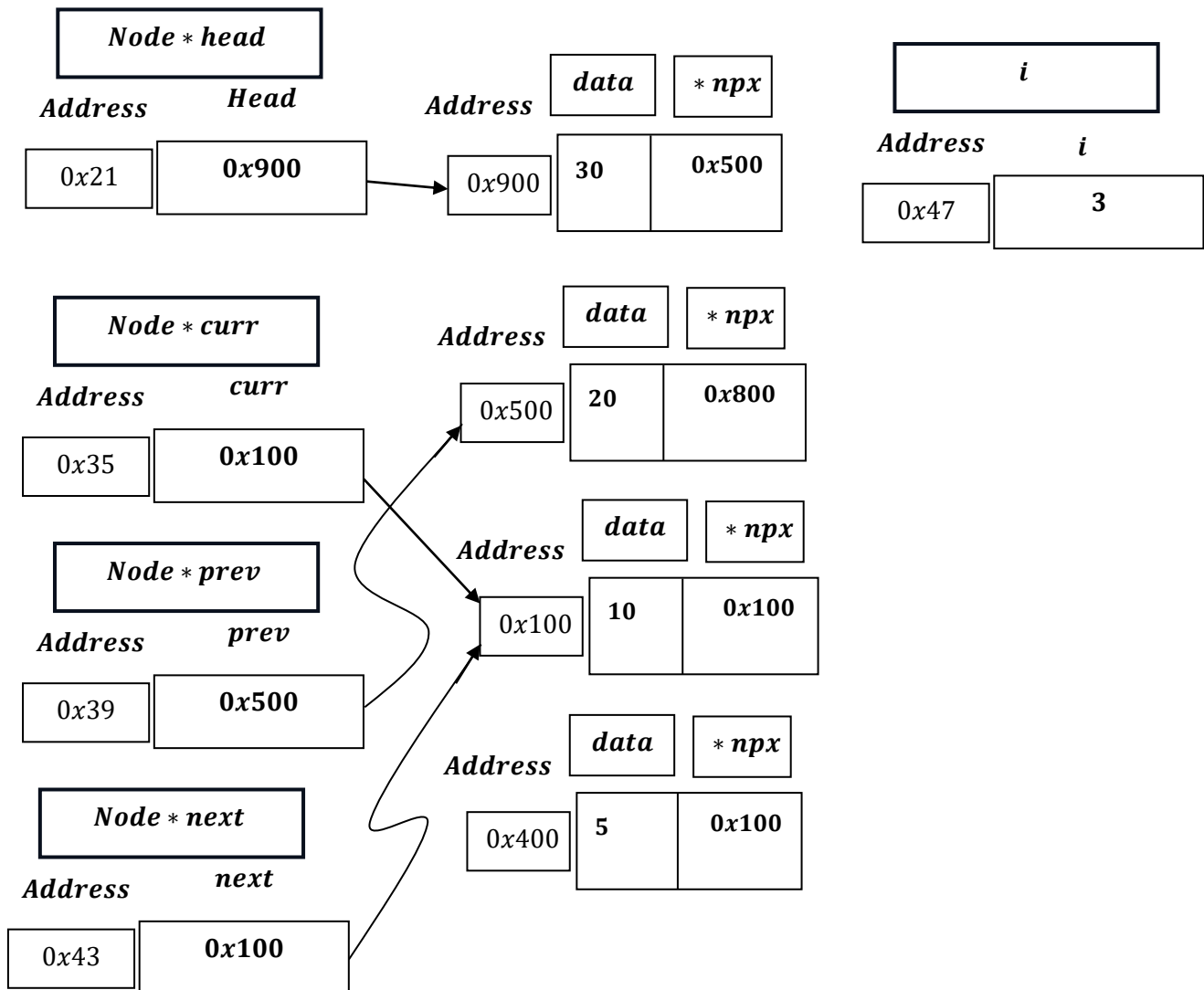
***curr*: 0x500 $\neq$ *nullptr*: 0x000 AND *i* = 2 < *pos*: 4 :**

***next* = $XOR(prev: 0x900, curr \rightarrow npx: 0x800)$ = $1001_2 \oplus 1000_2 = 0001_2 = 0x100$**

***prev* = *curr* = 0x500**

***curr* = *next* = 0x100**

***i* = *i* + 1 = 2 + 1 = 3.**



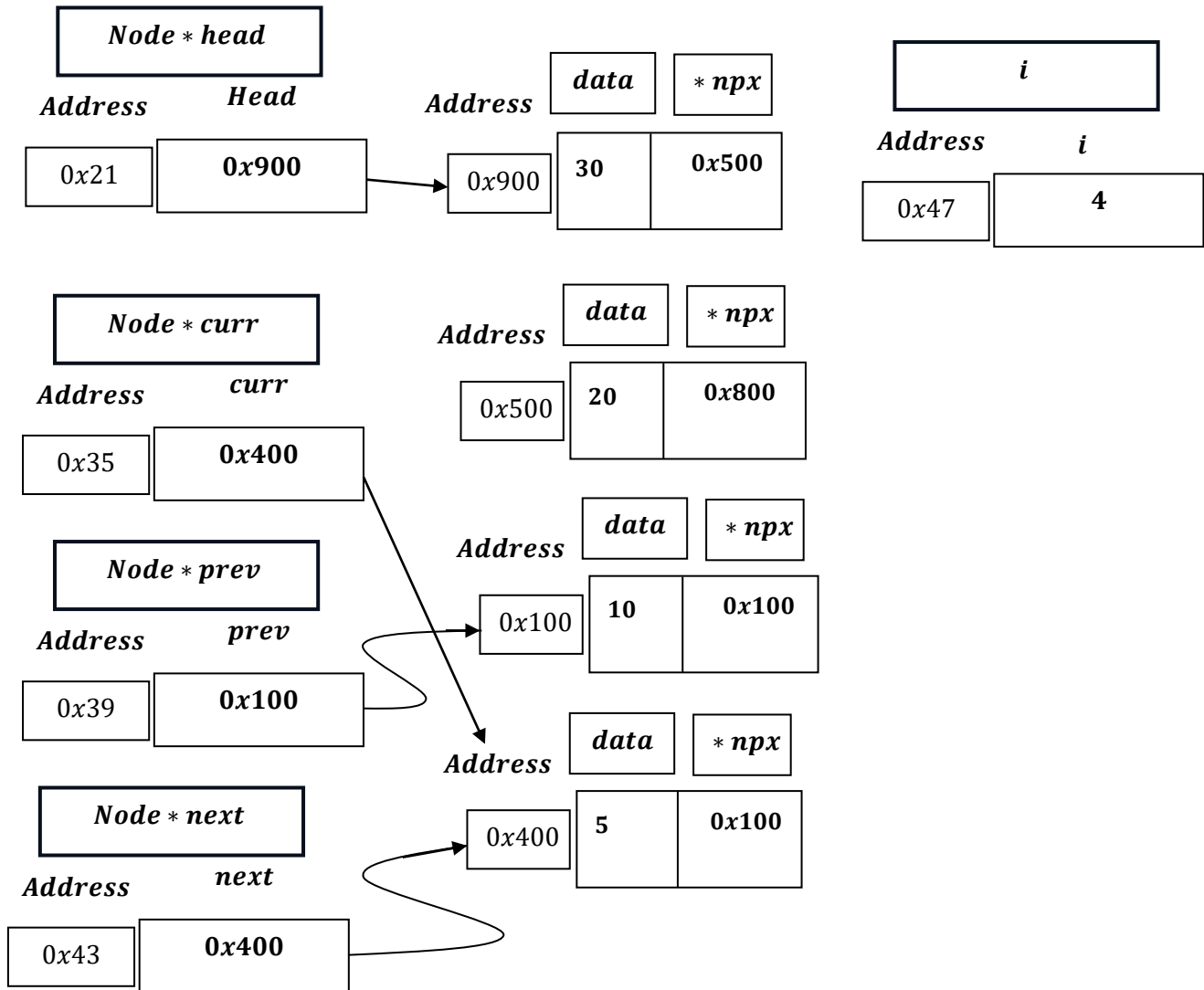
$curr: 0x100 \neq nullptr: 0x000$ AND $i = 3 < pos: 4$:

$next = XOR(prev: 0x500, curr \rightarrow npx: 0x100) = 0101_2 \oplus 0001_2 = 0100_2 = 0x400$

$prev = curr = 0x100$

$curr = next = 0x400$

$i = i + 1 = 3 + 1 = 4.$

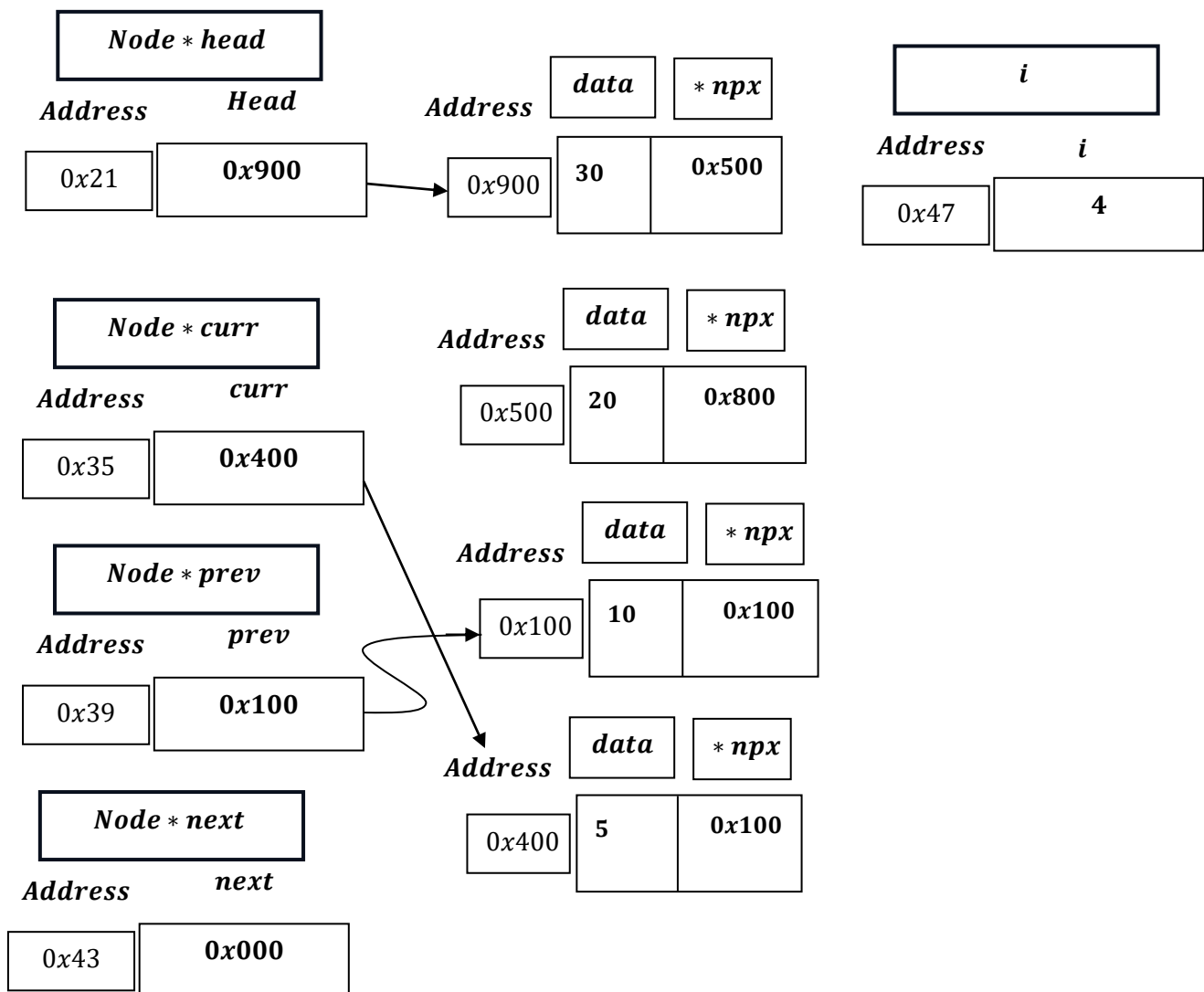


**$curr: 0x400 \neq nullptr: 0x000$ [is TRUE] AND $i = 4 < pos: 4$ [is False]
 $\Rightarrow$ Condition is False.**

```
next = XOR(prev, curr->npx);
```

$next = XOR(prev: 0x100 \oplus curr \rightarrow npx: 0x100);$

$0001_2 \oplus 0001_2 = 0000_2 = 0_{10} = 0x000.$



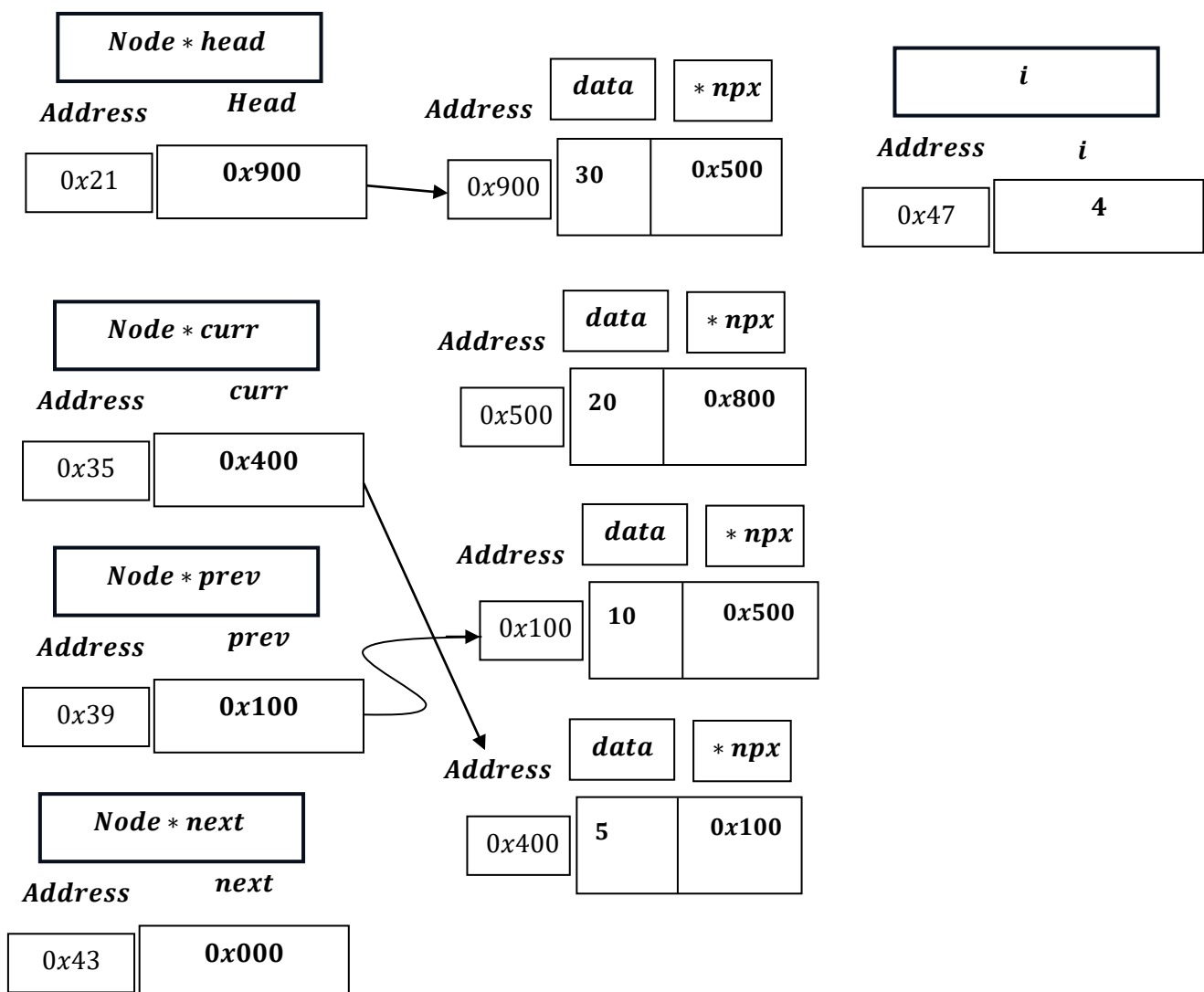
```
Node *prevPrev = XOR(prev->npx, curr);
prev->npx = XOR(prevPrev, next);
```


$prevPrev = XOR(prev \rightarrow npx: 0x100, curr: 0x400)$

$= 0001_2 \oplus 0100_2 = 0101_2 = 5_{10} = 0x500$

$prev \rightarrow npx = XOR(prevPrev: 0x500, next: 0x000)$

$= 0101_2 \oplus 0000_2 = 0101_2 = 5_{10} = 0x500$



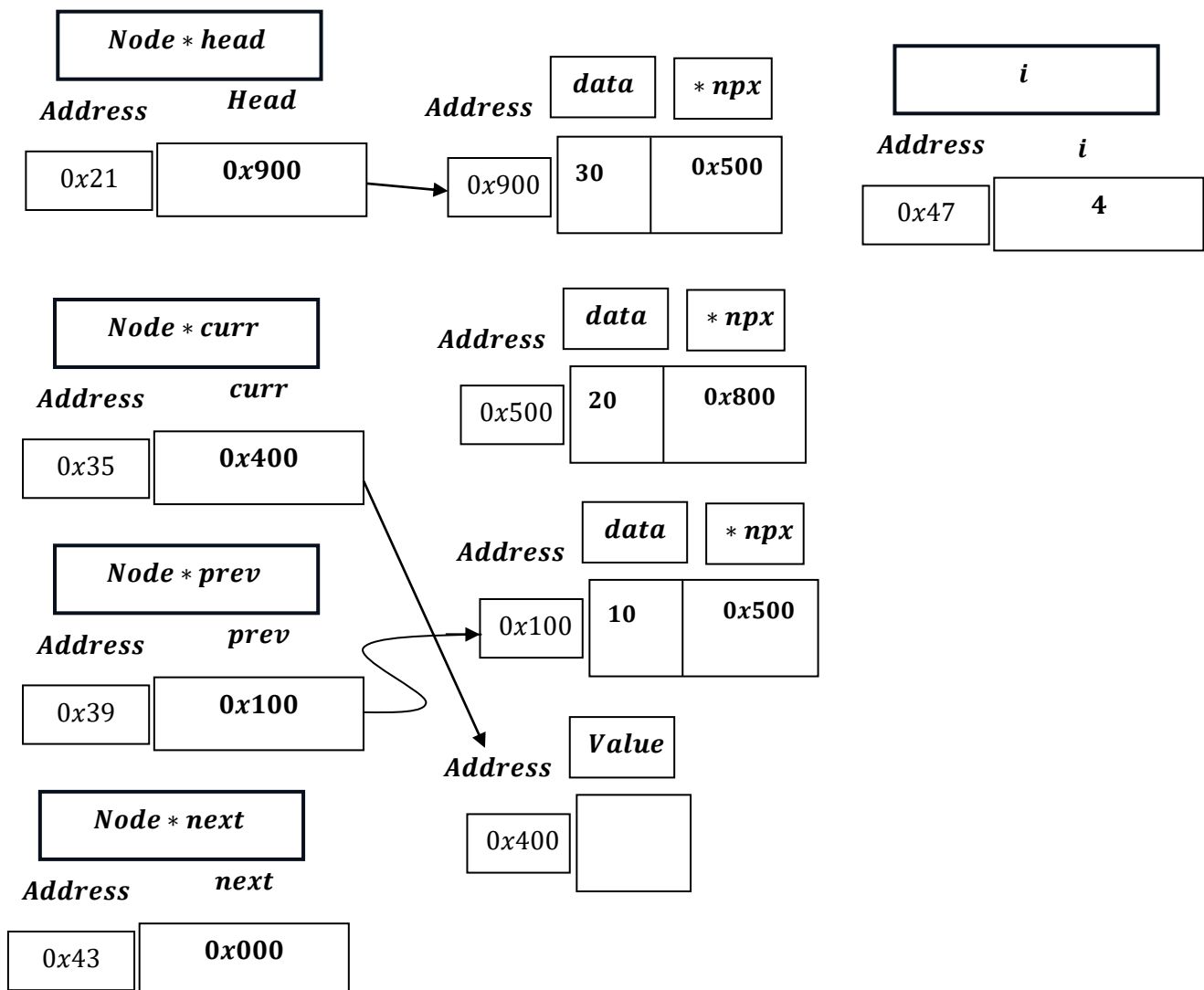
```

if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(prev, nextNext);
}

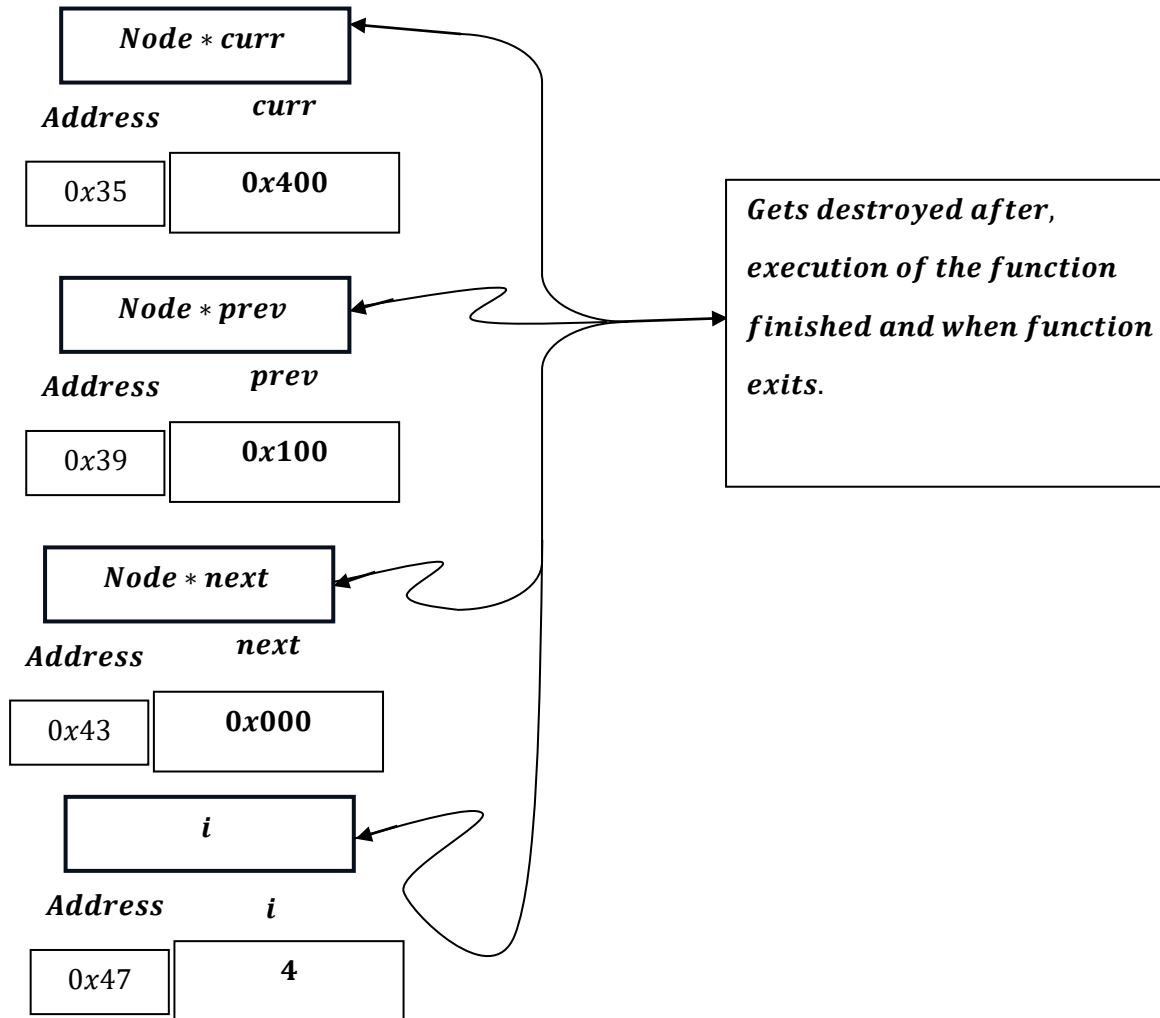
```

if(next:0x000 ≠ nullptr:0x000) → is false.

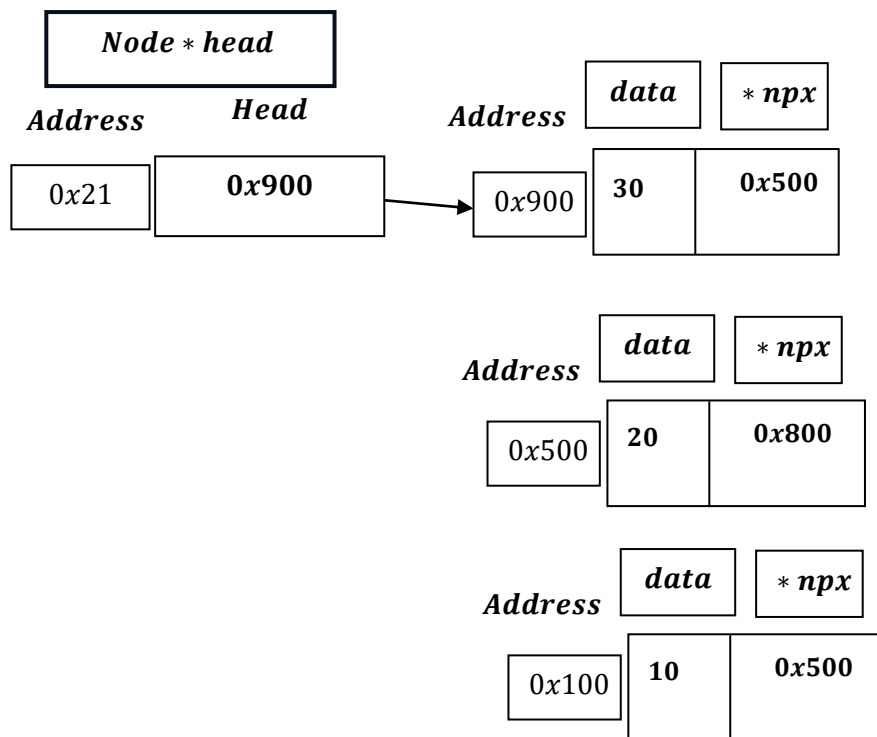
```
free(curr);
```



As , functions ends:



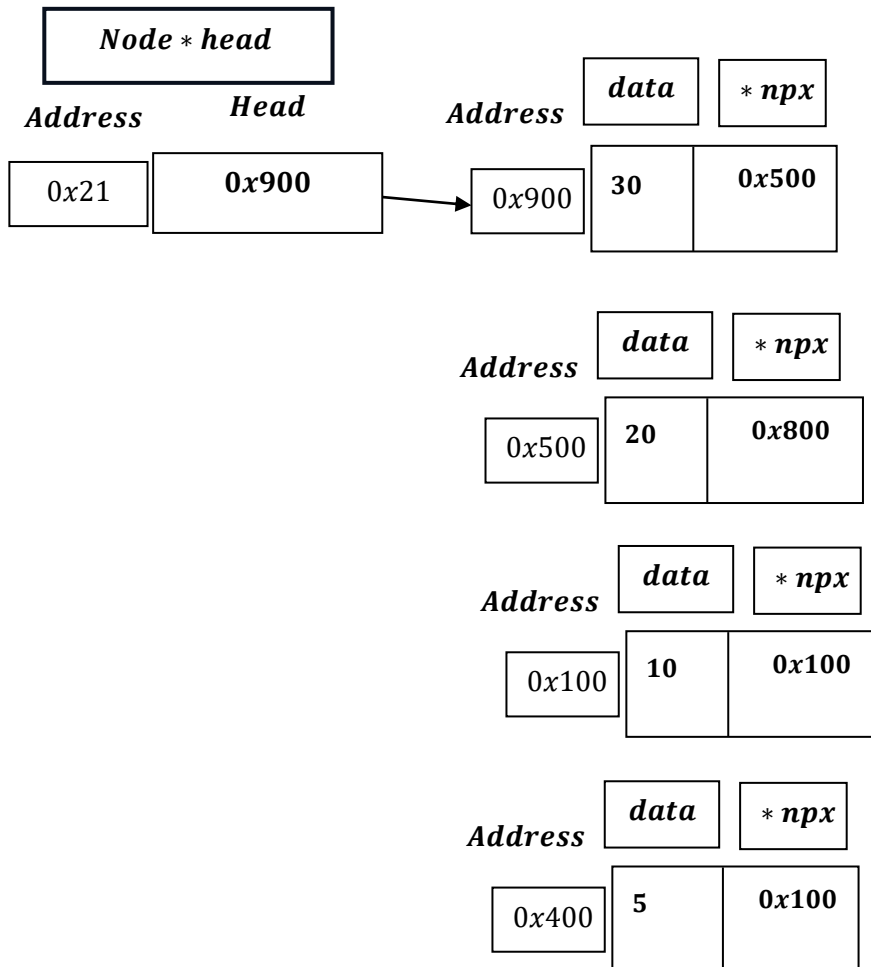
And we are left with:



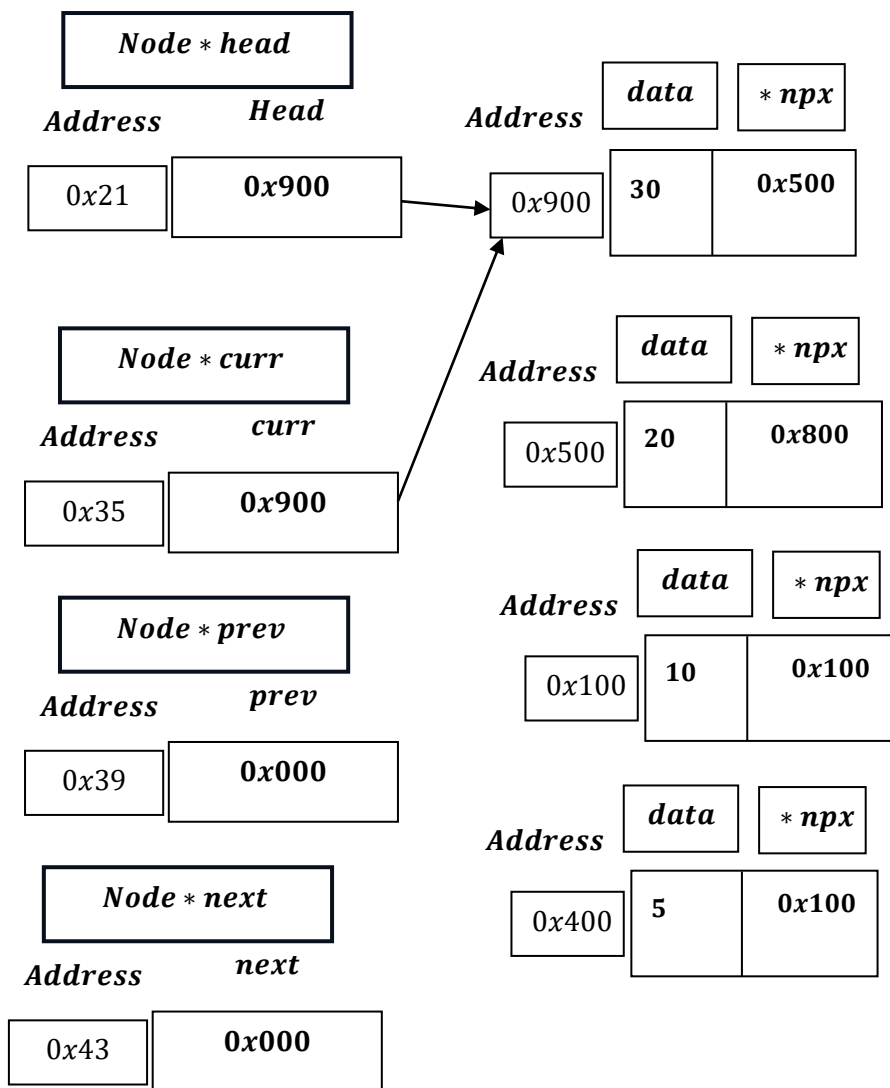
Memory Efficient Doubly Linked List – Delete At Position

–Program Analysis – Part 3 and Time Complexity

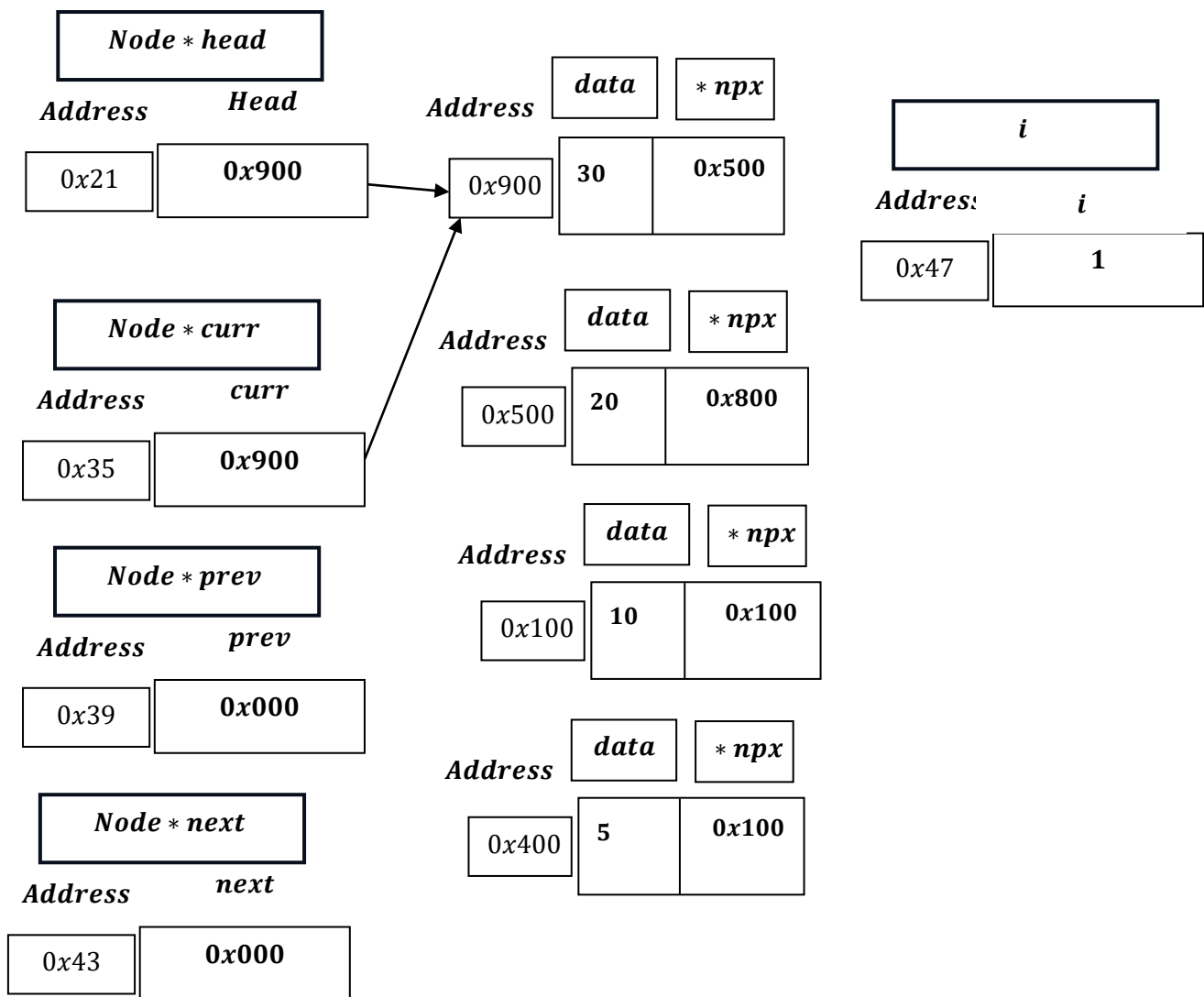
Position not Found.



```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;
```



```
int i = 1;
```



```
while (curr != nullptr && i < pos)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

pos: 5.

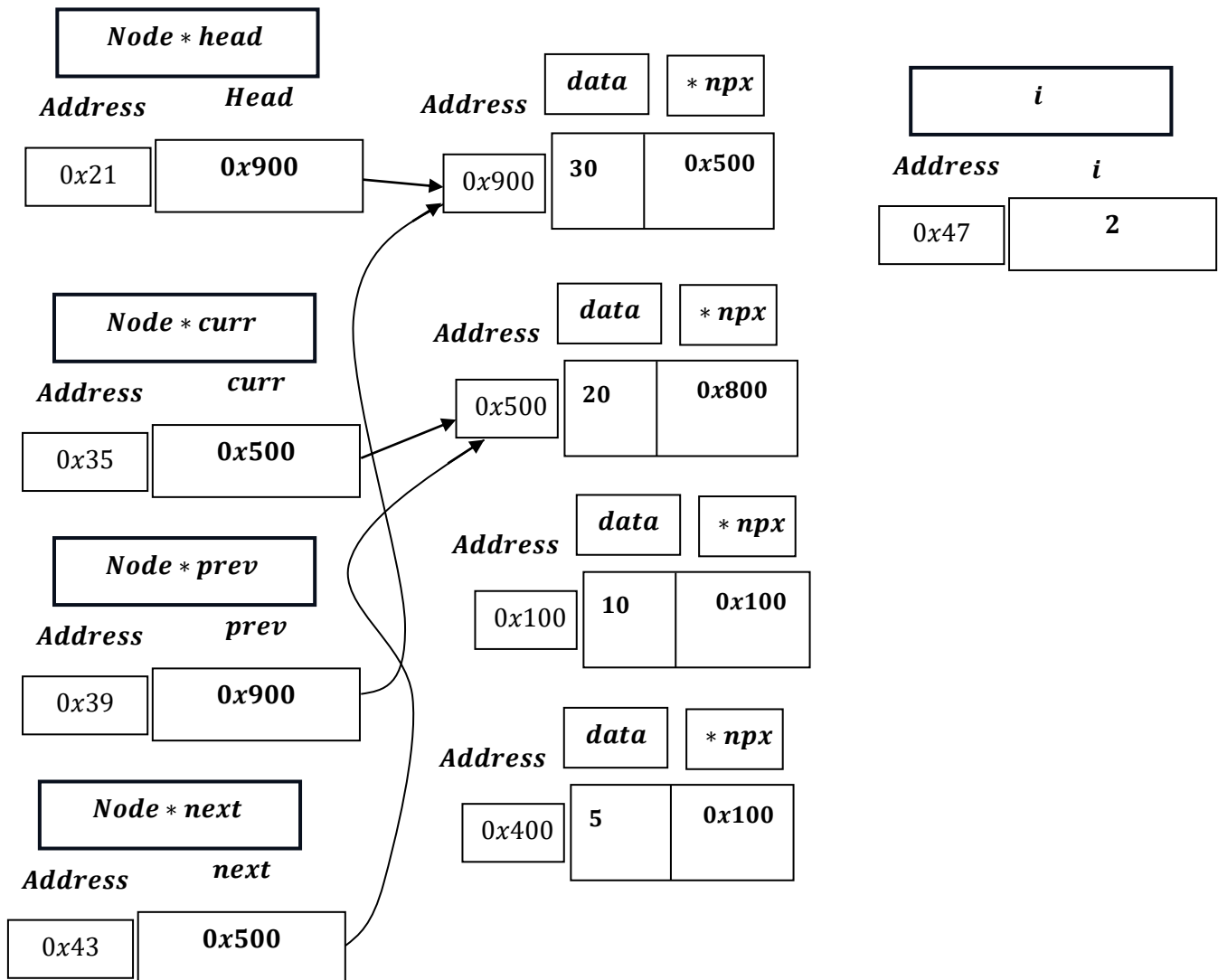
***curr: 0x900* $\neq$ *nullptr: 0x000* AND *i = 1* < *pos: 5* :**

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x000 $\oplus$ 0x500 = 0x500.

prev = curr = 0x900

curr = next = 0x500

i = i + 1 = 1 + 1 = 2.



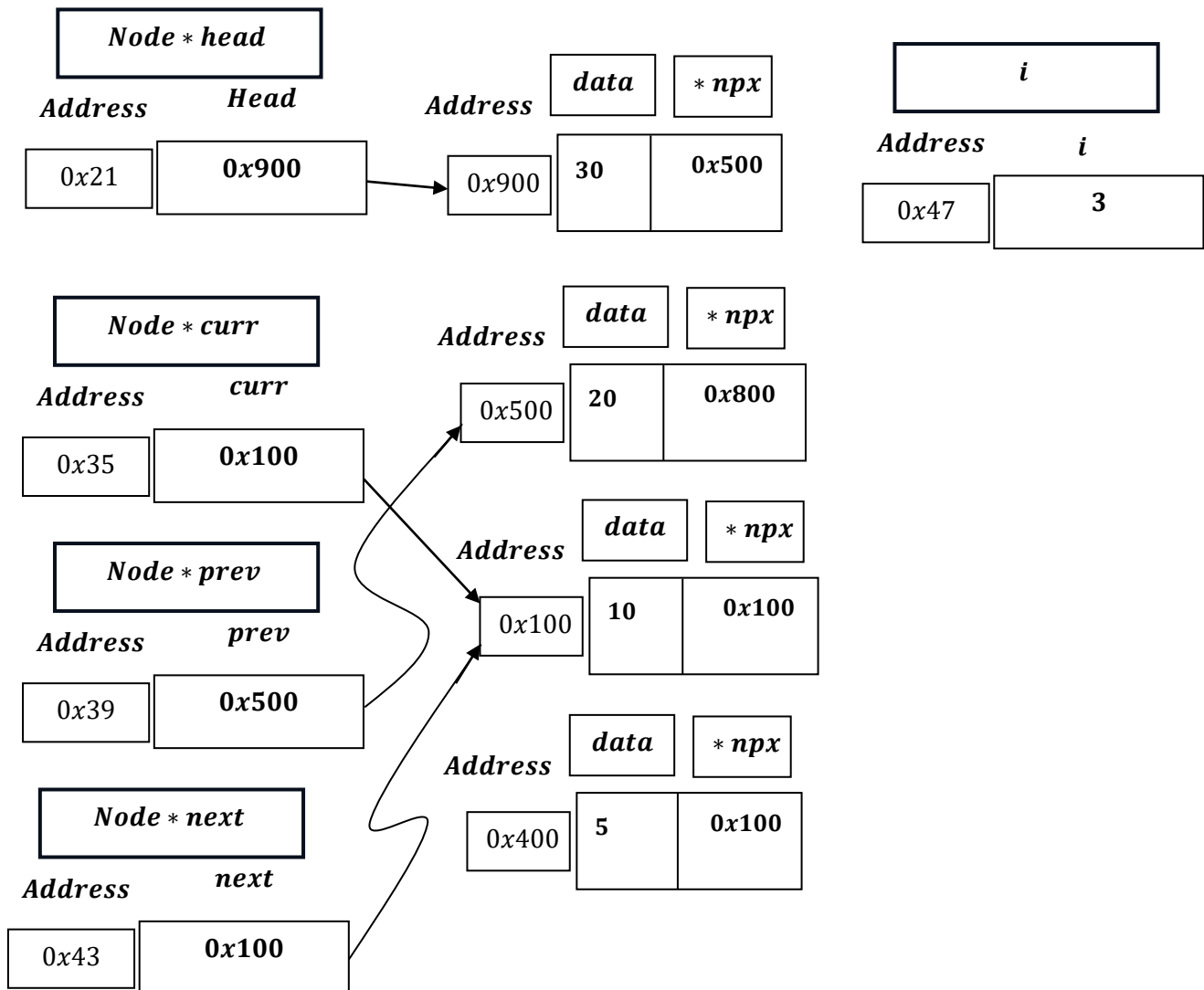
***curr*: 0x500 $\neq$ *nullptr*: 0x000 AND *i* = 2 < *pos*: 5 :**

***next* = $XOR(prev: 0x900, curr \rightarrow npx: 0x800)$ = $1001_2 \oplus 1000_2 = 0001_2 = 0x100$**

***prev* = *curr* = 0x500**

***curr* = *next* = 0x100**

***i* = *i* + 1 = 2 + 1 = 3.**



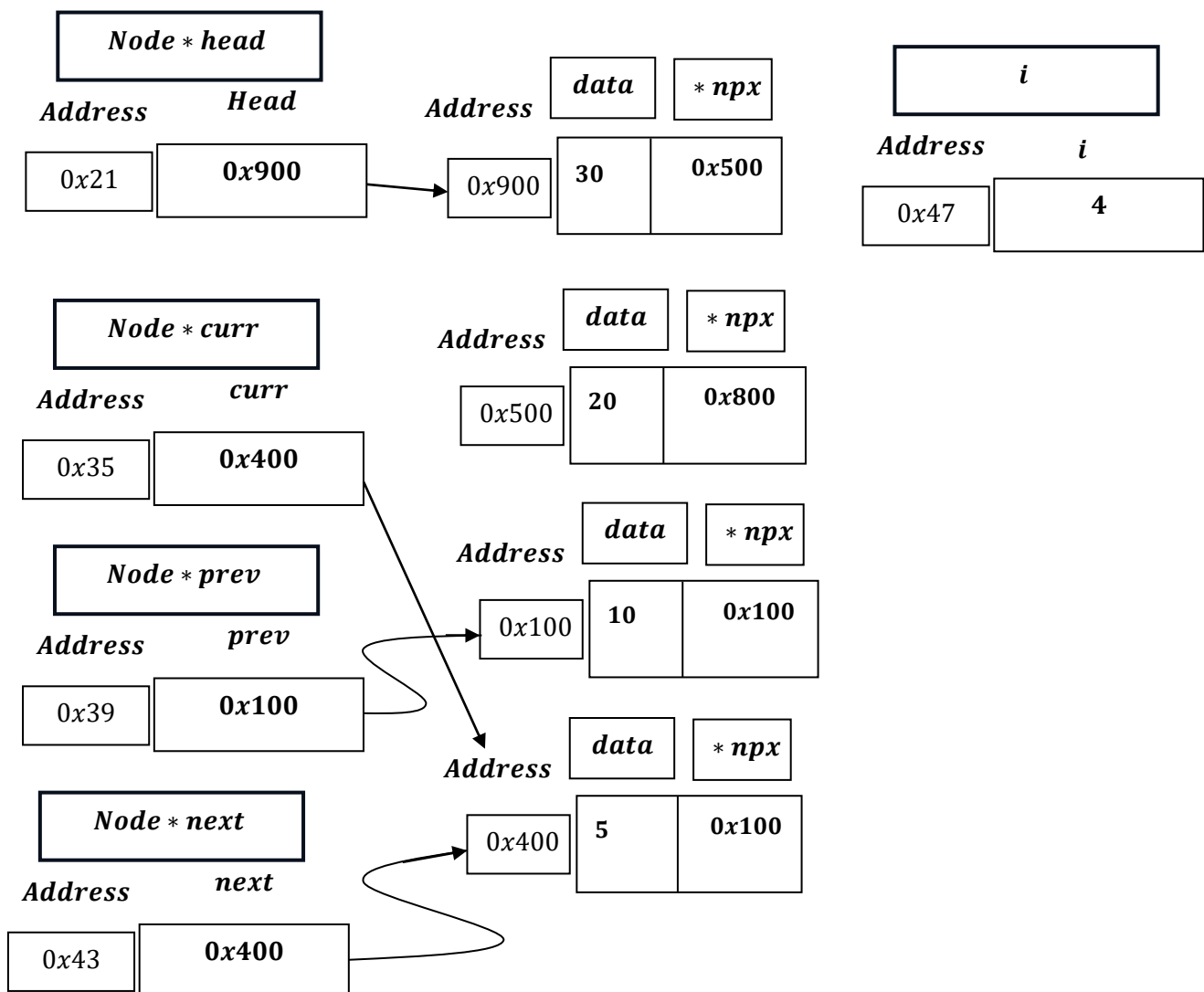
$curr: 0x100 \neq nullptr: 0x000$ AND $i = 3 < pos: 5$:

$next = XOR(prev: 0x500, curr \rightarrow npx: 0x100) = 0101_2 \oplus 0001_2 = 0100_2 = 0x400$

$prev = curr = 0x100$

$curr = next = 0x400$

$i = i + 1 = 3 + 1 = 4.$



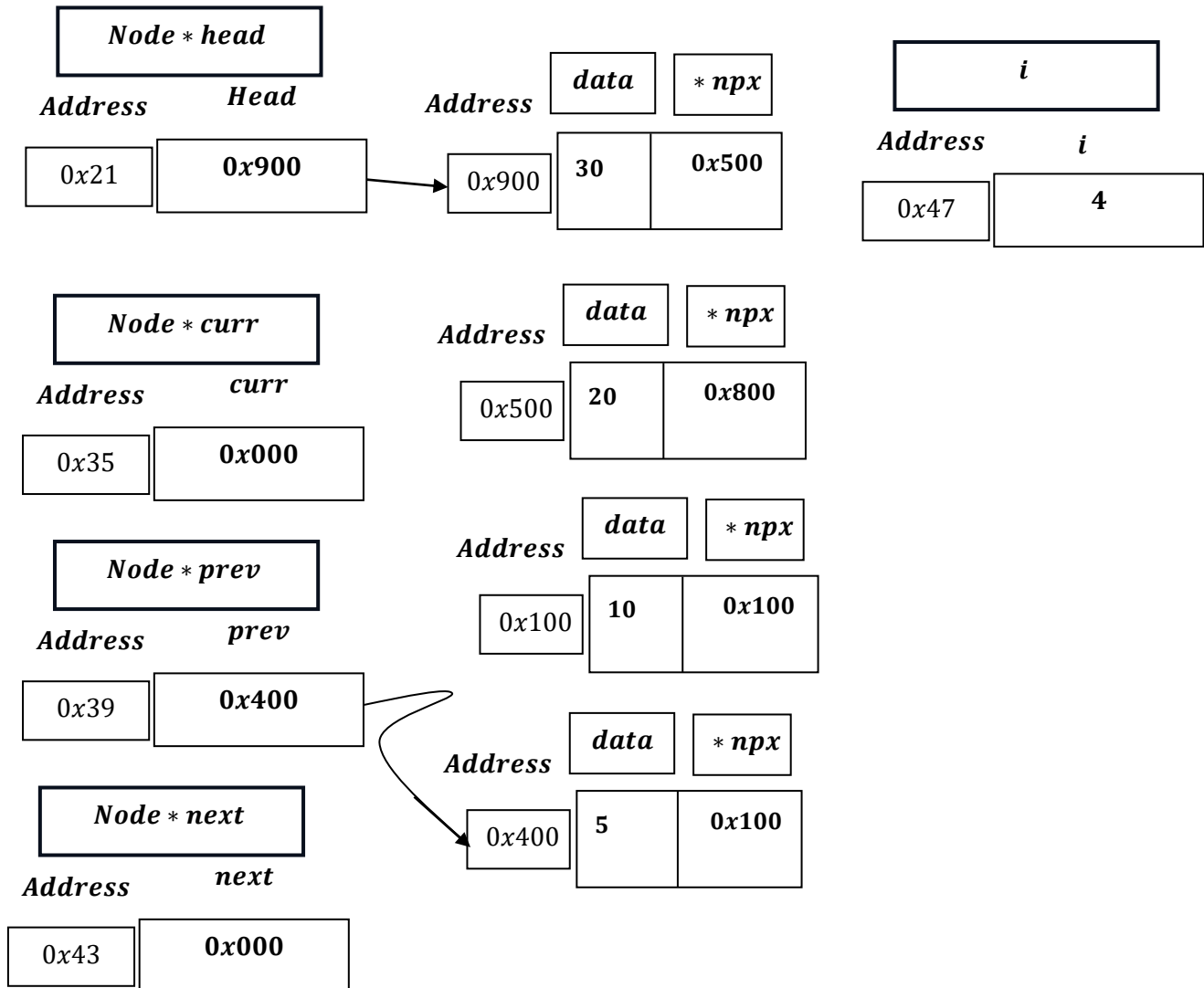
curr: 0x400 $\neq$ nullptr: 0x000 AND $i = 4 < pos: 5$:

next = XOR(prev: 0x100, curr $\rightarrow$ npx: 0x100) = $0001_2 \oplus 0001_2 = 0000_2 = 0x000$

prev = curr = 0x400

curr = next = 0x000

$i = i + 1 = 4 + 1 = 5$.



curr: 0x000 $\neq$ nullptr: 0x000 [False] AND $i = 5 < pos: 5$ [False] $\rightarrow$ Condition fails and loop exits.

```
if (curr == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

curr: 0x000 = nullptr: 0x000:

Print: ``Position out of bounds!``

Return Void and exit from function.

Time Complexity of Delete At Position

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    // Traverse to the node at 'pos'
    while (curr != nullptr && i < pos)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }
}
```

$O(1)$

$O(1)$

$O(1)$

**Run `n`
time in
worst time
complexity.**

$O(1)$

```

next = XOR(prev, curr->npx);
Node *prevPrev = XOR(prev->npx, curr);
prev->npx = XOR(prevPrev, next);
}

if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(prev, nextNext);
}

cout << "Deleted " << curr->data << " from position " <<
pos << "." << endl;
free(curr);
}

```

Complexity analysis for the code above:

- The first three lines of the first block are grouped together and labeled $O(1)$.
- The second block (the `if` statement) is grouped together and labeled $O(1)$.
- The `cout` and `free` statements are grouped together and labeled $O(1)$.

Best Case(Absolute Best Case – When List is Empty)

**List is Empty i. e. when head = nullptr, therefore ,
time complexity is : $\Omega(1)$.**

Worst Time Complexity(Absolute Worst Case = `Pos` is not found).

**while (curr != nullptr && i < pos) → runs $n + 1$ times when
`pos` is not found:**

next = XOR(prev, curr → npx); –runs `n` times.

prev = curr; –runs `n` times.

curr = next; –runs `n` times.

i = i + 1; –runs `n` times.

As we put forward the inner statements of the loop to calculate for worst case complexity :

$$4n = 4 \times O(n) = O(n).$$

Average Case

Deleted position from 2 to N, while loop runs from 1 to N – 1.

And when the position is uniformly distributed from 1 to N:

(Average Case i. e. Undetermined Position)

→ If the position is 1, the loop's inner statement runs 0 time.

→ If the position is 2, the loop's inner statement runs 1 time.

→ If the position is 3, the loop's inner statement runs 2 time.

.

.

→ If the position is (N), the loop's inner statement runs (N – 1) times.

As when $i = N < N$ becomes false, hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$$

Now, as we take all the possibilities then, we take the best case also i. e.

$O(1)$, when position = 1. Considering $T(P)$ is time complexity of position P .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ P - 1 & , \text{when } P = 2, 3, 4, \dots, N. \end{cases}$$

Now, If there is `N` no. of nodes there we can delete data at N positions.

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of deletion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N} \times 1 + \frac{1}{N} \times 2 + \dots + \frac{1}{N} \times N - 1 \right) + \frac{1}{N} \times 1$$

$$= \frac{1}{N} \times \{(1 + 2 + 3 + \dots + N - 1) + 1\}$$

$$= \frac{1}{N} \times \left\{ \sum_{i=1}^{N-1} i + 1 \right\}$$

$$= \frac{1}{N} \times \left(\frac{N(N-1)}{2} + 1 \right)$$

$$= \frac{N-1}{2} + \frac{1}{N}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N-1}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

Now, taking out the dominant term of numerator and denominator:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant ' $\frac{1}{2}$ ', we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms , largest growth rate is the result:

→ $\Theta(N)$ grows linearly with ' N ' .

→ $\Theta\left(\frac{1}{N}\right)$ decreases to a constant , i. e. when N grows larger , $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant} \right].$$

As $\Theta(N)$'s growth rate is higher , therefore , average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} + \frac{1}{N} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Approach]:
(Simplest approach)

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

**.
.**

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$, for position 1 to N.

Average case complexity : Probability of deletion $\times$ $\left(\begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N} \times \left(\sum_{i=1}^N i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$\text{or, } \frac{N}{2} + \frac{1}{2}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

Now removing the constant '\(\frac{1}{2}\)', we get:

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N+1}{2}\right)$ or $\Theta(N)$.

Now,

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Delete At Position</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

Memory Efficient Doubly Linked List

–Delete At Position

–Program Analysis and Time Complexity

Program:

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    // Traverse to the node at 'pos'
    while (curr != nullptr && i < pos)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }
```

```

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    // We are deleting 'curr'.
    // Logic: Connect 'prev' directly to 'next'.

    next = XOR(prev, curr->npx); // Decode node after curr

    // 1. Update prev
    // prev->npx was (prevPrev ^ curr). Needs to be
    (prevPrev ^ next).
    Node *prevPrev = XOR(prev->npx, curr);
    prev->npx = XOR(prevPrev, next);

    // 2. Update next (if exists)
    // next->npx was (curr ^ nextNext). Needs to be (prev ^
    nextNext).
    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(prev, nextNext);
    }

    cout << "Deleted " << curr->data << " from position " <<
pos << "." << endl;
    free(curr);
}

.....

    case 7:
        cout << "Enter position to delete: ";
        cin >> pos;
        deleteFromPosition(pos);
        break;

```

Program Analysis

1. When List Is Empty

```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

Node * head

Address

Head

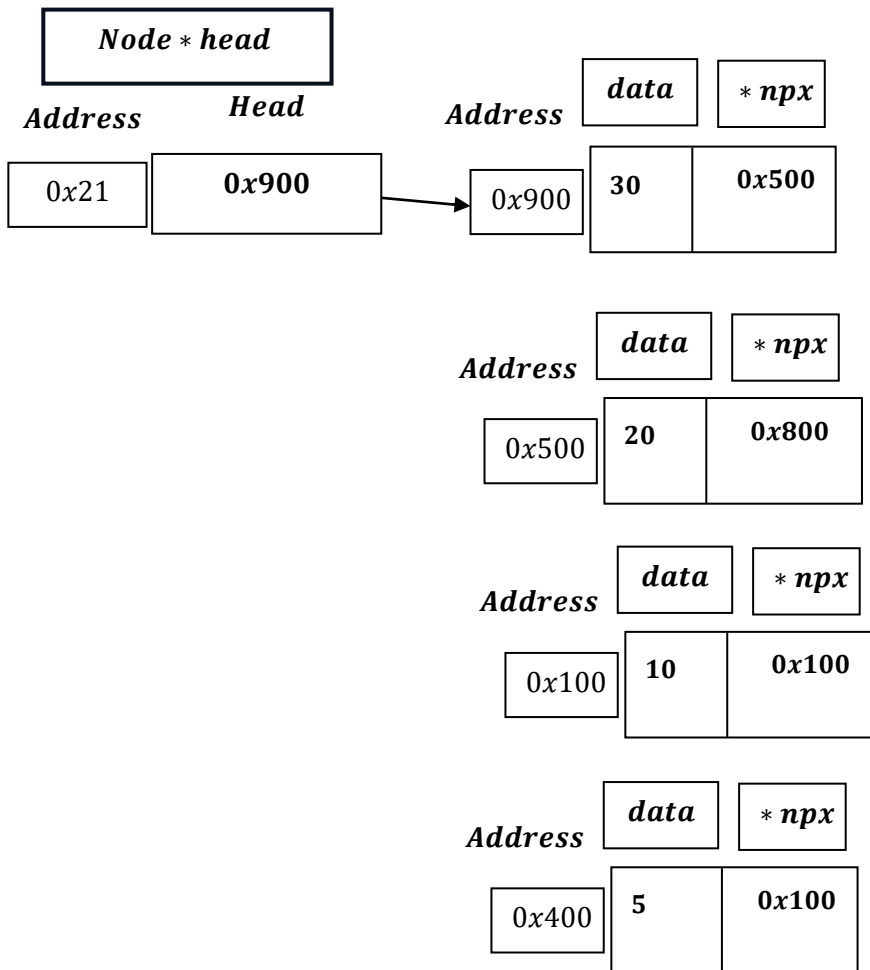
0x21

nullptr

Then , it prints : "List is empty.Nothing to delete."

Return Void and Exit.

2. When List Is Not Empty



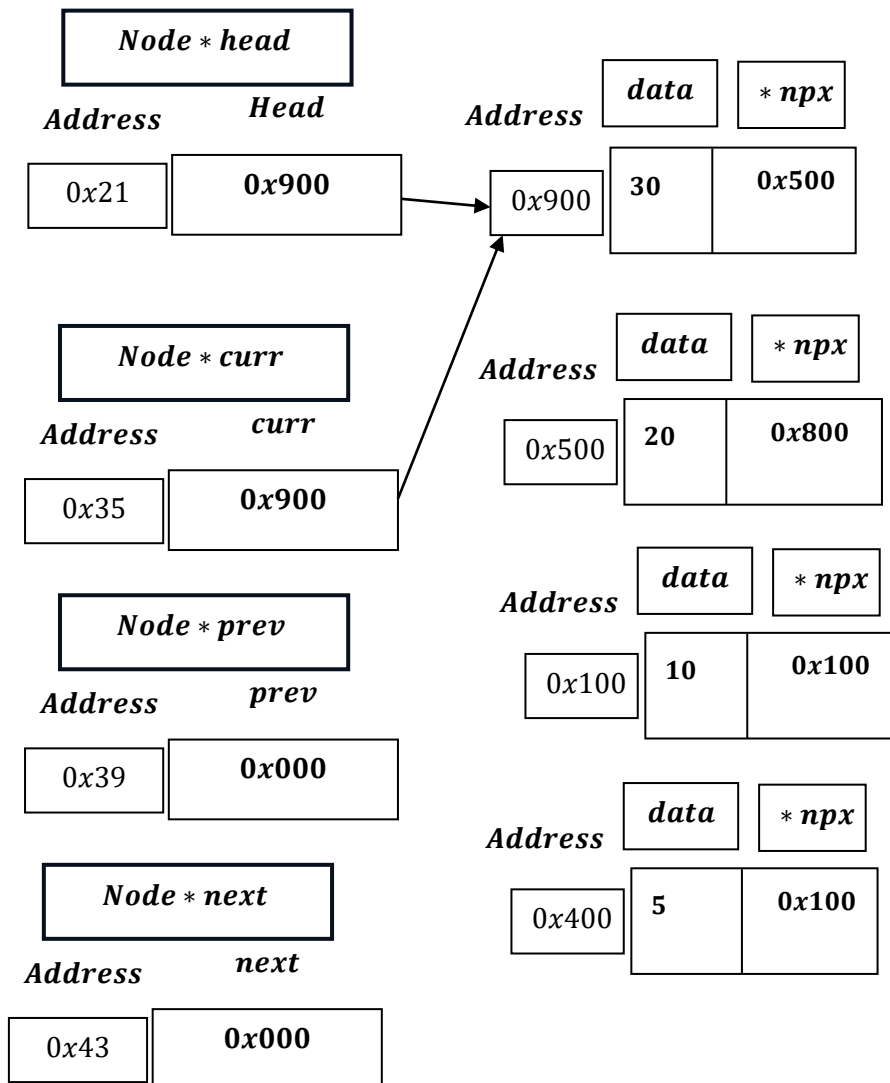
```
cout << "Enter position to delete: ";  
cin >> pos;  
deleteFromPosition(pos);  
break;
```

if pos = 1, then:

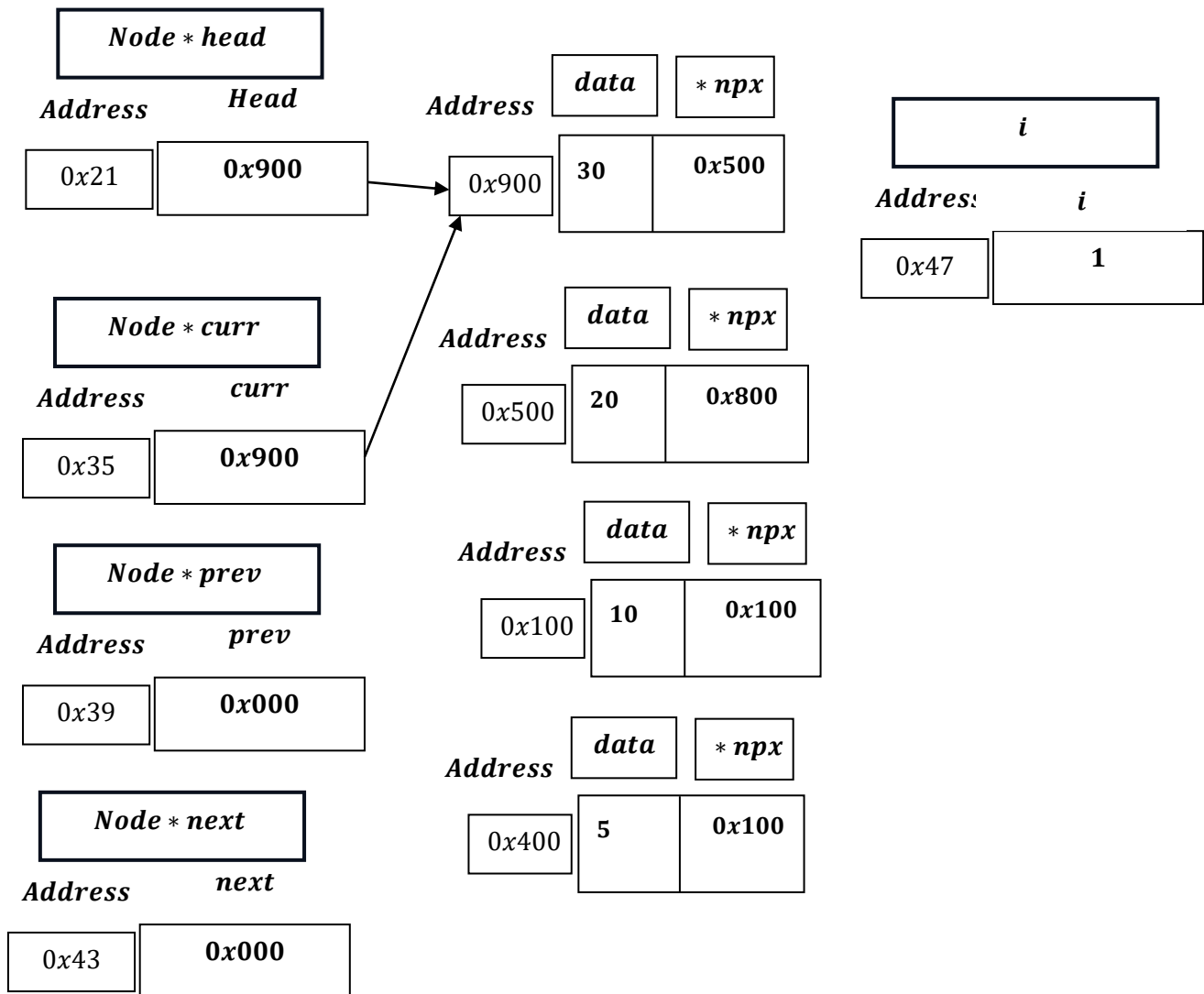
```
if (pos == 1)
{
    deleteFromBeginning();
    return;
}
```

Then it runs deleteFromBeginning() and return statement exit from the function.

```
Node *curr = head;
Node *prev = nullptr;
Node *next;
```



```
int i = 1;
```



```
while (curr != nullptr && i < pos)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

Lets say pos: 3.

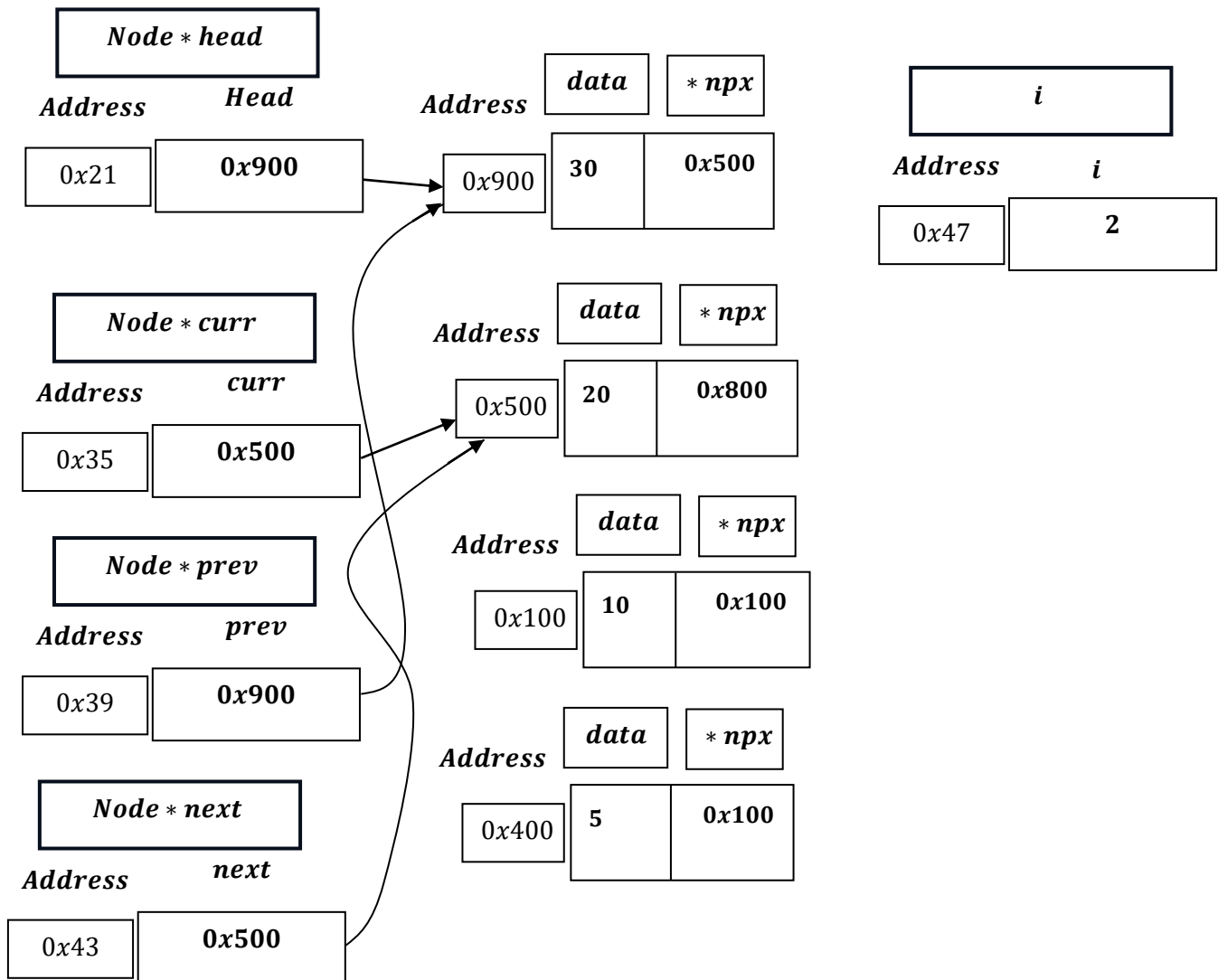
***curr: 0x900* $\neq$ *nullptr: 0x000* AND *i = 1* < *pos: 3* :**

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x000 $\oplus$ 0x500 = 0x500.

prev = curr = 0x900

curr = next = 0x500

i = i + 1 = 1 + 1 = 2.



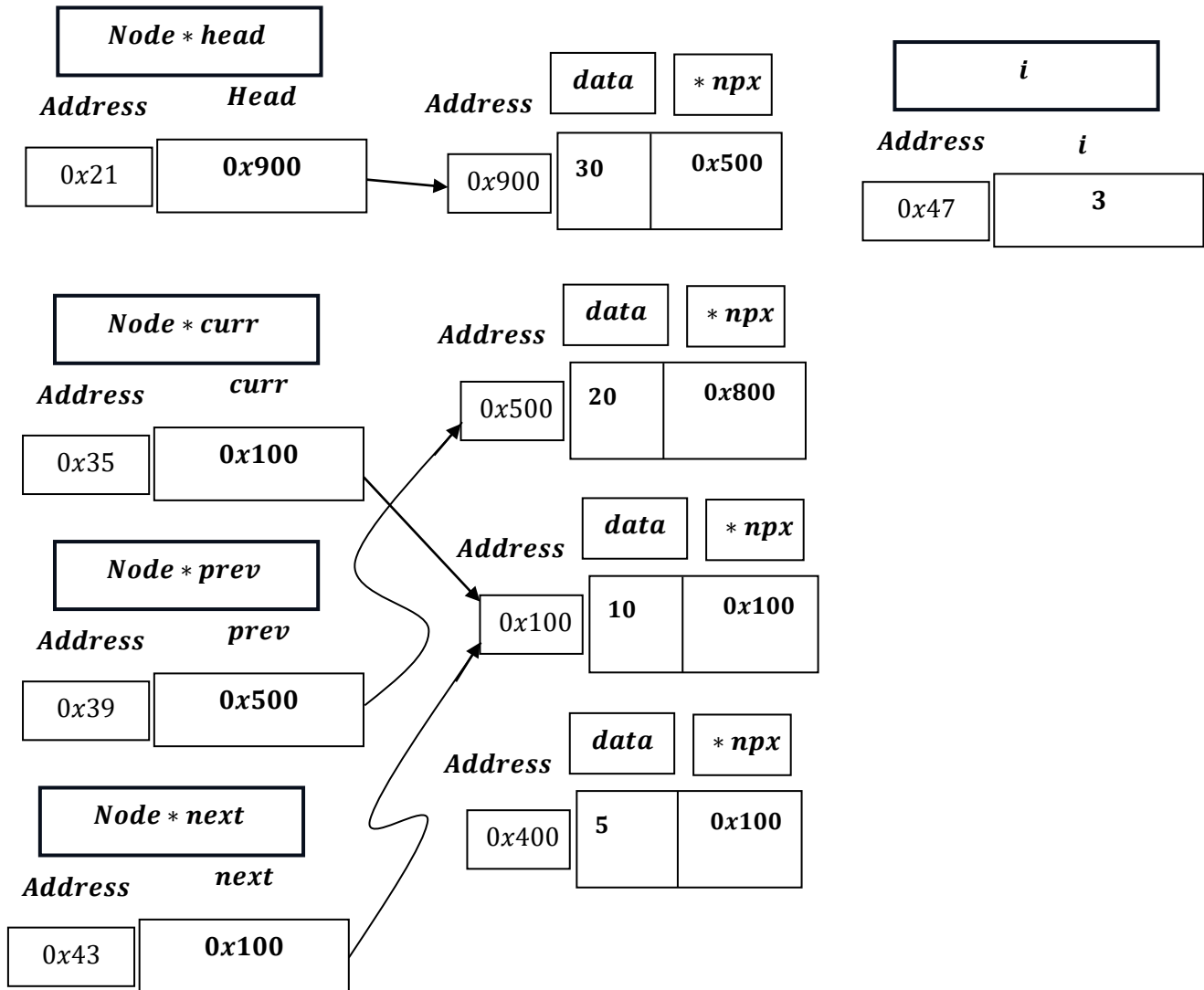
curr: 0x500 $\neq$ nullptr: 0x000 AND $i = 2 < pos: 3$:

next = XOR(prev: 0x900, curr $\rightarrow$ npx: 0x800) = $1001_2 \oplus 1000_2 = 0001_2 = 0x100$

prev = curr = 0x500

curr = next = 0x100

$i = i + 1 = 2 + 1 = 3$.

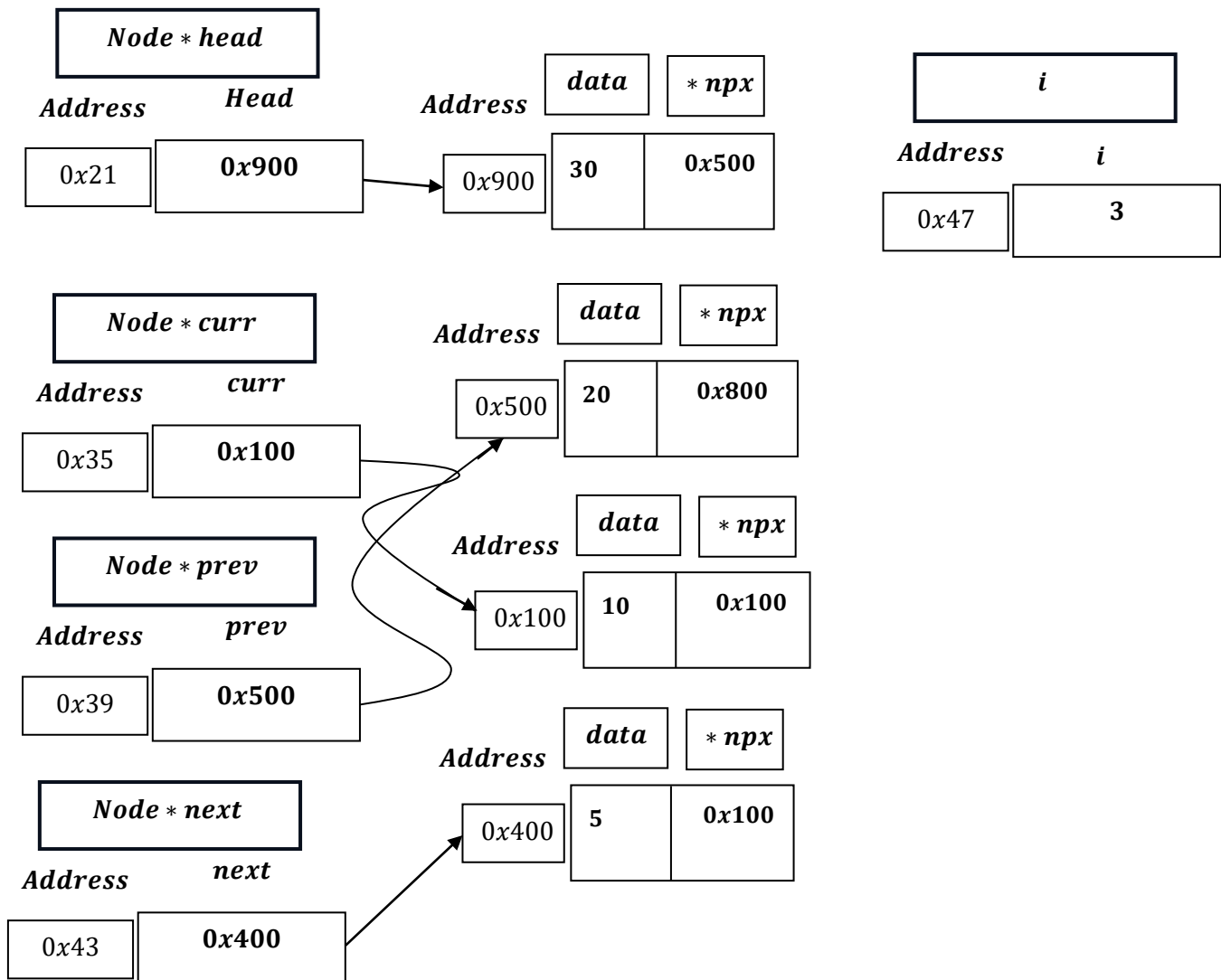


***curr: 0x100 $\neq$ nullptr: 0x000 [is TRUE] AND $i = 3 < pos: 3$ [is False]
 $\Rightarrow$ Condition is False.***

```
next = XOR(prev, curr->npx);
```

$next = XOR(prev: 0x500 \oplus curr \rightarrow npx: 0x100);$

$0101_2 \oplus 0001_2 = 0100_2 = 4_{10} = 0x400.$



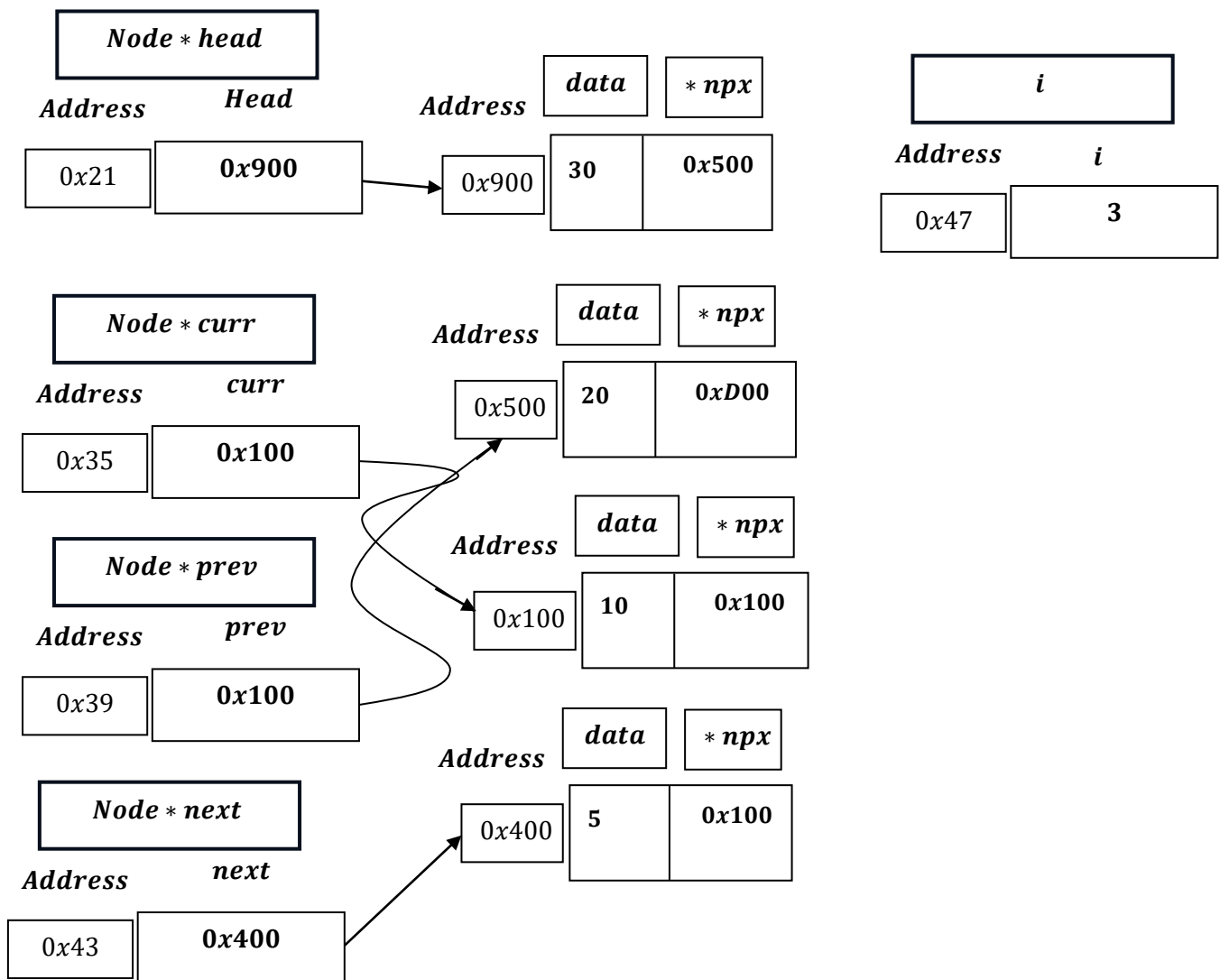
```
Node *prevPrev = XOR(prev->npx, curr);
prev->npx = XOR(prevPrev, next);
```

$prevPrev = XOR(prev \rightarrow npx: 0x800, curr: 0x100)$

$= 1000_2 \oplus 0001_2 = 1001_2 = 9_{10} = 0x900$

$prev \rightarrow npx = XOR(prevPrev: 0x900, next: 0x400)$

$= 1001_2 \oplus 0100_2 = 1101_2 = 13_{10} = 0xD00$




```

if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(prev, nextNext);
}

```

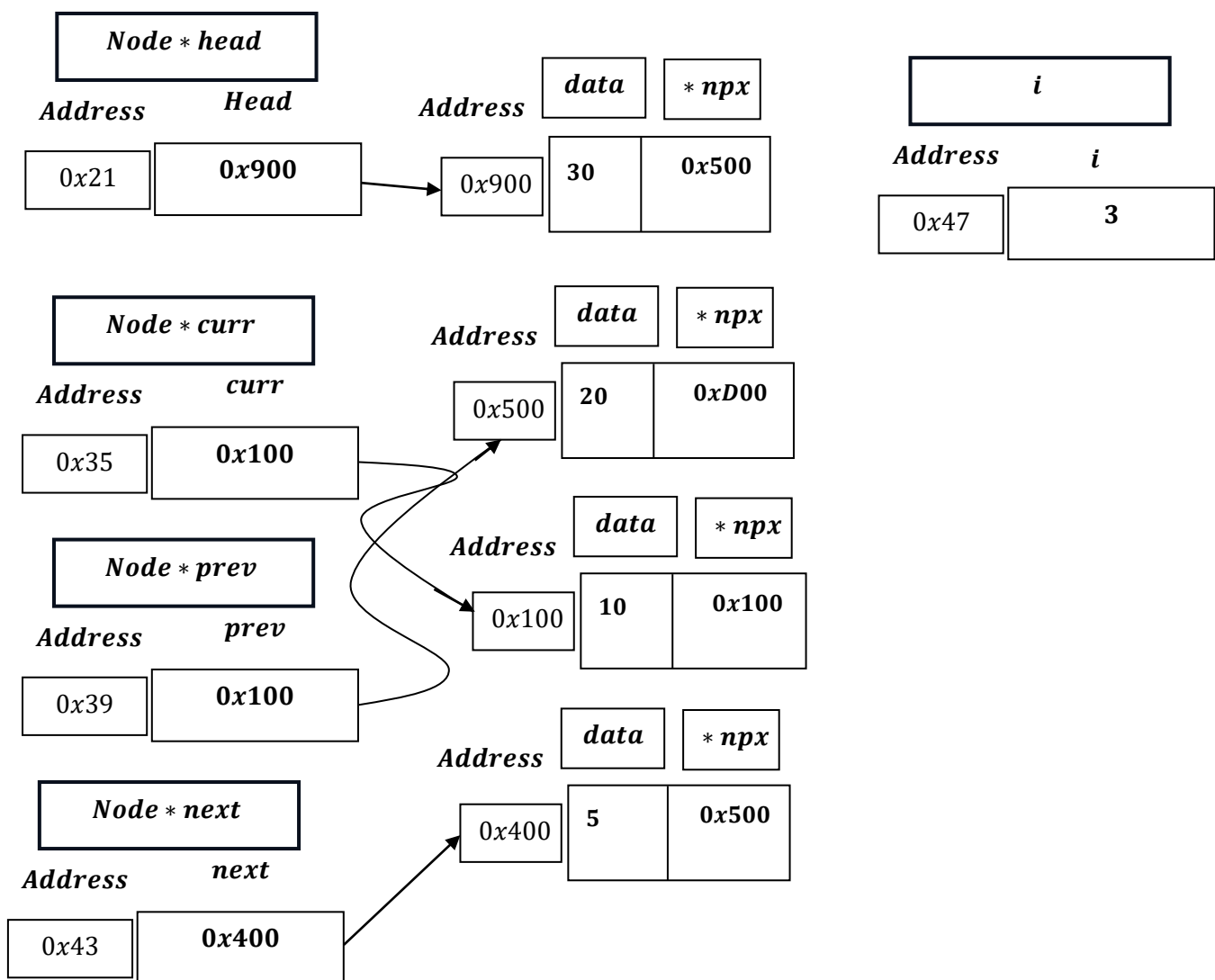
***next: 0x400* $\neq$ *nullptr: 0x000* :**

***nextNext* = *XOR*(*curr: 0x100*, *next* $\rightarrow$ *npx: 0x100*)**

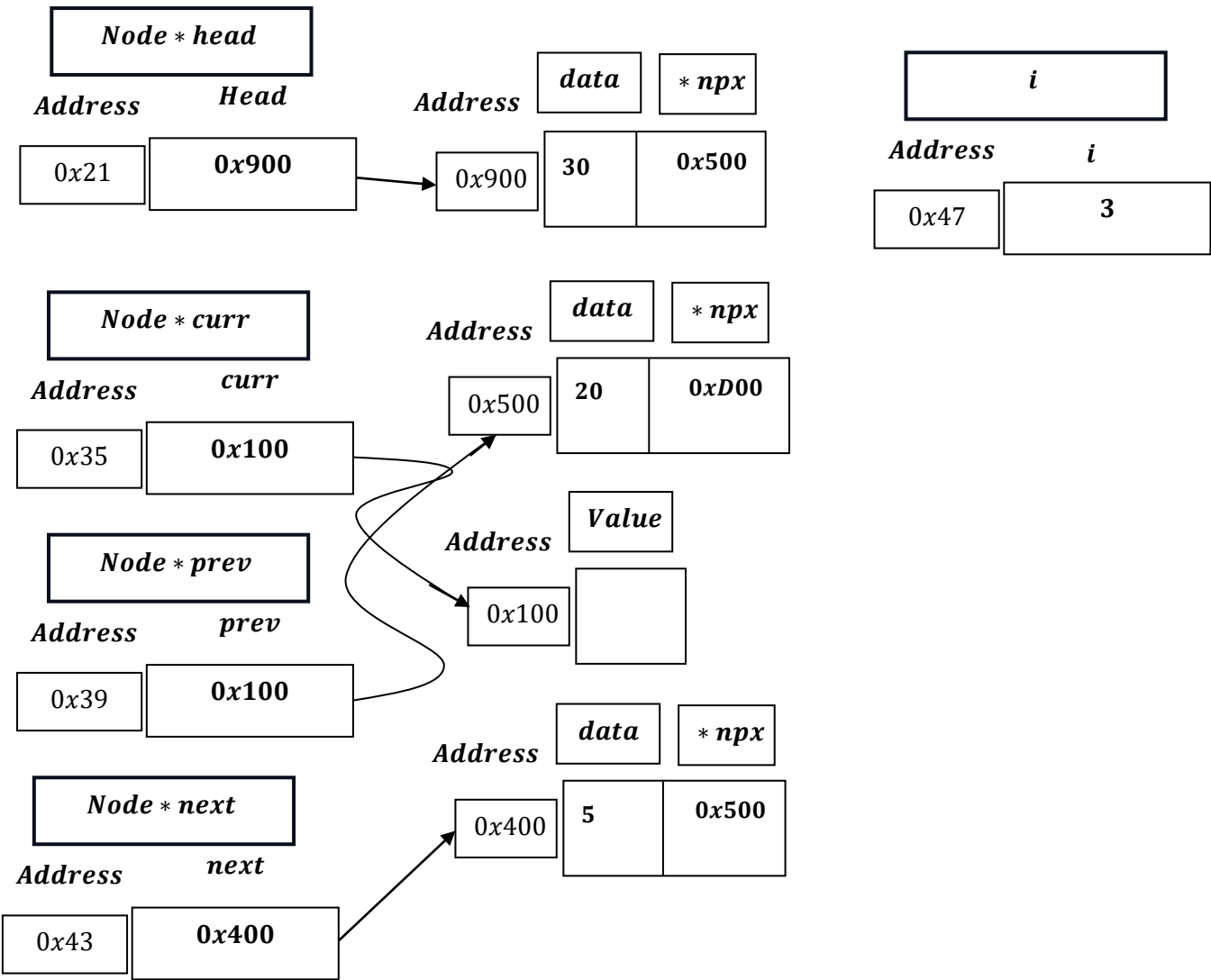
$= 0001_2 \oplus 0001_2 = 0000_2 = 0x000$.

***next* $\rightarrow$ *npx* = *XOR*(*prev: 0x500*, *nextNext: 0x000*)**

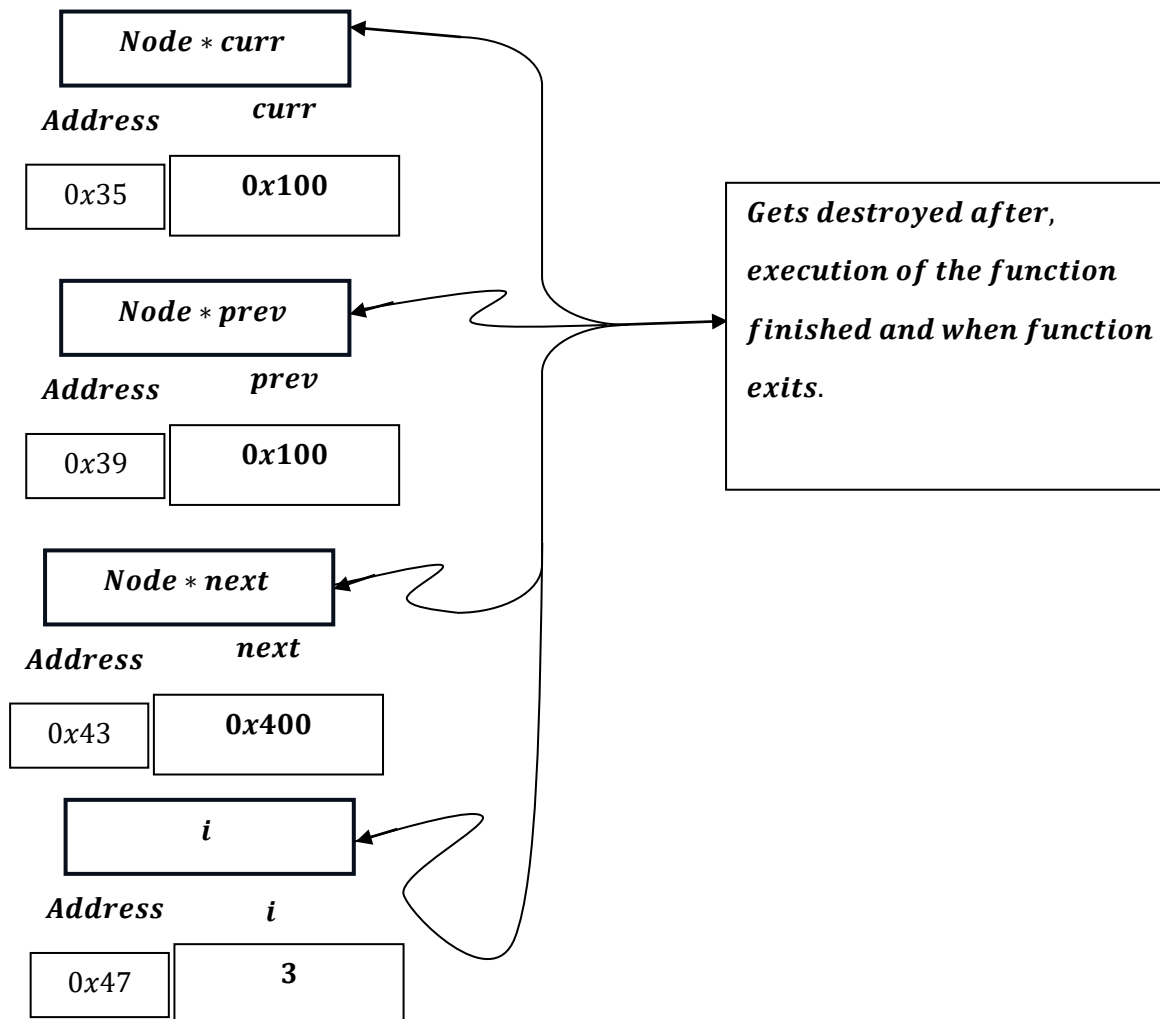
$= 0101_2 \oplus 0000_2 = 0101_2 = 5_{10} = 0x500$



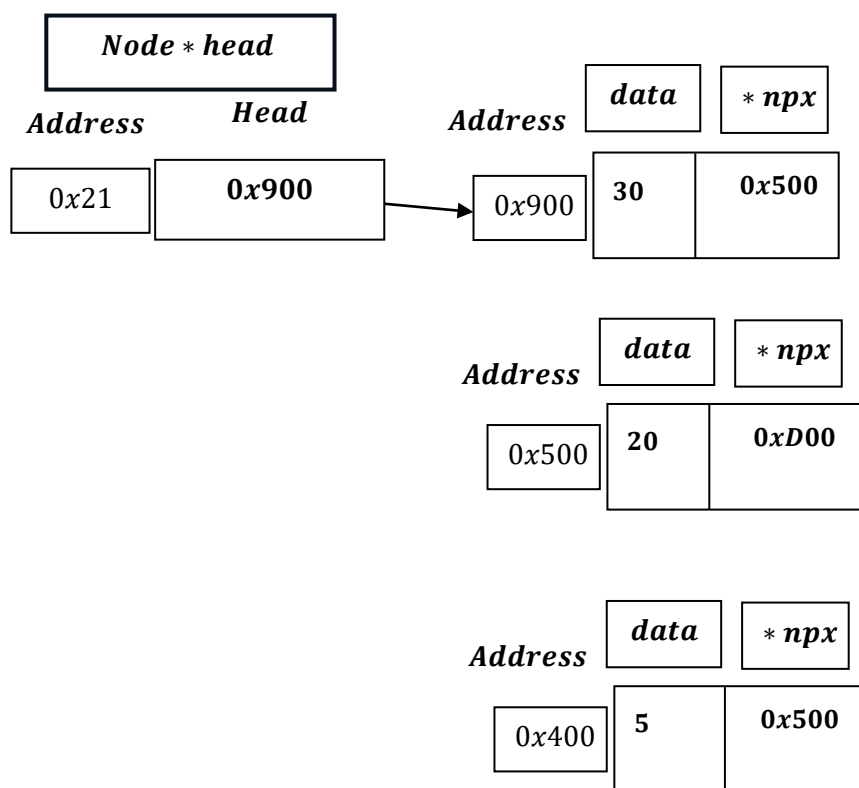
```
free(curr);
```



As , functions ends:



And we are left with:



Memory Efficient Doubly Linked List – Delete At Position
–Space Complexity Analysis and Algorithm

Program:

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    // Traverse to the node at 'pos'
    while (curr != nullptr && i < pos)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }
}
```

```

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    // We are deleting 'curr'.
    // Logic: Connect 'prev' directly to 'next'.

    next = XOR(prev, curr->npx); // Decode node after curr

    // 1. Update prev
    // prev->npx was (prevPrev ^ curr). Needs to be
    (prevPrev ^ next).
    Node *prevPrev = XOR(prev->npx, curr);
    prev->npx = XOR(prevPrev, next);

    // 2. Update next (if exists)
    // next->npx was (curr ^ nextNext). Needs to be (prev ^
    nextNext).
    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(prev, nextNext);
    }

    cout << "Deleted " << curr->data << " from position " <<
pos << "." << endl;
    free(curr);
}

.....

    case 7:
        cout << "Enter position to delete: ";
        cin >> pos;
        deleteFromPosition(pos);
        break;

```

Space Complexity Analysis: XOR Linked List (Delete From Position)

1. Input Space Complexity

- **Parameter – only Definition:**
 - **Function Parameter:** *int pos.*
 - **Analysis:** *The pos parameter is a single integer. It occupies a constant amount of space ($O(1)$) regardless of the size of the list or the value of the position.*
 - **Complexity:** **$O(1)$**
- **Full Data Definition:**
 - **Function Data:** *The parameter pos + the existing XOR linked list.*
 - **Analysis:** *The function operates on a data structure with n nodes. To perform the deletion, the function must be "aware" of the list context.*
 - **Complexity:** **$O(n)$**

2. Auxiliary Space Complexity

Definition: *This measures the temporary or extra memory used by the algorithm during its execution.*

- **Local Pointer Variables:** *The function declares several local pointers: curr, prev, next, prevPrev, and nextNext.*
 - **Analysis:** *These are all stored on the stack. On a 64 – bit system, each takes 8 bytes. The number of these pointers is fixed and does not increase even if the list size (n) increases or if pos is very large.*
- **Local Counter:** *The integer i used for counting nodes during traversal takes constant space.*
- **Traversal:** *The traversal is iterative (using a while loop). It does not use recursion, meaning no extra frames are added to the call stack as the list grows.*
- **XOR Operations:** *The bitwise calculations happen within CPU registers, requiring no extra memory allocation.*
- **Memory Release:** *free(curr) returns memory to the system. While this reduces total memory usage, the auxiliary space used to perform the logic remains constant.*

Conclusion: *The amount of extra memory used is fixed and does not grow as the size of the linked list (n) grows.*

- **Auxiliary Space Complexity: $O(1)$**

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Delete From Position (XOR Doubly Linked List)

XORDELPOS(START, POS):

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete."

Return and EXIT.

[End of If structure]

2. [DELETE FROM BEGINNING?]

If POS = 1, then:

Call XORDELBEG(START)

Translating XORDELBEG(START):

XORDELBEG(START):

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete."

Return and EXIT.

[End of If structure]

2. [INITIALIZE TEMPORARY POINTER]

Set TEMPPTR := START $\left(\begin{array}{l} \text{Store the current head to free} \\ \text{its memory later} \end{array} \right)$

3. [DECODE THE SECOND NODE]

Set NEXTTEMPPTR := NULL $\oplus$ NEXTXORPTR[START] $\left(\begin{array}{l} \text{Calculate the} \\ \text{address of the node} \\ \text{that will become} \\ \text{the new head} \end{array} \right)$

4. [UPDATE NEW HEAD'S LINKAGE]

If NEXTTEMPPTR $\neq$ NULL, then: $\left(\begin{array}{l} \text{If the list has more} \\ \text{than one node,} \\ \text{we must update} \\ \text{the successor's NPX} \end{array} \right)$

a. Set NEXTNEXTTEMPPTR :
 = TEMPPTR

$\oplus$ NEXTXORPTR[NEXTTEMPPTR]
 $\left(\begin{array}{l} \text{Decode the node following} \\ \text{the second node using} \\ \text{the current head} \\ \text{as the 'previous'} \\ \text{reference} \end{array} \right)$

b. Set NEXTXORPTR[NEXTTEMPPTR]:

= NULL $\oplus$ NEXTNEXTTEMPPTR
 $\left(\begin{array}{l} \text{Update the new head's NPX} \\ \text{to indicate its} \\ \text{previous neighbor} \\ \text{is now NULL} \end{array} \right)$

[End of If structure]

5. [UPDATE GLOBAL START]

Set START := NEXTTEMPPTR
 $\left(\begin{array}{l} \text{The second node officially} \\ \text{becomes the} \\ \text{first node} \\ \text{of the list} \end{array} \right)$

6. [FREE MEMORY AND PRINT]

Write: Deleted ,INFO[TEMPPTR], from the beginning.

FREE(TEMPPTR)
 $\left(\begin{array}{l} \text{Return the memory of the} \\ \text{deleted node to the system} \end{array} \right)$

7. EXIT

Return and **EXIT**.
 [End of If structure]

3. [INITIALIZE TRAVERSAL]

Set CURRPTR := START
 Set PREVPTR := NULL
 Set COUNT := 1

4. [TRAVERSE TO POSITION POS]

Repeat while CURRPTR $\neq$ NULL AND COUNT < POS:

- a. Set NEXTTEMPPTR := PREVPTR $\oplus$ NEXTXORPTR[CURRPTR]*
- b. Set PREVPTR := CURRPTR*
- c. Set CURRPTR := NEXTTEMPPTR*
- d. Set COUNT := COUNT + 1*

[End of Loop]

5. [CHECK BOUNDS]

If CURRPTR = NULL, then:

Write: "Position out of bounds!"

Return and EXIT.

[End of If structure]

6. [DECODE NEIGHBORS]

Set NEXTTEMPPTR :

$$= \text{PREVPTR} \oplus \text{NEXTXORPTR}[\text{CURRPTR}] \left(\begin{array}{c} \text{This decodes the node} \\ \text{following the} \\ \text{one to be deleted} \end{array} \right)$$

7. [UPDATE PREVIOUS NODE]

a. Set PREVPREVTEMPPTR := NEXTXORPTR[PREVPTR] $\oplus$ CURRPTR

b. Set NEXTXORPTR[PREVPTR]:

$$= \text{PREVPREVTEMPPTR}$$

$$\oplus \text{NEXTTEMPPTR} \left(\begin{array}{c} \text{Re - link the previous node to} \\ \text{bypass CURRPTR} \\ \text{and point to NEXTTEMPPTR} \end{array} \right)$$

8. [UPDATE NEXT NODE IF IT EXISTS]

If NEXTTEMPPTR $\neq$ NULL, then:

a. Set NEXTNEXTTEMPPTR := CURRPTR $\oplus$ NEXTXORPTR[NEXTTEMPPTR]

b. Set NEXTXORPTR[NEXTTEMPPTR]

$$:= \text{PREVPTR}$$

$$\oplus \text{NEXTNEXTTEMPPTR} \left(\begin{array}{c} \text{Re - link the successor node to bypass} \\ \text{CURRPTR and point back to PREVPTR} \end{array} \right)$$

[End of If structure]

9. [FREE MEMORY AND PRINT]

Write: "Deleted ", INFO[CURRPTR], " from position ", POS

FREE(CURRPTR)

10. EXIT

Memory Efficient Doubly Linked List

– Traverse In Order – Program Analysis and Time Complexity

Program:

```
void traverseInOrder()
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    cout << "List (Forward): ";
    while (curr != nullptr)
    {
        cout << curr->data << " <-> ";

        // Calculate next address using XOR formula
        next = XOR(prev, curr->npx);

        // Shift pointers forward
        prev = curr;
        curr = next;
    }
    cout << "NULL" << endl;
}

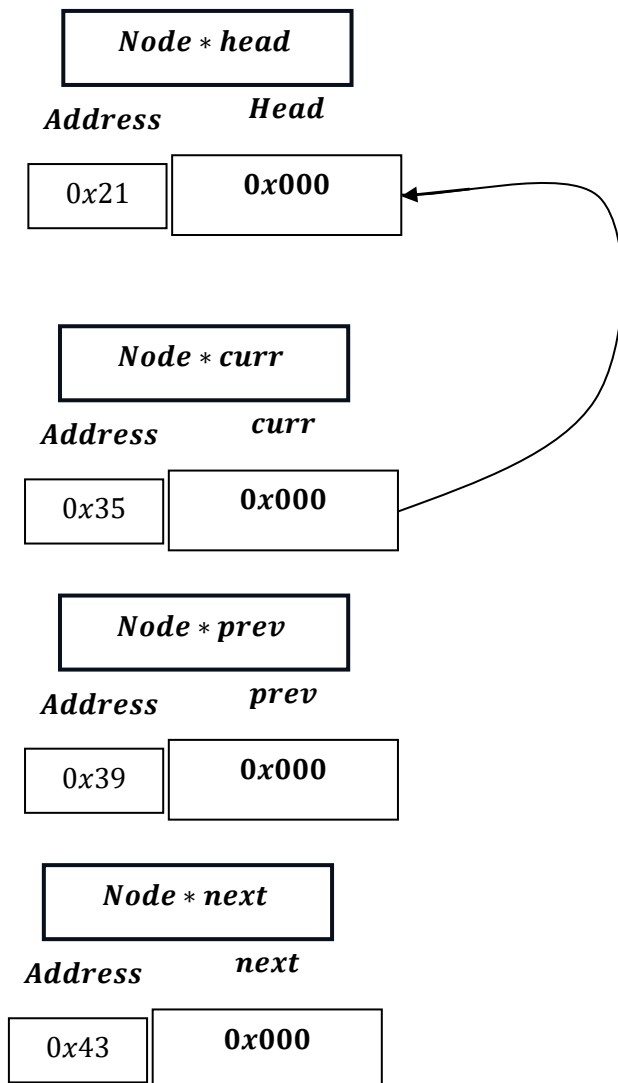
...

case 8:
    traverseInOrder();
    break;
```

Program Analysis:

```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;
```

1. When List Is Empty



```
cout << "List (Forward): ";
while (curr != nullptr)
{
    cout << curr->data << " <-> ";

    // Calculate next address using XOR formula
    next = XOR(prev, curr->npx);

    // Shift pointers forward
    prev = curr;
    curr = next;
}
cout << "NULL" << endl;
```

Print: ``List(Forward): ``

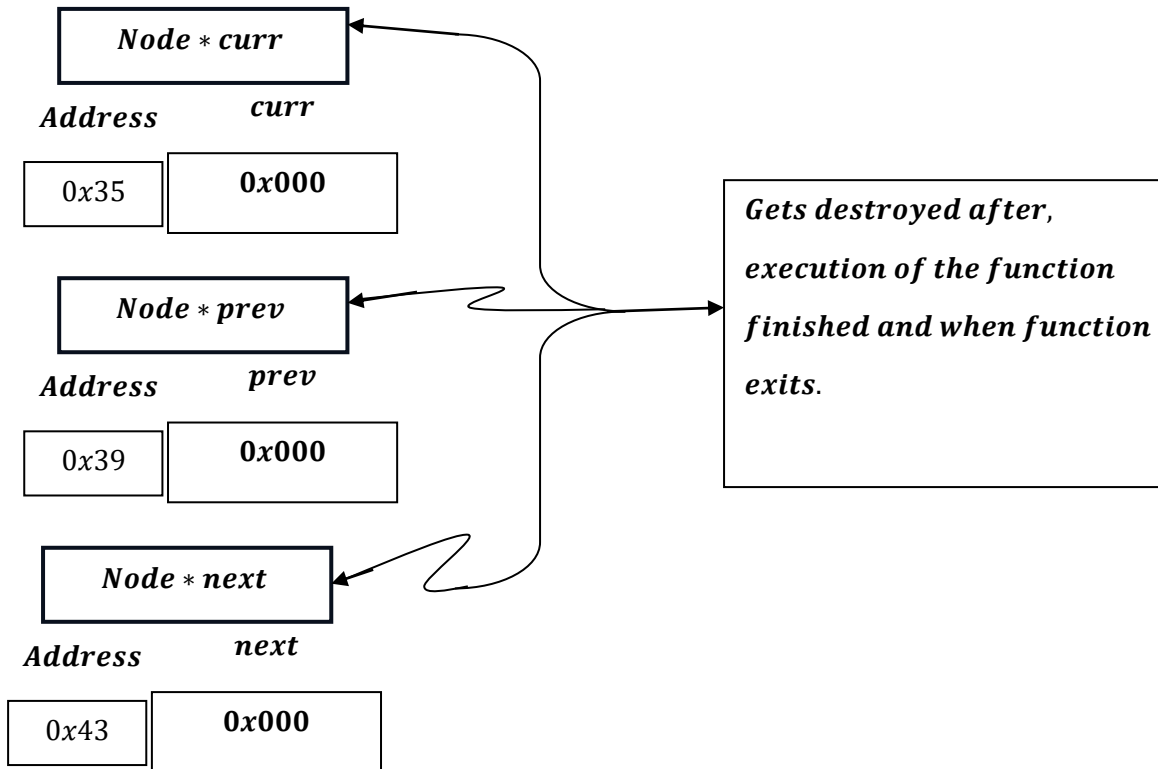
while(curr: 0x000 ≠ nullptr: 0x000) → False

Print: ``NULL``

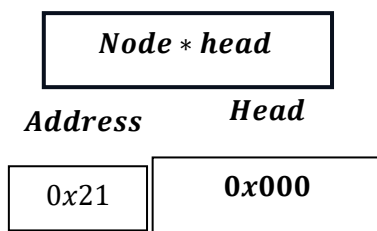
Hence, it Prints(Output):

ListForward: NULL.

As , functions ends:

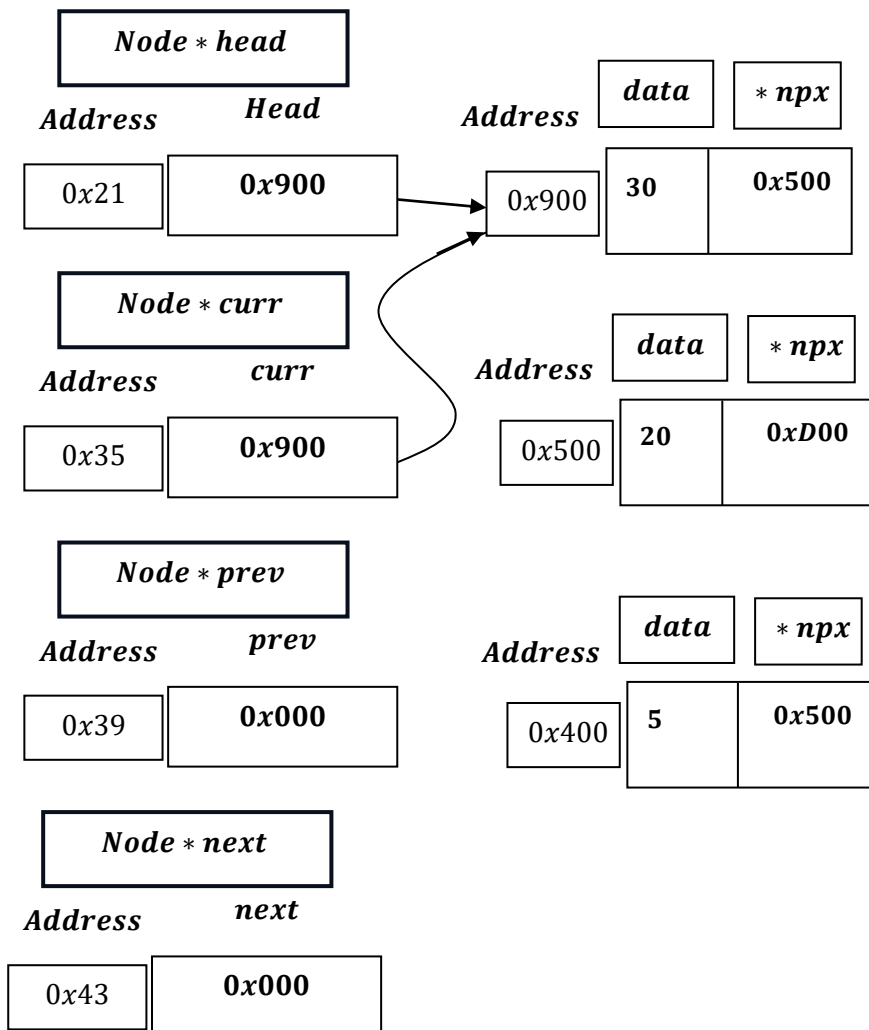


And we are left with:



2. When List Is Not Empty

```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;
```




```

cout << "List (Forward): ";
while (curr != nullptr)
{
    cout << curr->data << " <-> ";

    // Calculate next address using XOR formula
    next = XOR(prev, curr->npx);

    // Shift pointers forward
    prev = curr;
    curr = next;
}
cout << "NULL" << endl;

```

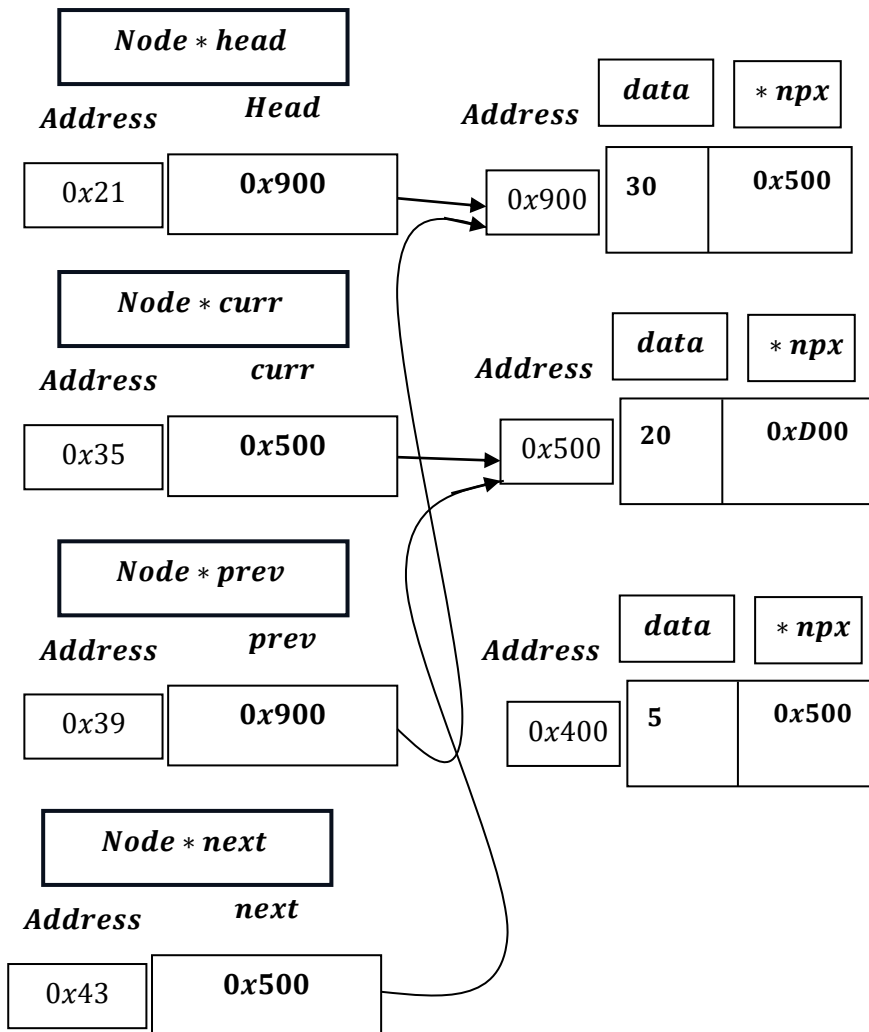
curr: 0x900 ≠ nullptr: 0x000 := is True:

Print Output: List(Forward): 30 <->

next = XOR(prev: 0x000, curr → npx: 0x500) = 0x500

prev = curr = 0x900

curr = next = 0x500



curr: 0x500 $\neq$ nullptr: 0x000 := is True:

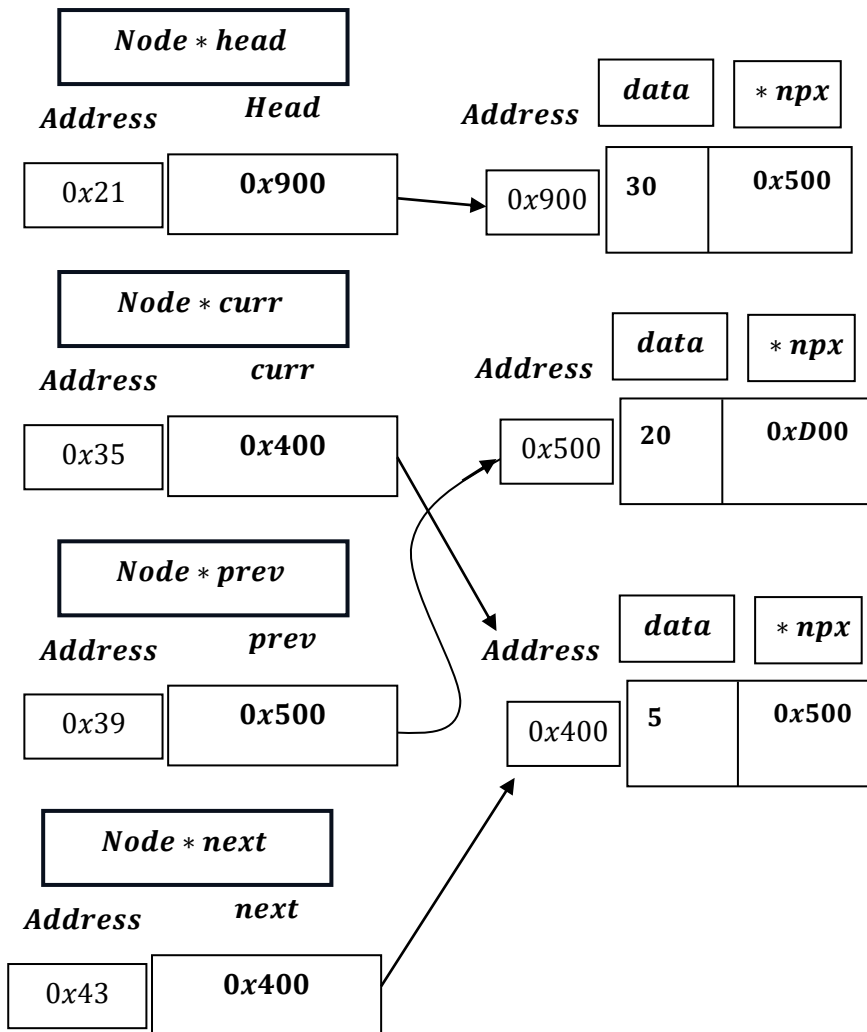
Print Output: List(Forward): 30 <-> 20 <->

next = XOR(prev: 0x900 , curr $\rightarrow$ npx: 0xD00)

= $1001_2 \oplus 1101_2 = 0100_2 = 4_{10} = 0x400$

prev = curr = 0x500

curr = next = 0x400



curr: 0x400 $\neq$ nullptr: 0x000 := is True:

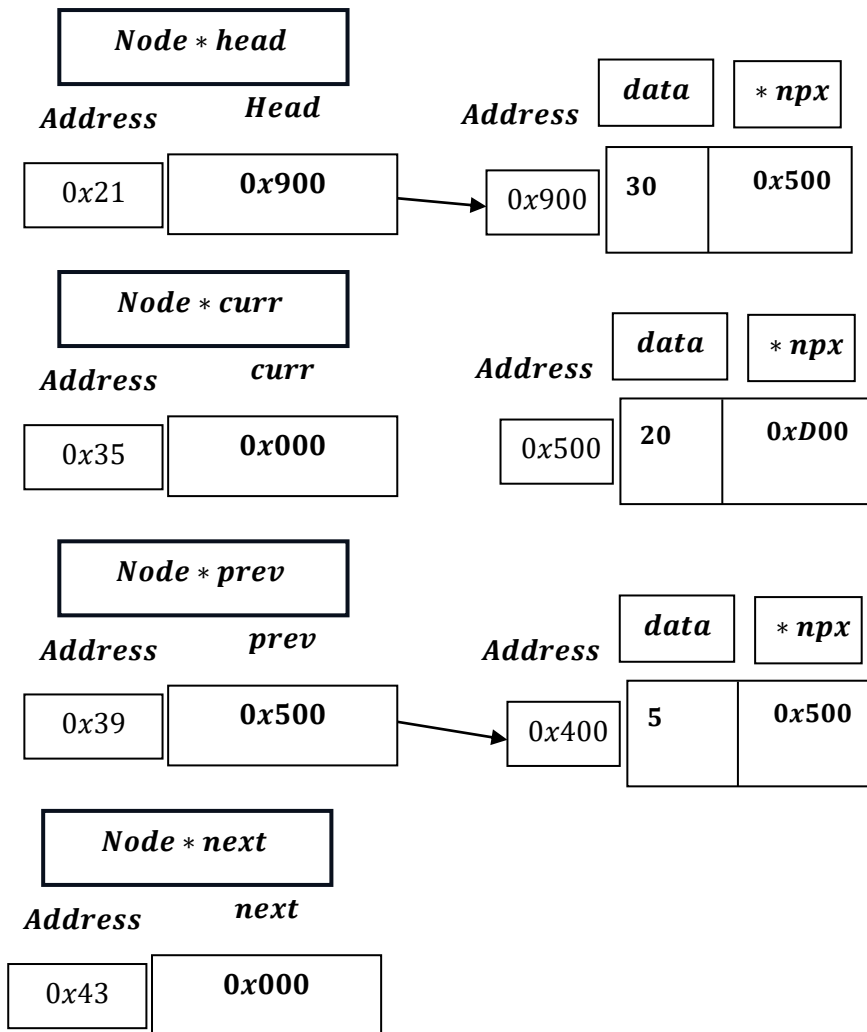
Print Output: List(Forward): 30 <-> 20 <-> 5 <->

next = XOR(prev: 0x500 , curr $\rightarrow$ npx: 0x500)

= 0101₂ $\oplus$ 0101₂ = 0000₂ = 0₁₀ = 0x000

prev = curr = 0x400

curr = next = 0x000

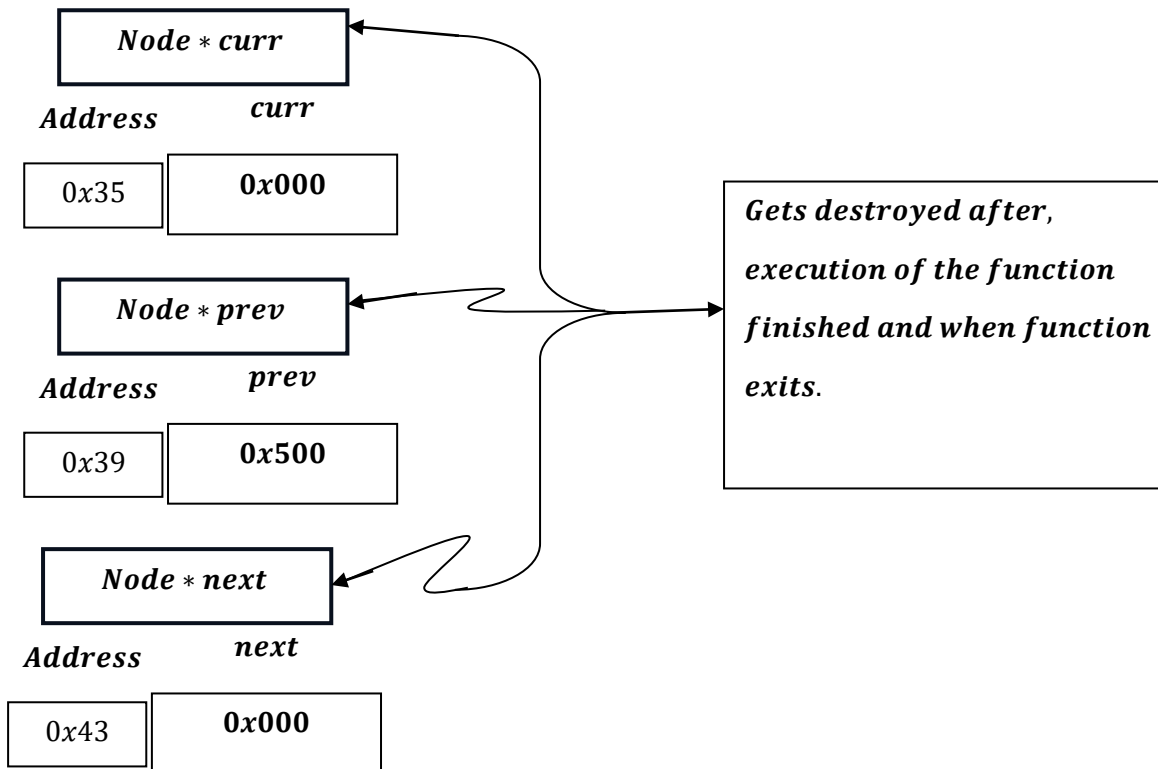


curr: 0x000 $\neq$ nullptr: 0x000 := is False, hence exit from loop.

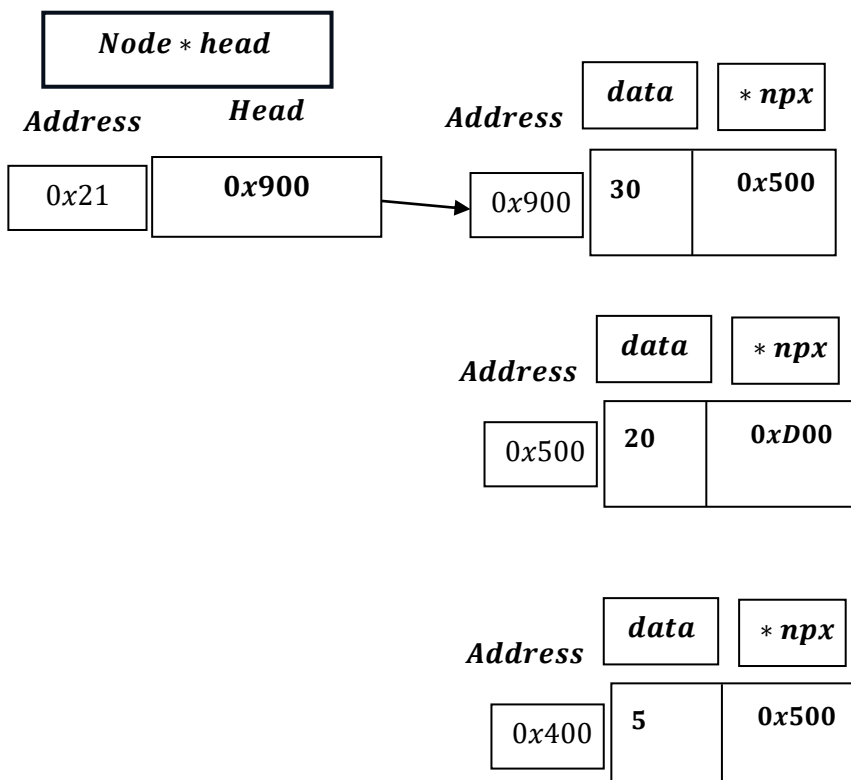
```
cout << "NULL" << endl;
```

Print Output: List(Forward): 30 < -> 20 < -> 5 < -> NULL.

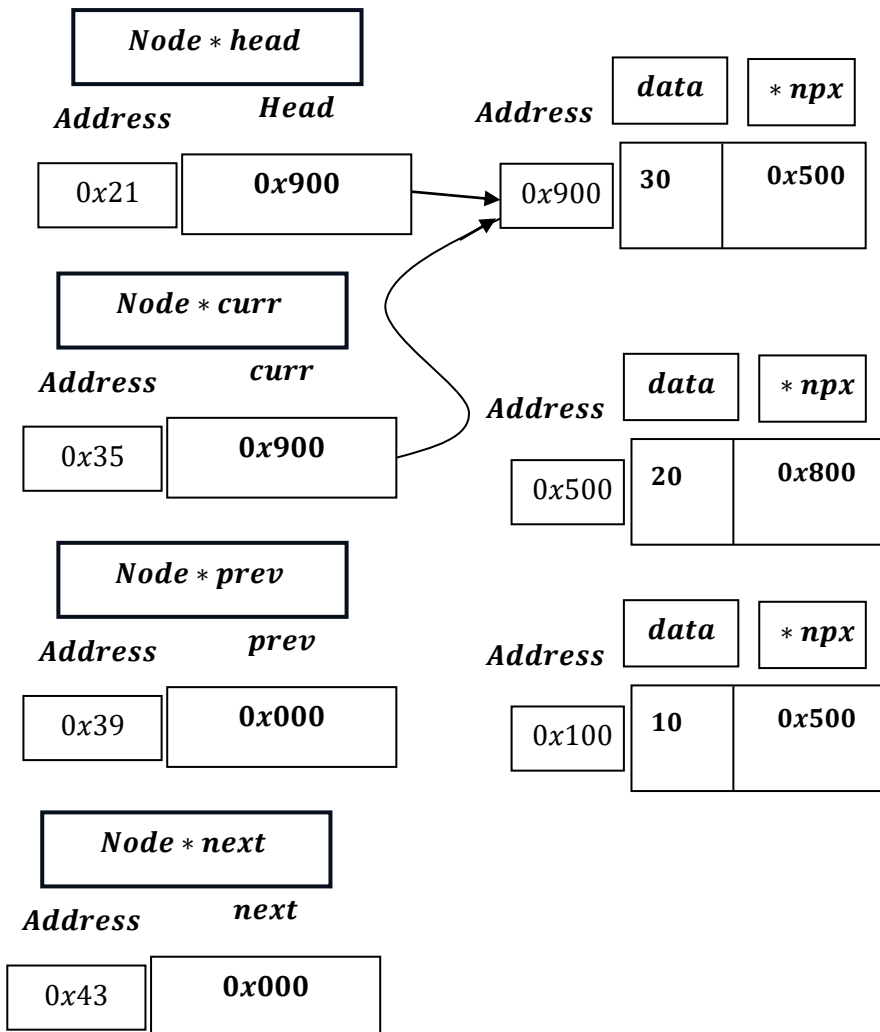
As , functions ends:



And we are left with:



Another Example:



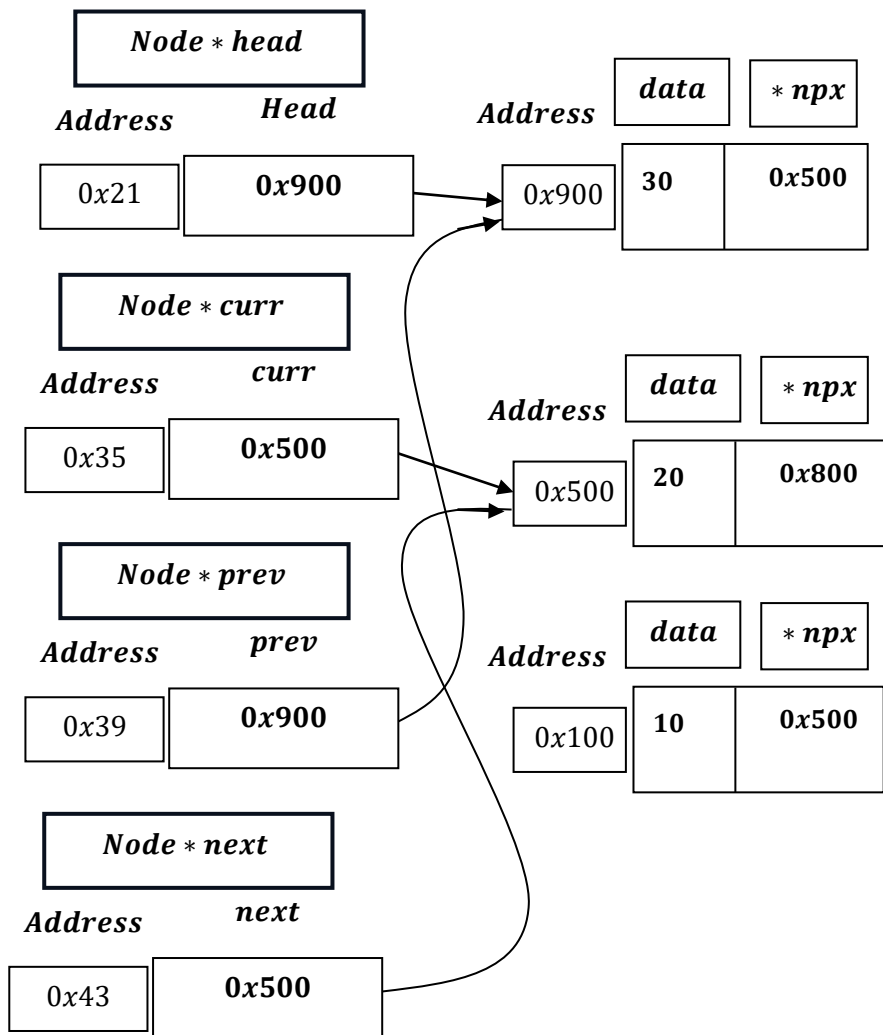
curr: 0x900 $\neq$ nullptr: 0x000 := is True:

Print Output: List(Forward): 30 < ->

next = XOR(prev: 0x000 , curr $\rightarrow$ npx: 0x500) = 0x500

prev = curr = 0x900

curr = next = 0x500



curr: 0x500 ≠ nullptr: 0x000 := is True:

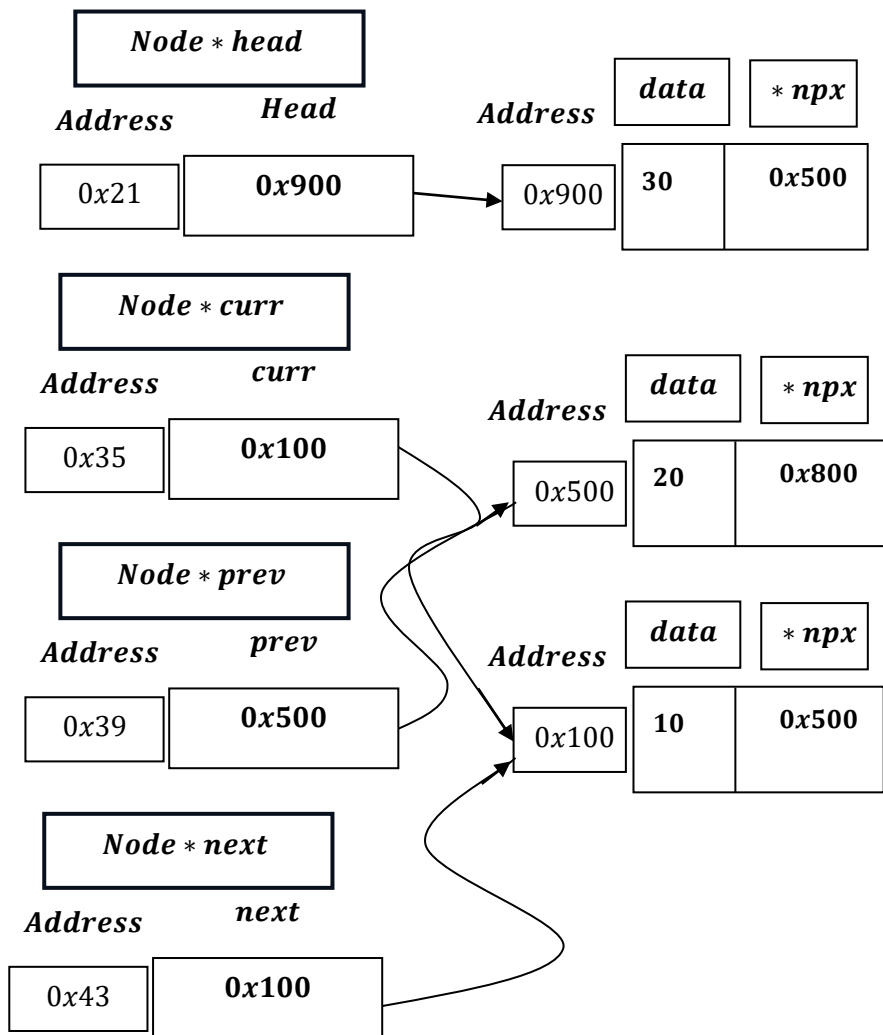
Print Output: List(Forward): 30 < - > 20 < - >

next = XOR(prev: 0x900, curr → npx: 0x800)

= 1001₂ ⊕ 1000₂ = 0001₂ = 1₁₀ = 0x100

prev = curr = 0x500

curr = next = 0x100



curr: 0x100 $\neq$ nullptr: 0x000 := is True:

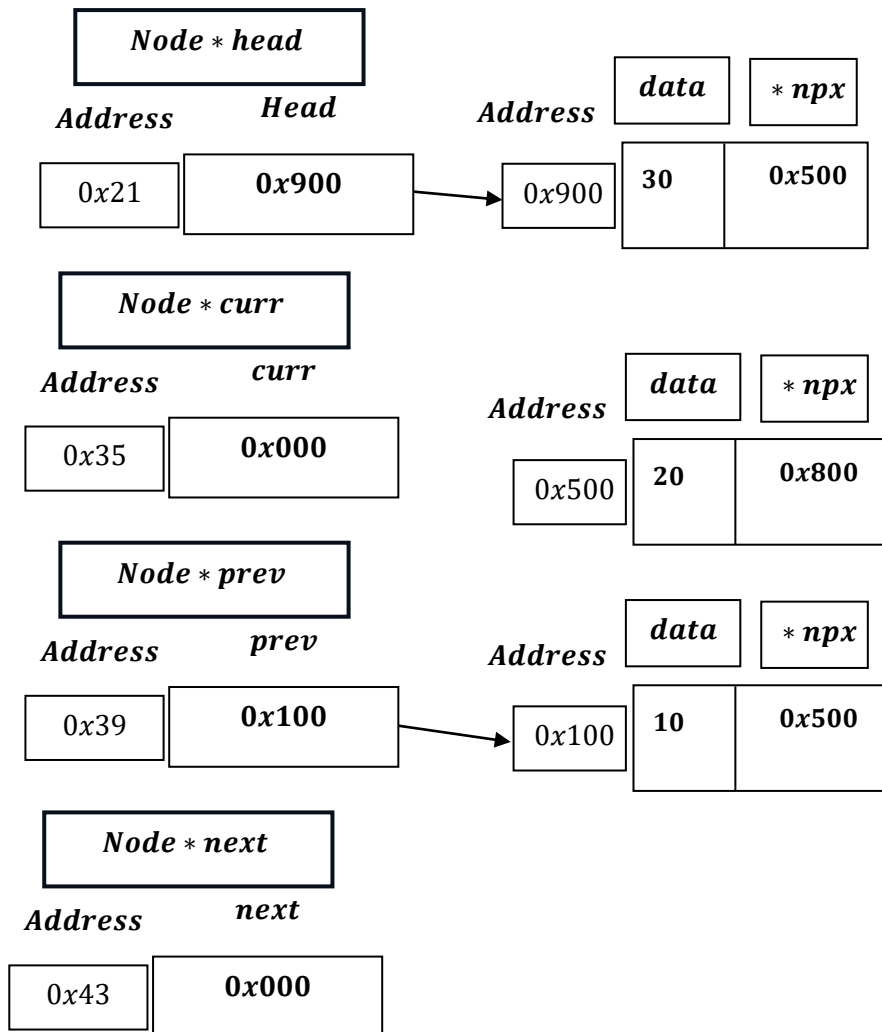
Print Output: List(Forward): 30 < -> 20 < -> 10 < ->

next = XOR(prev: 0x500, curr $\rightarrow$ npx: 0x500)

= 0101₂ $\oplus$ 0101₂ = 0000₂ = 0₁₀ = 0x000

prev = curr = 0x100

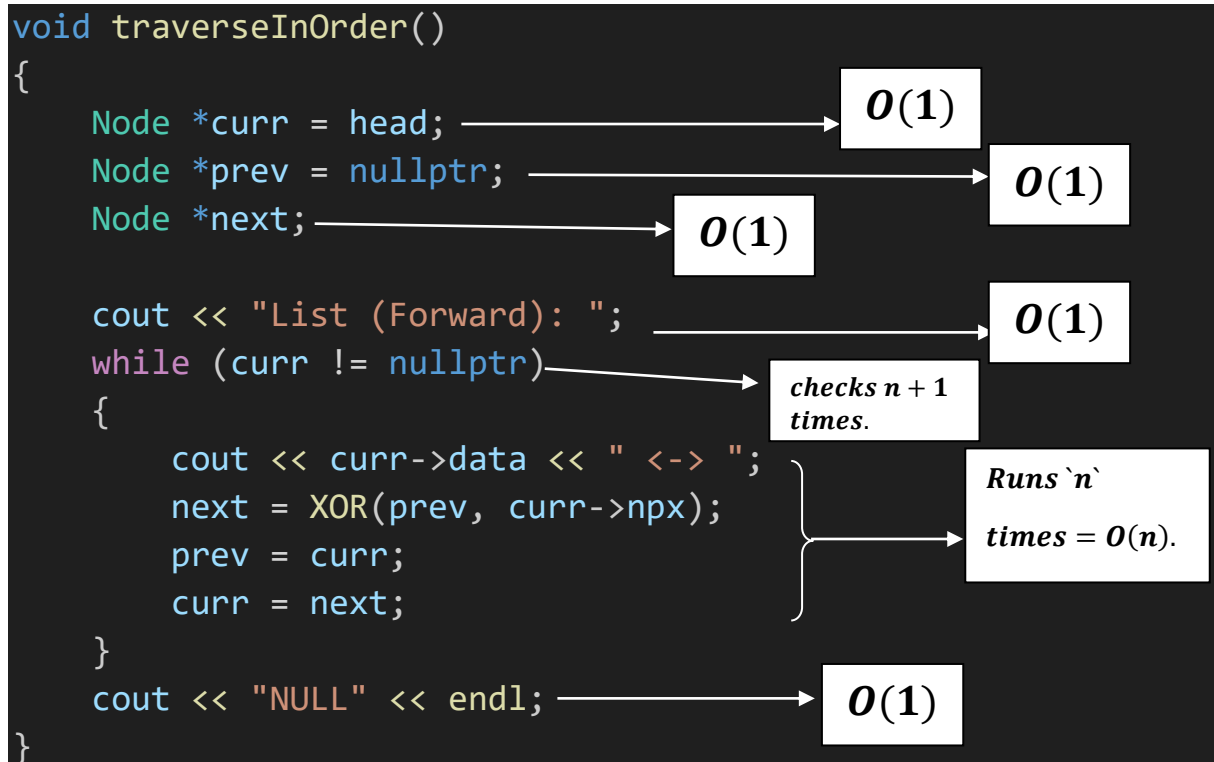
curr = next = 0x000



curr: 0x100 $\neq$ nullptr: 0x000 := is False. Hence:

Print: 30 < -> 20 < -> 10 < -> NULL.

Time Complexity



Best Case

When, List is empty , then we get our absolute best case: $\Omega(1)$.

Worst Case

Full linked list traversal:

$curr \neq nullptr$ [This check runs `n + 1` times , including when it becomes false.]:

$cout \ll curr \rightarrow data \ll " <-> "; \rightarrow runs : 1 + 1 + 1 + \dots n \text{ times} = n.$

$next = XOR(prev, curr \rightarrow npx); \rightarrow runs : 1 + 1 + 1 + \dots n \text{ times} = n.$

$prev = curr; \rightarrow runs: 1 + 1 + 1 + \dots n \text{ times} = n.$

$curr = next; \rightarrow runs: 1 + 1 + 1 + \dots n \text{ times} = n .$

As , we focus , to the run of inner – statements of loop, hence :

$\therefore \text{Runs} = n + n + n + n = 4n = 4 \times O(n) = O(n)$.

Average Case

*We can say , if List is Empty ,
, we get our best case scenario, and when list is not empty and there present multiple nodes , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the deletion to be made , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.*

In the case of TraverseInOrder, where list is traversed till it reach the end of list, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the traversal position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence TraverseInOrder doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Traverse In Order</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

Memory Efficient Doubly Linked List
– Traverse In Order – Space Complexity and Algorithm

Program:

```
void traverseInOrder()
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    cout << "List (Forward): ";
    while (curr != nullptr)
    {
        cout << curr->data << " <-> ";

        // Calculate next address using XOR formula
        next = XOR(prev, curr->npx);

        // Shift pointers forward
        prev = curr;
        curr = next;
    }
    cout << "NULL" << endl;
}

...

case 8:
    traverseInOrder();
    break;
```

Space Complexity Analysis: XOR Linked List (In – Order Traversal)

1. Input Space Complexity

- ***Parameter – only Definition:***
 - ***Function Parameters:*** None.
 - ***Analysis:*** No data is passed into the function as arguments.
 - ***Complexity:*** $O(1)$
- ***Full Data Definition:***
 - ***Function Data:*** The entire XOR linked list structure.
 - ***Analysis:*** The function must access all n nodes in the list to print their values. Therefore, the data structure it operates on is proportional to the number of elements.
 - ***Complexity:*** $O(n)$

2. Auxiliary Space Complexity

Explanation: This measures the temporary memory used by the algorithm during execution.

- ***Local Pointer Variables:*** The function uses three local pointers: *curr*, *prev*, and *next*.
 - ***Analysis:*** These are stack variables that occupy a constant amount of space (e. g., 24 bytes on a 64 – bit system). Regardless of whether the list has 10 nodes or 10 million nodes, only these three pointers are needed to keep track of the traversal.
- ***Iterative Process:*** The traversal uses a while loop rather than recursion. This ensures that the function call stack does not grow with the number of nodes.
- ***XOR Logic:*** The calculation of the next address happens in CPU registers using bitwise operations, requiring no extra memory allocation.

Conclusion: The amount of extra memory used is fixed and does not grow as the size of the linked list (n) grows.

- ***Auxiliary Space Complexity:*** $O(1)$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Traverse In – Order (XOR Doubly Linked List)

XORTRAVERSE(START):

1. [INITIALIZE TRAVERSAL POINTERS]

Set CURRPTR := START

Set PREVPTR := NULL

2. [PRINT HEADER]

Write: "List (Forward): "

3. [TRAVERSE AND PRINT NODES]

Repeat while CURRPTR $\neq$ NULL:

a. **[DISPLAY DATA]**

Write: INFO[CURRPTR], " < -> "

b. **[DECODE NEXT NODE ADDRESS]**

Set NEXTTEMPPTR :

= PREVPTR

$\oplus$ NEXTXORPTR[CURRPTR] $\left(\begin{array}{l} \text{Use the address of the} \\ \text{previous node to unlock} \\ \text{the address of the} \\ \text{next node} \end{array} \right)$

c. **[SHIFT POINTERS FORWARD]**

Set PREVPTR := CURRPTR

Set CURRPTR := NEXTTEMPPTR

[End of Loop]

4. [PRINT TERMINATOR]

Write: "NULL"

5. EXIT

Memory Efficient Doubly Linked List
–Destroy List – Program Analysis and Time Complexity

Program:

```
void destroyList()
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    while (curr != nullptr)
    {
        next = XOR(prev, curr->npx);
        free(curr);
        prev = curr;
        curr = next;
    }
    head = nullptr;
    cout << "List destroyed." << endl;
}

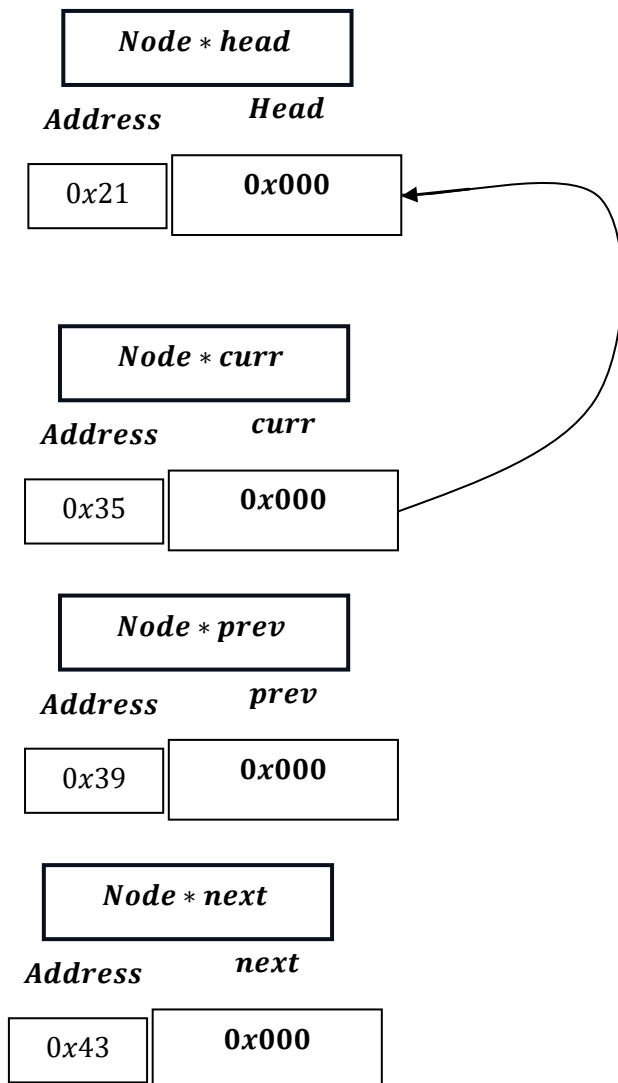
... .

    case 9:
        destroyList();
        break;
```

Program Analysis:

```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;
```

1. When List Is Empty



```

while (curr != nullptr)
{
    next = XOR(prev, curr->npx);
    free(curr);
    prev = curr;
    curr = next;
}
head = nullptr;
cout << "List destroyed." << endl;

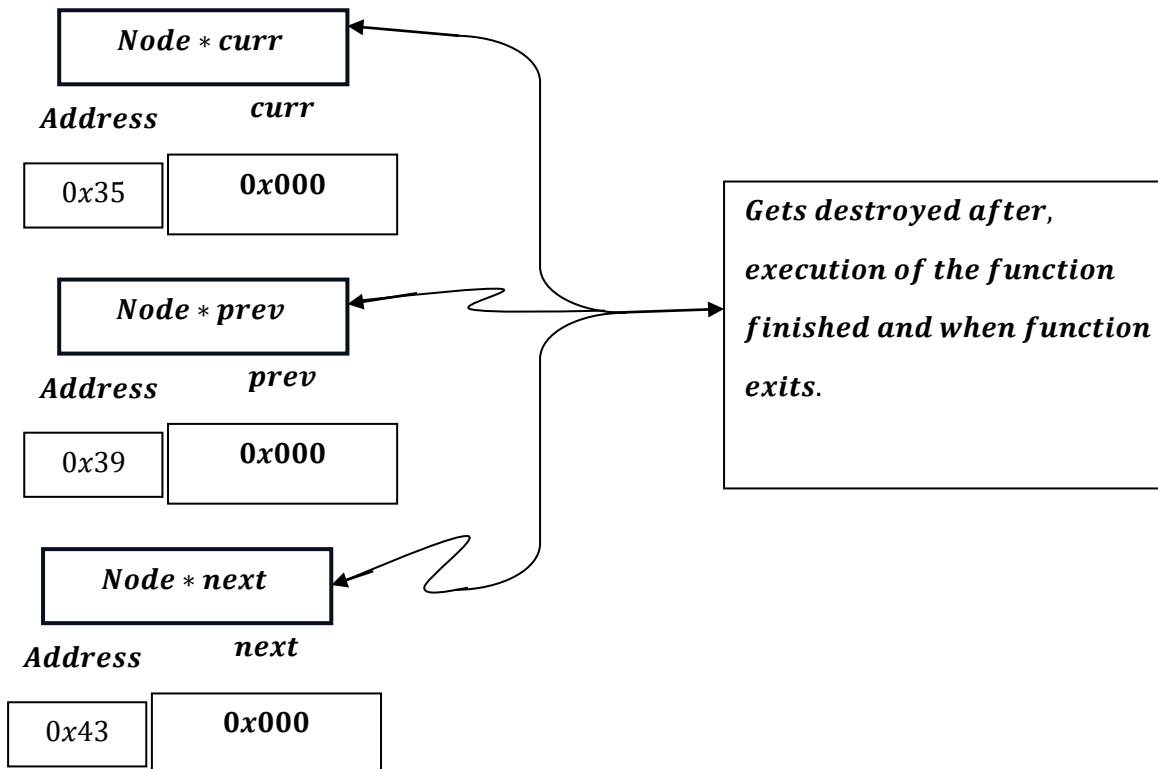
```

while(curr: 0x000 ≠ nullptr: 0x000) → False.

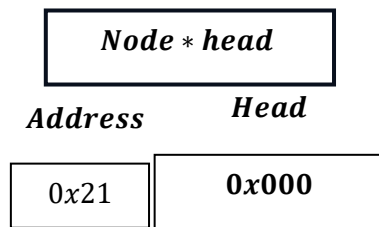
head = nullptr. $\left[\begin{array}{l} \text{just an re - assignment, here no need to show} \\ \text{in a picture.} \end{array} \right]$

Print: ``List destroyed.``

As , functions ends:

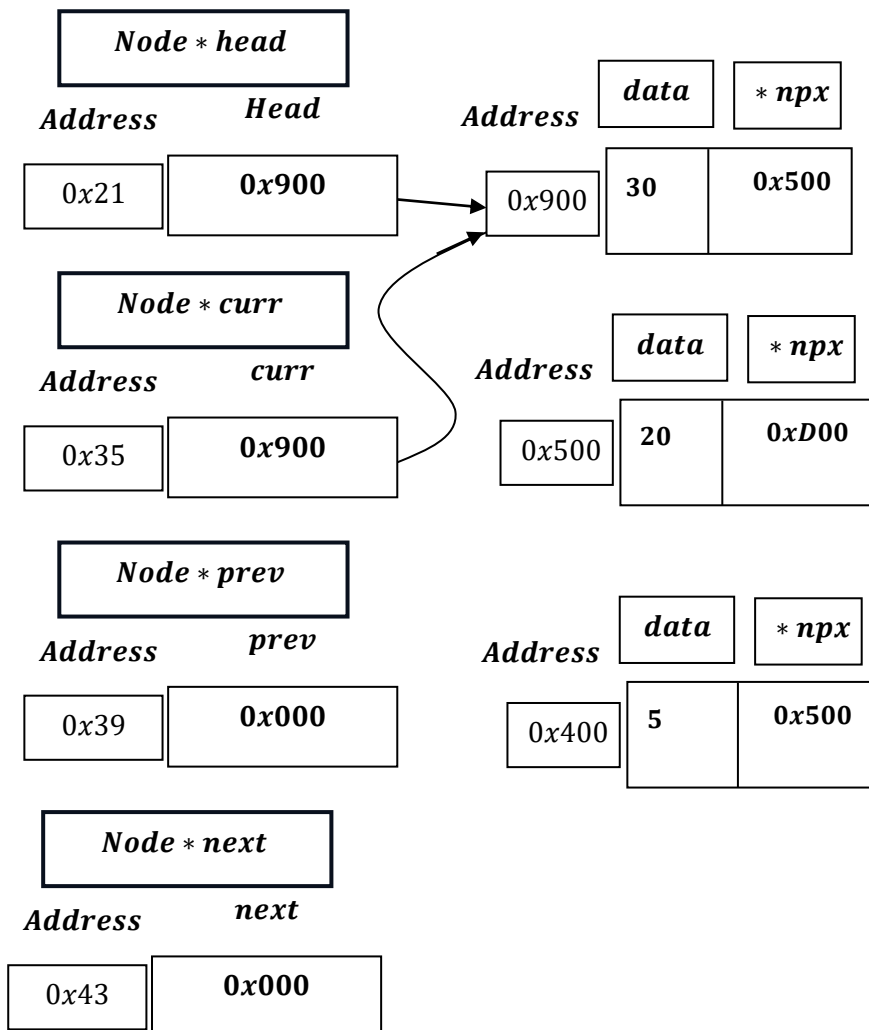


And we are left with:



2. When List Is Not Empty

```
Node *curr = head;  
Node *prev = nullptr;  
Node *next;
```



```

while (curr != nullptr)
{
    next = XOR(prev, curr->npx);
    free(curr);
    prev = curr;
    curr = next;
}

```

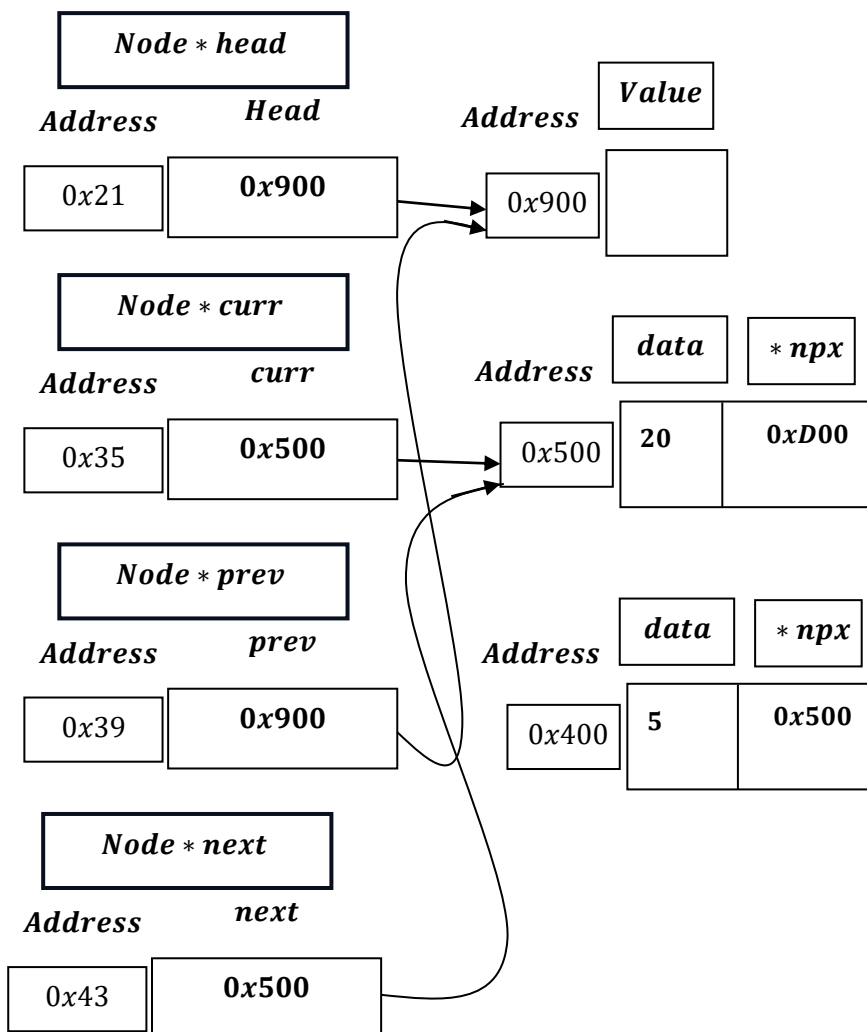
curr: 0x900 $\neq$ nullptr: 0x000 := is True:

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x500

free(curr);

prev = curr = 0x900

curr = next = 0x500



curr: 0x500 $\neq$ nullptr: 0x000 := is True:

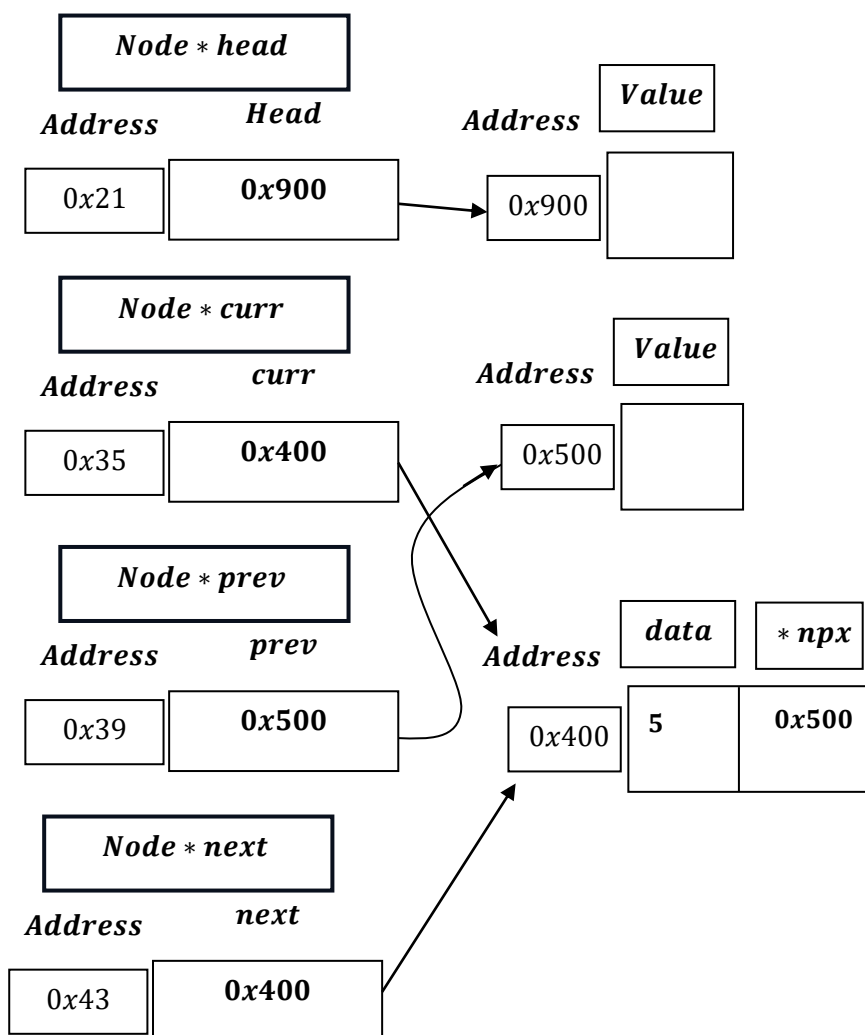
next = XOR(prev: 0x900 , curr $\rightarrow$ npx: 0xD00)

= $1001_2 \oplus 1101_2 = 0100_2 = 4_{10} = 0x400$

free(curr);

prev = curr = 0x500

curr = next = 0x400



curr: 0x400 $\neq$ nullptr: 0x000 := is True:

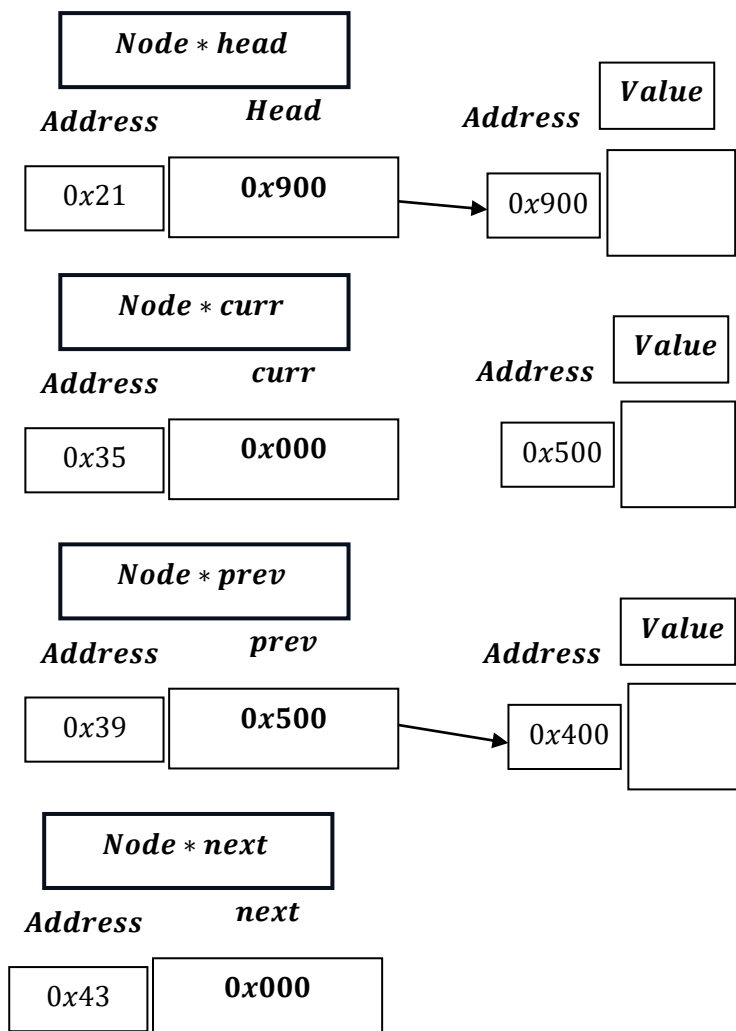
next = XOR(prev: 0x500 , curr $\rightarrow$ npx: 0x500)

= 0101₂ $\oplus$ 0101₂ = 0000₂ = 0₁₀ = 0x000

free(curr);

prev = curr = 0x400

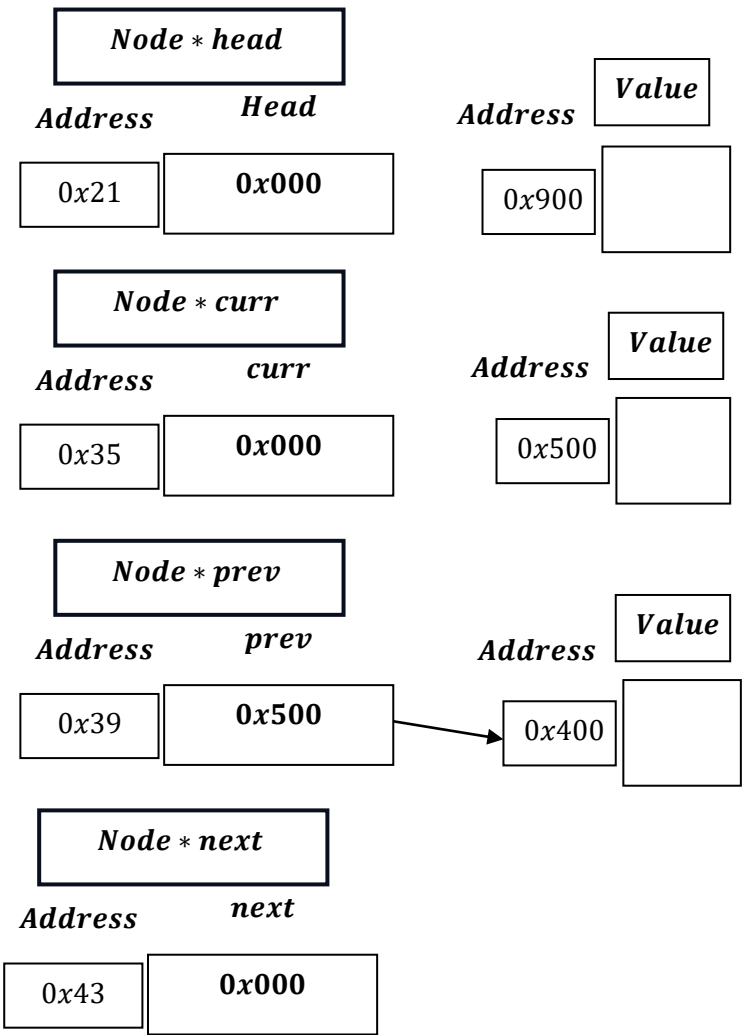
curr = next = 0x000



curr: 0x000 $\neq$ nullptr: 0x000 := is False, hence exit from loop.

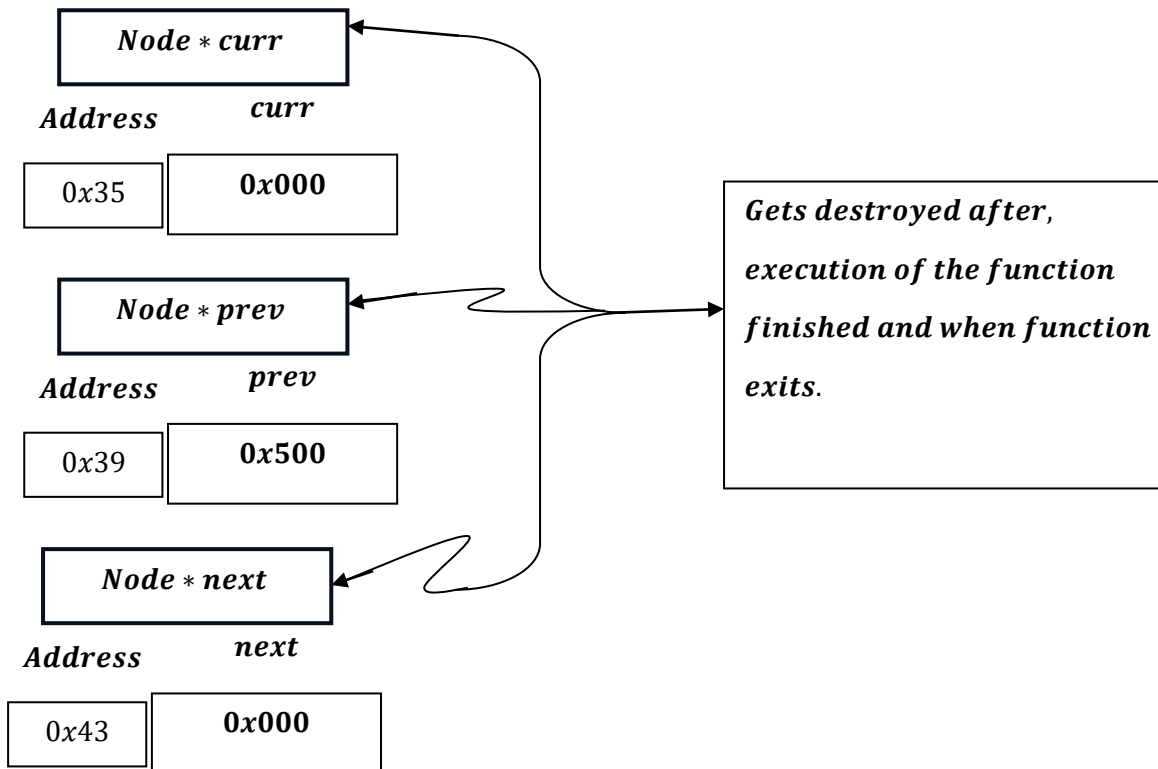

```
head = nullptr;
```

```
head = nullptr = 0x000;
```

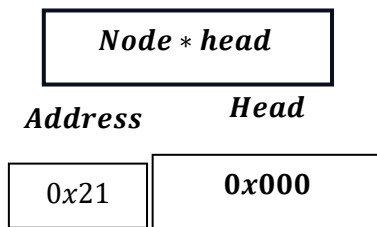


```
Print: Output: ``List Destroyed.``
```

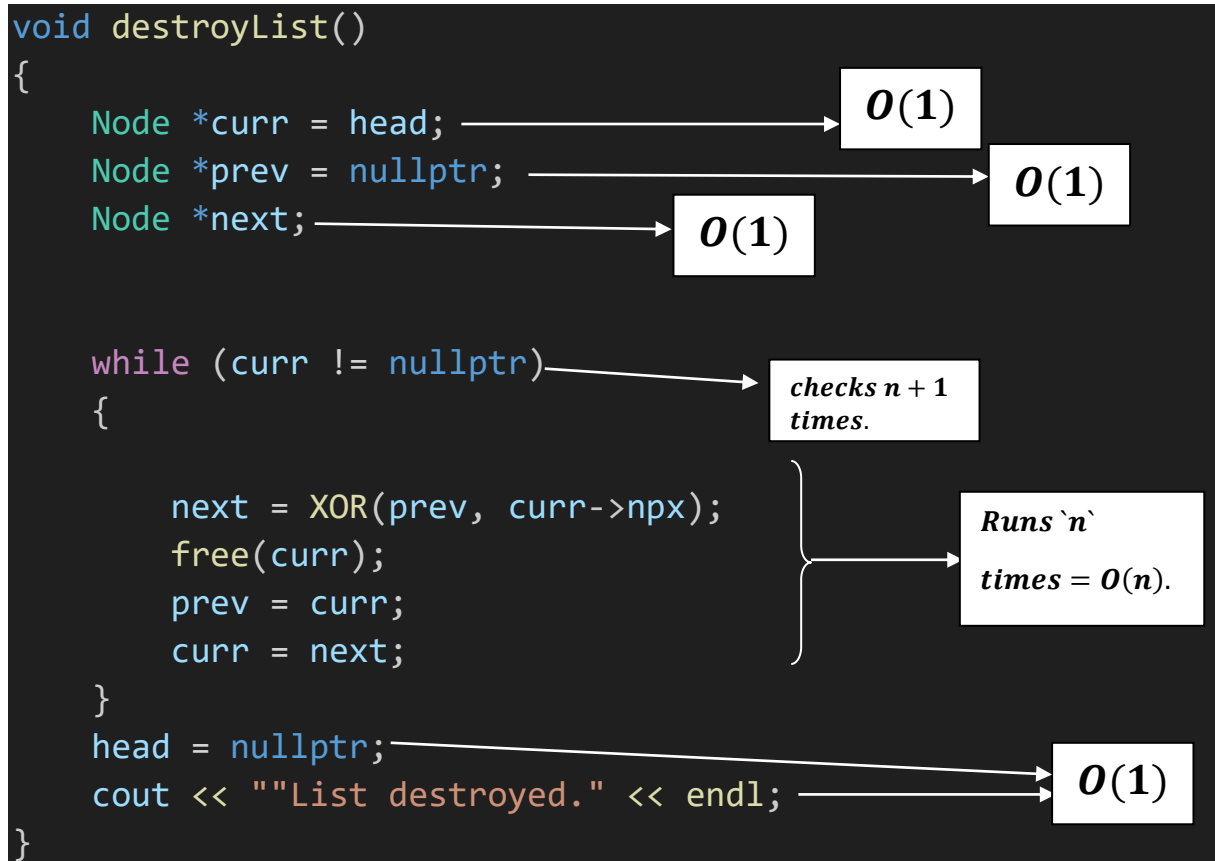
As , functions ends:



And we are left with:



Time Complexity



Best Case

When, List is empty , then we get our absolute best case: $\Omega(1)$.

Worst Case

Full linked list traversal:

$curr \neq nullptr$ $\left[\begin{array}{l} \text{This check runs } n + 1 \text{ times,} \\ \text{including when it becomes false.} \end{array} \right]$:

$next = XOR(prev, curr \rightarrow npx); \rightarrow$ runs : $1 + 1 + 1 + \dots n$ times = n .

$free(curr); \rightarrow$ runs : $1 + 1 + 1 + \dots n$ times = n .

$prev = curr; \rightarrow$ runs: $1 + 1 + 1 + \dots n$ times = n .

$curr = next; \rightarrow$ runs: $1 + 1 + 1 + \dots n$ times = n .

As , we focus , to the run of inner – statements of loop, hence :

$\therefore \text{Runs} = n + n + n + n + n = 5n = 5 \times O(n) = O(n)$.

Average Case

We can say , if List is Empty ,

, we get our best case scenario, and when list is not empty and there present multiple nodes , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the deletion to be made , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.

In the case of DestroyList, where list is traversed till it reach the end of list, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case).

Since the traversal position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence DestroyList doesn't have a true average case.

No meaningful probabilistic average exists here.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Destroy List</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

Memory Efficient Doubly Linked List – Destroy List
–Space Complexity Analysis and Algorithm

Program:

```
void destroyList()
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    while (curr != nullptr)
    {
        next = XOR(prev, curr->npx);
        free(curr);
        prev = curr;
        curr = next;
    }
    head = nullptr;
    cout << "List destroyed." << endl;
}

... .

    case 9:
        destroyList();
        break;
```

Space Complexity Analysis: XOR Linked List (Destroy List)

1. Input Space Complexity

- **Parameter – only Definition:**
 - **Function Parameters:** None.
 - **Analysis:** No data is passed into the function; it operates solely on the global head.
 - **Complexity:** $O(1)$
 - **Full Data Definition:**
 - **Function Data:** The entire XOR linked list structure existing in memory.
Analysis: The function must iterate through all n nodes to free them. The input context is the size of the list currently in the heap.
 - **Complexity:** $O(n)$
-

2. Auxiliary Space Complexity

Explanation: This measures the temporary memory used by the algorithm during its execution.

- **Local Pointer Variables:** The function uses three local pointers: *curr*, *prev*, and *next*.
 - **Analysis:** These are fixed – size stack variables. Even if the list has a million nodes, we only ever need these three pointers to manage the current deletion and calculate the next address.
- **Iterative Process:** The list is destroyed using a while loop. Since it is not a recursive destruction (which would create $O(n)$ stack frames), it uses constant stack space.
- **XOR Operations:** The next node calculation is done using bitwise math in CPU registers, requiring no extra memory.
- **Heap Impact:** *free(curr)* removes nodes from the heap. While the **Total Memory** of the program decreases, the **Auxiliary Space** (extra memory needed to run the algorithm) does not change.

Conclusion: The amount of extra memory used is fixed and does not scale with the number of nodes n .

- **Auxiliary Space Complexity:** $O(1)$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Destroy List (XOR Doubly Linked List)

XORDESTROY(START):

1. [INITIALIZE TRAVERSAL]

Set CURRPTR := START

Set PREVPTR := NULL

2. [ITERATE THROUGH THE LIST]

Repeat while CURRPTR $\neq$ NULL:

a. [DECODE NEXT NODE ADDRESS]

Set NEXTTEMPPTR :

$= \text{PREVPTR} \oplus \text{NEXTXORPTR}[\text{CURRPTR}] \left(\begin{array}{c} \text{Calculate the address of the next} \\ \text{node before deleting} \\ \text{the current one} \end{array} \right)$

b. [SAVE CURRENT ADDRESS AS PREV]

Set TEMPPTR := CURRPTR $\left(\begin{array}{c} \text{Temporarily hold the address} \\ \text{of the node to be freed} \end{array} \right)$

c. [FREE CURRENT NODE]

FREE(TEMPPTR) $\left(\begin{array}{c} \text{Release the memory of the} \\ \text{current node back} \\ \text{to the system} \end{array} \right)$

d. [SHIFT POINTERS FORWARD]

Set PREVPTR := TEMPPTR $\left(\begin{array}{c} \text{Even though the node is freed,} \\ \text{its address is still} \\ \text{used as the `key`} \\ \text{for the next XOR operation} \end{array} \right)$

Set CURRPTR := NEXTTEMPPTR

[End of Loop]

3. [RESET HEAD]

Set START := NULL $\left(\begin{array}{c} \text{Indicate that} \\ \text{the list is now} \\ \text{empty} \end{array} \right)$

4. [PRINT MESSAGE]

Write: "List destroyed."

5. EXIT

Memory Efficient Doubly Linked List –Operation Understanding

1. Insert At Beginning

```
newNode->npx = XOR(nullptr, head);
```

This exhibit the identity property : $X \oplus 0 = X$.

It stores the address stored in head ,the address of first node : A.

When List is not Empty:

```
if (head != nullptr)
{
    Node *next = XOR(nullptr, head->npx);
    head->npx = XOR(newNode, next);
}
```

Here we need a pointer next to hold the identity property : $X \oplus 0 = X$. Here X is $head \rightarrow npx$, this is current address that becomes node B, Actually it is :

The next pointer of A is : $NULL \oplus B$

$head \rightarrow npx = XOR(newNode, next)$.

It maintains the property : Next pointer of B is: $A \oplus C$.

That is the first address where data and npx is stored,

the npx pointer will store the XOR result : $XOR(newNode, next)$

i. e. $XOR(newNode: Node A, next: Node C)$

, This XOR result will help to traverse to next node.

2. Insert At End

1. When List is Empty:

```
if (head == nullptr)
{
    newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
    head = newNode;
    cout << data << " inserted at the end." << endl;
    return;
}
```

Here: $newNode \rightarrow npx = XOR(nullptr, nullptr)$;

That is when there will be one node the npx pointer of the node will carry: nullptr.

2. When List is not Empty:

```
while (curr != nullptr)
{
    next = XOR(prev, curr->npx);
    if (next == nullptr){ break; }
    prev = curr;
    curr = next;
}
```

1. $next = XOR (prev: NULL, curr \rightarrow npx: \text{Address of Node B})$

2. $next = XOR(prev: \text{Address of A}, curr \rightarrow npx: C)$

$\Rightarrow \text{Address of Node C}$

Note: C is the XOR result.

May not be address of Node C i.e. Result of $XOR(\text{Address of Node A}, \text{Address of Node C})$. The only time Node B's npx will contain Address of Node A When there will be only two node exists.

3. $next = XOR(prev: \text{Address of B}, curr \rightarrow npx: D)$

$\Rightarrow \text{Address of Node D}$

Note: D is the XOR result..

May not be address of Node D i.e. Result of $XOR(\text{Address of Node B}, \text{Address of Node D})$. The only time Node C's npx will contain Address of Node B when there will be only three node exists. Similarly Node D's npx will contain Address of Node C, when there will be only four node exist.

... etc.

Similarly ,when the loop fails when we get situation like:

$next = (prev: \text{Address of Node A}, curr \rightarrow npx: \text{Address of Node A}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Two Node} \end{array} \right]$

$next = (prev: \text{Address of Node B}, curr \rightarrow npx: \text{Address of Node B}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Three Node} \end{array} \right]$

$next = (prev: \text{Address of Node C}, curr \rightarrow npx: \text{Address of Node C}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Four Node} \end{array} \right]$

... etc.

Again ,we have:

```
curr->npx = XOR(prev, newNode);  
newNode->npx = XOR(curr, nullptr);
```

Suppose we have 3 nodes in the list. Hence $curr \rightarrow npx$, indicates Node C's npx and Node D to be inserted at last , hence:

$curr \rightarrow npx = XOR(prev: \text{Address of B}, newNode: \text{Address of D});$

Node C's npx: Address of B $\oplus$ Address of D.

$newNode \rightarrow npx = XOR(curr: \text{Address of C} , NULL)$

i. e., Node D's npx : Address of C $\oplus$ NULL = Address of C.

Similarly,

Suppose we have 4 nodes in the list. Hence $curr \rightarrow npx$, indicates Node D's npx and Node E to be inserted at last, hence:

$curr \rightarrow npx = XOR(prev: Address\ of\ C, newNode: Address\ of\ E);$

Node D's npx: $Address\ of\ C \oplus Address\ of\ E$.

$newNode \rightarrow npx = XOR(curr: Address\ of\ D, NULL)$

i.e., Node E's npx : $Address\ of\ D \oplus NULL = Address\ of\ D$.

This is goes on ...

2. Insert At Position

```
while (curr != nullptr && i < pos - 1)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

1. $next = XOR (prev: NULL , curr \rightarrow npx: Address\ of\ Node\ B)$

2. $next = XOR(prev: Address\ of\ A , curr \rightarrow npx: C)$

$\Rightarrow Address\ of\ Node\ C$

Note: *C* is the XOR result.

$\left[\begin{array}{l} \text{May not be address of} \\ \text{Node C i.e. Result of} \\ \text{XOR} \left(\begin{array}{l} \text{Address of Node A,} \\ \text{Address of Node C} \end{array} \right) \\ \text{The only time Node B's npx} \\ \text{will contain Address of Node A} \\ \text{When there will be only two node} \\ \text{exists.} \end{array} \right]$

3. $\text{next} = \text{XOR}(\text{prev: Address of B}, \text{curr} \rightarrow \text{npx: D})$

$\Rightarrow$ Address of Node D

Note: *D* is the XOR result..

$\left[\begin{array}{l} \text{May not be address of} \\ \text{Node D i.e. Result of} \\ \text{XOR} \left(\begin{array}{l} \text{Address of Node B,} \\ \text{Address of Node D} \end{array} \right) \\ \text{The only time Node C's npx} \\ \text{will contain Address of Node B} \\ \text{when there will be only three node} \\ \text{exists. Similarly Node D's npx will} \\ \text{contain Address of Node C, when there will} \\ \text{be only four node exist.} \end{array} \right]$

... etc.

Similarly, when the loop fails when we get situation like:

$\text{next} = (\text{prev: Address of Node A}, \text{curr} \rightarrow \text{npx: Address of Node A});$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Two Node} \end{array} \right]$

$\text{next} = (\text{prev: Address of Node B}, \text{curr} \rightarrow \text{npx: Address of Node B});$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Three Node} \end{array} \right]$

$\text{next} = (\text{prev: Address of Node C}, \text{curr} \rightarrow \text{npx: Address of Node C});$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Four Node} \end{array} \right]$

... etc.

Again , we have:

```
next = XOR(prev, curr->npx);
```

If the `pos` goes from first to $n - 1$

Eg: $next = (prev: Address\ of\ B \oplus curr \rightarrow npx(npx\ of\ Node\ C): D),$

where, D is : $Address\ of\ B \oplus Address\ of\ D$, hence,

$\Rightarrow next = (Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D))$

And , we know: by property $(B \oplus D) \oplus D = B$ (since , $D \oplus D = 0$)

Hence, we get: $(Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D))$

We get $next = Address\ of\ D$.

At n th (last node, say Node D)

$\Rightarrow next = (prev: Address\ of\ Node\ C \oplus curr \rightarrow npx(npx\ of\ Node\ D): D),$

where, D is : $Address\ of\ C \oplus NULL = Address\ of\ C$, hence,

$\Rightarrow next = XOR(prev: Address\ of\ Node\ C \oplus curr \rightarrow npx: Address\ of\ C)$

$\Rightarrow next = NULL$.


```
newNode->npx = XOR(curr, next);
```

If the `pos` goes from first to $n - 1$

Say, $\text{newNode} \rightarrow \text{npx} = \text{XOR}(\text{curr: Address of C, next: Address of Node D})$;

(As newNode will sit between Node C and Node D), basically :

Node D will become Node E and New Node becomes Node D,

as, it is following, the process:

The next pointer of D is: $C \oplus E$.

***i. e. NPX of New Node(Node D) will XOR result of Address of C,
and Address of Node E (which was Node D earlier).***

At nth (last node, say Node D)

$\text{newNode} \rightarrow \text{npx} = \text{XOR}(\text{curr: Address of D, next: NULL})$,

As, New Node will become Node E, hence,

The next pointer(NPX) of Node E (last Node) will be : $(D \oplus \text{NULL})$.

```
curr->npx = XOR(prev, newNode);
```

Now, say we need to enter a Node between Node C and Node D and we have series of nodes : Node A, Node B, Node C and Node D.

And after entering A new Node between Node C and Node D ,the New Node becomes Node D, and Node D becomes Node E.

Hence ,we need to update NPX (Next Pointer through XORing process) of Node C and Node E. Here ``curr → npx`` is the NPX of Node C.

Now why XORing address of prev and new Node? NewNode will become Node D and Prev hold address of Node B and we know:

The next pointer of C is : Address of Node B $\oplus$ Address of Node D

*curr → npx(NPX of Node C) = prev: Address of Node B $\oplus$
newNode: (Added Node which becomes Node D)Address of Node D.*

If $next \neq NULLPTR$, when we does not insert newNode to the last, newNode doesn't becomes last node.

```
if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}
```

Suppose we want to enter after Node A:

$$next = (Address\ of\ A \oplus (Address\ of\ A \oplus Address\ of\ C))$$

$$next = Address\ of\ C$$

$$Nextnext = XOR(Curr: Address\ of\ B, next \rightarrow npx(NPX\ of\ NODE\ C): D)$$

$$Where, D = (Address\ of\ B \oplus Address\ of\ D)$$

$$\therefore Nextnext = Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D)$$

$$= Address\ of\ D$$

$$next \rightarrow npx = XOR\left(newNode: \begin{pmatrix} [Newly\ entered\ Node\ at\ Pos: B] \\ Address\ of\ newNode(Pos: B) \end{pmatrix} \oplus Address\ of\ D\right)$$

Here,

Next $\rightarrow$ NPX is Node C's NPX and the above is the exact update we needed.

Insert After Element

```
while (curr != nullptr && curr->data != element)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
}
```

1. $next = XOR (prev: NULL, curr \rightarrow npx: \text{Address of Node B})$

2. $next = XOR(prev: \text{Address of A}, curr \rightarrow npx: C)$

$\Rightarrow$ Address of Node C

Note: C is the XOR result.

May not be address of Node C i.e. Result of $XOR(\text{Address of Node A}, \text{Address of Node C})$
The only time Node B's npx will contain Address of Node A When there will be only two node exists.

3. $next = XOR(prev: \text{Address of B}, curr \rightarrow npx: D)$

$\Rightarrow$ Address of Node D

Note: D is the XOR result..

May not be address of Node D i.e. Result of $XOR(\text{Address of Node B}, \text{Address of Node D})$
The only time Node C's npx will contain Address of Node B when there will be only three node exists. Similarly Node D's npx will contain Address of Node C, when there will be only four node exist.

... etc.

Similarly ,when the loop fails when we get situation like:

$next = (prev: \text{Address of Node A}, curr \rightarrow npx: \text{Address of Node A}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Two Node} \end{array} \right]$

$next = (prev: \text{Address of Node B}, curr \rightarrow npx: \text{Address of Node B}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Three Node} \end{array} \right]$

$next = (prev: \text{Address of Node C}, curr \rightarrow npx: \text{Address of Node C}); \left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Four Node} \end{array} \right]$

... etc.

Again ,we have:

```
next = XOR(prev, curr->npx);
```

If the `pos` goes from first to $n - 1$

Eg: $next = (prev: \text{Address of B} \oplus curr \rightarrow npx(\text{npx of Node C}): D),$

where, D is : $\text{Address of B} \oplus \text{Address of D}$, hence,

$\Rightarrow next = (\text{Address of B} \oplus (\text{Address of B} \oplus \text{Address of D}))$

And ,we know: by property $(B \oplus D) \oplus D = B$ (since , $D \oplus D = 0$)

Hence,we get: $(\text{Address of B} \oplus (\text{Address of B} \oplus \text{Address of D}))$

We get $next = \text{Address of D}$.

At nth (last node, say Node D)

$\Rightarrow next = (prev: Address\ of\ Node\ C \oplus curr \rightarrow npx(npx\ of\ Node\ D): D),$

where, D is : Address of C $\oplus$ NULL = Address of C, hence,

$\Rightarrow next = XOR(prev: Address\ of\ Node\ C \oplus curr \rightarrow npx: Address\ of\ C)$

$\Rightarrow next = NULL.$

```
newNode->npx = XOR(curr, next);
```

If the `pos` goes from first to n - 1

Say, newNode $\rightarrow$ npx = XOR(curr: Address of C, next: Address of Node D);

(As newNode will sit between Node C and Node D), basically :

Node D will become Node E and New Node becomes Node D,

as, it is following, the process:

The next pointer of D is: C $\oplus$ E.

***i. e. NPX of New Node(Node D) will XOR result of Address of C,
and Address of Node E (which was Node D earlier).***

At nth (last node, say Node D)

newNode $\rightarrow$ npx = XOR(curr: Address of D, next: NULL),

As, New Node will become Node E, hence,

The next pointer(NPX) of Node E (last Node) will be : (D $\oplus$ NULL).

```
curr->npx = XOR(prev, newNode);
```

Now, say we need to enter a Node between Node C and Node D and we have series of nodes : Node A, Node B, Node C and Node D.

And after entering A new Node between Node C and Node D , the New Node becomes Node D, and Node D becomes Node E.

Hence , we need to update NPX (Next Pointer through XORing process) of Node C and Node E. Here ``curr → npx`` is the NPX of Node C.

Now why XORing address of prev and new Node? NewNode will become Node D and Prev hold address of Node B and we know:

The next pointer of C is : Address of Node B $\oplus$ Address of Node D

*curr → npx(NPX of Node C) = prev: Address of Node B $\oplus$
newNode: (Added Node which becomes Node D)Address of Node D.*

If $next \neq NULLPTR$, when we does not insert newNode to the last, newNode doesn't becomes last node.

```
if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
}
```

Suppose we want to enter after Node A:

$$next = (Address\ of\ A \oplus (Address\ of\ A \oplus Address\ of\ C))$$

$$next = Address\ of\ C$$

$$Nextnext = XOR(Curr: Address\ of\ B, next \rightarrow npx(NPX\ of\ NODE\ C): D)$$

$$Where, D = (Address\ of\ B \oplus Address\ of\ D)$$

$$\therefore Nextnext = Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D)$$

$$= Address\ of\ D$$

$$next \rightarrow npx = XOR\left(newNode: \begin{pmatrix} [Newly\ entered\ Node\ at\ Pos: B] \\ Address\ of\ newNode(Pos: B) \end{pmatrix} \oplus Address\ of\ D\right)$$

Here,

Next $\rightarrow$ NPX is Node C's NPX and the above is the exact update we needed.

Memory Efficient Doubly Linked List – Deletion, Traversal, Destroy Operation Understanding

1. Deletion At Beginning

```
Node *next = XOR(nullptr, head->npx);
```

This exhibit the identity property : $X \oplus 0 = X$.

***It stores the address of 2nd Node i.e. $head \rightarrow npx$,
, the address of second node : B.***

As we know that ,:

The next pointer of A is : $NULL \oplus \text{Address of B}$

```
Node *temp = head;
if (next != nullptr)
{
    Node *nextNext = XOR(temp, next->npx);
    next->npx = XOR(nullptr, nextNext);
}
```

***Here Temp pointer hold the address of Node A present
at Head.***

As next $\neq$ NullPtr :

$$\text{nextNext} = \text{XOR}(\text{prev: Address of Node A}, \text{curr} \rightarrow \text{npx: C})$$

Note: C is the XOR result.

***May not be address of Node C i.e. Result of XOR (Address of Node A, Address of Node C)
The only time Node B's npx will contain Address of Node A When there will be only two node exists.***

Hence, nextNext

$$= \text{Address of Node A} \oplus \left(\text{Address of Node A} \oplus \text{Address of Node C} \right)$$

And we know by property:

$$(B \oplus D) \oplus D = B \text{ (since , } D \oplus D = 0)$$

$$\therefore \text{nextNext} = \text{Address of C.}$$

```
next->npx = XOR(nullptr, nextNext);
```

$$\text{next} \rightarrow \text{npx} = \text{XOR}(\text{NULLPTR}, \text{nextNext});$$

Hence it is very much needed 2nd Node (i.e. Node B) becomes Node A and hence next pointer of Node B must contain address of Node C.

Hence, $next \rightarrow npx = NULL \oplus nextNext = \text{Address of Node C}.$
 $next \rightarrow npx = \text{Address of Node C}.$

And , then we free Node A, the first Node of the list.

-- This process continues ...

2. Deletion At End

```
while (true)
{
    next = XOR(prev, curr->npx);
    if (next == nullptr)
        break; // curr is the last node
    prev = curr;
    curr = next;
}
```

1. $next = XOR (prev: NULL , curr \rightarrow npx: \text{Address of Node B})$

2. $next = XOR(prev: \text{Address of A} , curr \rightarrow npx: C)$

$\Rightarrow \text{Address of Node C}$

Note: C is the XOR result.

May not be address of Node C i.e. Result of $XOR (\text{Address of Node A} , \text{Address of Node C})$
The only time Node B's npx will contain Address of Node A When there will be only two node exists.

3. $next = XOR(prev: Address\ of\ B, curr \rightarrow npx: D)$
 $\Rightarrow Address\ of\ Node\ D$

Note: D is the XOR result..

$\left[\begin{array}{l} \text{May not be address of} \\ \text{Node } D \text{ i.e. Result of} \\ XOR \left(\begin{array}{l} \text{Address of Node } B, \\ \text{Address of Node } D \end{array} \right) \\ \text{The only time Node } C's \text{ npx} \\ \text{will contain Address of Node } B \\ \text{when there will be only three node} \\ \text{exists. Similarly Node } D's \text{ npx will} \\ \text{contain Address of Node } C, \text{ when there will} \\ \text{be only four node exist.} \end{array} \right]$

... etc.

Similarly ,when the loop fails when we get situation like:

$next = (prev: Address\ of\ Node\ A, curr \rightarrow npx: Address\ of\ Node\ A);$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Two Node} \end{array} \right]$

$next = (prev: Address\ of\ Node\ B, curr$
 $\rightarrow npx: Address\ of\ Node\ B);$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Three Node} \end{array} \right]$

$next = (prev: Address\ of\ Node\ C, curr \rightarrow npx: Address\ of\ Node\ C);$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Four Node} \end{array} \right]$

... etc.

When we have more than 1 node(List with more than that of Single Node)

```
if (prev != nullptr)
{
    Node *prevPrev = XOR(prev->npx, curr);
    prev->npx = XOR(prevPrev, nullptr);
}
```

Suppose we have a List of Node A, Node B, Node C and Node D. Node D will be deleted and prev pointer points to Node C and next pointer of Node C must point to previous Node B's by logic:

The next pointer of Node D is: Address of Node C $\oplus$ NULL.

$prevPrev = prev \rightarrow npx(\text{Node C's } npx) \oplus curr: \text{Address of D}$
Here, Node C's $npx = \text{Address of B} \oplus \text{Address of D}$.

Hence , we can write:

$\Rightarrow prevPrev = (\text{Address of B} \oplus \text{Address of D}) \oplus \text{Address of D}$.

By using logic: $(B \oplus D) \oplus D = B$ (since , $D \oplus D = 0$), we get:

$\Rightarrow prevPrev = \text{Address of B}$.

$\therefore$ We know, the last node's next pointer would represent like:

The next pointer of D is $C \oplus NULL$

Hence : $prev \rightarrow npx(\text{Node C's } NPX) = XOR(prevPrev, NULL)$

$\Rightarrow$ *Next Pointer of Node C = Address of B $\oplus$ NULL*
= Address of B.

And then we free the `curr` pointer variable pointed to last node.

3. Deletion At Position

```
while (curr != nullptr && i < pos)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

1. $next = XOR (prev: NULL, curr \rightarrow npx: \text{Address of Node B})$

2. $next = XOR(prev: \text{Address of A}, curr \rightarrow npx: C)$

$\Rightarrow$ Address of Node C

Note: C is the XOR result.

May not be address of Node C i.e. Result of $XOR(\text{Address of Node A}, \text{Address of Node C})$
The only time Node B's npx will contain Address of Node A When there will be only two node exists.

3. $next = XOR(prev: Address\ of\ B, curr \rightarrow npx: D)$
 $\Rightarrow Address\ of\ Node\ D$

Note: D is the XOR result..

$\left[\begin{array}{l} \text{May not be address of} \\ \text{Node } D \text{ i.e. Result of} \\ XOR \left(\begin{array}{l} \text{Address of Node } B, \\ \text{Address of Node } D \end{array} \right) \\ \text{The only time Node } C's \text{ npx} \\ \text{will contain Address of Node } B \\ \text{when there will be only three node} \\ \text{exists. Similarly Node } D's \text{ npx will} \\ \text{contain Address of Node } C, \text{ when there will} \\ \text{be only four node exist.} \end{array} \right]$

... etc.

Similarly ,when the loop fails when we get situation like:

$next = (prev: Address\ of\ Node\ A, curr \rightarrow npx: Address\ of\ Node\ A);$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Two Node} \end{array} \right]$

$next = (prev: Address\ of\ Node\ B, curr$
 $\rightarrow npx: Address\ of\ Node\ B);$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Three Node} \end{array} \right]$

$next = (prev: Address\ of\ Node\ C, curr \rightarrow npx: Address\ of\ Node\ C);$ $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Four Node} \end{array} \right]$

... etc.

```
next = XOR(prev, curr->npx); // Decode node after curr
```

If the `pos` goes from first to $n - 1$

Eg: $next = (prev: Address\ of\ B \oplus curr \rightarrow npx(npx\ of\ Node\ C): D),$

where, D is : $Address\ of\ B \oplus Address\ of\ D$, hence,

$\Rightarrow next = (Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D))$

And , we know: by property $(B \oplus D) \oplus D = B$ (since , $D \oplus D = 0$)

Hence, we get: $(Address\ of\ B \oplus (Address\ of\ B \oplus Address\ of\ D))$

We get $next = Address\ of\ D$.

At nth (last node, say Node D)

$\Rightarrow next = (prev: Address\ of\ Node\ C \oplus curr \rightarrow npx(npx\ of\ Node\ D): D),$

where, D is : $Address\ of\ C \oplus NULL = Address\ of\ C$, hence,

$\Rightarrow next = XOR(prev: Address\ of\ Node\ C \oplus curr \rightarrow npx: Address\ of\ C)$

$\Rightarrow next = NULL$.


```
Node *prevPrev = XOR(prev->npx, curr);  
prev->npx = XOR(prevPrev, next);
```

When curr is not pointing to end of the list $\left[\begin{array}{l} \text{The last Node} \\ \text{not to be deleted} \end{array} \right]$

*Suppose we have a List of Node A, Node B, Node C and Node D.
Node C will be deleted and prev pointer points to Node B and
Node C will get deleted then ,next pointer logic :*

The next pointer of B is: Address of Node A $\oplus$ Address of Node C.

Where as Curr points to Node C means ,Curr holds Address of Node C.

*Hence ,prevPrev =
prev $\rightarrow$ npx: (Address of Node A $\oplus$ Address of Node C)
 $\oplus$ curr: Address of Node C.

= (Address of Node A $\oplus$ Address of Node C) $\oplus$ Address of Node C.

 $\Rightarrow$ And, as we know : $(B \oplus D) \oplus D = B$ (since , $D \oplus D = 0$)

 $\therefore$ (Address of Node A $\oplus$ Address of Node C) $\oplus$ Address of Node C.
= Address of Node A .*

As, Node C will get deleted , the Nodes will be left is:

Node A, Node B and Node D , hence,

NPX of Node B is: Address of A $\oplus$ Address of D.

Hence,

$prev \rightarrow npx = prevPrev : \text{Address of Node A} \oplus next: \text{Address of D}.$

The exact XORing we need to update Node B.

When we want to delete the last node

Suppose we have a List of Node A, Node B, Node C and Node D. Node D will be deleted and prev pointer points to Node C and next pointer of Node C must point to previous Node B's by logic:

The next pointer of Node D is: Address of Node C $\oplus$ NULL.

Suppose we have a List of Node A, Node B, Node C and Node D. Node D will be deleted and prev pointer points to Node C and next pointer of Node C must point to previous Node B's by logic:

The next pointer of Node D is: Address of Node C $\oplus$ NULL.

$prevPrev = prev \rightarrow npx(\text{Node C's } npx) \oplus curr: \text{Address of D}$

Here, Node C's $npx = \text{Address of B} \oplus \text{Address of D}.$

Hence , we can write:

$$\Rightarrow prevPrev = (Address\ of\ B \oplus Address\ of\ D) \oplus Address\ of\ D.$$

By using logic: $(B \oplus D) \oplus D = B$ (since , $D \oplus D = 0$), we get:

$$\Rightarrow prevPrev = Address\ of\ B.$$

$\therefore$ We know, the last node's next pointer would represent like:

$$The\ next\ pointer\ of\ D\ is\ C \oplus NULL$$

$$Hence : prev \rightarrow npx(Node\ C's\ NPX) = XOR(prevPrev, NULL)$$

$$\Rightarrow Next\ Pointer\ of\ Node\ C = Address\ of\ B \oplus NULL$$

$$= Address\ of\ B.$$

And then we free the `curr` pointer variable pointed to last node.

```
if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(prev, nextNext);
}
```

*When $next \neq NULL$ (when last node is to be Deleted, $next = NULL$,):
(when last node is not be Deleted, $next \neq NULL$):*

Suppose there is a list composed of Node A, Node B, Node C and Node D.

If Node C is to get deleted then, we need to update :

$\rightarrow$ next pointer of Node B.

$\rightarrow$ and , next pointer of Node D.

We have already updated , $\rightarrow$ next pointer of Node B.

Now, we have to, $\rightarrow$ update next pointer of Node D.

next pointer of Node D, was $NULL \oplus \text{Address of C}$,

now, it will be : next pointer of Node D, was $NULL \oplus \text{Address of B}$,

Note: Next $\rightarrow$ NPX is NODE D's NPX as NEXT pointer points to Node D.

nextNext = Curr: Address of C $\oplus$ next $\rightarrow$ npx: Address of C , and we know:

By Property : $X \oplus X = 0$, we get:

nextNext = Curr: Address of C $\oplus$ next $\rightarrow$ npx: Address of C = NULL.

And next,

```
next->npx = XOR(prev, nextNext);
```

*next $\rightarrow$ npx(NPX of NODE D) = prev: Address of B $\oplus$ nextNext
: NULL .*

next $\rightarrow$ npx(NPX of NODE D) = Address of B .

And after freeing curr i.e. NODE C , current NODE D becomes NODE C.

List Traversal and Destroy List

```
while (curr != nullptr)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
}
```

1. $next = XOR (prev: NULL, curr \rightarrow npx: \text{Address of Node B})$

2. $next = XOR(prev: \text{Address of A}, curr \rightarrow npx: C)$

$\Rightarrow \text{Address of Node C}$

Note: C is the XOR result.

*May not be address of
Node C i.e. Result of
 $XOR(\text{Address of Node A},$
 $\text{Address of Node C})$
The only time Node B's npx
will contain Address of Node A
When there will be only two node
exists.*

3. $next = XOR(prev: \text{Address of B}, curr \rightarrow npx: D)$

$\Rightarrow \text{Address of Node D}$

Note: D is the XOR result..

*May not be address of
Node D i.e. Result of
 $XOR \left(\text{Address of Node B}, \right.$
 $\left. \text{Address of Node D} \right)$
The only time Node C's npx
will contain Address of Node B
when there will be only three node
exists. Similarly Node D's npx will
contain Address of Node C, when there will
be only four node exist.*

... etc.

Similarly ,when the loop fails when we get situation like:

next = (prev: Address of Node A, curr → npx: Address of Node A); $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Two Node} \end{array} \right]$

*next = (prev: Address of Node B, curr
→ npx: Address of Node B);* $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Three Node} \end{array} \right]$

next = (prev: Address of Node C, curr → npx: Address of Node C); $\left[\begin{array}{l} \text{When} \\ \text{There is only} \\ \text{Four Node} \end{array} \right]$

... etc.

Memory Allocation(Theoretical Vs Reality)

We have seen:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *npx(a pointer to another Node) is typically 8 bytes.*

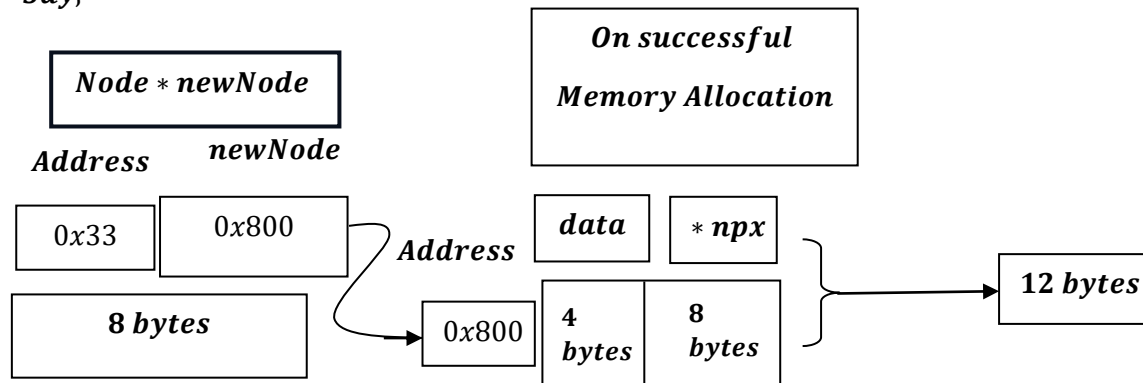
→ *Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npx) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

→ *newNode will try to allocate memory for data i.e. 4 bytes ,npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



The above is Theoretical , But if we do a reality check:

Theoretical vs. Real-World Allocation

In a real programming environment (like the C ++ environments you typically work in)

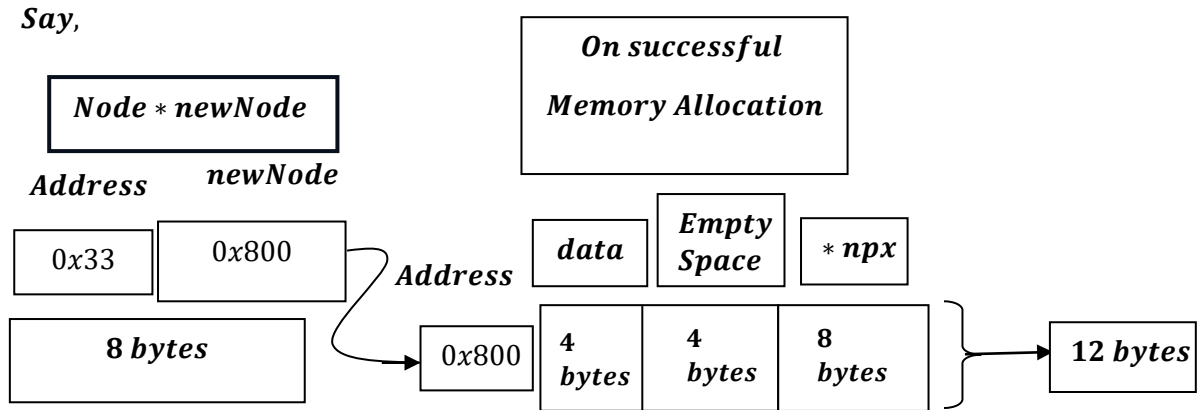
, the allocation might not be exactly 12 bytes because of

how CPUs access memory:

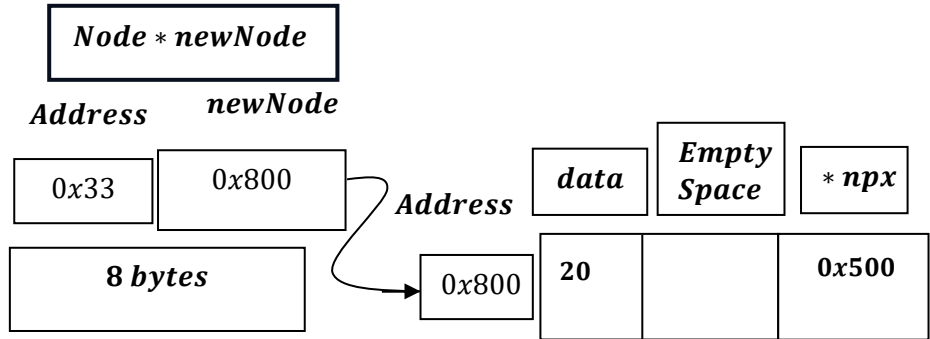
- **Memory Alignment:** Most 64 – bit compilers align data to 8 – byte boundaries to improve performance.
- **Padding:** To align the 8 – byte `npx` pointer, the compiler will often add 4 bytes of "padding" after the 4 – byte data integer.
- **Actual Size:** Consequently, `sizeof(Node)` would likely return 16 bytes on your system, rather than the 12 bytes shown in the document.

<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0	4 bytes	data	The integer value (e. g., 2).
4	4 bytes	Padding	Empty space added by the compiler.
8	8 bytes	npx	The XORed address (e. g., 0x500).

It will look like:



Like:



In a 64 – bit environment, the compiler aligns memory to the size of the largest primitive member in a structure – in this case, the 8 – byte npx pointer.

Because the data integer only occupies 4 bytes, the compiler inserts empty space (padding) to ensure the pointer starts at an address that is a multiple of 8.

Why it is Memory Efficient?

1. The Real Specialty: Memory Efficiency

*In a standard Doubly Linked List, each node requires **two** pointers: next and prev.*

- ***Standard DLL Node:** $[prev | data | next] \rightarrow 2 \text{ pointers} + 1 \text{ data field.}$*
- ***XOR Node:** $[npx | data] \rightarrow 1 \text{ pointer field} + 1 \text{ data field.}$*

By using the XOR bitwise operation ($A \oplus B$)

, we compress two addresses into a single memory space (npx).

<i>Feature</i>	<i>Standard Doubly Linked List</i>	<i>XOR Linked List</i>
<i>Pointers per Node</i>	<i>2 (next, prev)</i>	<i>1 (npx)</i>
<i>Memory Usage</i>	<i>Higher</i>	<i>Lower (saves ~50% pointer space)</i>
<i>Traversal</i>	<i>Bidirectional (Simple)</i>	<i>Bidirectional (Requires bitwise math)</i>
<i>Readability</i>	<i>High</i>	<i>Low (Complex logic)</i>

2. Does it reduce LOC? (Spoiler: Usually No)

Actually, the XOR Linked List often requires **more lines of code** than a standard list. This is because:

- **Pointer Arithmetic:** We cannot simply say `temp = temp → next`. We must perform an XOR calculation using the current node and the previous node's address every time you move.
- **Type Casting:** C++ pointers cannot be XORed directly. We must cast them to `uintptr_t`, perform the math, and cast them back to a pointer.
- **Helper Functions:** We usually need to write a specific XOR() helper function to keep the code readable.

Comparison Example (Traversal Logic):

Standard DLL:

```
current = current ->next;
```

XOR Linked List:

```
Node *next = XOR(prev, curr->npx);  
prev = curr;  
curr = next;
```

3. Why use it if it's harder to code?

The XOR Linked List is a niche optimization used in environments where **every byte of RAM counts**, such as:

1. **Embedded Systems:** Devices with extremely limited memory (kilobytes of RAM).
2. **Large – scale Databases:** When storing billions of nodes, saving 4 – 8 bytes per node adds up to gigabytes of saved memory.
3. **Old – school Game Development:** Where hardware limitations forced developers to be extremely clever with bit manipulation.

The XOR Linked List is a Space – Optimization technique, not a code – shortening technique. It makes your program use less RAM at the cost of making the code more difficult to read and maintain.
